邊緣人工智慧
Edge AI

(General-Purpose) AI Accelerators

吳凱強
Kai-Chiang Wu



Outline

- **Benchmarking Metrics**
- GEMM Accelerator Design
- DNN Accelerator Design
- Energy



How Can We Compare Designs?

- Key metrics
  - Accuracy
  - Throughput
  - Latency
  - Energy consumption
  - Power consumption
  - Hardware cost
- Importance of each of these metrics
- Factors that affect each metric



Accuracy

- **Indicate the quality of the result for a given task**
  - **The units used to measure accuracy depend on the task**
  - **Image classification**
    - Top-1, Top-5 accuracy
  - **Object detection**
    - Mean Average Precision (mAP)
- **Factors that affect accuracy**
  - **Difficulty of the task and dataset**
    - Object detection or semantic segmentation is more difficult than classification
  - **Evaluating hardware performance using well-studied, widely used DNN models, tasks, and datasets**



Accuracy

- **Motivated by the impact of the SPEC benchmarks for general-purpose computing**
- **MLPerf**
  - **Serve as a common set of well-studied DNN models to evaluate hardware performance**
  - **Put together a broad suite of DNN models**
    - Various types of DNNs
    - Various tasks
    - Image classification, object detection, semantic segmentation, translation, speech-to-text, recommendation, sentiment analysis, and reinforcement learning
  - **Enable fair comparison of various software frameworks, hardware accelerators**



Throughput and Latency

- **Throughput**
  - **Amount of data that can be processed/completed in a given time period**
  - **Reported as** $\frac{Number\ of\ operations}{seconds}$
  - **Applications that action needs to be promptly taken based on the analysis require high throughput**
    - E.g., Real-time video processing, security, terrorist prevention, medical diagnosis, drug discovery
- **Latency (response time)**
  - $time_{output\ generated} - time_{input\ arrives\ to\ a\ system}$
  - **Real-time interactive applications require low latency**
    - E.g., AR, autonomous navigation, robotics



Throughput vs. Latency

- **Pitfall**
  - Throughput and latency can be directly derivable from one another
- **Batching input data**
  - Increase throughput but increase latency in real-time video processing
  - E.g., at 30 frames per second and a batch of 100 frames, some frames will experience at least 3.3 second delay
- **Real-time navigation requires low latency but can bear low throughput**

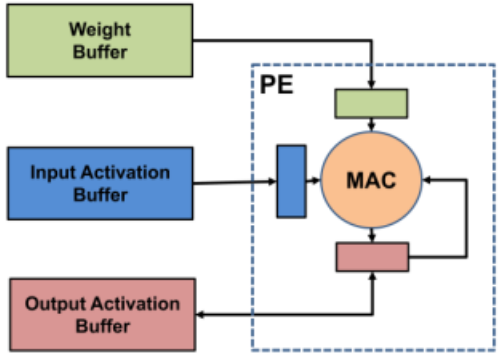


Factors that Affect Throughput

$$\frac{Inferences}{second} = \frac{operations}{seconds} \times \frac{1}{\frac{operations}{inference}}$$

Dictated by DNN hardware and DNN models

Dictated by the DNN model



Consider a system comprised of multiple processing elements (PEs)
PE – a simple or primitive core that performs a single MAC operation

$$\frac{operations}{seconds} = \left(\frac{1}{\frac{cycles}{operation}} \times \frac{cycle}{second}\right) \times numberof\ PEs \times utilization\ of\ PEs$$

Peak throughput for a single PE

Amount of parallelism

degradation due to the inability of the architecture to effectively utilize the PEs

Increase Throughput

- **Increase $\frac{cycle}{second}$ → higher clock frequency → reducing critical path**
  - **Affected by design of the MAC**
- **Increase $numberof\ PEs$ → multiple MACs perform in parallel**
  - **If Area cost of the system is fixed**
    - Increasing the area density of the PEs
    - Trading off on-chip storage area for more PEs
      - **Affect the utilization of the PEs**
  - **Increasing the density of PEs**
    - Reducing the logic associated with delivering operands to a MAC
      - **Multiple MACs share control logic**
      - **E.g., SIMD and SIMT with CPUs and GPUs**



Increase Throughput

$$utilization\ of\ PEs = \frac{number\ of\ active\ PEs}{number\ of\ PEs} \times utilization\ of\ active\ PEs$$

Ability to distribute the workload to PEs

How efficiently those active PEs are processing the workload.

- **Number of active PEs – number of PEs that receive work**
  - **Distribute the workload to as many PEs as possible**
- **Ability to distribute the workload**
  - **Determined by the flexibility of the architecture**
  - **Network-on-Chip (NoC) matters!**



Bandwidth Requirements

- Affected by the amount of data reuse available in DNN models
- Amount of data reuse can be exploited by the memory hierarchy and dataflow
- Dataflow determines the order of operations and where data is stored and reused
- E.g., Larger batch size → higher amount of data reuse



Interplay Between

- Reduce the likelihood that a PE needs to wait for data is to store some data locally near or within the PE
  - Increasing the chip area of on-chip storage
  - Reducing the number of PEs

How to balance areas of PEs and on-chip storage?



How DNN Models Affect

- $\frac{operations}{inference}$ **depends on the DNN model only**
- $\frac{operations}{seconds}$ **depends on both DNN model and hardware**
  - **Efficient DNN models**
    - Reduce the number of MAC operations
      → reduce number of operations per inference
    - Wide range of layer shapes
      → Poor utilization of PEs
      → Reduce the overall operations per second
- **Number of ineffectual operations is a function of both the DNN model and the input data**
  - **E.g., MAC accumulates anything multiplied by zero → sparsity (ineffectual operation)**



Effectual and Ineffectual Operations

$$\frac{operations}{cycle} = \frac{EO + UIO}{cycle} \times \frac{EO}{EO + UIO} \times \frac{1}{\frac{EO}{operations}}$$

EO: $effectual\ operations$
UIO: $unexploited\ ineffectual\ operations$

Constant for a given hardware accelerator design

Ability of the hardware to exploit ineffectual operations

Related to amount of sparsity and depends on the DNN model

- **Amount of sparsity increase** $\rightarrow \frac{operations}{cycle}$ **increase**
  - **Requires additional hardware** $\rightarrow$ **increase critical path, reduce area density of PE** $\rightarrow$ **reduce** $\frac{operations}{second}$
  - **Trade off**

Reducing Precision (e.g., Quantization)

- Increase $\frac{operations}{second}$
- Lower precision (fewer bits)
  → lower bandwidth requirement
  → increase utilization of PEs
- Multiple levels of precision requires addition hardware
  → increase critical path, reduce area density of PE
  → decrease $\frac{operations}{second}$



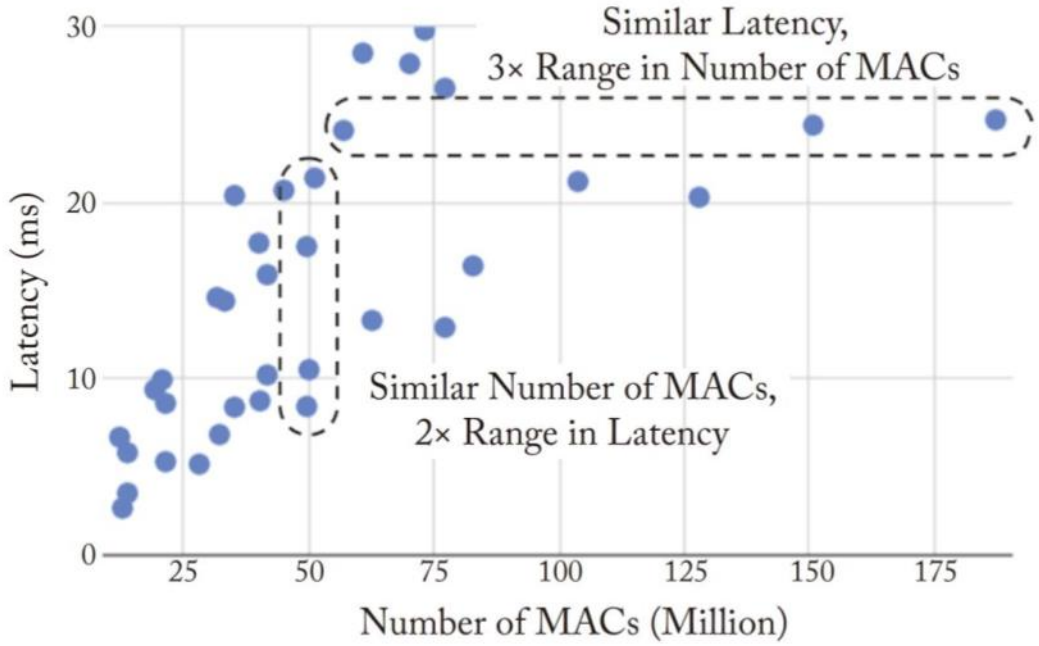
Factors that Affect Inferences per Second

Factor	Hardware	DNN Models	Input Data
Operations per inference		V	
Operations per cycle	V		
Cycles per second	V		
Number of PEs	V		
Number of active PEs	V	V	
Utilization of active PEs	V	V	
Eff ectual operations out of (total) operations		V	V
Eff ectual operations plus unexploited ineffectual operations per cycle	V		



MAC Alone as an Performance Estimation

- **Measured on Pixel phone**



#MACs is not a good predictor of latency …

Design efficient DNN models with hardware in the loop!

How Can We Compare Designs?

- Key metrics
  - Accuracy
  - Throughput
  - Latency
  - Energy consumption
  - Power consumption
  - Hardware cost
- Importance of each of these metrics
- Factors that affect each metric



Energy Efficiency and Power Consumption

- **Energy Efficiency**
  - Given unit of energy, how much data that can be processed
  - Edge processing requires high energy efficiency
  - $\frac{number\ of\ operations}{joule}$
- **Power consumption**
  - Amount of energy consumed per unit time
  - Increase power consumption $\rightarrow$ increase heat dissipation
  - Thermal Design Power (TDP) limits max power consumption
  - $\frac{joules}{second}\ (watt)$



Relation between Throughput

$$\frac{inferences}{second} \leq \max\left(\frac{joules}{second}\right) \times \frac{inferences}{joules}$$

- increasing the $\frac{inferences}{joules}$

  $\rightarrow$ increase $\frac{inferences}{second}$

  $\rightarrow$ increase throughput

Factors that Affect Energy Efficiency

$$\frac{inferences}{joules} = \frac{operations}{joules} \times \frac{1}{\frac{operations}{inference}}$$

Dictated by both the hardware and DNN model

Dictated by DNN model

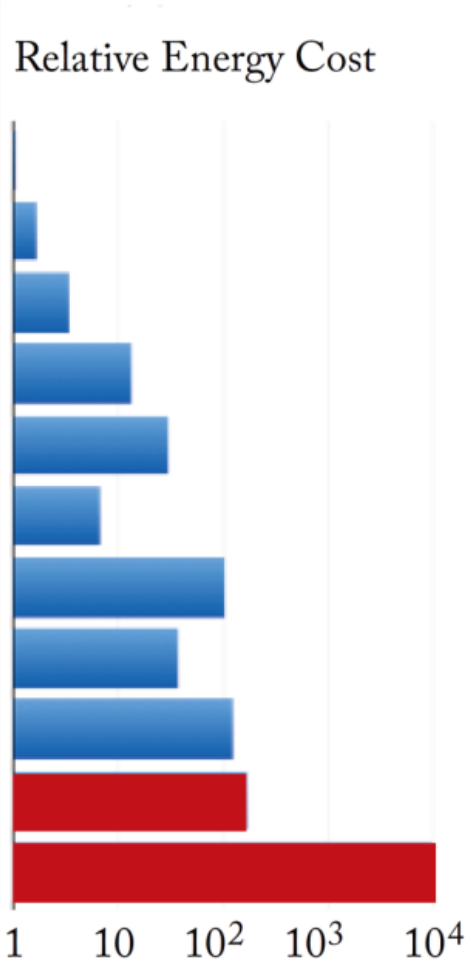
- $Energy_{total} = Energy_{data} + Energy_{MAC}$
- $\frac{joules}{operation} = \alpha \times C \times V_{DD}^2$

α: switching activity
C: switching capacitance
V_{DD}: supply Voltage

Energy Consumption

- 45nm process
- Energy consumption is dominated by the data movement
  - Capacitance of data movement tends to be much higher than the capacitance for arithmetic operations (PEs)
- Larger memories and longer interconnects
  - Consume more energy
- Optimizing data movement is the key!

Operation:	Energy (pJ)
8b Add	0.03
16b Add	0.05
32b Add	0.1
16b FP Add	0.4
32b FP Add	0.9
8b Multiply	0.2
32b Multiply	3.1
16b FP Multiply	1.1
32b FP Multiply	3.7
32b SRAM Read (8KB)	5
32b DRAM Read	640



Operations per Joule

$$\frac{operations}{joules} = \frac{EO + UIO}{joules} \times \frac{EO}{EO + UIO} \times \frac{1}{\frac{EO}{operations}}$$

Constant for a given hardware accelerator design

Ability of the hardware to exploit ineffectual operations

Related to amount of sparsity and depends on the DNN model

- **A function of the ability of the hardware to exploit sparsity to avoid performing ineffectual MAC operations**

Hardware Cost

- Process technology
- Amount of on-chip storage and compute → Area
  - PEs in DNN accelerator
  - Cores in CPU and GPU
  - Digital Signal Processing (DSP) Engines in FPGA
- Off-chip bandwidth
  - Complexity of the packaging
  - PCB design
  - Off-cihp DRAM, NVLink, etc.



Metrics for DNN Hardware

- Accuracy
  - Quality of result for a given task
- Throughput
  - Analytics on high volume data
  - Real-time performance (e.g., video at 30 fps)
- Latency
  - For interactive applications (e.g., autonomous navigation)
- Energy and Power
  - Edge and embedded devices have limited battery capacity
  - Data centers have stringent power ceilings due to cooling costs
- Hardware Cost
  - Money



Specifications to Evaluate Metrics

- Accuracy
  - Difficulty of dataset and/or task should be considered
- Throughput
  - Number of cores (include utilization along with peak performance)
  - Runtime for running specific DNN models
- Latency
  - Include batch size used in evaluation
- Energy and Power
  - Power consumption for running specific DNN models
  - Include external memory access
- Hardware Cost
  - On-chip storage, number of cores, chip area + process technology



Example: Metrics of Eyeriss Chip

ASIC Specs	Input
Process Technology	65nm LP TSMC (1.0V)
Total Core Area (mm^2)	12.25
Total On-Chip Memory (kB)	192
Number of Multipliers	168
Clock Frequency (MHz)	200
Core area (mm^2) /multiplier	0.073
On-Chip memory (kB) / multiplier	1.14
Measured or Simulated	Measured

Metric	Units	Input
Name of CNN Model	Text	AlexNet
Top-5 error classification on ImageNet	#	19.8
Supported Layers		All CONV
Bits per weight	#	16
Bits per input activation	#	16
Batch Size	#	4
Runtime	ms	115.3
Power	mW	278
Off-chip Access per Image Inference	MBytes	3.85
Number of Images Tested	#	100



Comprehensive Coverage

- All metrics should be reported for fair evaluation of design tradeoffs
- Examples of what can happen if certain metric is omitted:
  - Without the accuracy given for a specific dataset and task, one could run a simple DNN and claim low power, high throughput, and low cost – however, the processor might not be usable for a meaningful task
  - Without reporting the off-chip bandwidth, one could build a processor with only multipliers and claim low cost, high throughput, high accuracy, and low chip power – however, when evaluating system power, the off-chip memory access would be substantial
- Are results measured or simulated? On what test data?



Evaluation Process

- The evaluation process for whether a DNN system is a viable solution for a given application might go as follows:

1. Accuracy determines if it can perform the given task
2. Latency and throughput determine if it can run fast enough and in real-time
3. Energy and power consumption will primarily dictate the form factor of the device where the processing can operate
4. Cost, which is primarily dictated by the chip area, determines how much one would pay for this solution



Outline

- Benchmarking Metrics
- **GEMM Accelerator Design**
  - **GEMM**
  - Systolic and Other Variation
- DNN Accelerator Design
- Energy



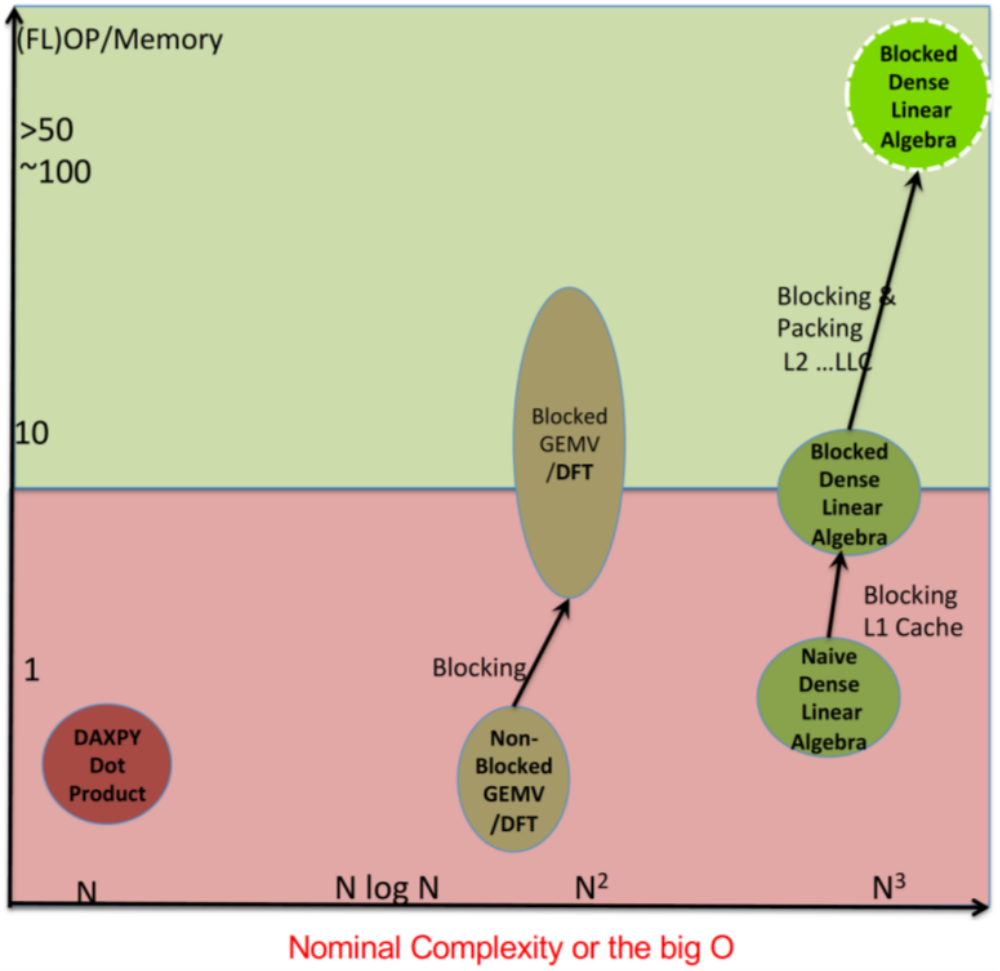
Basic Linear Algebra Subprograms (BLAS)

- The building blocks of most of scientific computing
- Application codes either call these routines directly, indirectly through other libraries that themselves call the BLAS, or via environments like Matlab
- Three Levels:
  - Level 1: vector-vector operations
  - Level 2: matrix-vector operations
  - Level 3: matrix-matrix operations
- Implementing level-3 BLAS in terms of GEneral Matrix-Matrix multiplication (GEMM) kernels
  - Only a small set of kernels needed to be highly optimized



Basic Linear Algebra Subprograms (BLAS)

BLAS Level	Example	# of mem refs	# of flops	Flop /Memory
1	**Axpy** Dot product	3n	2n	2/3
2	**GEMV** Matrix-vector mult	n^2	$2n^2$	2
3	**GEMM** Matrix-matrix mult	$4n^2$	$2n^3$	n/2



Native GEMM Implementation

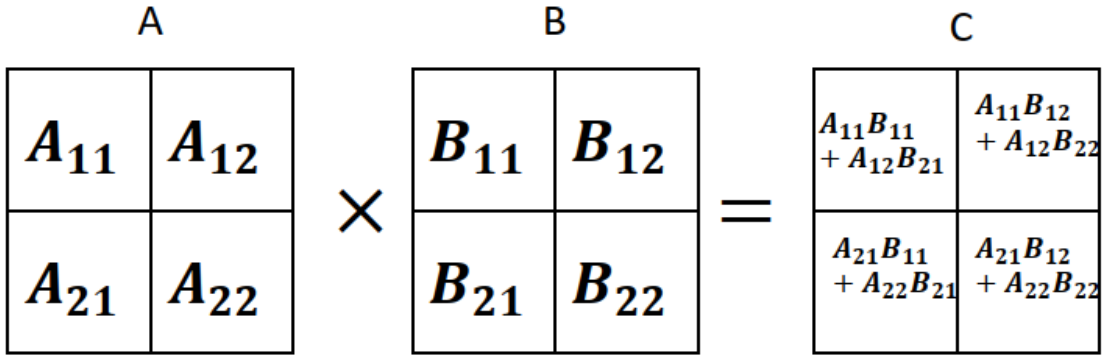
```
1   C = C+ A*B
2   for i=1~n
3       for j=1~n
4           for k=1~n
5               C(i,j)=C(i,j)+A(i,k)*B(k,j)
```

- Flops = $2 \times n^3 \rightarrow O(n^3)$
- Operate on $3 \times n^2$ words of memory
- Questions
  - What is the performance of this code?
  - Where is the cause of problem with this code?
  - How can we improve it ?
  - What is an easy way to increase the performance with minimum code change?



Recursive GEMM

- Divide & conquer
- How does it work with non-squared matrices?
- Why does it improve performance?



$$\begin{pmatrix} A_1 \\ A_2 \end{pmatrix} B = \begin{pmatrix} A_1 B \\ A_2 B \end{pmatrix}$$

Case 1

$$(A_1, A_2)\begin{pmatrix} B_1 \\ B_2 \end{pmatrix} = (A_1 B + A_2 B)$$

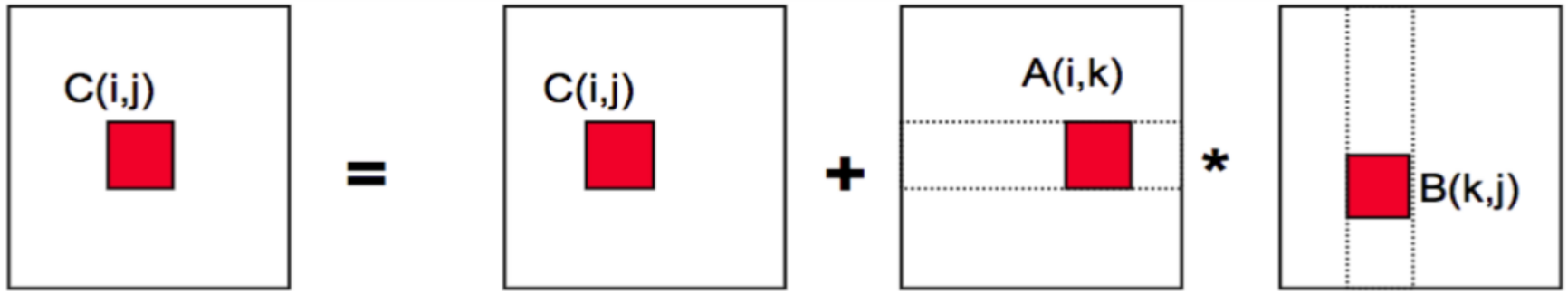
Case 2

$$A(B_1, B_2) = (A\,B_1, A\,B_2)$$

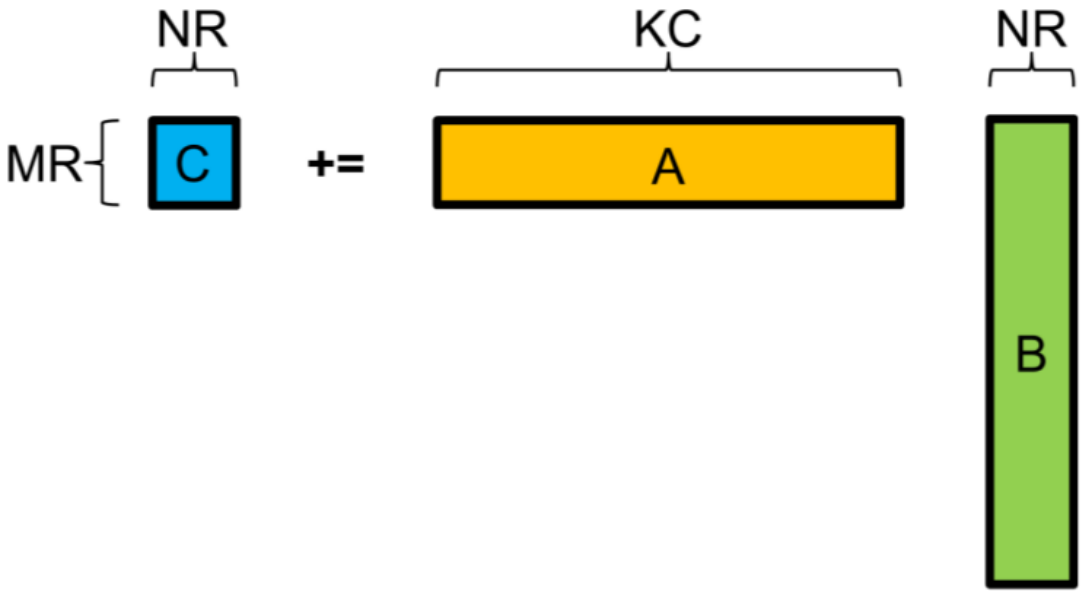
Case 3

Blocked GEMM

```
1   Consider A,B,C to be n-by-n matrix viewed as N-by-N Matrices
     of b-by-b subblock where b=n/N is called the block size
2   C = C+ A*B
3   for i=1~N
4       for j=1~N
5           {read block C(i,j) into fast memory}
6           for k=1~N
7               {read block A(i,j) into fast memory}
8               {read block B(i,j) into fast memory}
9               C(i,j)=C(i,j)+A(i,k)*B(k,j)
10          {write block C(i,j) back to slow memory}
```

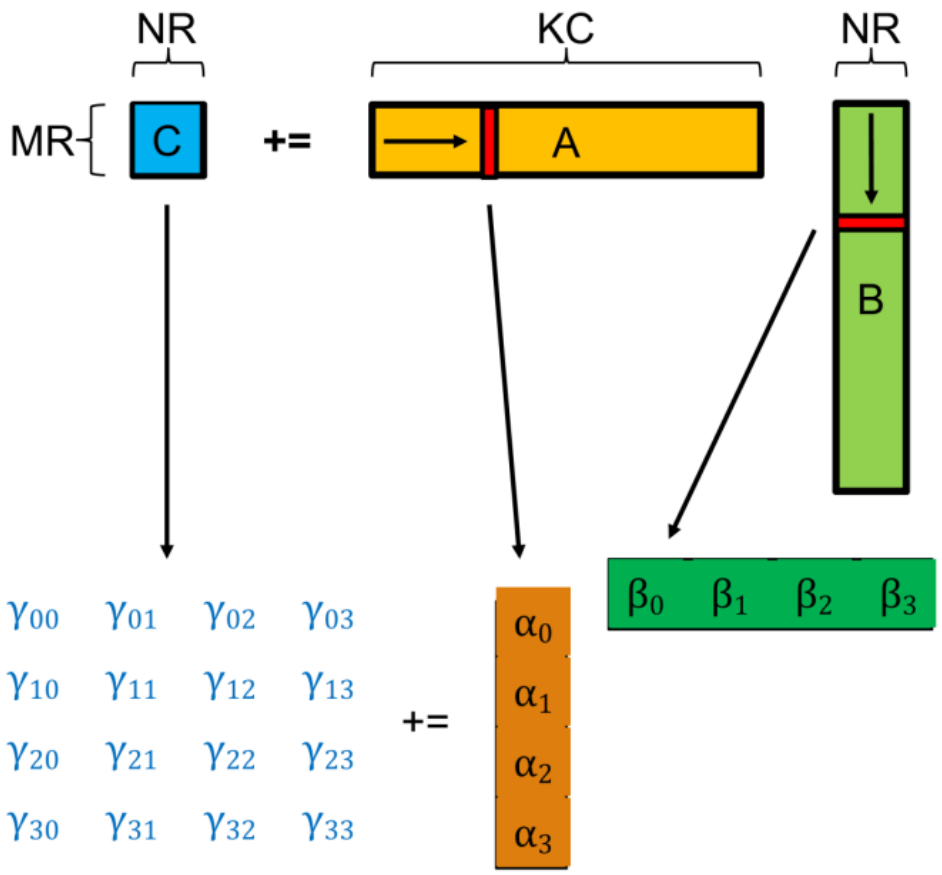


The GEMM Micro-kernel




```
1   for ( 0 to NC: NR )
2       for ( 0 to MC: MR )
3           for ( 0 to KC: 1 )
4               // block-dot product
```

The GEMM Micro-kernel



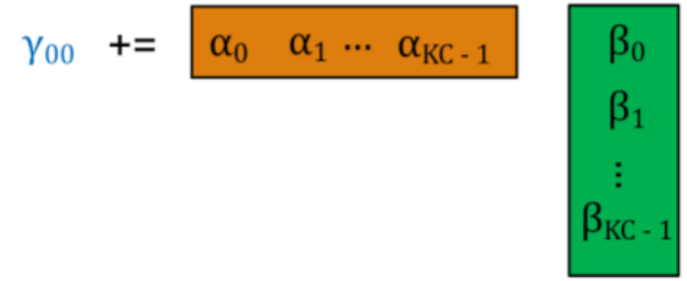

```
1  for ( 0 to NC: NR )
2      for ( 0 to MC: MR )
3          for ( 0 to KC: 1 )
4              // block-dot product
```

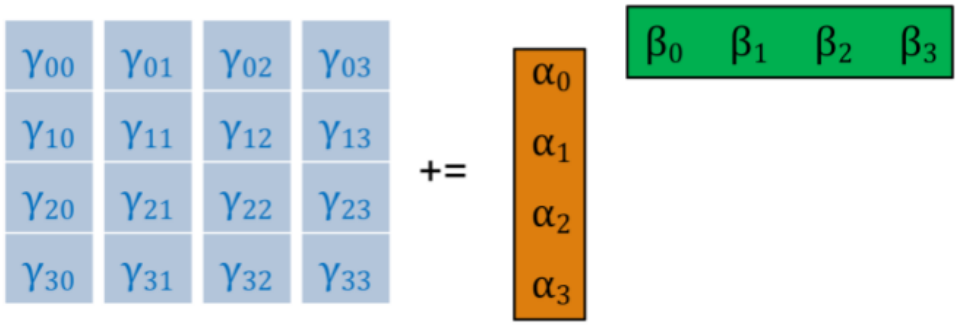
Typical micro-kernel loop iteration ("block-dot product")

- **Load column of packed A**
- **Load row of packed B**
- **Compute outer product**
- **Update C (kept in registers)**

The GEMM Micro-kernel

- **Why MR x NR block-dot product?**
  - **Why not a simple scalar dot product?**
  - **Ratio of floating-point operations to memory operations**



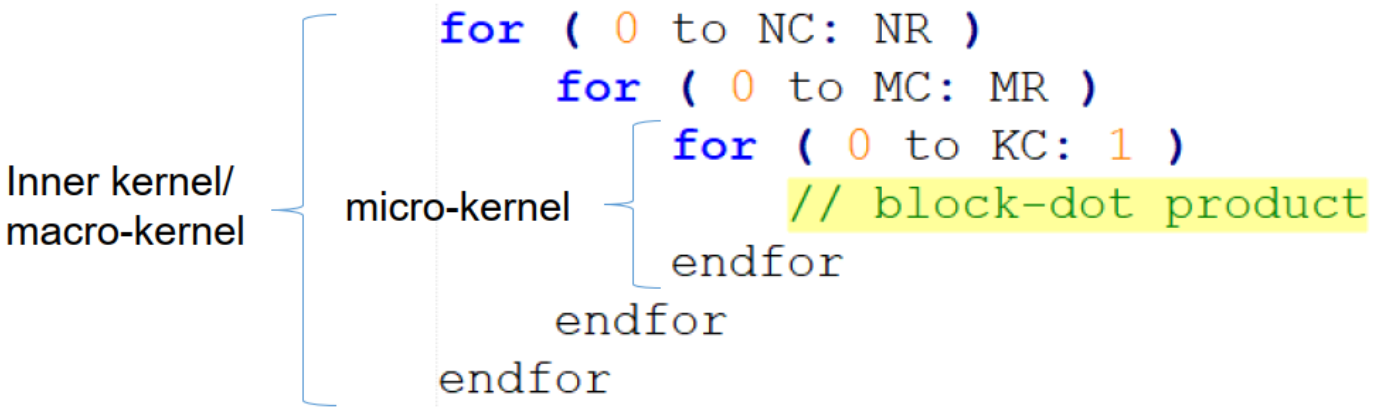


$$\frac{2K_c}{2K_c + 2} \approx 1$$

$$\frac{2M_R N_R K_c}{(M_R + N_R)K_c + 2M_r N_r} \approx \frac{2M_R N_R}{(M_R + N_R)}$$

The GEMM Micro-kernel

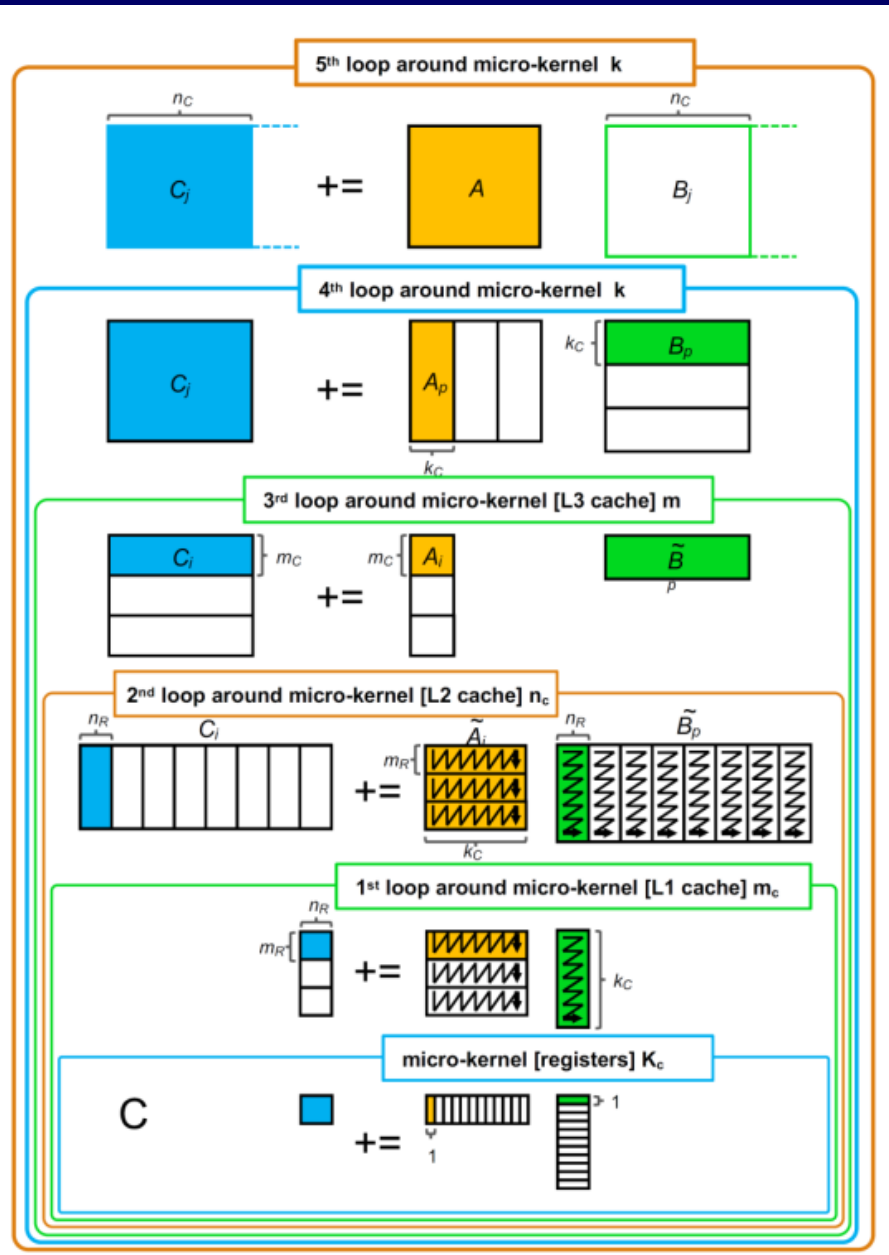
- BLIS exposes outer two loops of the inner kernel
  - Key observation: Virtually all of the differences between level-3 inner kernels reside in the outer two loops (e.g. loop bounds)
  - All inner kernels (which we call “macro-kernels”) are provided as part of the BLIS framework (C99)
  - Inner-most exposed loop simply calls micro-kernel



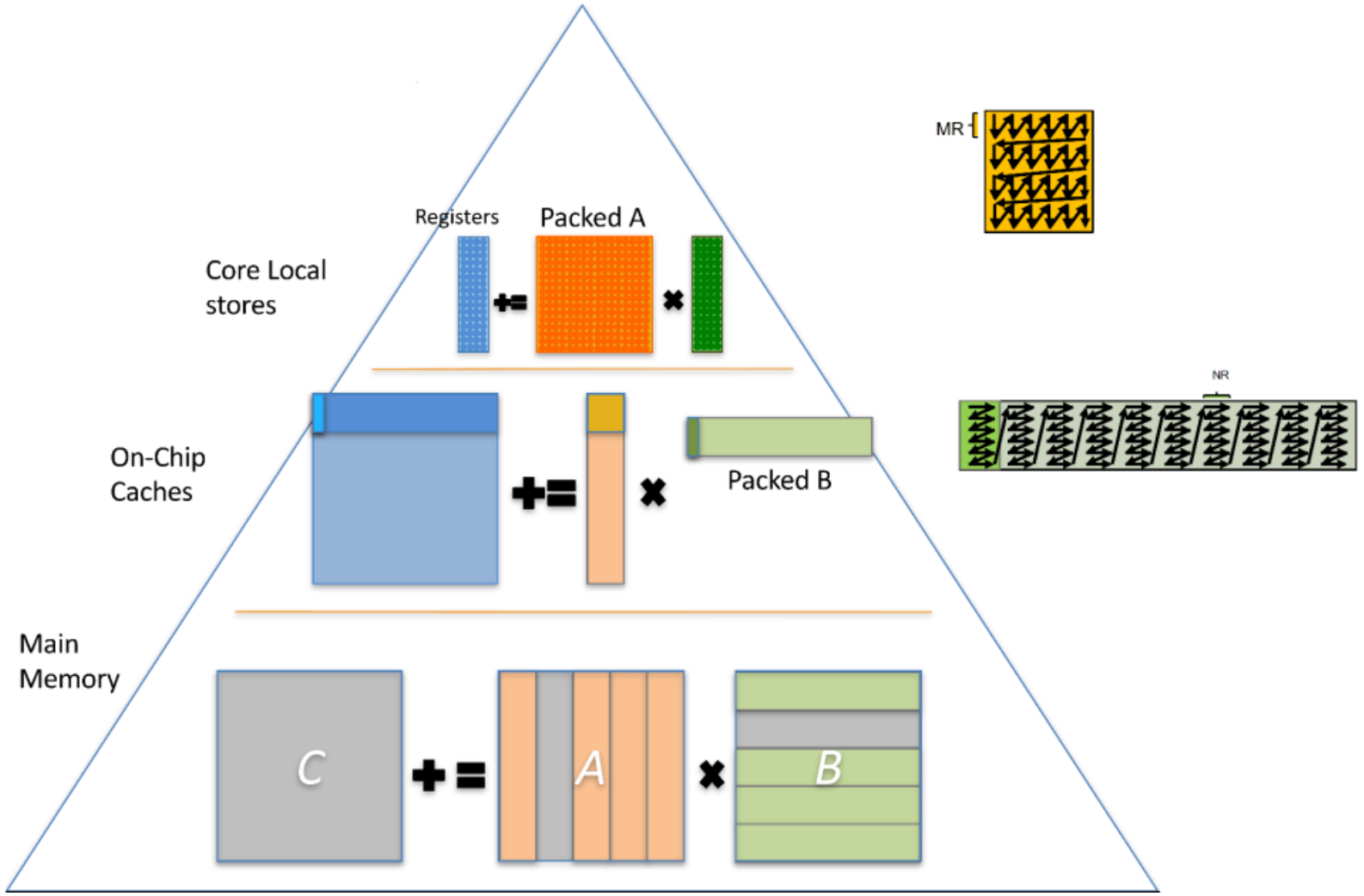
GEMM Loops

- **Rules for each new character**
  - **Buffers**
  - **Re-fetch rate**
- $m_r\ n_r\ k_c\ m_c\ n_c\ m\ k\ n$
- $m_0\ n_0\ k_0\ m_1\ n_1\ m_2\ k_1\ n_2$

for $j_c = 0, \ldots, n-1$ **in steps of** n_c
  for $p_c = 0, \ldots, k-1$ **in steps of** k_c
    $B(p_c : p_c + k_c - 1, j_c : j_c + n_c - 1) \rightarrow B_c$
    for $i_c = 0, \ldots, m-1$ **in steps of** m_c
      $A(i_c : i_c + m_c - 1, p_c : p_c + k_c - 1) \rightarrow A_c$
      for $j_r = 0, \ldots, n_c - 1$ **in steps of** n_r
        for $i_r = 0, \ldots, m_c - 1$ **in steps of** m_r
          for $p_r = 0, \ldots, k_c - 1$ **in steps of** 1
            $C_c(i_r : i_r + m_r - 1, j_r : j_r + n_r - 1)$
            $\mathrel{+}= A_c(i_r : i_r + m_r - 1, p_r)$
            $\cdot\ B_c(p_r, j_r : j_r + n_r - 1)$



Memory Hierarchy



Design Hardware for GEMM

- Systolic Arrays
  - Systolic Arrays
  - GEMM Systolic solution
- Other
  - Other parallel GEMM algorithm



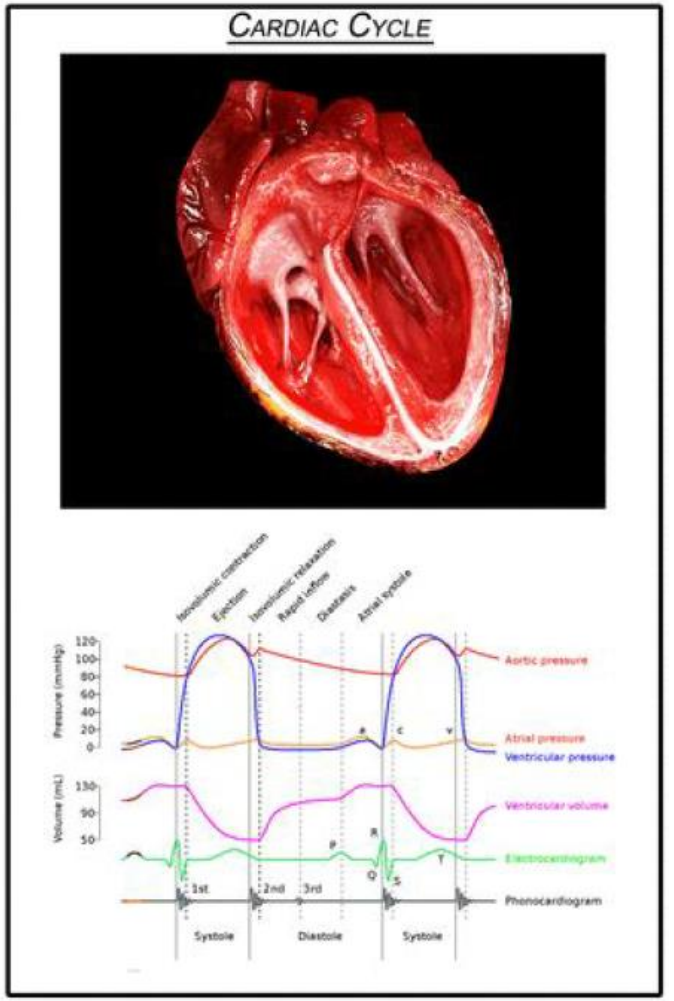
Outline

- Benchmarking Metrics
- **GEMM Accelerator Design**
  - GEMM
  - **Systolic and Other Variation**
- DNN Accelerator Design
- Energy



Systolic Arrays

- Systolic arrays are hardware structures built for fast and efficient operation of regular algorithms that perform the same task with different data at different time instants.
  Systolic arrays replace a pipeline structure with an array of processing elements that can be programmed to perform a common operation.



Definitions of Systolic Array

- **“Imagine n simple processors arranged in a row or an array and connected in such a manner that each processor may exchange information with only its neighbors to the right and left. The processors at either end of the row are used for input and output. Such a machine constitutes the simplest example of a systolic array.”**



Bayoumi, M. A., & Ling, N. (1999). Specification and verification of systolic arrays. World Scientific.

Definitions of Systolic Array

- **"Systolic Arrays are regular arrays of simple finite state machines, where each finite state machine in the array is identical...A systolic algorithm relies on data from different directions arriving at cells in the array at regular intervals and being combined."**



Brown, Andrew (1991). VLSI Circuits and Systems in Silicon. McGraw-Hill Boo Company.

Definitions of Systolic Array

- **"By pipelining, processing may proceed concurrently with input and output, and consequently overall execution time is minimized. Pipelining plus multiprocessing at each stage of a pipeline should lead to the best-possible performance."**



Dewdney, A.K. (1993) The (New) Turing Omnibus. Henry Holt and Company.

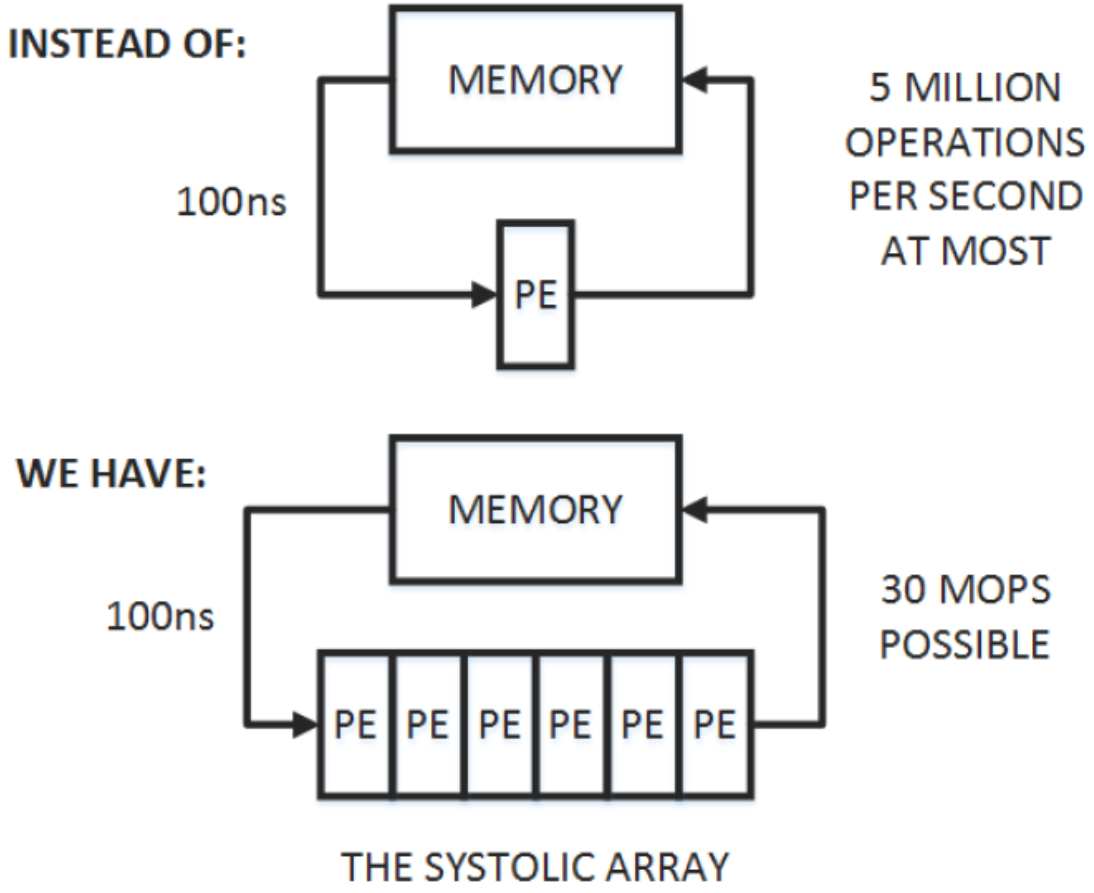
Systolic Arrays

- This is a form of pipelining, sometimes in more than one dimension
- The term systolic was first used in this context by H.T. Kung, then at CMU; it refers to the “pumping” action of a heart
- Machines have been constructed based on this principle, notable the iWARP, fabricated by Intel



Systolic Arrays

- **Balancing computation with I/O**



Systolic Architectures

- Replace single processor with an array of regular processing elements
- Orchestrate data flow for high throughput with less memory access
- Different from linear pipelining
  - Nonlinear array structure, multidirectional data flow, each PE may have (small) local instruction and data memory
- Different from SIMD: each PE may do something different
- Initial motivation: VLSI Application-Specific Integrated Circuits (ASICs)
- Represent algorithms directly by chips connected in regular pattern



Systolic Matrix Multiplication

- Processors are arranged in a 2-D grid
- Each processor accumulates one element of the product
- The elements of the matrices to be multiplied are pumped through the array



Data Preparation for Systolic

- **We need to modify the input data, like so:**

Flip columns 1 & 3 ⟶

$$\begin{matrix} a_{13} & a_{12} & a_{11} \\ a_{23} & a_{22} & a_{21} \\ a_{33} & a_{32} & a_{31} \end{matrix}$$

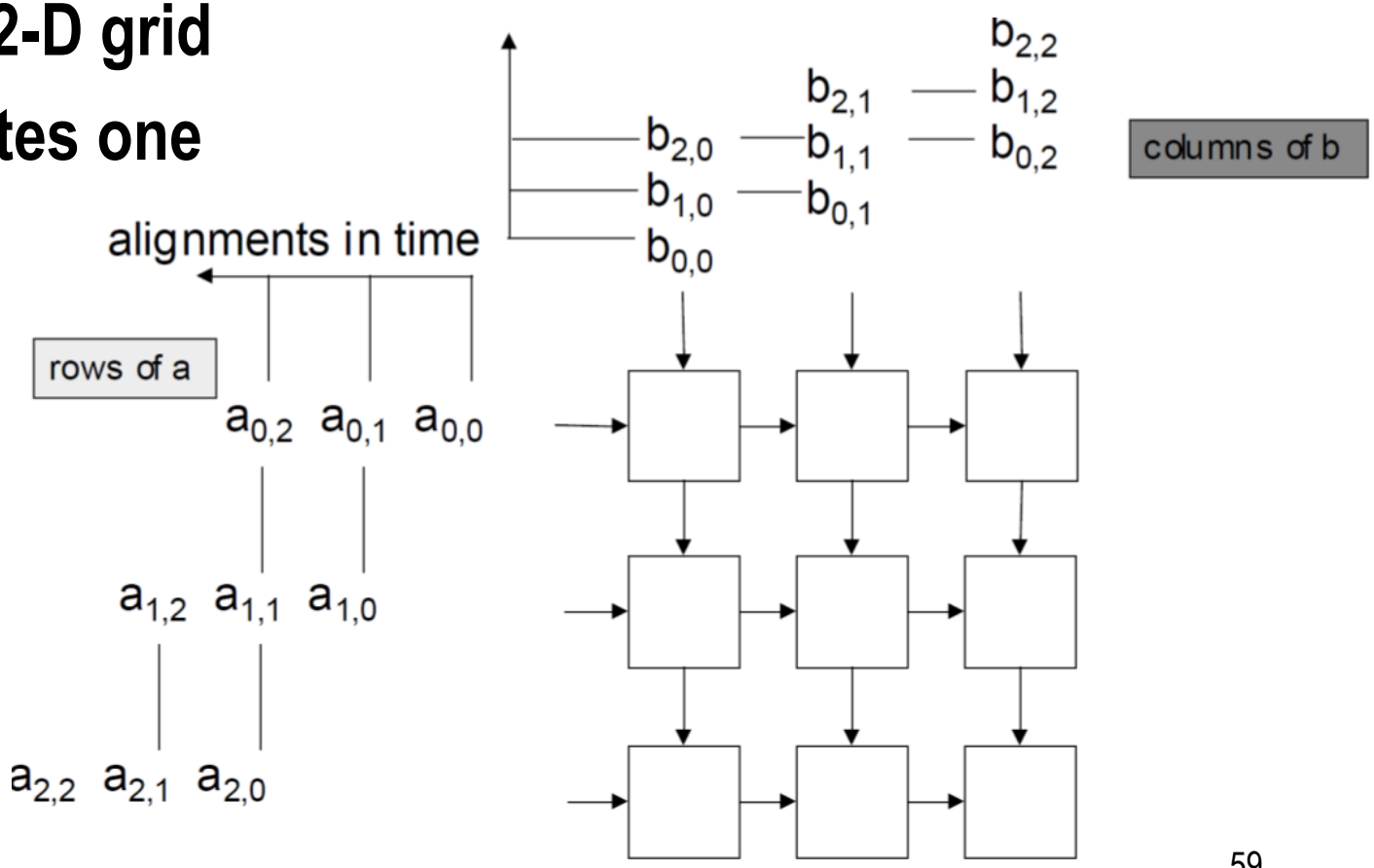
Flip rows 1 & 3 ⟶

$$\begin{matrix} b_{31} & b_{32} & b_{33} \\ b_{21} & b_{22} & b_{23} \\ b_{11} & b_{12} & b_{13} \end{matrix}$$

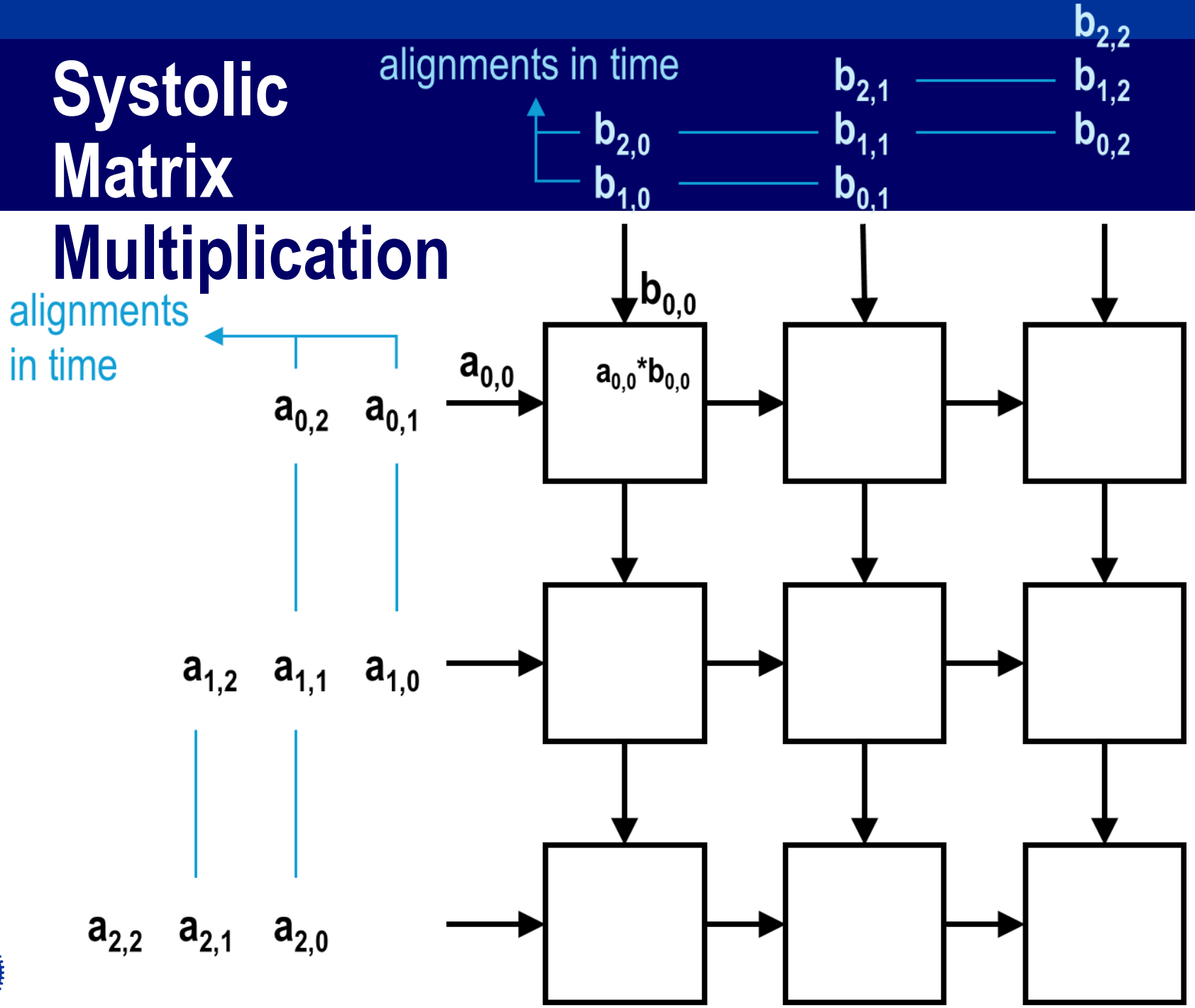
and finally stagger the data sets for input.

Systolic Matrix Multiplication

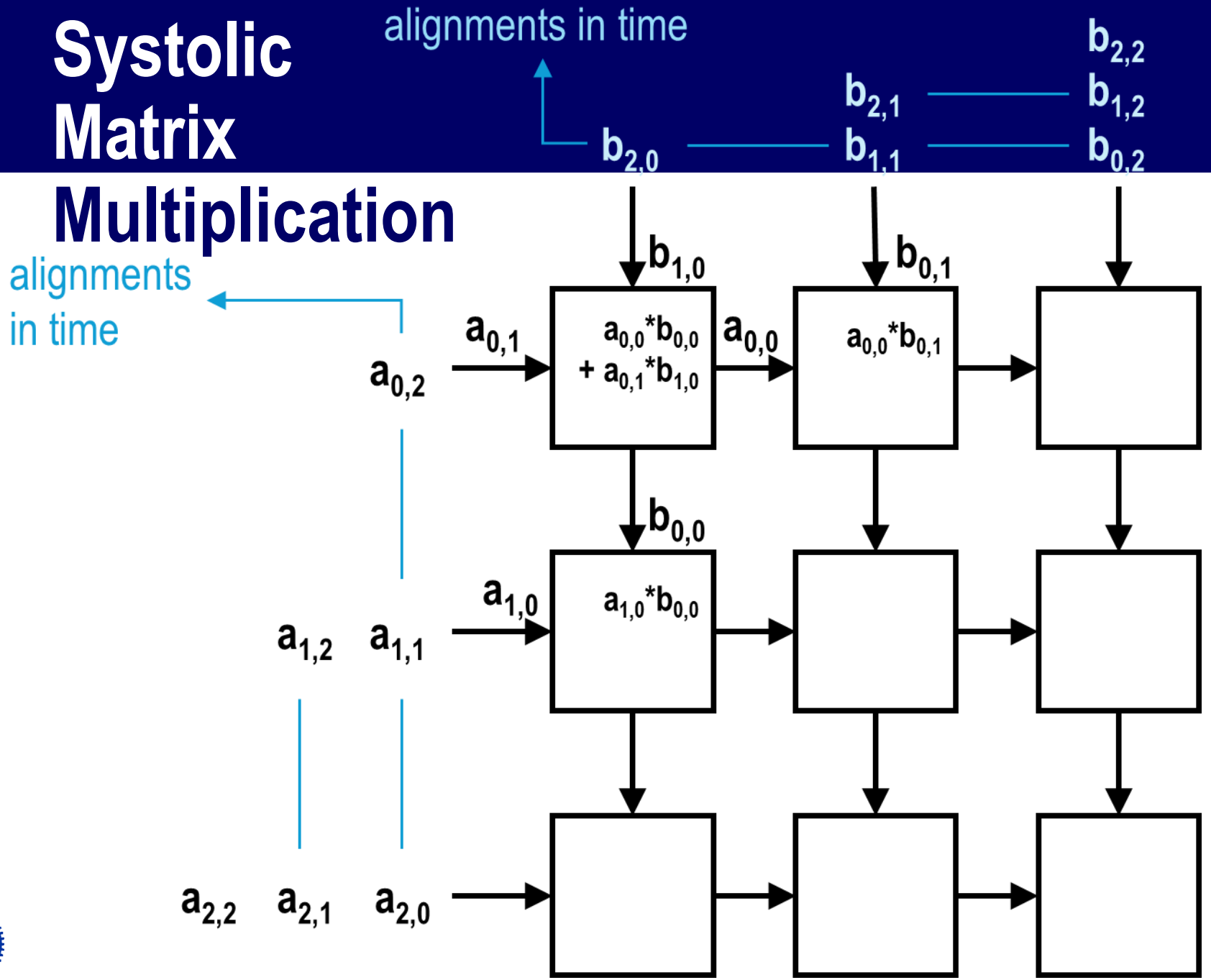
- 3x3 matrices multiplications
- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product



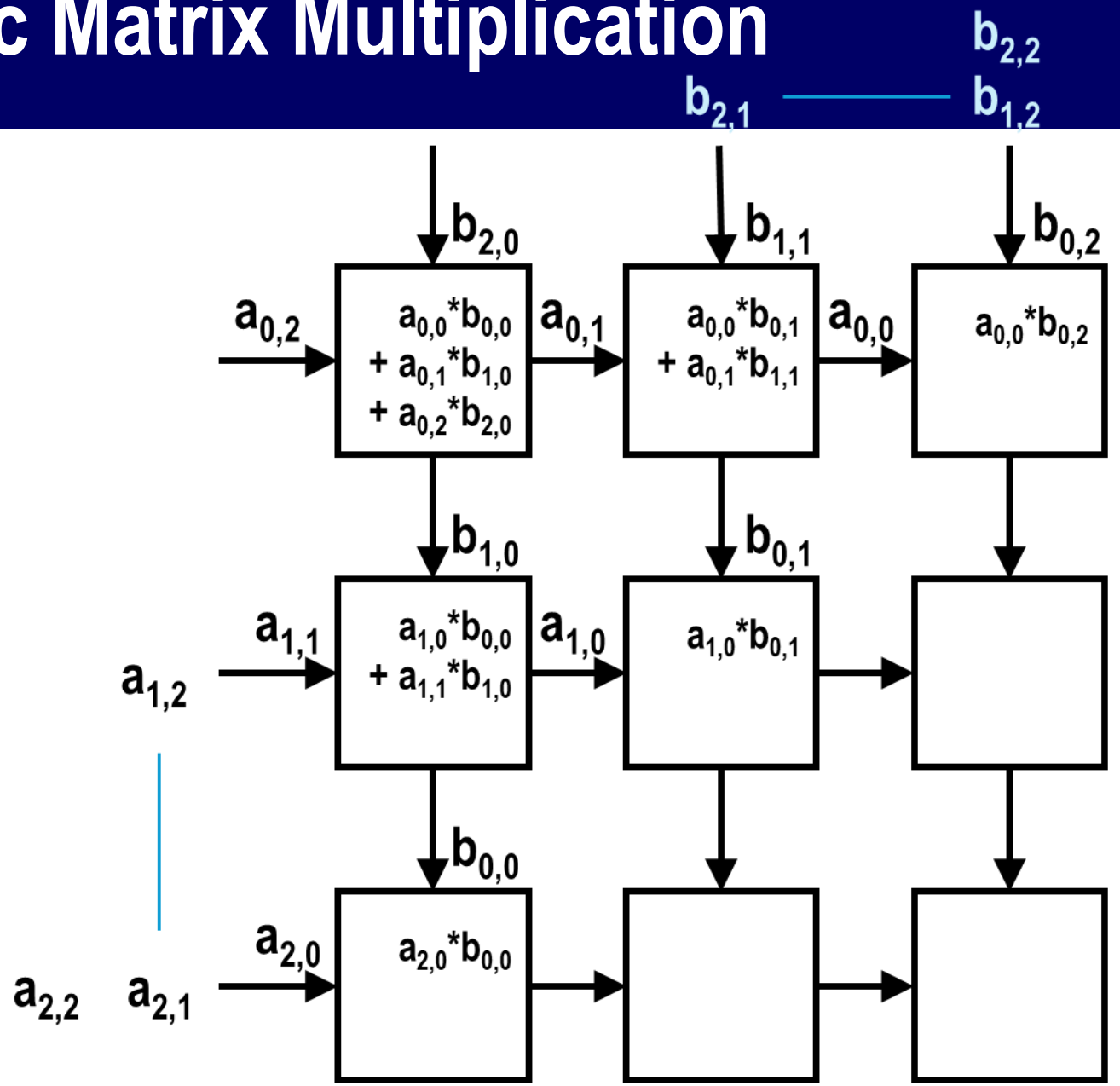
Systolic Matrix Multiplication



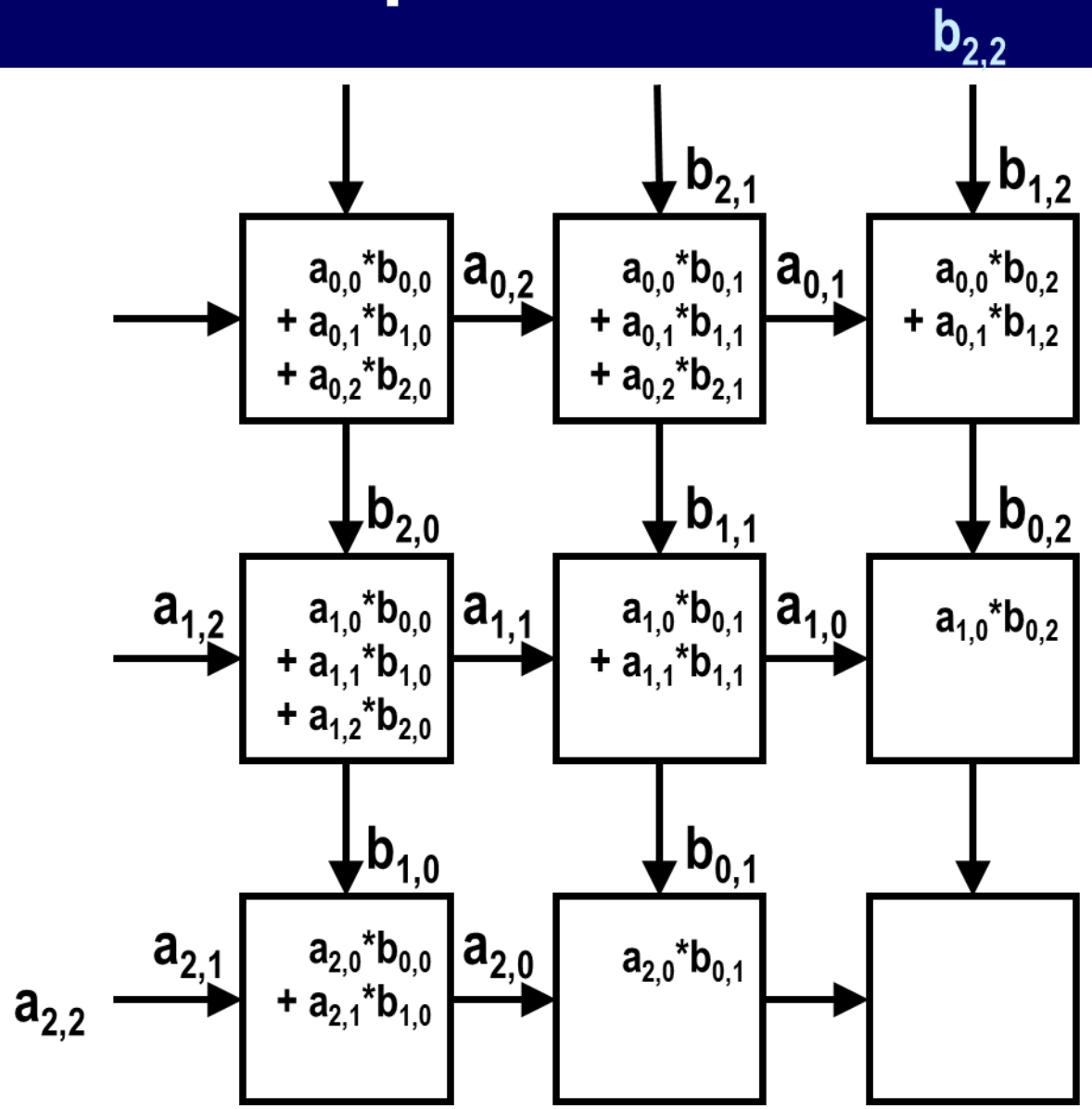
Systolic Matrix Multiplication



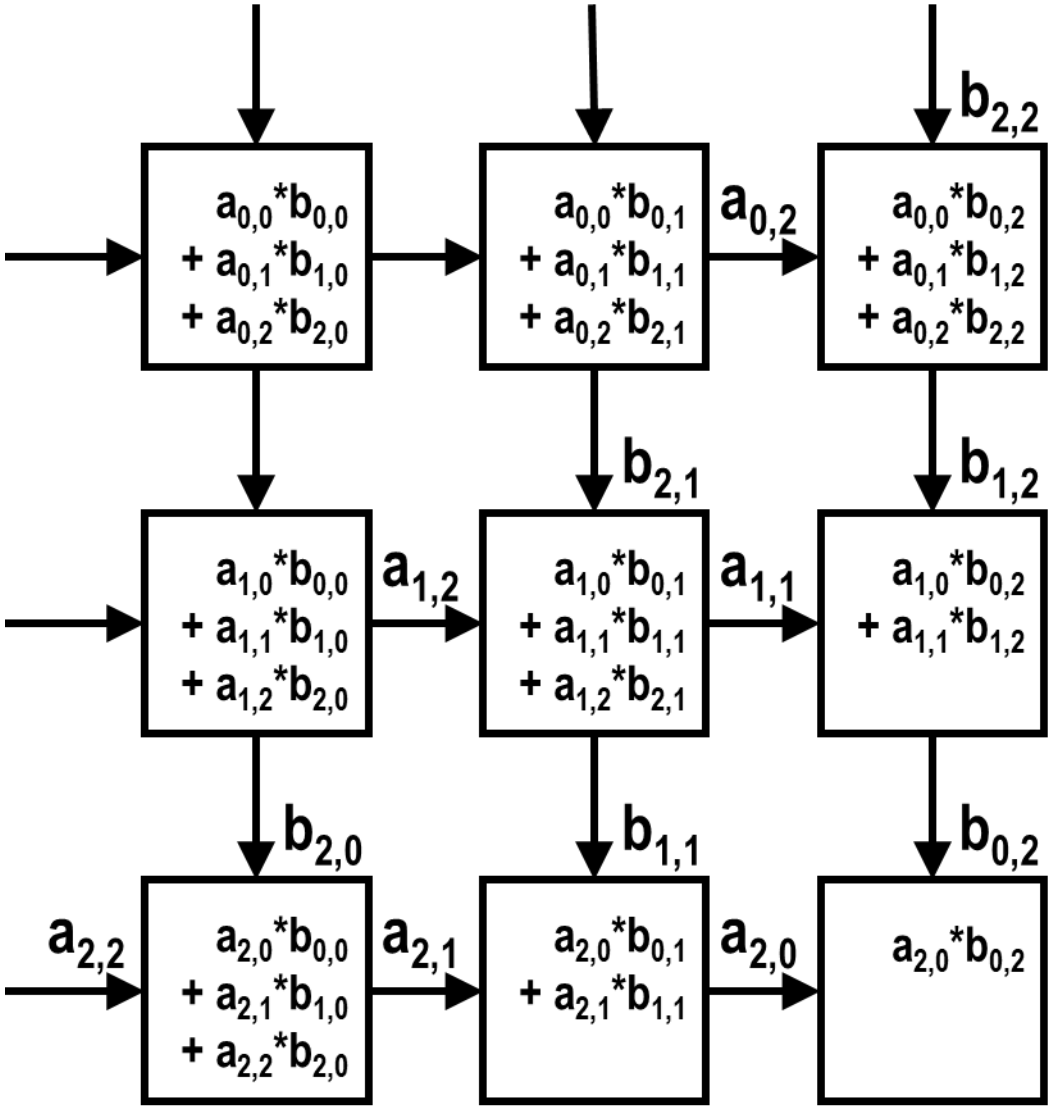
Systolic Matrix Multiplication



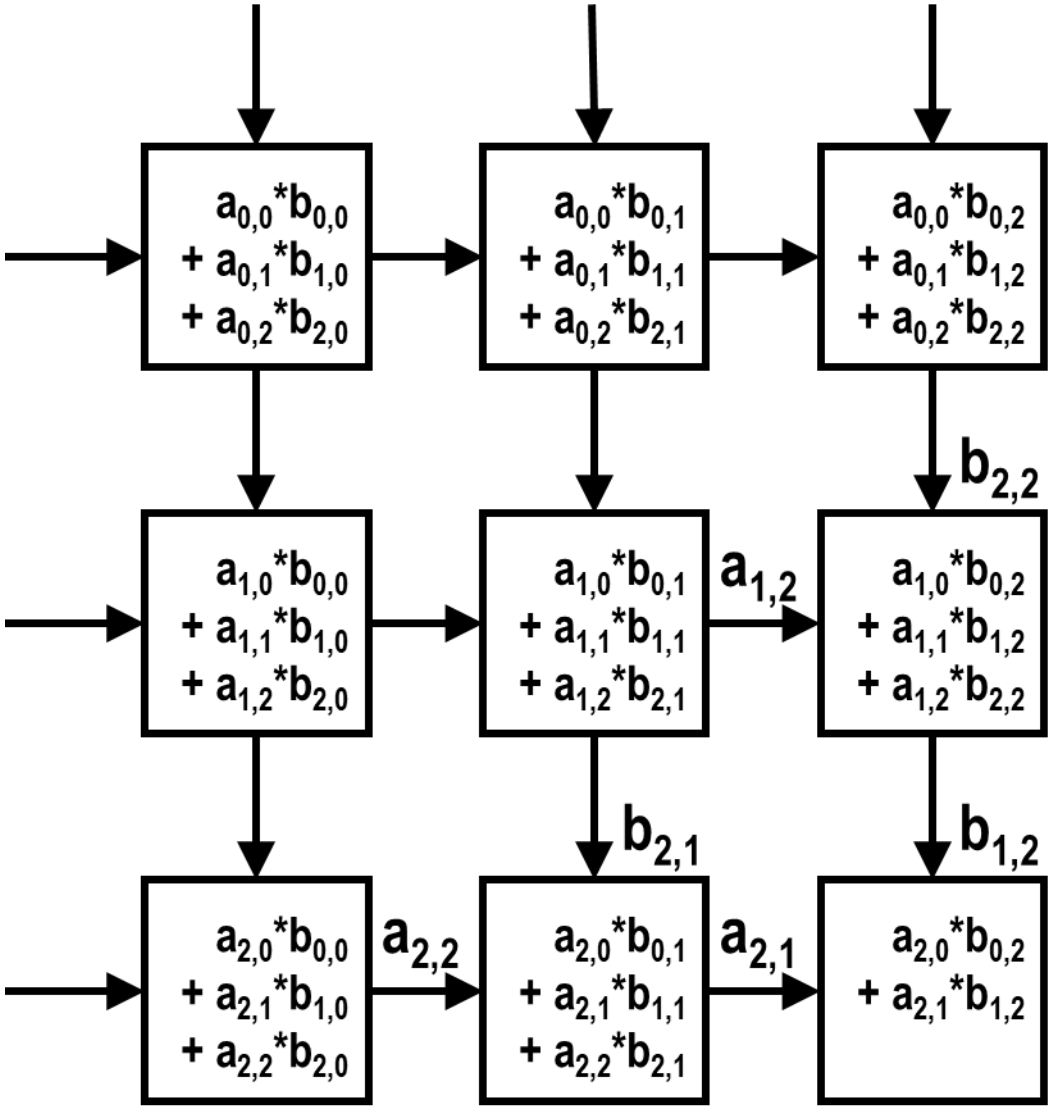
Systolic Matrix Multiplication



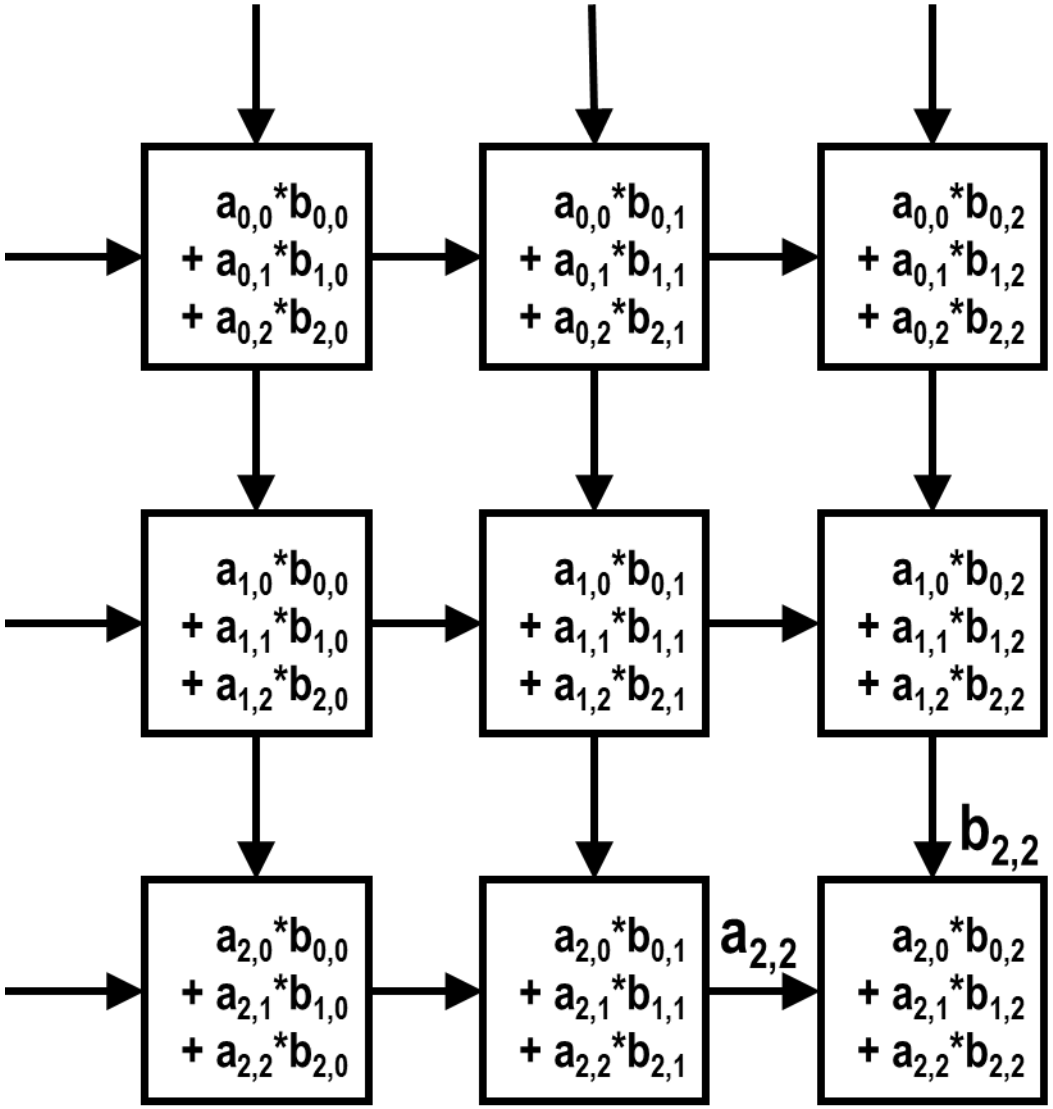
Systolic Matrix Multiplication



Systolic Matrix Multiplication



Systolic Matrix Multiplication

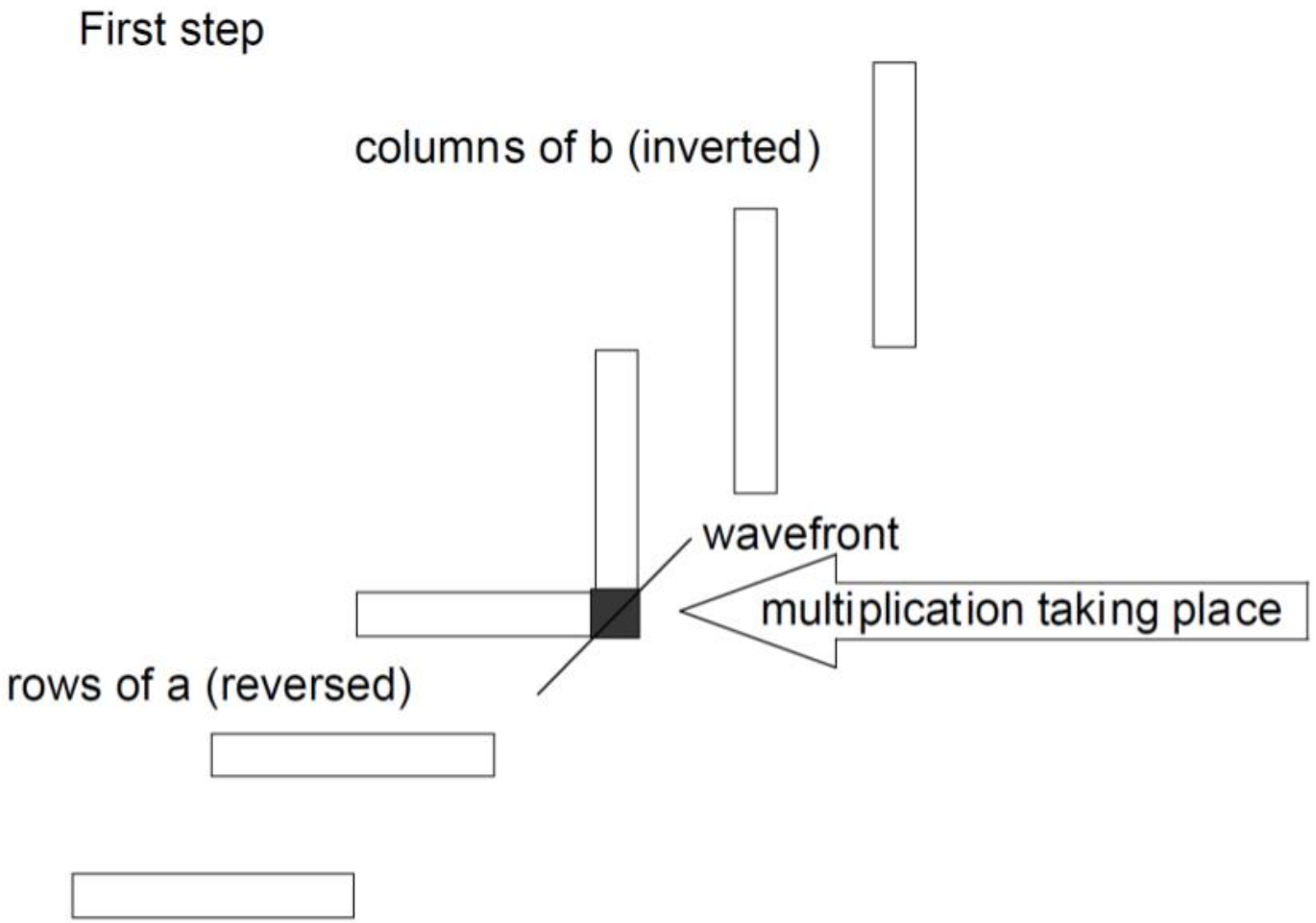


Strips View of Systolic Array

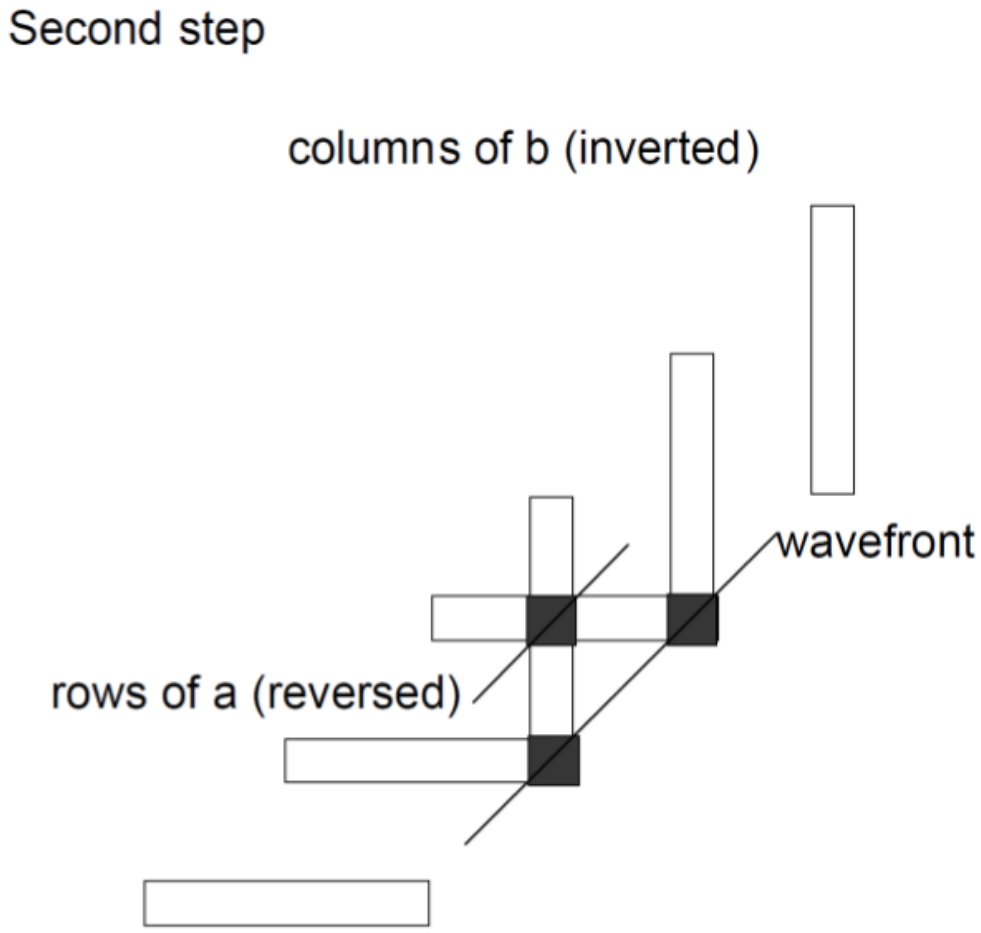
- Let's take another view of systolic multiplication
  - Consider the rows and columns of the matrices to be multiplied as strips that are slide past each other.
- The strips are staggered so that the correct elements are multiplied at each time step.



Strips View of Systolic Array

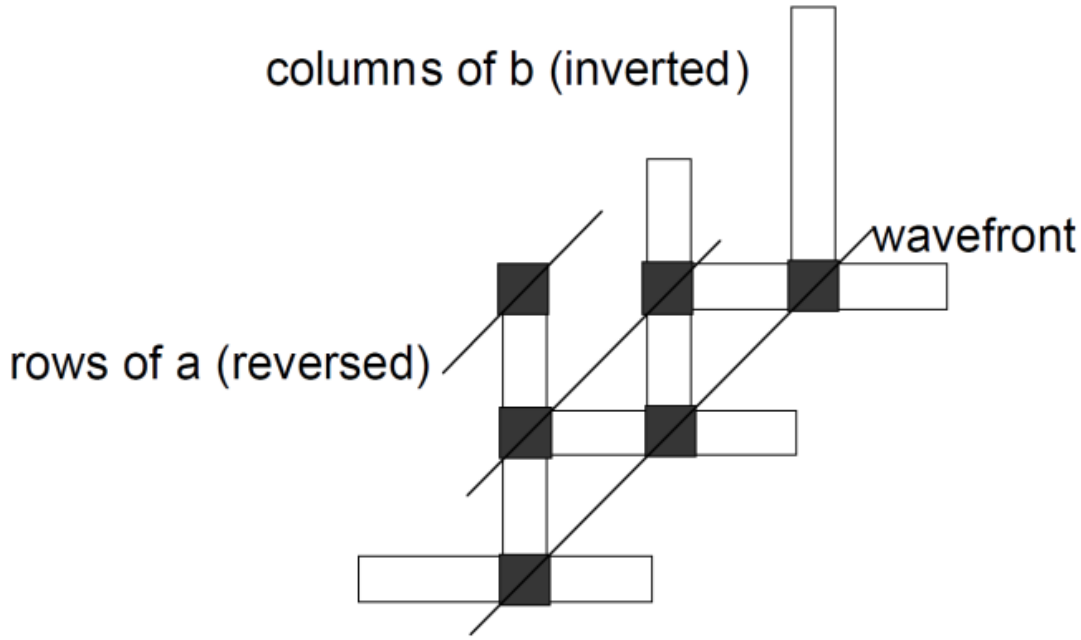


Strips View of Systolic Array

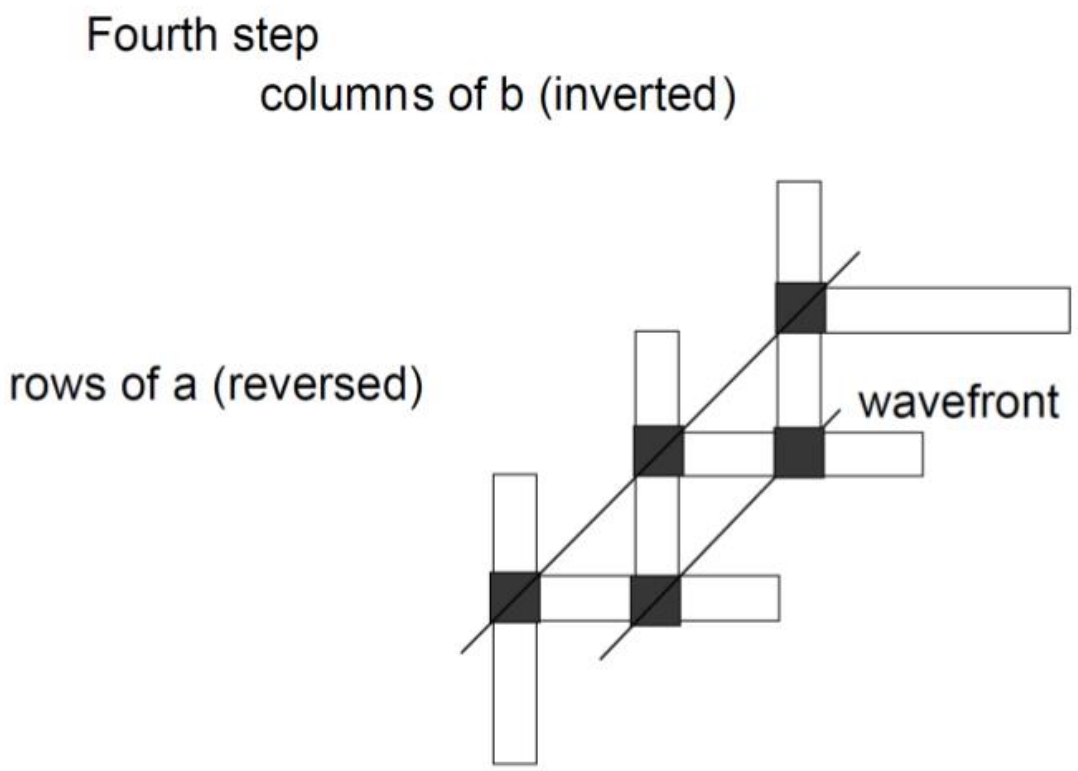


Strips View of Systolic Array

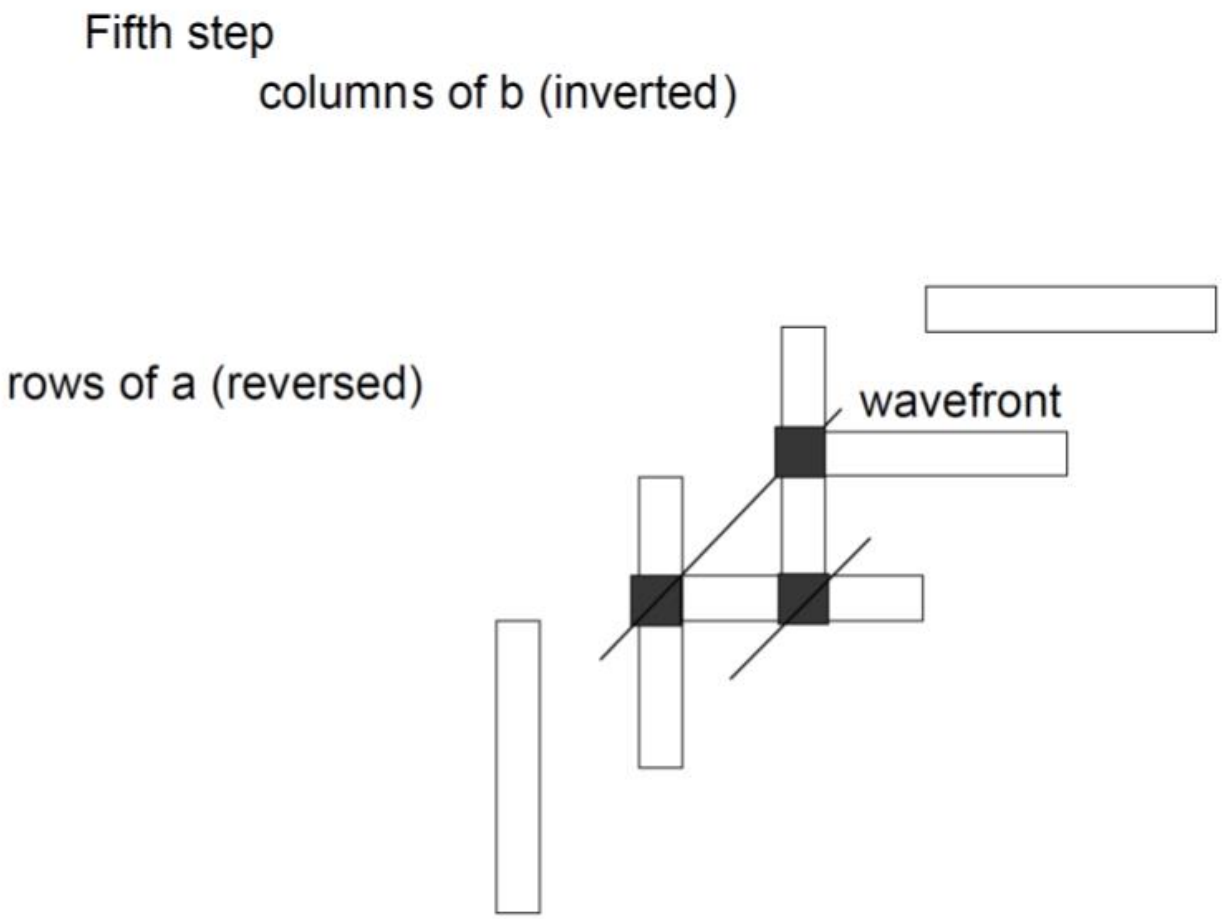
Third step



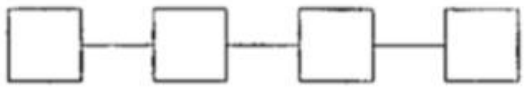
Strips View of Systolic Array



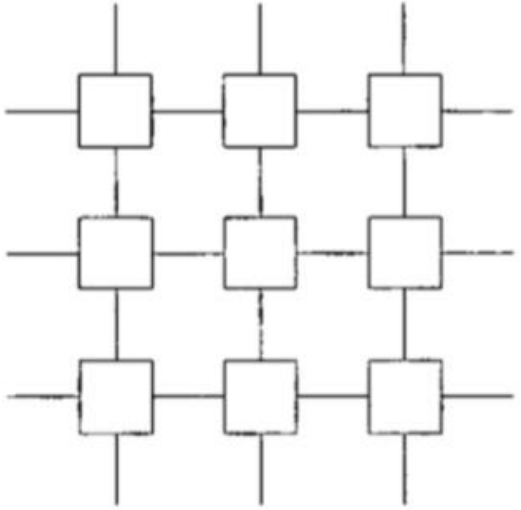
Strips View of Systolic Array



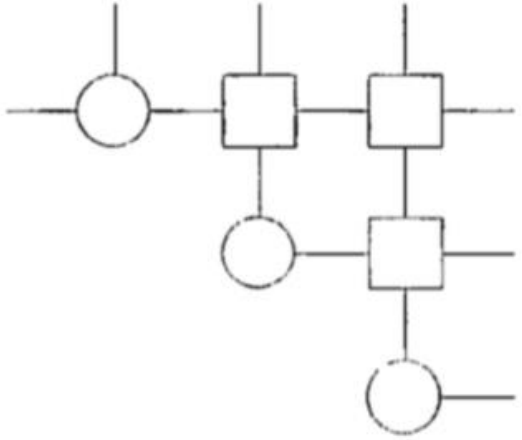
Systolic Array Architectures



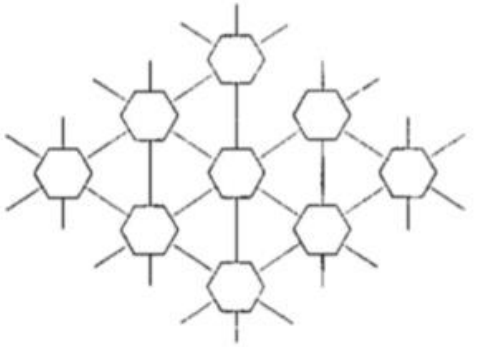
Linear Array



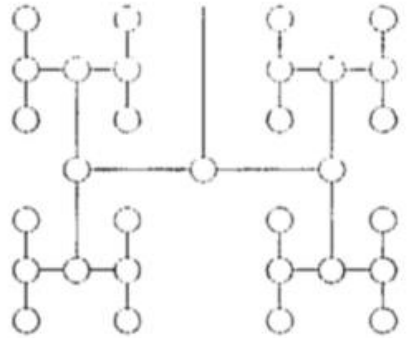
Orthogonal Array



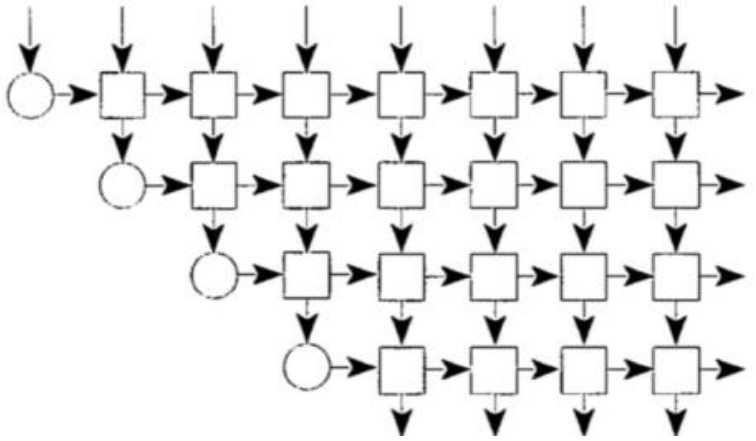
Triangular Array



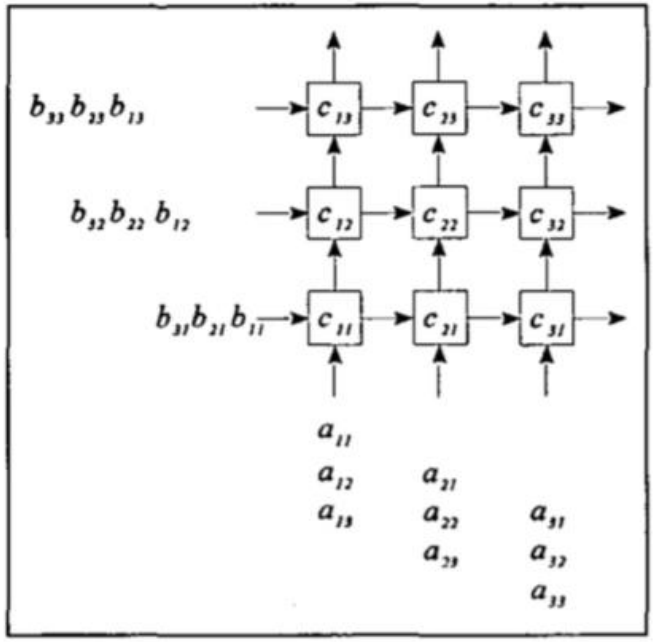
Hexagonal Array



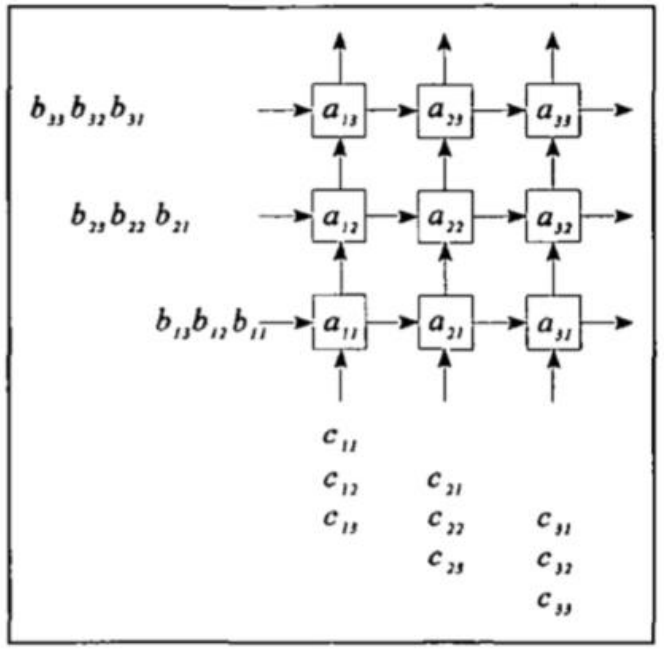
Binary Tree



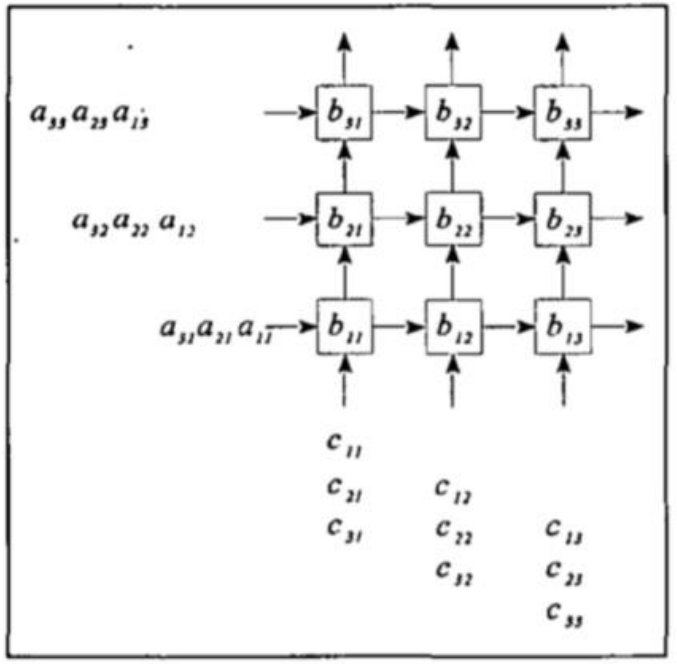
[C] = [A]x[B] Variation



- Keep C resident
- Stream A,B



- Keep A resident
- Stream B,C



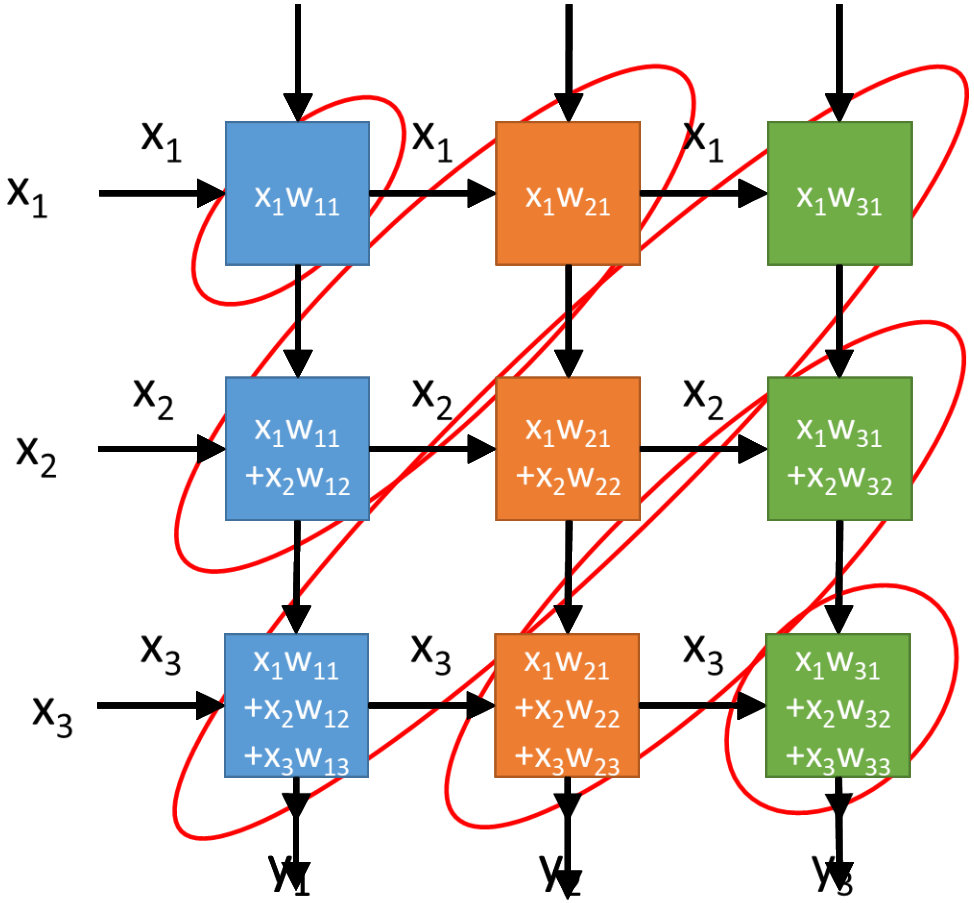
- Keep B resident
- Stream A,C

Systolic Array – Example

Step = 4

y_2 is done.

$$[x_1 \quad x_2 \quad x_3]\begin{bmatrix} w_{11} & w_{21} & w_{31} \\ w_{12} & w_{22} & w_{32} \\ w_{13} & w_{23} & w_{33} \end{bmatrix}$$



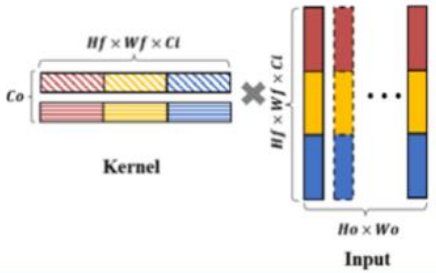
Outline

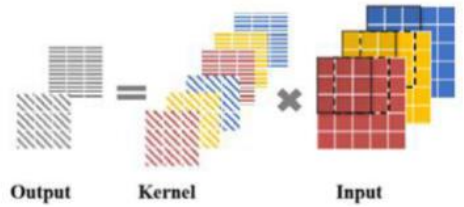
- Benchmarking Metrics
- GEMM Accelerator Design
- **DNN Accelerator Design**
  - **Locality and Data Reuse**
  - Dataflow Taxonomy
  - Network-on-Chip
  - Optimization
- Energy



Direct Convolution is Better

- **Higher performance, zero memory overheads**

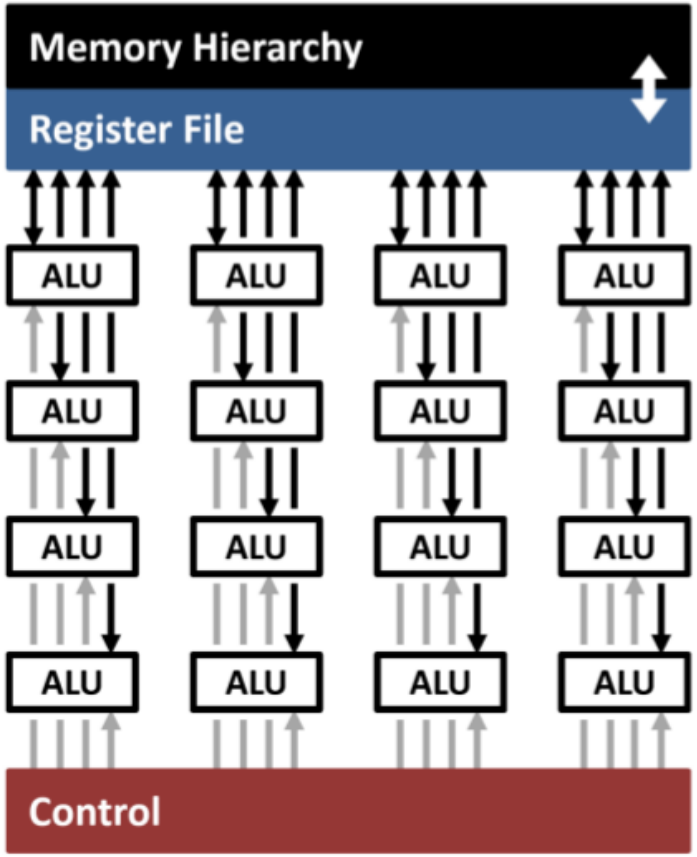




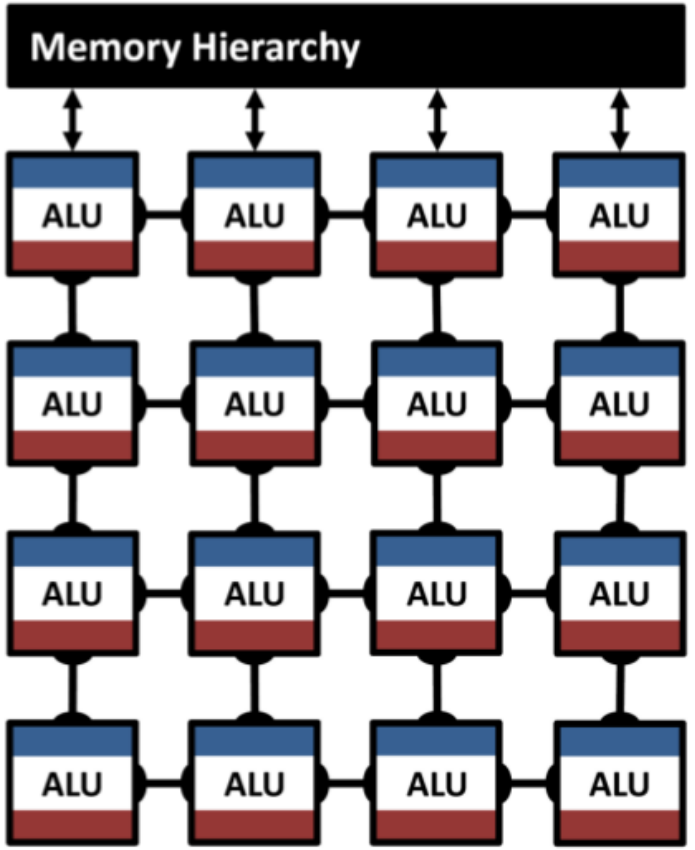
	Matrix-Matrix-Multiplication	Direct Convolution
Packing	**Yes** • Over 10x additional memory for image data • Performance penalty	**No** • Zero memory overheads • No performance penalty
Computation Performance	**Less than** expected theoretic peak of GEMM	Close to system's theoretic peak

Highly-Parallel Compute Paradigms

Temporal Architecture
(SIMD/SIMT)



Spatial Architecture
(Dataflow Processing)



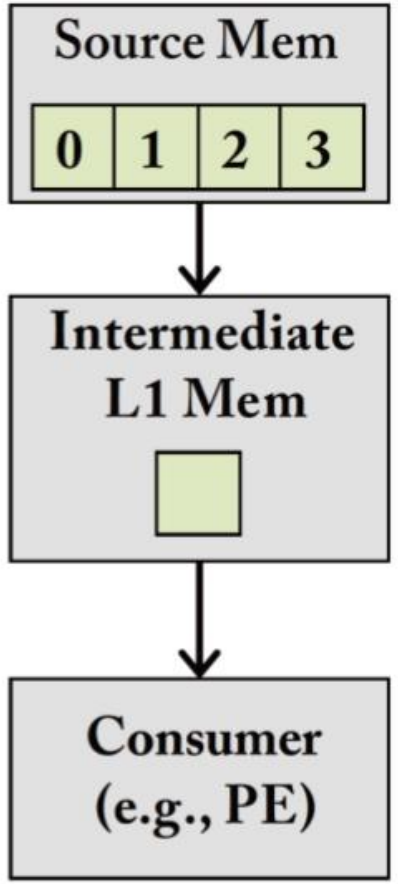
Locality: Temporal and Spatial Reuse

- Temporal reuse: the same data is used more than once over time by the same consumer.
- Spatial reuse: the same data is used by more than one consumer at different spatial locations of the hardware.



Temporal Reuse

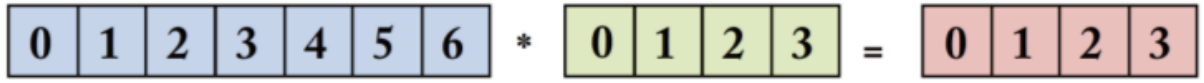
- The same data value is used more than once by the same consumer
  - E.g. PE
- Adding an intermediate memory level to the memory hierarchy
  - Used multiple times at the intermediate level
  - Reduces the overall energy cost



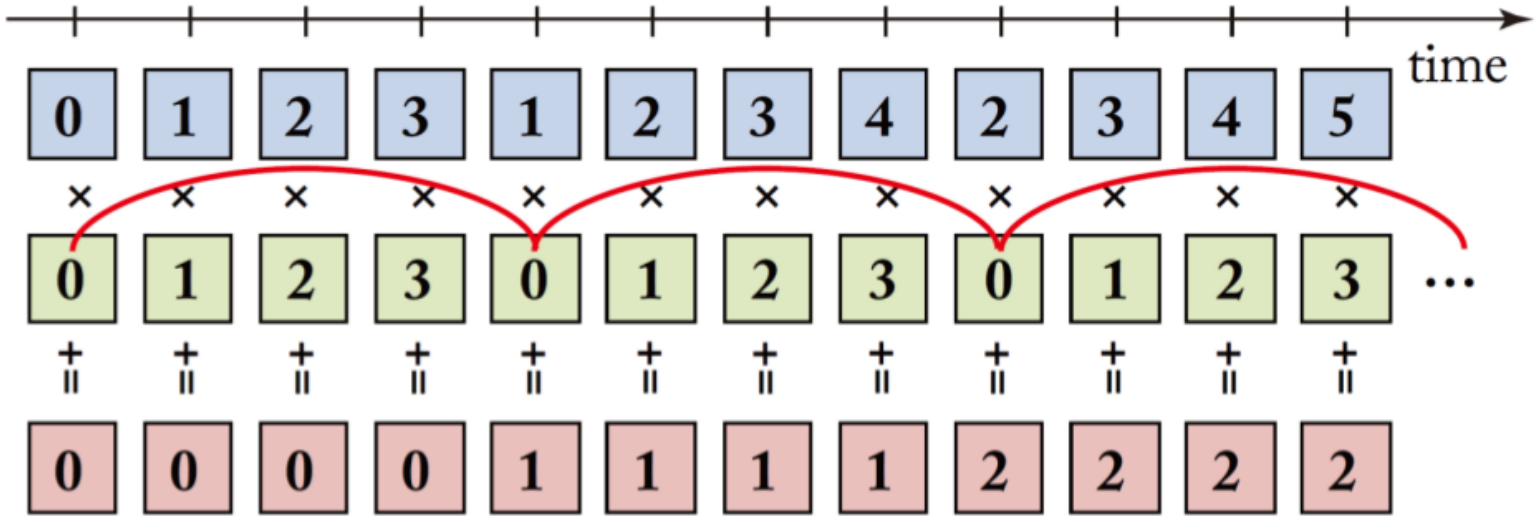
* Only storage for weights is shown

1D CNN Temporal Reuse Example

- Example 1D convolution

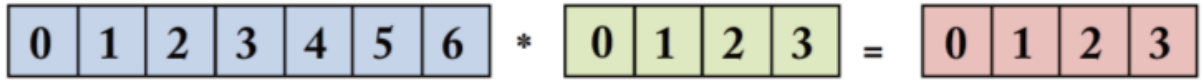


- Operation ordering = 1, reuse distance = 4

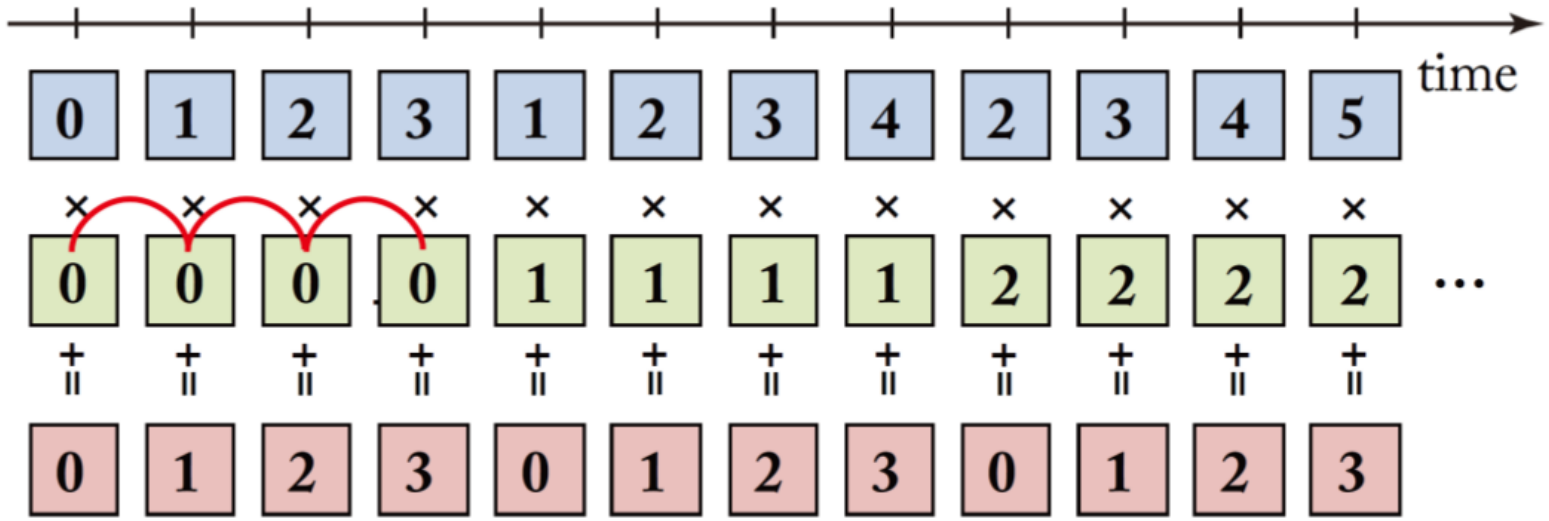


1D CNN Temporal Reuse Example

- Example 1D convolution



- Operation ordering = 2, reuse distance = 1



Temporal Reuse Distance

- Temporal reuse distance
  - The maximum number of data accesses required by the consumer in between the accesses to the same data value
  - Storage capacity of the intermediate memory level limits the maximum reuse distance
  - Storage capacity >= maximum reuse distance
    - Temporal reuse can be exploited
  - Not limit to weights
- Reduce the reuse distance of one data type often increase the reuse distance of other data types
- Exploited by multiple levels of the memory hierarchy



Spatial Reuse

- The same data value is used by more than one consumer at different spatial locations of the hardware
  - E.g., a group of PEs
- Exploited by reading the data once from the source memory level and multicasting it to all of the consumers
- Reducing the number of accesses to the source memory level
  - Reduces the overall energy cost
- Reducing the bandwidth required from the source memory level
  - Helps to keep the PEs busy and therefore increases performance



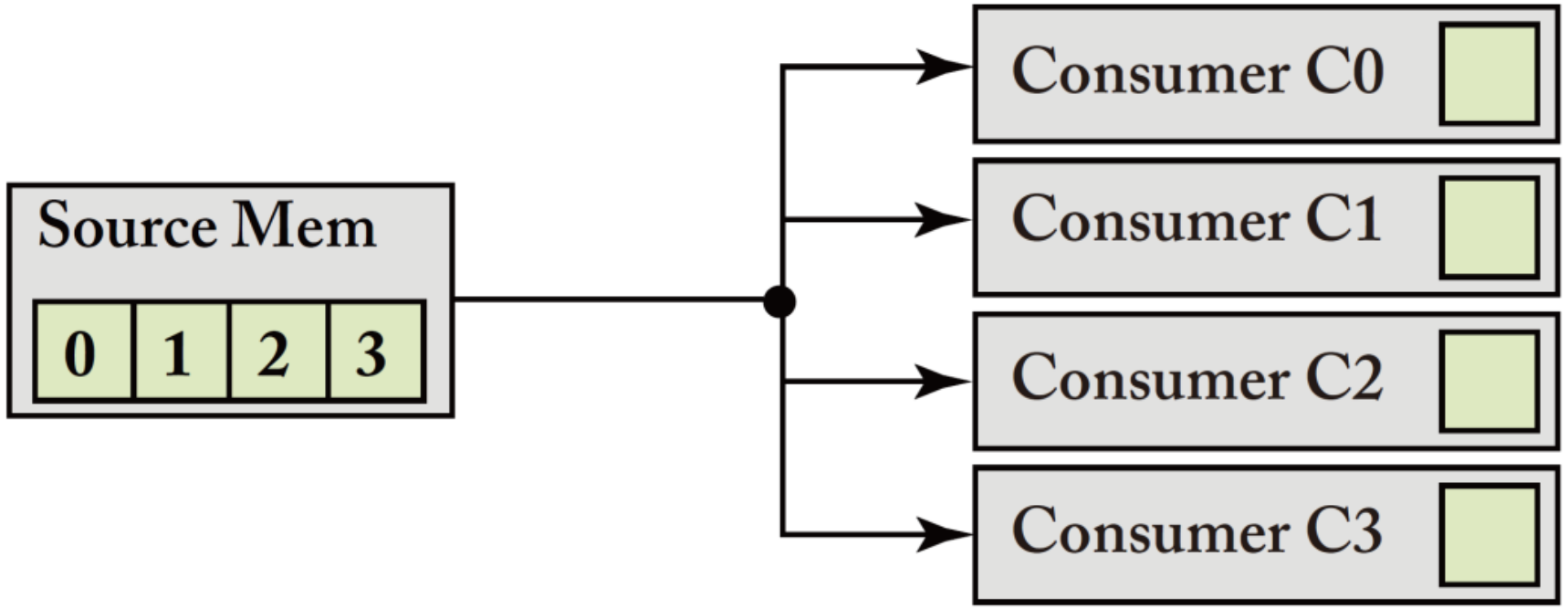
Spatial Reuse Distance

- If no storage capacity in a group of consumers
  - Spatial reuse can only be exploited by the subset of consumers that can process the data in the same cycle as the multicast of the data
- Spatial Reuse Distance
  - The maximum number of data accesses in between any pair of consumers that access the same data value

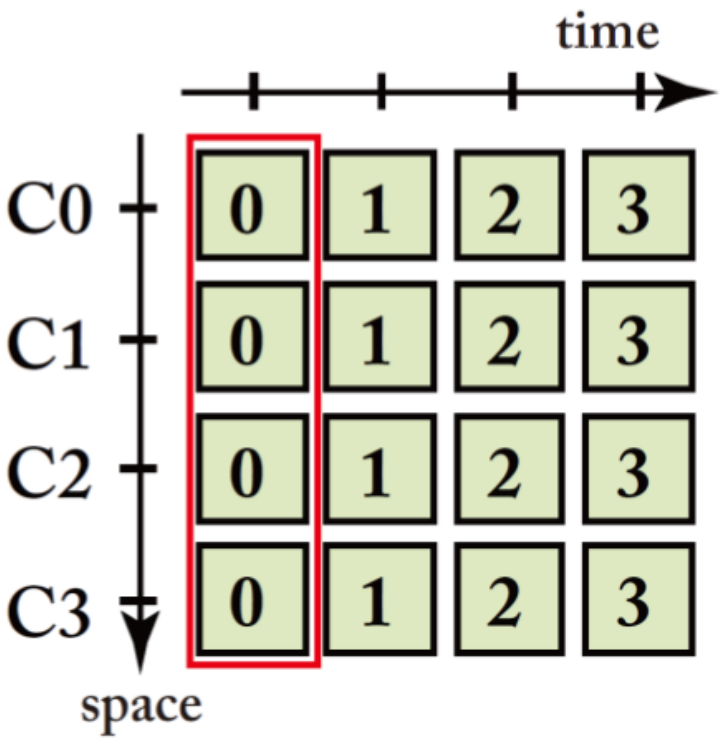


Memory Hierarchy for Spatial Reuse

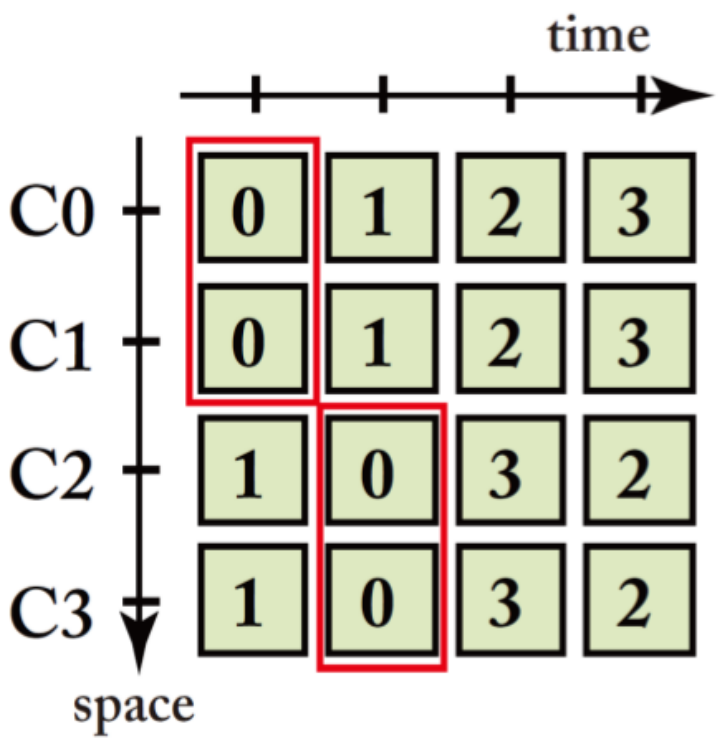
* Only storage for weights is shown



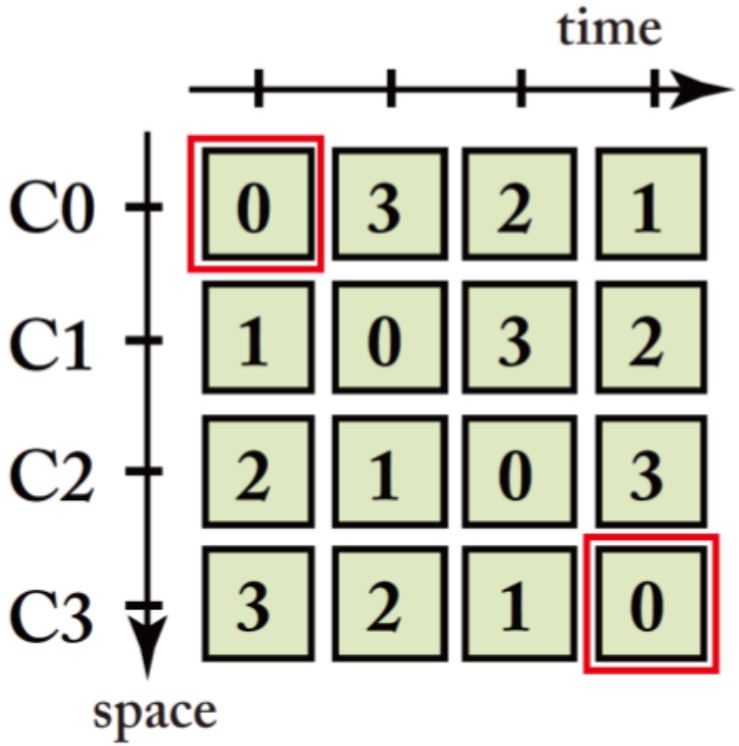
Spatial Reuse Example



Operation ordering = 1,
reuse distance = 0



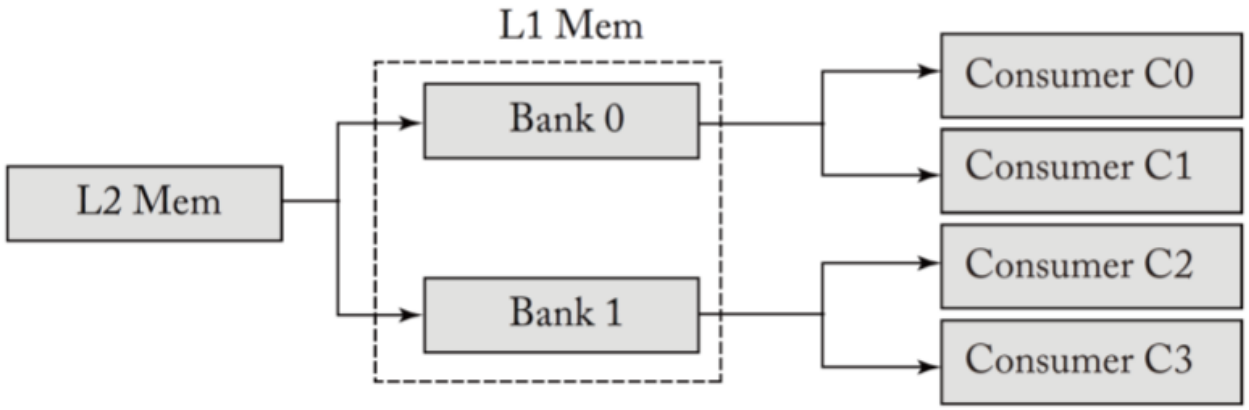
Operation ordering = 2,
reuse distance = 1



Operation ordering = 3,
reuse distance = 3

Routing Data to Multiple Destination

- Network-on-Chip(NoC)
  - Support the data distribution patterns as derived from the ordering of operations
- Exploiting spatial reuse at higher levels of the memory hierarchy creates more duplicated data in the hardware
- Supporting multicast to all consumers in a single level at a large scale can also be expensive
- Trade-off



physical connectivity of the NoC between the L1 memory and the consumers limits the multicast from any banks in L1 to all consumers

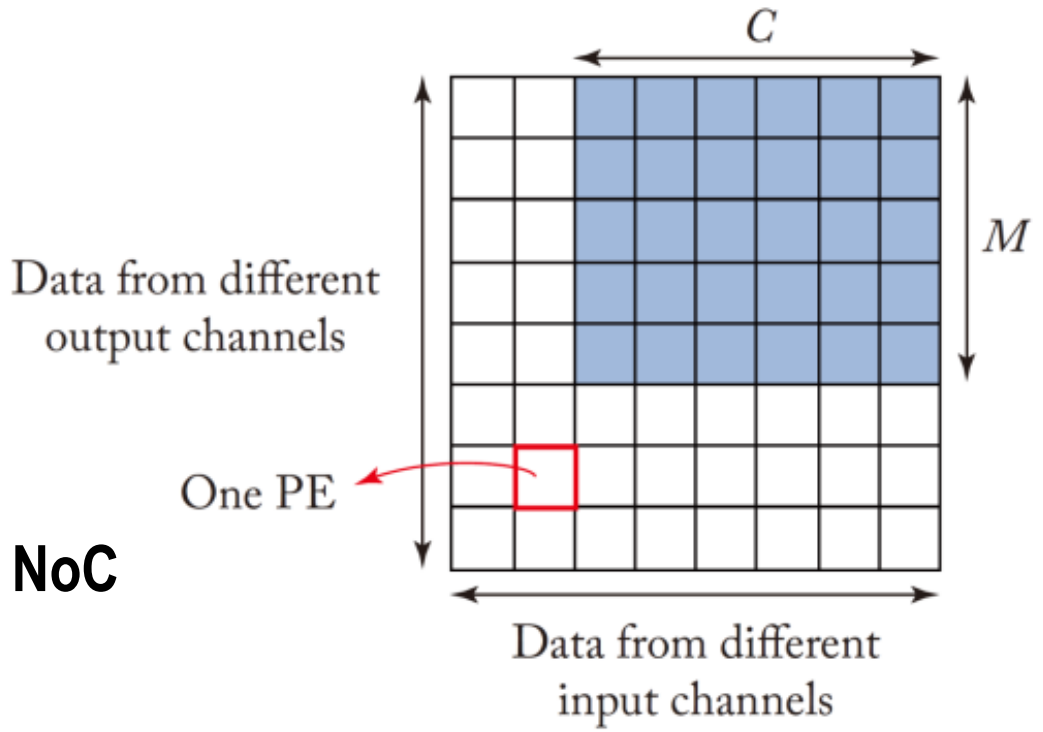
Reducing Reuse Distance

- **Data stay stationary over time in the sequence of operations**
  - **Reduce temporal reuse distance**
- **Data tiling (or blocking)**
  - **Temporal tiling**
    - Reduce temporal reuse distance
    - Reducing the reuse distance of specific data types
    - Make it smaller than the storage capacity of a certain memory level in the memory hierarchy
  - **Spatial tiling**
    - Reduce spatial reuse distance
    - Reusing the same data value by as many consumers as possible
    - Reducing the reuse distance so that one multicast can serve as many consumers as possible given a fixed amount of storage capacity at each consumer

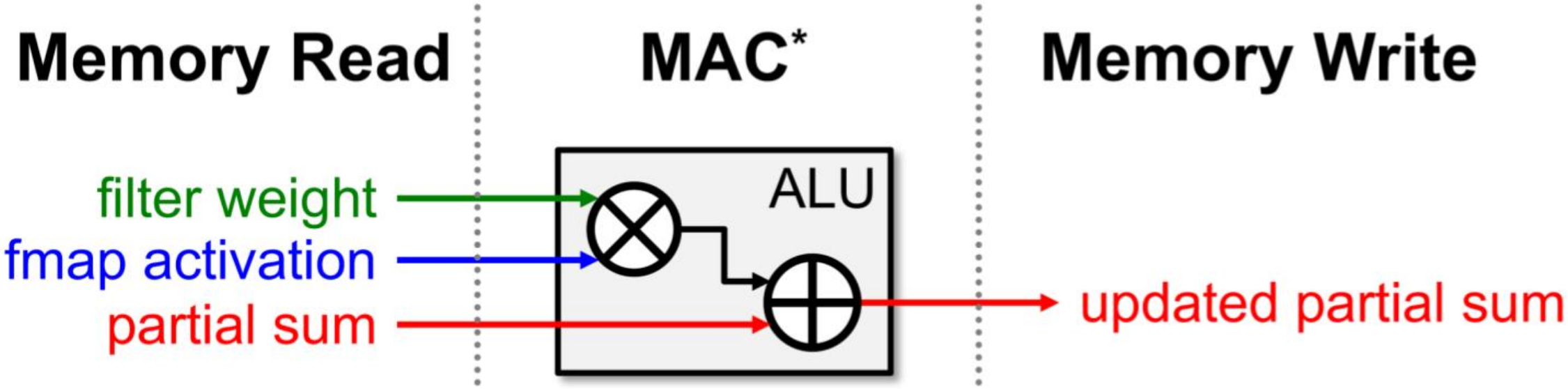


Under-utilization of Spatial tiling

- **Under-utilization when**
  - **M< Number of PEs in a column or N <Number of PEs in a row**
    - M: number of output channels
    - N: number of input channels
- **Solution**
  - **Multiple tiles of data run at the same time**
  - **Sufficient flexibility of data delivery and the NoC**
    - Provide sufficient bandwidth

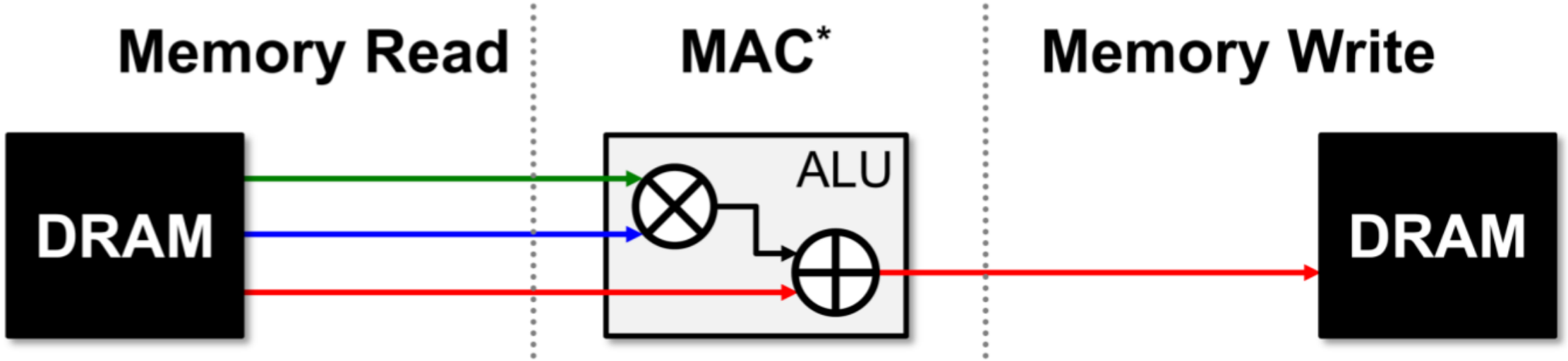


PE with Memory Access



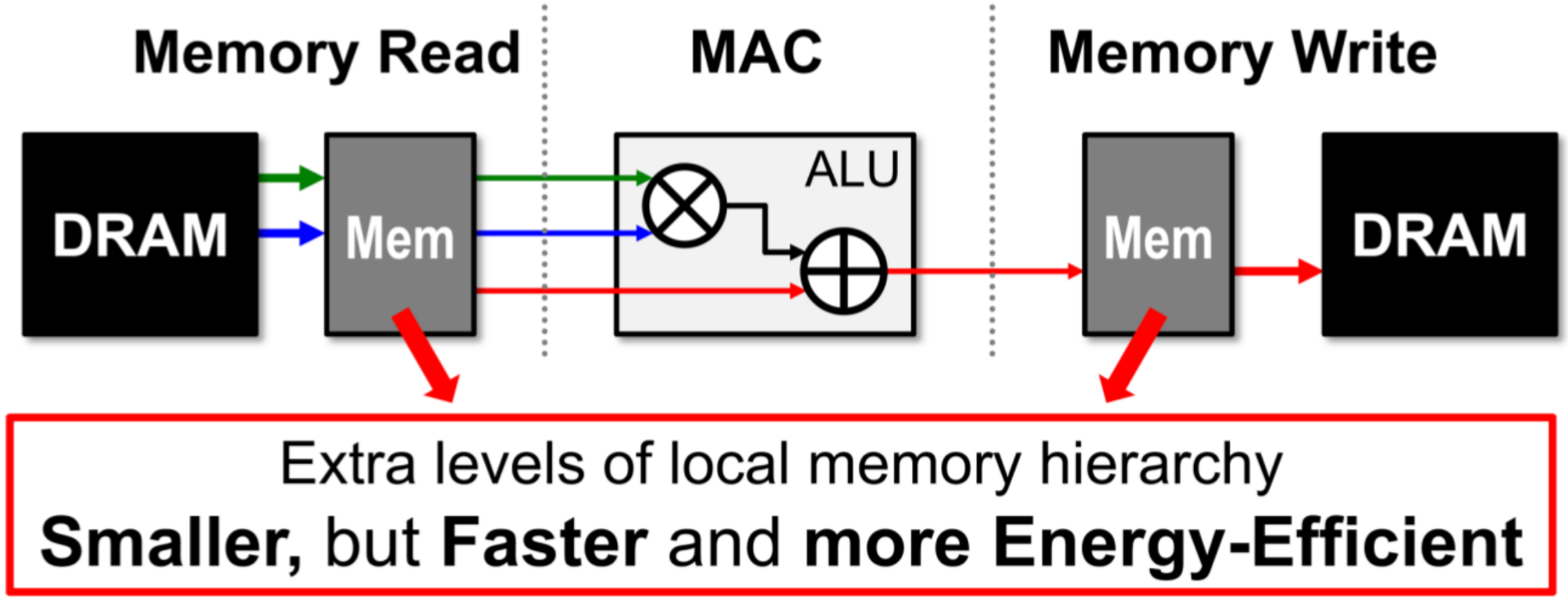
DRAM Access

- Worse Case: all memory R/W are DRAM accesses

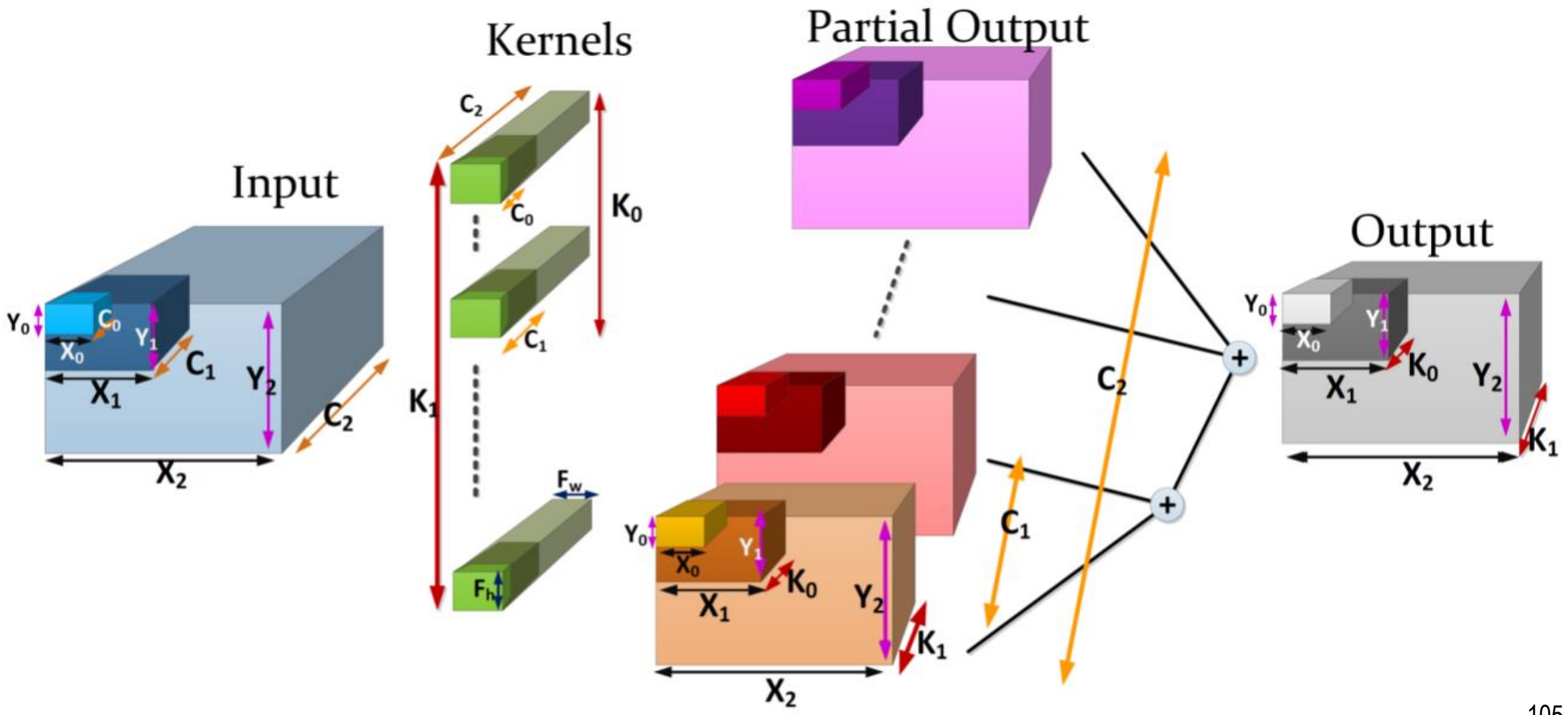


Leverage Local Memory for Data Reuse

- **Temporal reuse: the same data is used more than once over time by the same consumer**



Hierarchical Blocking of a Single Convolutional Layer



Formal Derivation of CONV

- **Loop representation string**
  - Rules for each new character
  - Buffer
  - Re-fetch rate
- $Y_0 K_0 C_0 X_0 K_1 C_1 Y_1 X_1$

```
for xi = 1: X0 : X1 do
    for yi = 1 : Y0 : Y1 do
        for ci = 1 : C0 : C1 do
            for ki = 1 : K0 : K1 do
                for xii = xi : 1 : xi + X0 do
                    for cii = ci : 1 : ci + C0 do
                        for kii = ki : 1 : ki + K0 do
                            for yii = yi : 1 : yi + Y0 do
                                compute a convolution window;
                                out[yii][xii][kii]+ = img[yii:yii+Fh][xii:xii+Fw][cii] x kernel[cii][kii];
```



Recursive Formulation

- **Fixed order as follows**
- $__Y_{i-1}X_{i-1}C_{i-1}K_{i-1}\ \cdots$
- **From left to right observing**

Buffer Refetch Rate: the number of times a piece of data is fetched from a certain buffer after initially being loaded into that buffer

Buffer Name	Buffer Size	Buffer Refetch Rate

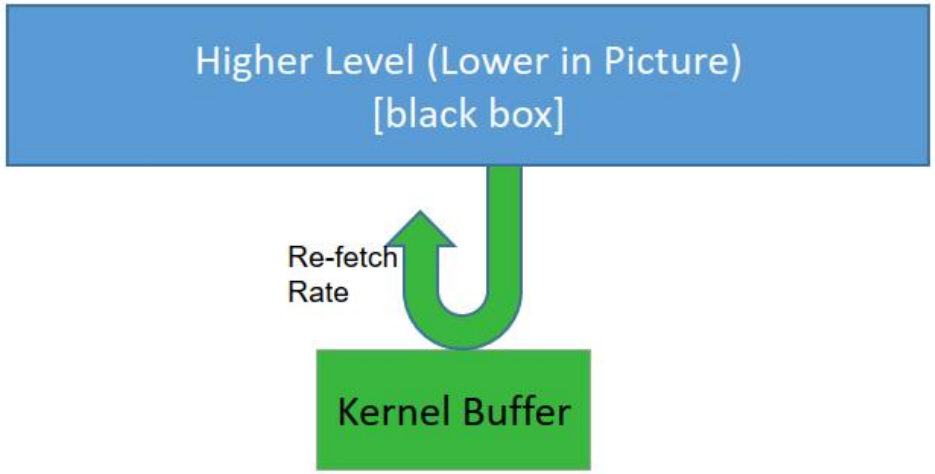
Higher Level (Lower in Picture)
[black box]

Recursive Formulation

- **Fixed order as follows**
- $__Y_{i-1}X_{i-1}C_{i-1}K_{i-1}Y_iX_i \cdots$
- **From left to right observing**
  - **X or Y: new KB**

Buffer Refetch Rate: the number of times a piece of data is fetched from a certain buffer after initially being loaded into that buffer

Buffer Name	Buffer Size	Buffer Refetch Rate
KB_i	$C_{i-1}K_{i-1}F_hF_w$	$(X_iY_i)/(X_{i-1}Y_{i-1})$

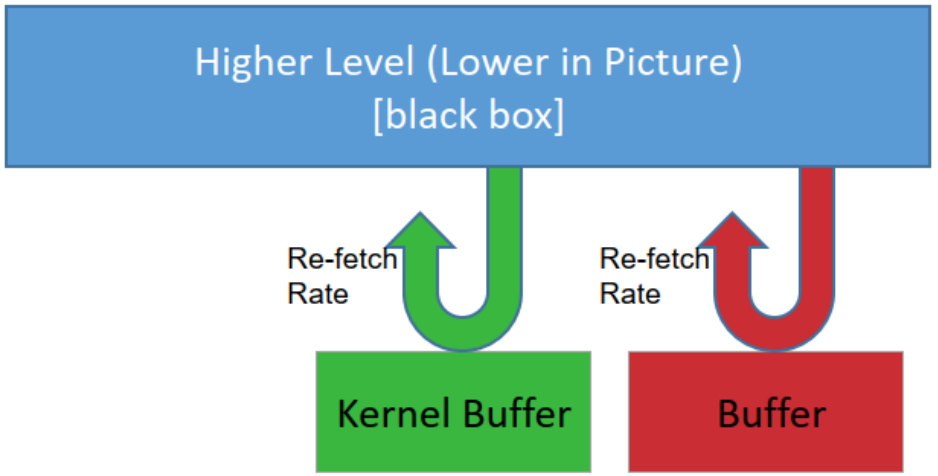


Recursive Formulation

- **Fixed order as follows**
- $____Y_{i-1}X_{i-1}C_{i-1}K_{i-1}\boldsymbol{Y_iX_iC_i}\ \ldots$
- **From left to right observing**
  - **X or Y: new KB**
  - **C: new OB**

Buffer Refetch Rate: the number of times a piece of data is fetched from a certain buffer after initially being loaded into that buffer

Buffer Name	Buffer Size	Buffer Refetch Rate
KB_i	$C_{i-1}K_{i-1}F_hF_w$	$(X_iY_i)/(X_{i-1}Y_{i-1})$
OB_i	$Y_iX_iK_{i-1}$	$2C_i/C_{i-1}$

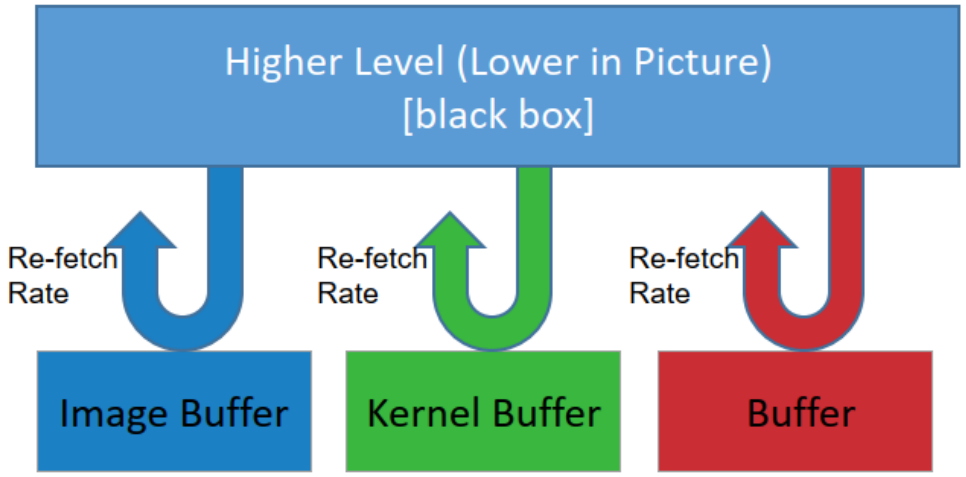


Recursive Formulation

- **Fixed order as follows**
- $____Y_{i-1}X_{i-1}C_{i-1}K_{i-1}\boldsymbol{Y_iX_iC_iK_i}\ldots$
- **From left to right observing**
  - **X or Y: new KB**
  - **C: new OB**
  - **K: new IB**

Buffer Refetch Rate: the number of times a piece of data is fetched from a certain buffer after initially being loaded into that buffer

Buffer Name	Buffer Size	Buffer Refetch Rate
KB_i	$C_{i-1}K_{i-1}F_hF_w$	$(X_iY_i)/(X_{i-1}Y_{i-1})$
OB_i	$Y_iX_iK_{i-1}$	$2C_i/C_{i-1}$
IB_i	$(Y_i+F_h-1)(X_i+F_w-1)C_i$	$(K_i(Y_i+F_h-1)(X_i+F_w-1))/(K_{i-1}Y_iX_i)$



How Does Parallelism Work?

- Unroll Loops in Space
- Replicate Compute Units
- Perform Loops in parallel
  - SIMD
  - Multiple Cores
  - Multiple CPU/GPUs

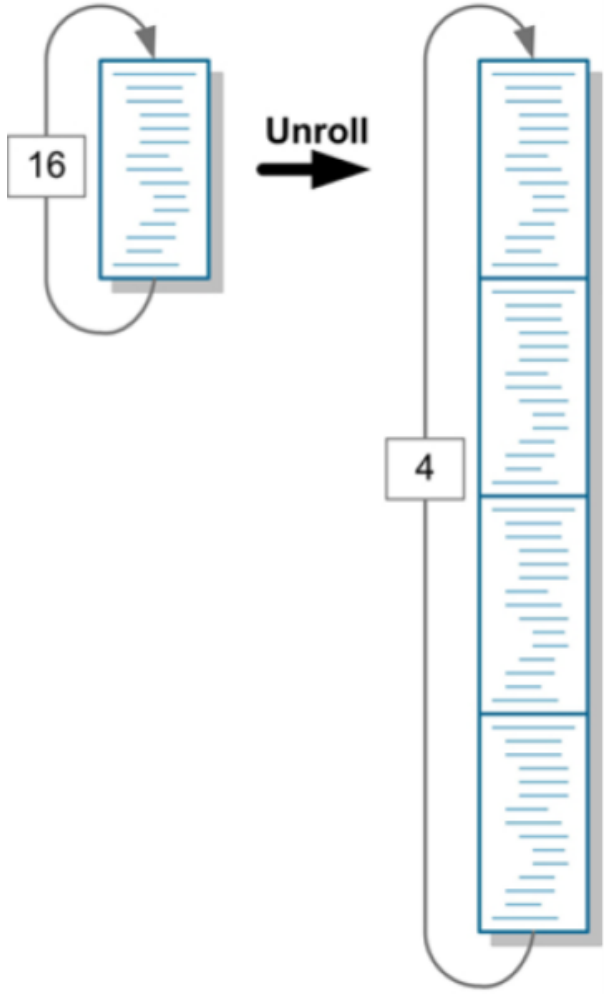
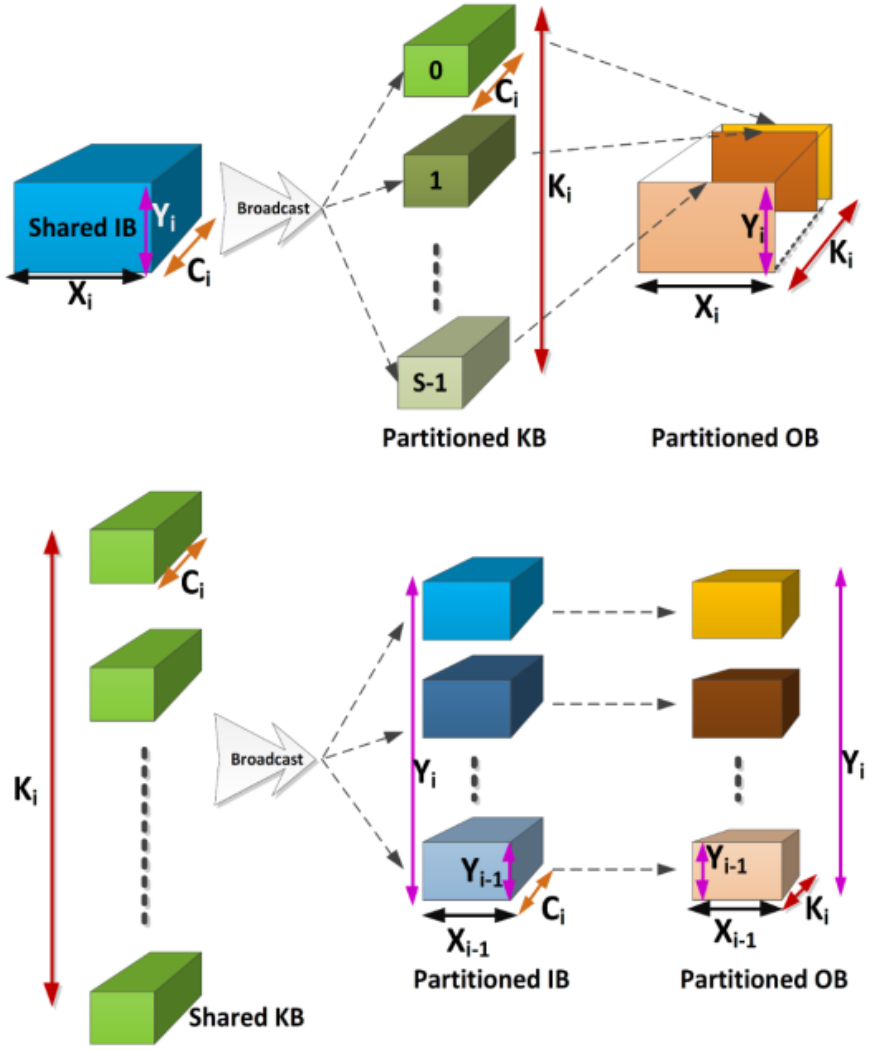


Figure 5. Loop unrolled four times.

Multi-Core Parallelism and Partitioning

- Across channels (C)
  - Reduction
  - Synchronization
- Across kernels (K)
  - Synchronization
- Across image (X or Y)
  - Independent

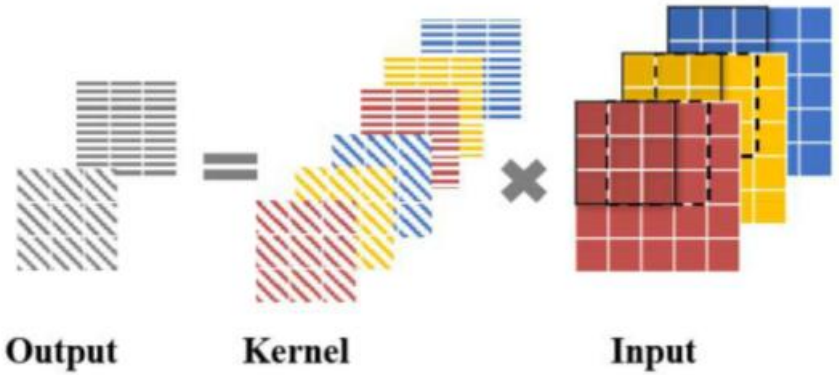
- Broadcast operation
- Partitioned memory
- Nested parallelism



High Performance Direct Convolution

- **Map application to system architecture to fully exploit system capacity**

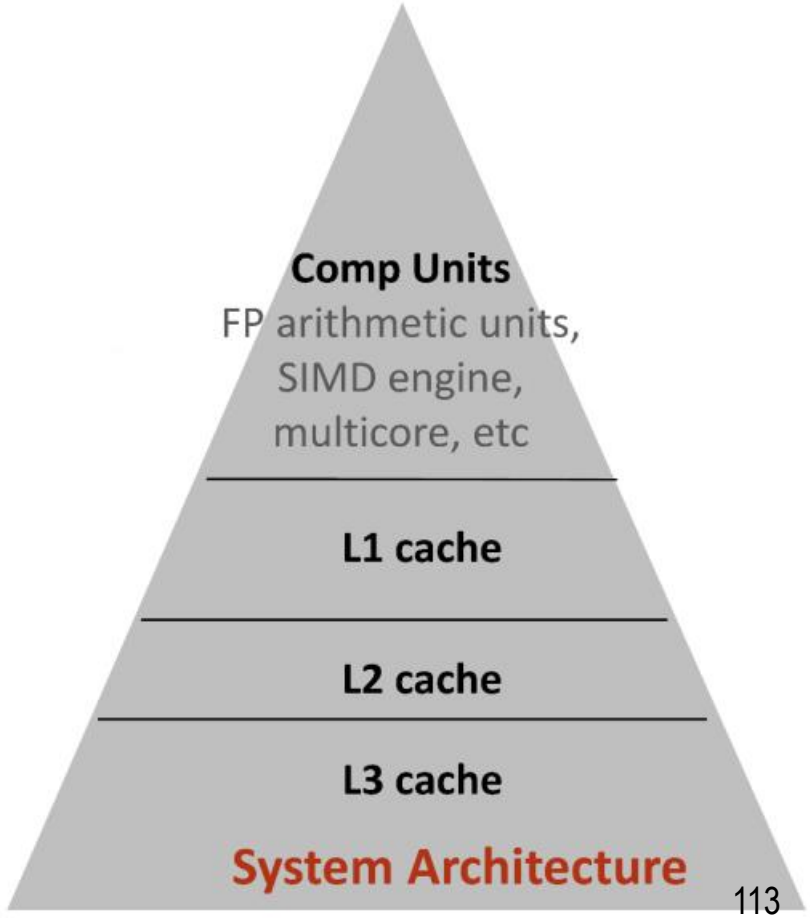
High-dimensional data tensor



Multi-level nested loops and various data access patterns

```
for i = 1 to C_i do
  for j = 1 to C_o do
    for k = 1 to W_o do
      for ℓ = 1 to H_o do
        for m = 1 to W_f do
          for n = 1 to H_f do
            O_{j,k,ℓ} += I_{i,k×s+m,ℓ×s+n} × F_{i,j,m,n}
```

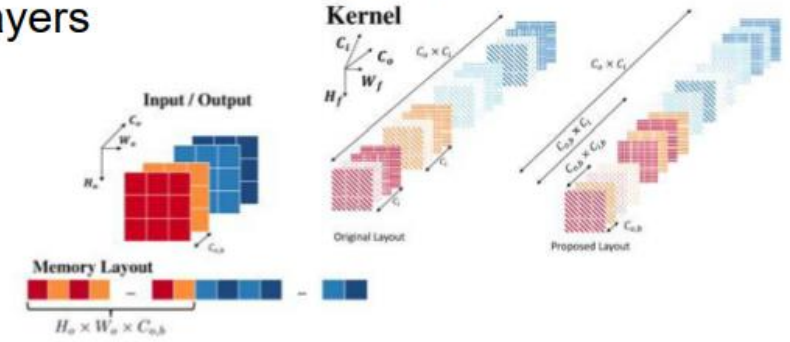
Direct Convolution



High Performance Direct Convolution

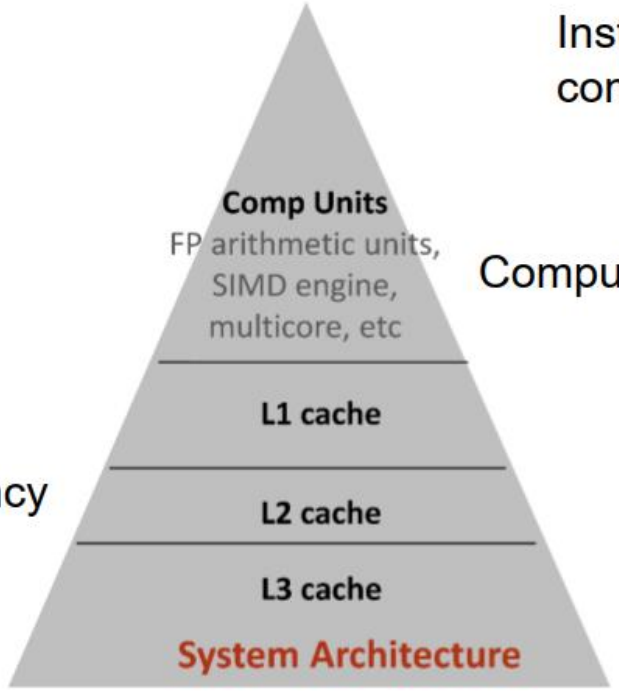
- **Analytical model to achieve system theoretical peak performance**

Convolution-friendly data layout that avoids format reshaping between layers



Parallelism to exploit core-level concurrency

for $j' = 1$ **to** $C_o/C_{o,b}$ **in Parallel do**
  for $i' = 1$ **to** $C_i/C_{i,b}$ **do**



Instruction scheduling to saturate the computation pipeline

$$\mathcal{E} \geq N_{\text{vec}} N_{\text{fma}} L_{\text{fma}},$$

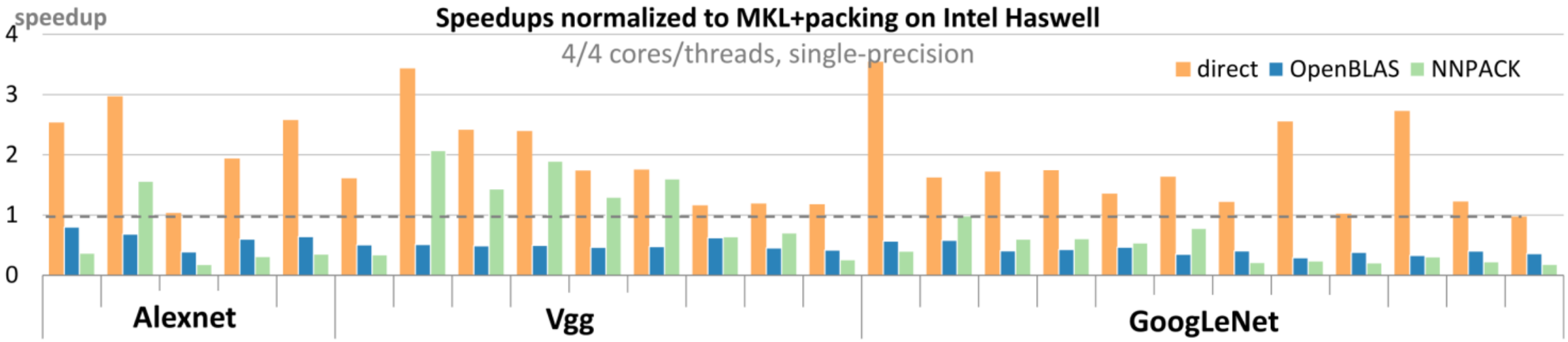
Computation reordering that maximizes data reuse

Algorithm 2 Reorder Convolution Algorithm

Input: Input $\mathcal{I}$, Kernel Weights $\mathcal{F}$, stride s;
Output: Output $\mathcal{O}$
for $\ell = 1$ **to** H_o **do**
  for $n = 1$ **to** H_f **do**
    for $m = 1$ **to** W_f **do**
      for $i = 1$ **to** C_i **do**
        for $k = 1$ **to** W_o **do**
          for $j = 1$ **to** C_o **do**
            $\mathcal{O}_{j,k,\ell} += \mathcal{I}_{i,k\times s+m,\ell\times s+n} \times \mathcal{F}_{i,j,m,n}$

Loop tiling and blocking to utilize the memory hierarchies

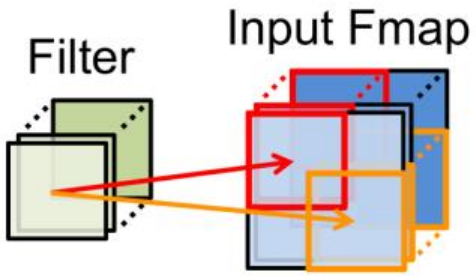
Better Performance



Types of Data Reuse in DNN

Convolutional Reuse

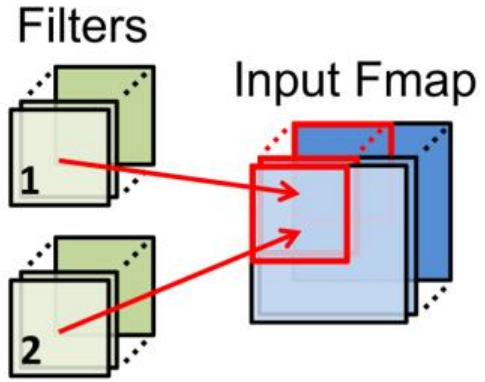
CONV layers only
(sliding window)



Reuse: Activations
Filter weights

Fmap Reuse

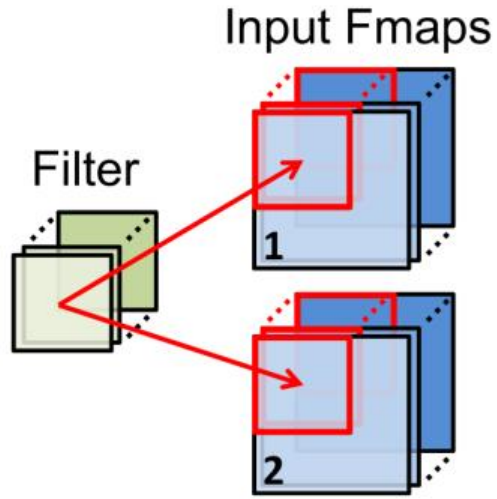
CONV and FC layers



Reuse: Activations

Filter Reuse

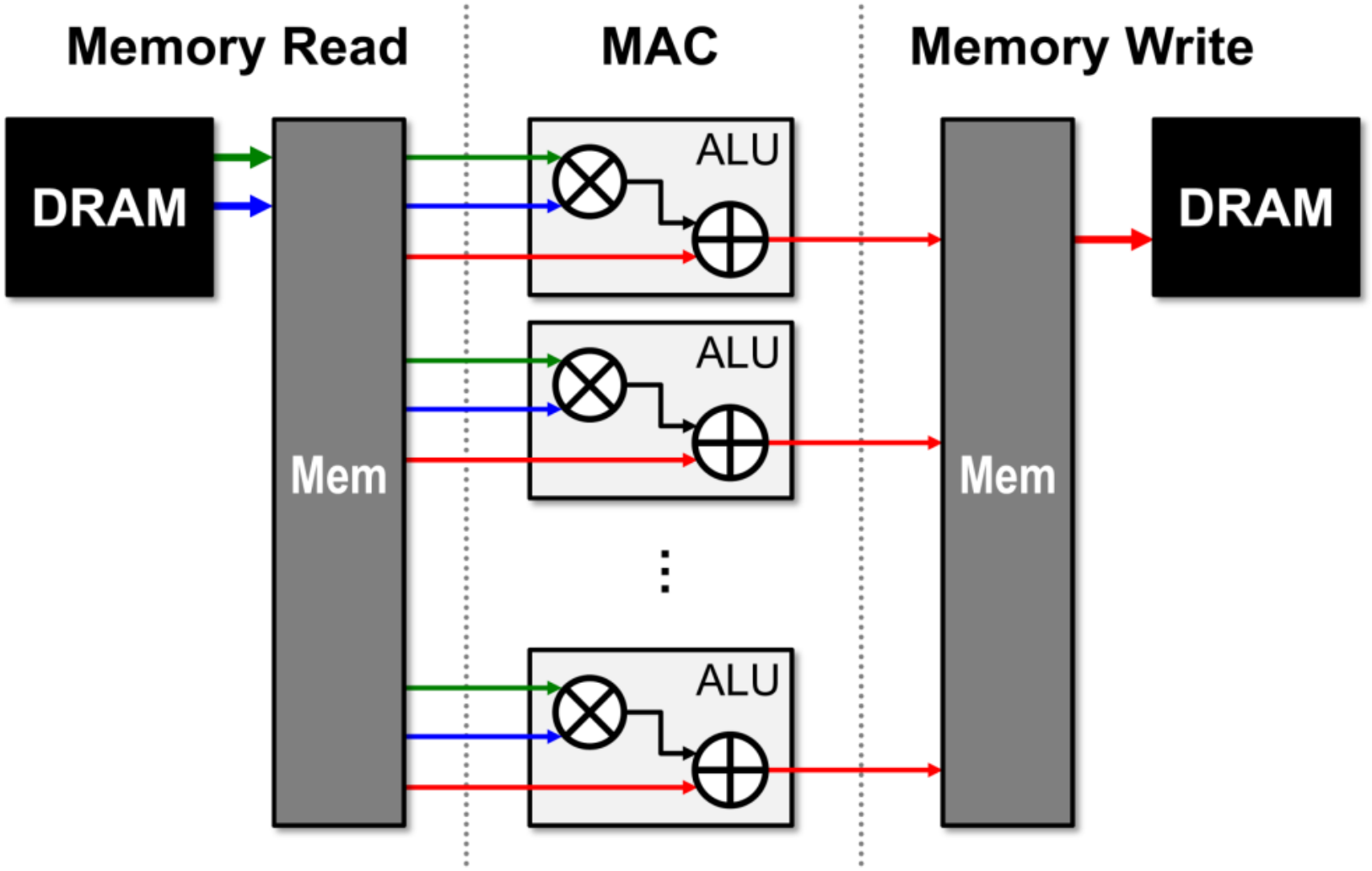
CONV and FC layers
(batch size > 1)



Reuse: Filter weights

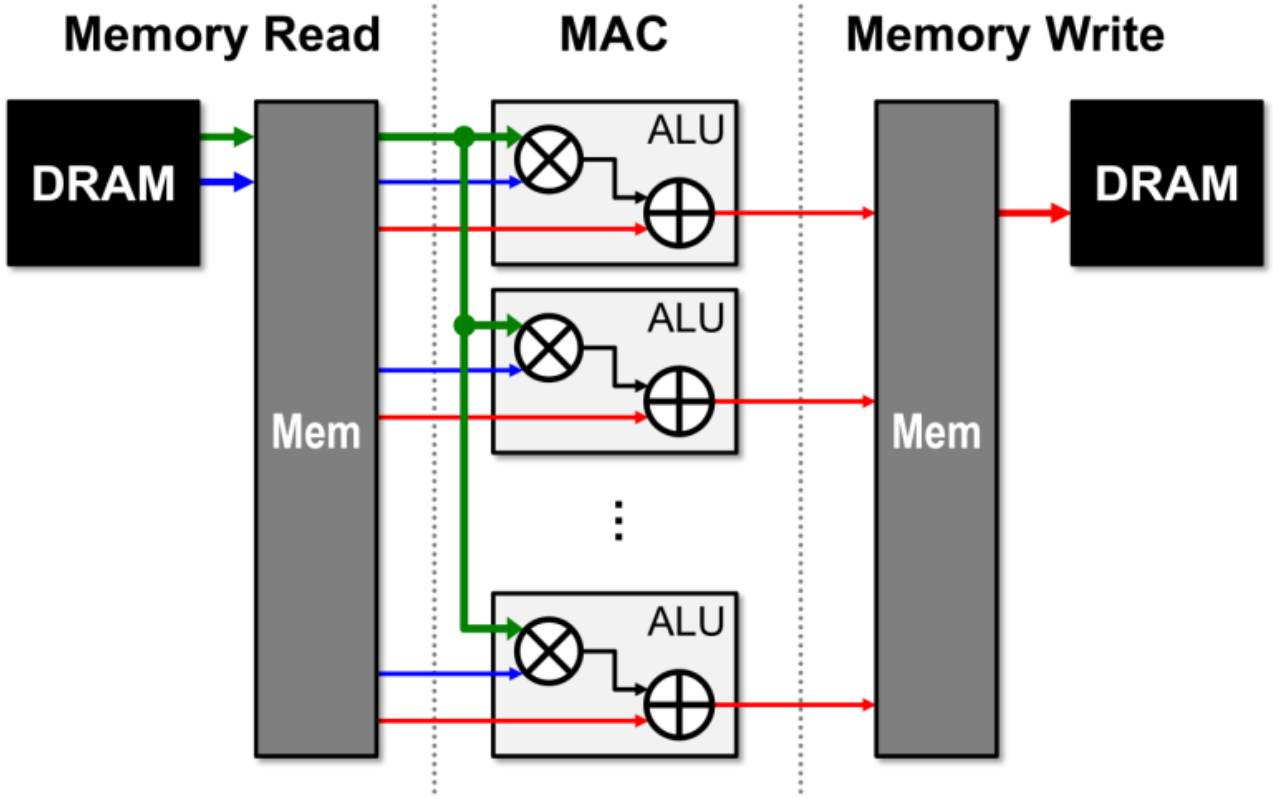
If all data reuse is exploited, DRAM accesses in AlexNet can be reduced from 2896M to 61M (best case)

Parallelism: Parallel MAC Units

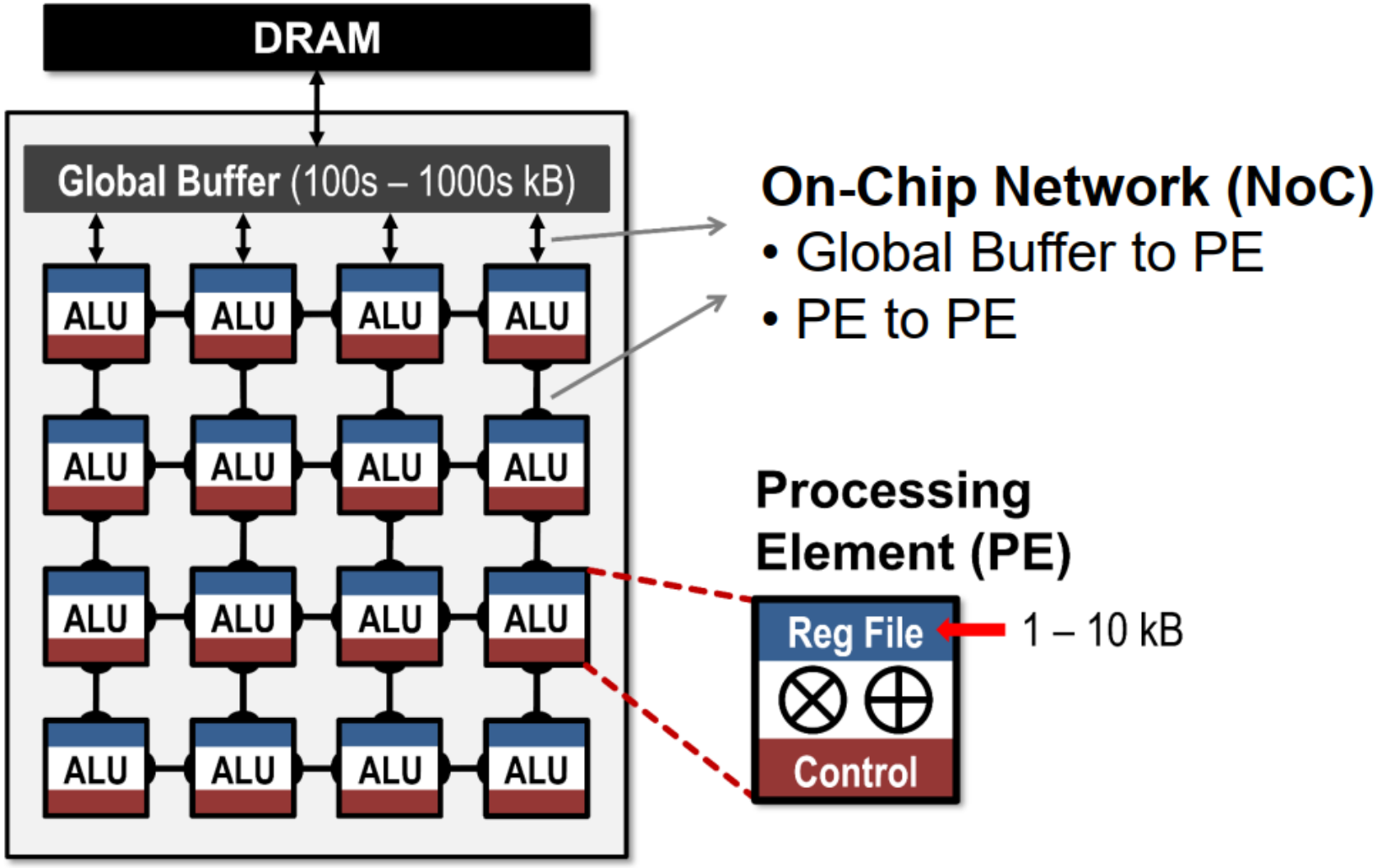


Parallelism: Spatial Data Reuse

- Spatial reuse: the same data is used by more than one consumer at different spatial locations of the hardware.

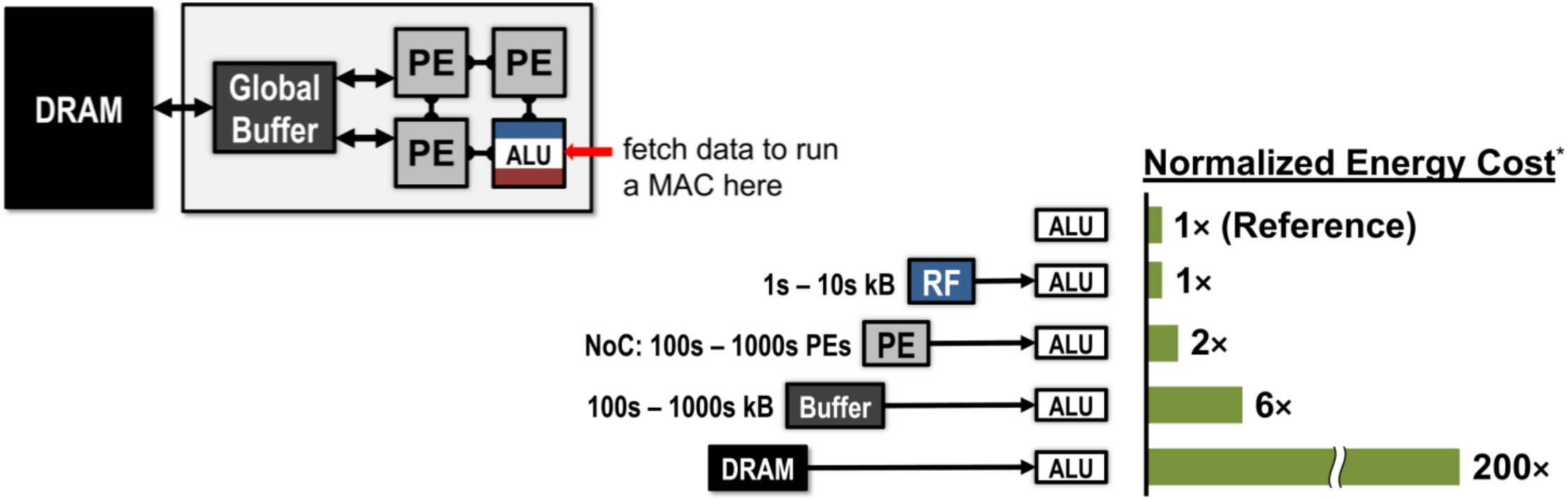


Spatial Architecture for DNN



Multi-Level Low-Cost Data Access

- A Dataflow is required to maximally exploit data reuse with the low-cost memory hierarchy and parallelism



Recap CNN Decoder Ring

Shape Parameter	Description
N	Number of **input fmaps/output fmaps** (batch size)
C	Number of 2-D **input fmaps /filters** (channels)
H	Height of **input fmap** (activations)
W	Width of **input fmap** (activations)
R	Height of 2-D **filter** (weights)
S	Width of 2-D **filter** (weights)
M	Number of 2-D **output fmaps** (channels)
E	Height of **output fmap** (activations)
F	Width of **output fmap** (activations)



Dataflow and Loop Nest

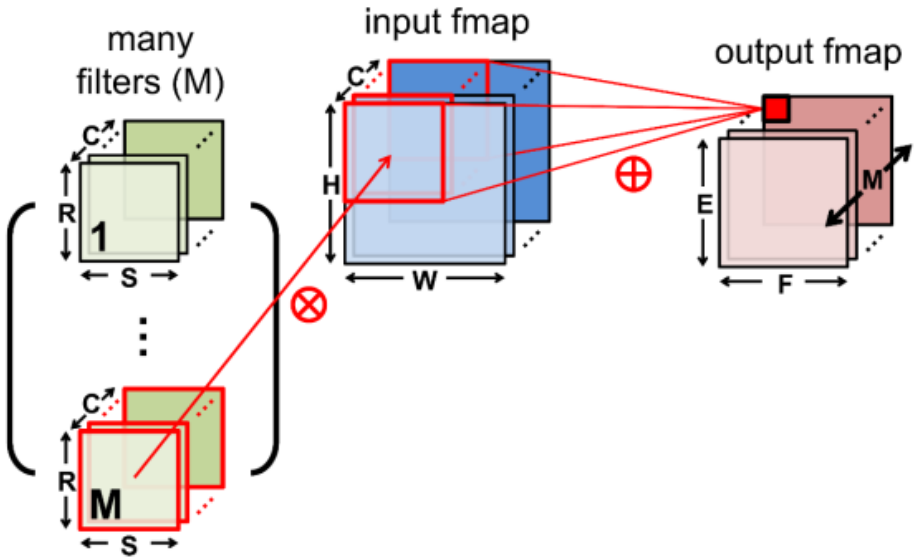
$$O_{nmef} = \left(\sum_{crs} I_{nc(e+r)(f+s)} F_{mcrs} \right) + b_m$$

- No ordering or notion of parallelism on the individual calculations
  - But are important for Hardware for performance and efficiency computation
- Dataflow
  - Specifying an ordering and which calculation run in parallel
  - Computation order
  - Data movement order
- Loop Nest
  - Describe Various properties of a given DNN accelerator design
  - Describe Dataflow
  - for: temporal for, describes the temporal execution order
  - spatial_for: describes parallel execution



Data Reuse in DNN

- Total # of MACs:
  - RSCEFM
- Input Activation
  - Size: HWC
  - Reuse:~RSM
- Weight
  - Size: RSCM
  - Reuse: EF
- Output Activation
  - Size: EFM
  - Reuse: RSC



1D Convolution Example




```
# i[W] - input activations
# f[R] - filter weights
# o[E] - output activations
for r in range(R):
    for e in range(E):
        w = e+r
        o[e] += i[w] * f[r]
```

Weight Stationary (WS) Dataflow

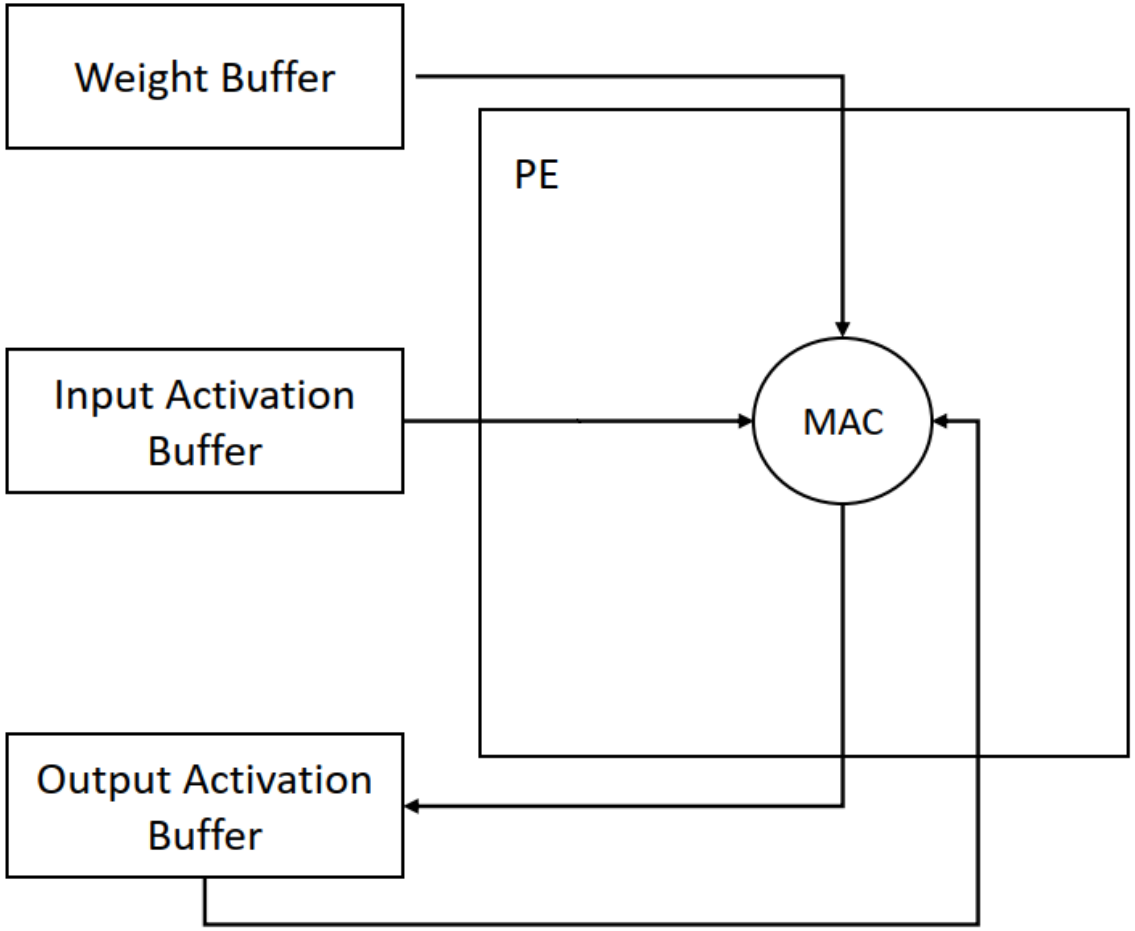
```
# i[W] - input activations
# f[R] - filter weights
# o[E] - output activations
for e in range(E):
    for r in range(S):
        w = e+r
        o[e] += i[w] * f[r]
```

Output Stationary (OS) Dataflow

```
# i[W] - input activations
# f[R] - filter weights
# o[E] - output activations
for w in range(W):
    for r in range(R):
        e = w-r
        o[e] += i[w] * f[r]
```

Input Stationary (IS) Dataflow

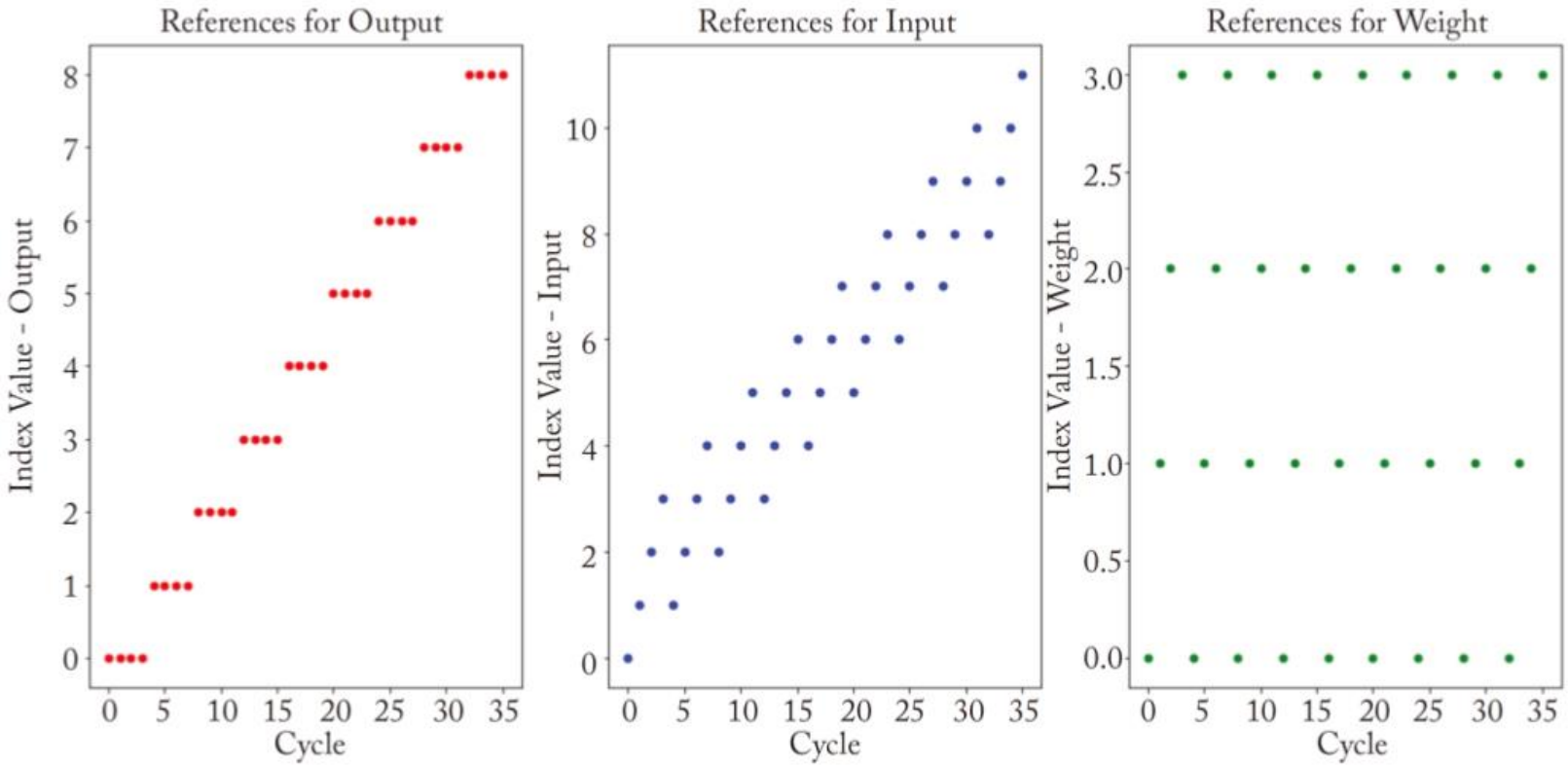
Single Processing Engine(PE) Setup



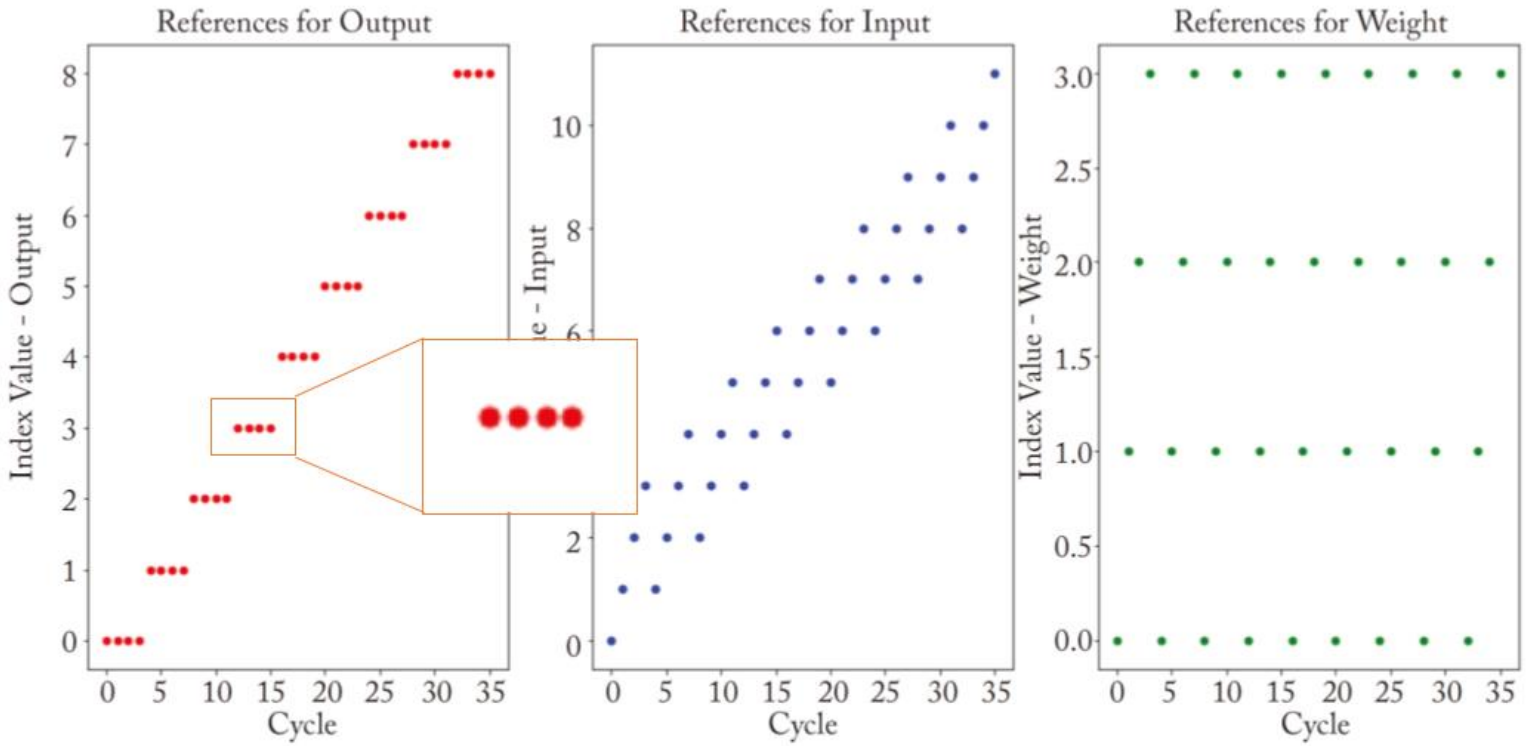
Output Stationary – Reference Pattern

```
for (e = 0; e < E; e++)
    for (r = 0; r < R; r++)
        O[e] += I[e+r] * W[r];
```

Layer Shape
- H = 12
- R = 4
- E = 9

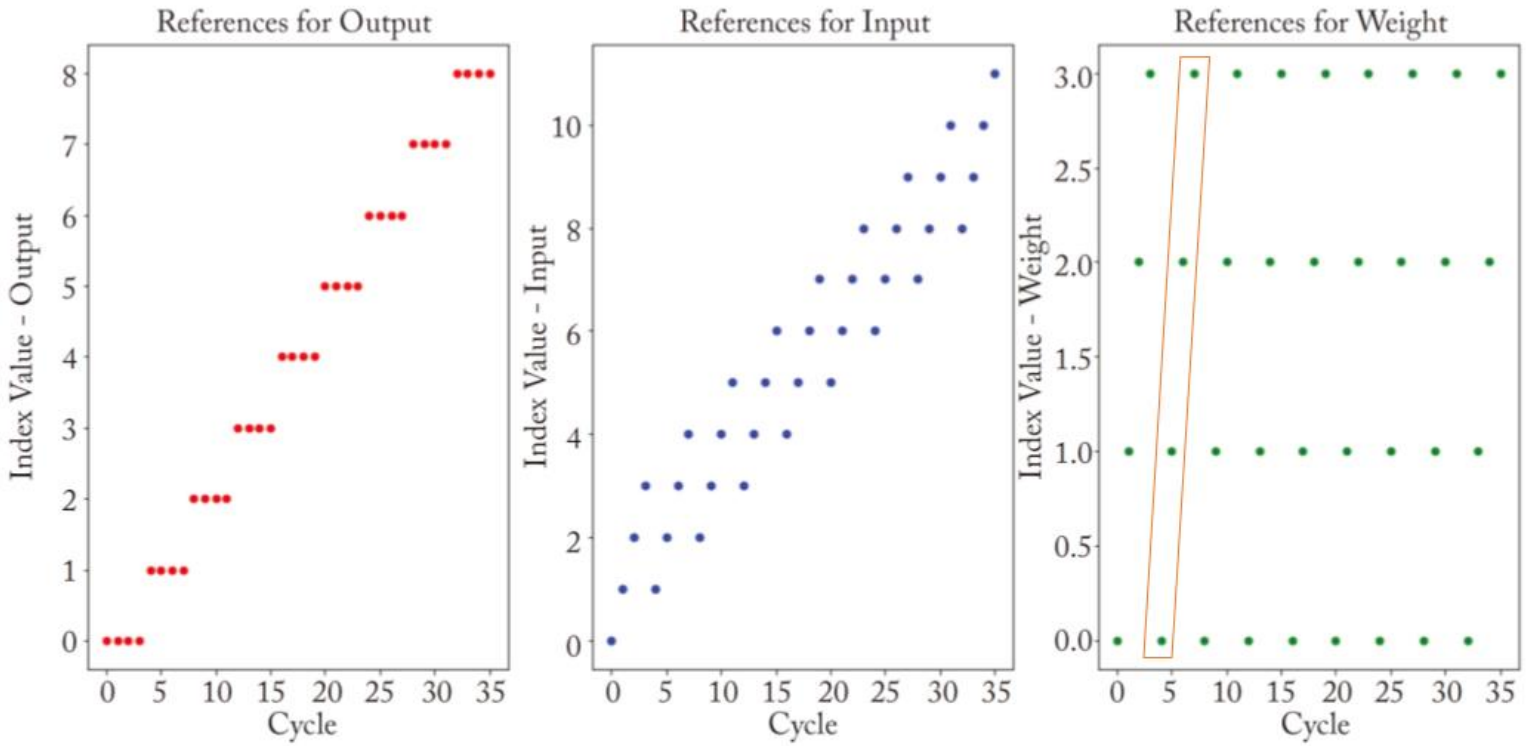


Output Stationary – Reference Pattern



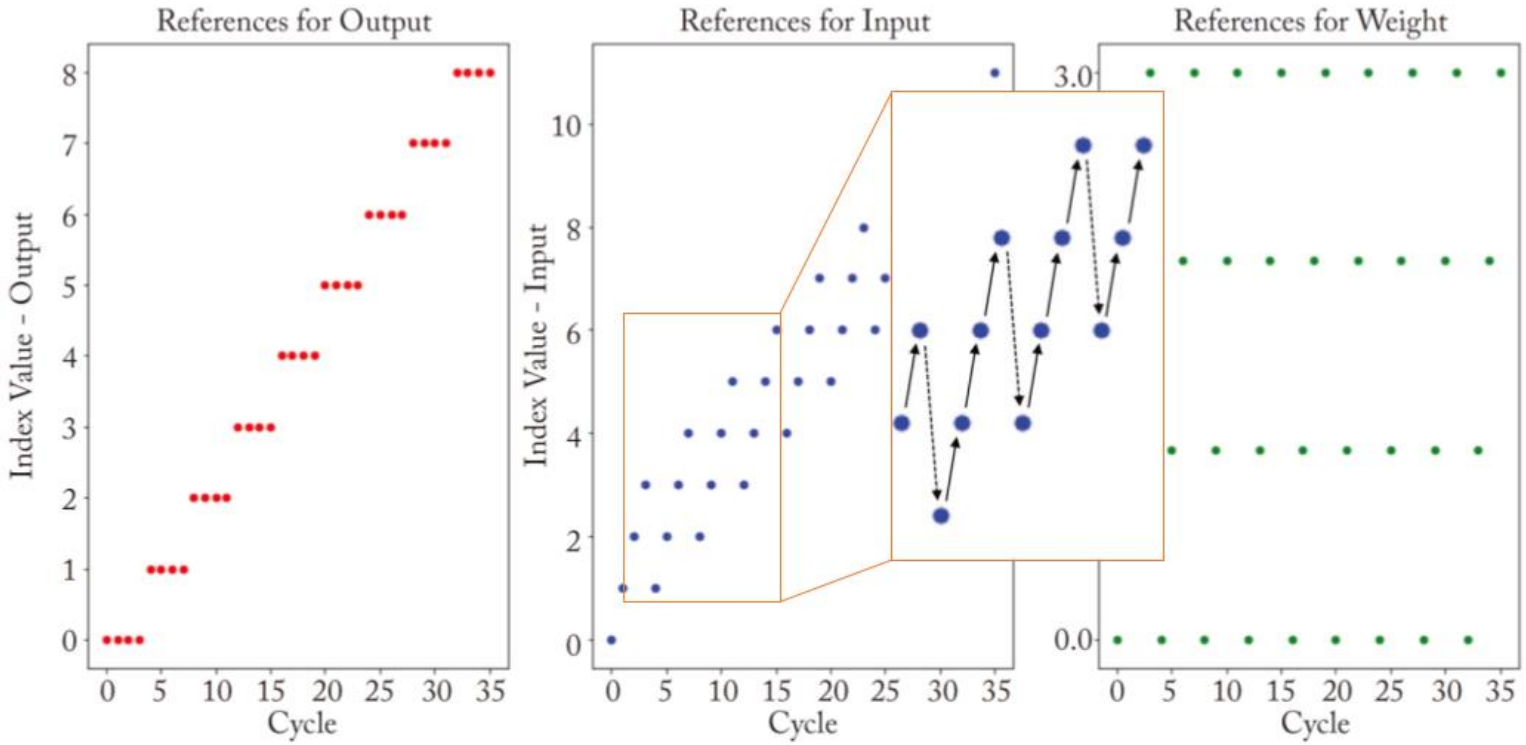
- Single output is reused many times (R)

Output Stationary – Reference Pattern



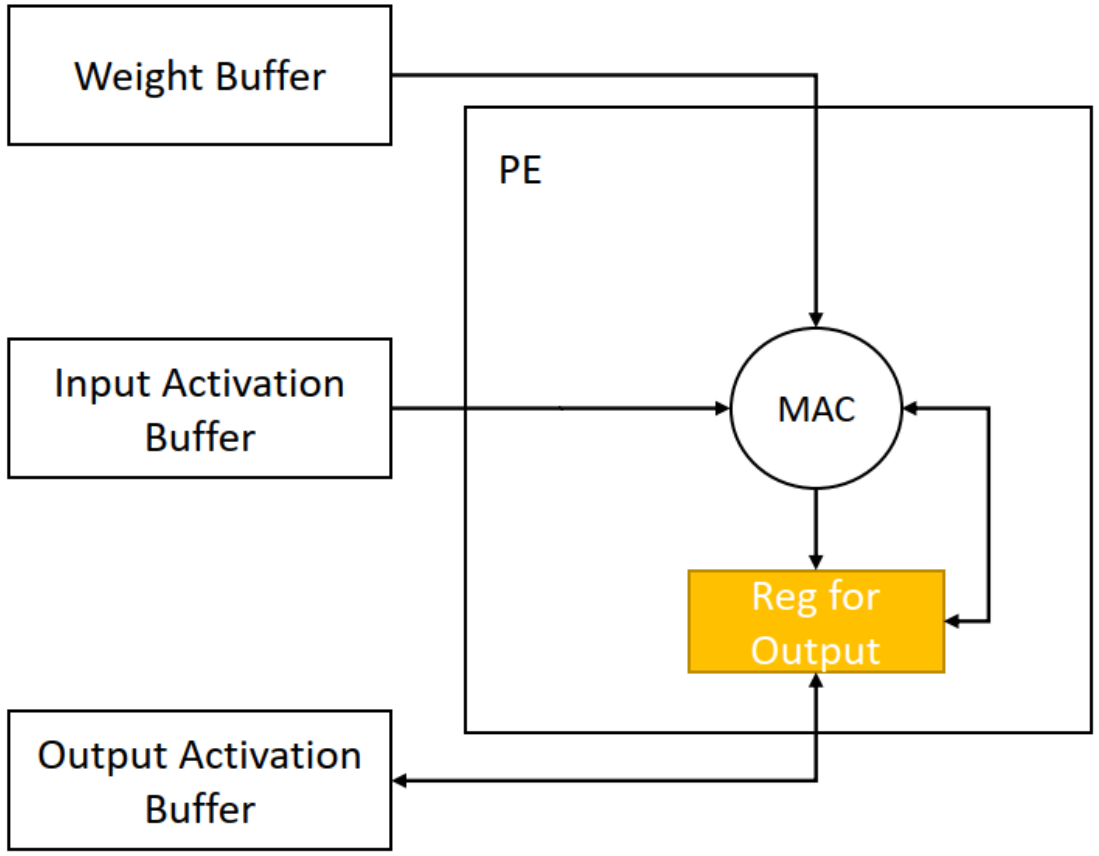
- Single output is reused many times (R)
- All weights reused repeatedly

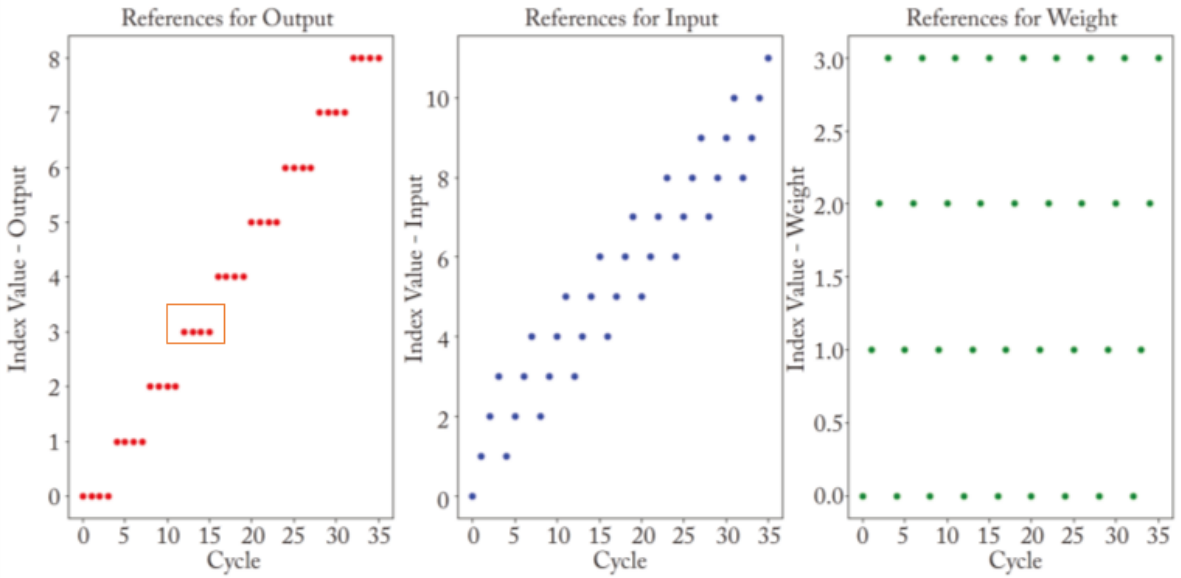
Output Stationary – Reference Pattern



- Single output is reused many times (R)
- All weights reused repeatedly
- Sliding window of inputs (size = R)

Buffer Access Pattern (OS)

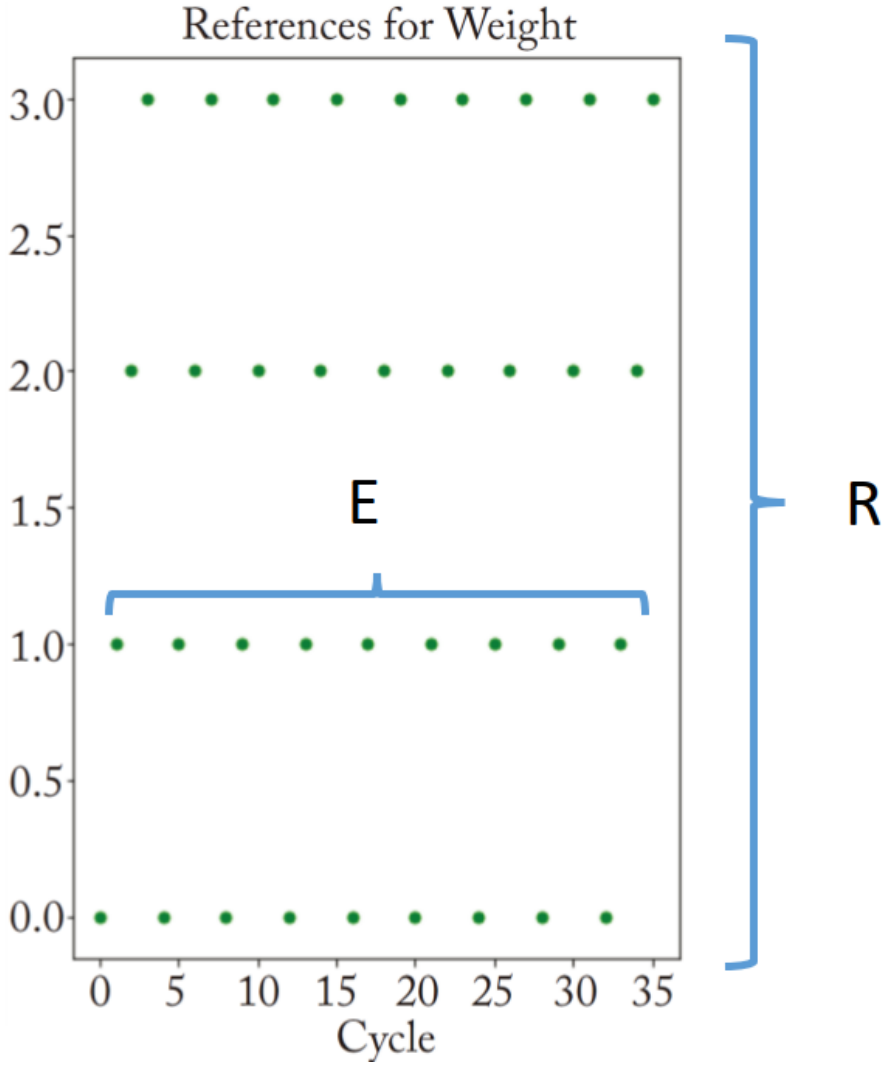




Buffer Data Accesses

```
for (e = 0; e < E; e++)
    for (r = 0; r < R; r++)
        O[e] += I[e+r] * W[r];
```

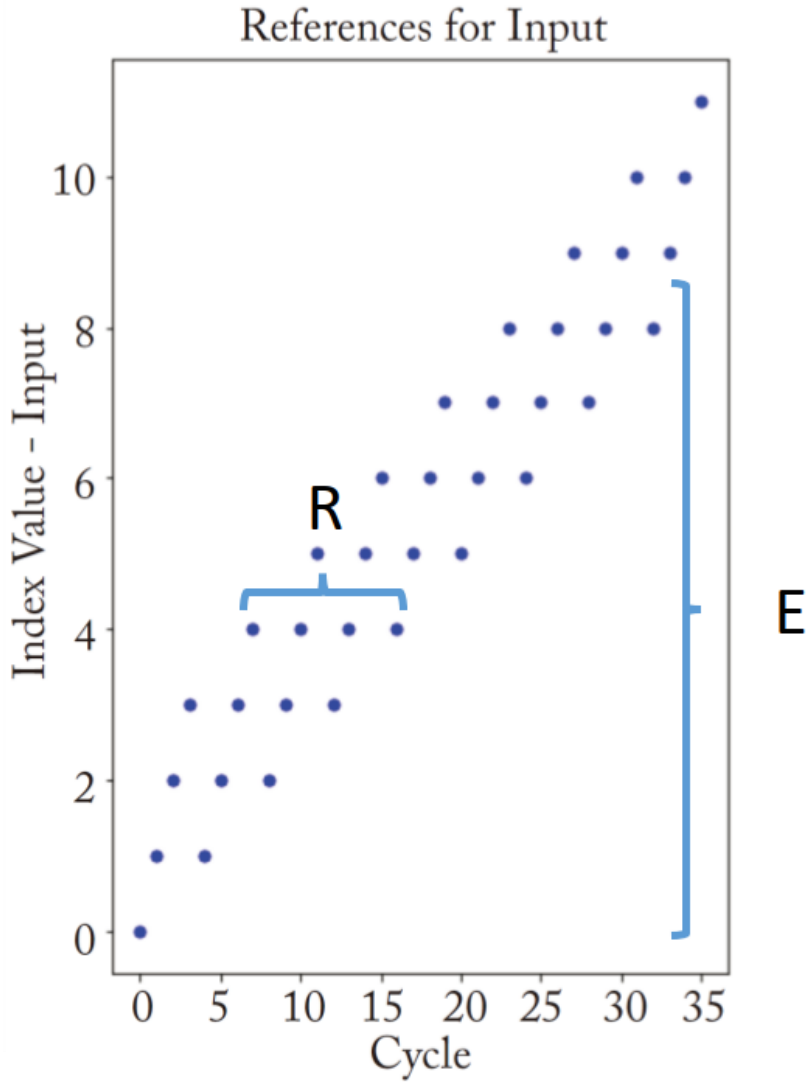
	OS
MACs	E*R
Weight Reads	E*R
Input Reads	
Output Reads	
Output Writes	



Buffer Data Accesses

```
for (e = 0; e < E; e++)
    for (r = 0; r < R; r++)
        O[e] += I[e+r] * W[r];
```

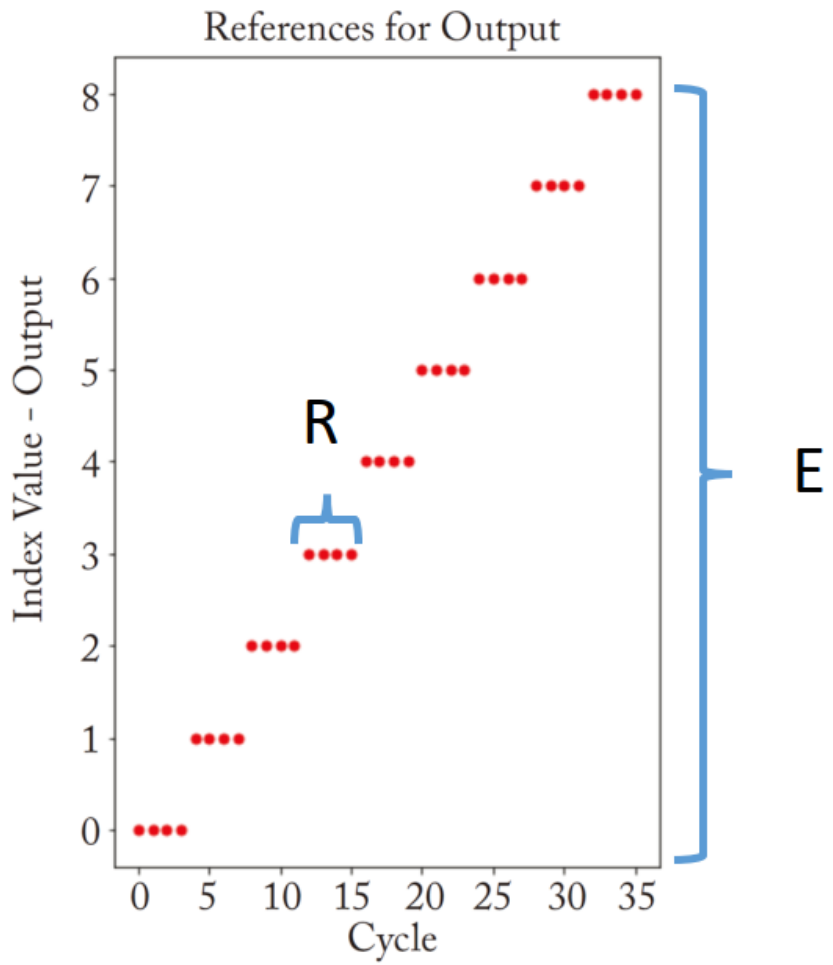
	OS
MACs	E*R
Weight Reads	E*R
Input Reads	E*R
Output Reads	
Output Writes	



Buffer Data Accesses

```
for (e = 0; e < E; e++)
    for (r = 0; r < R; r++)
        O[e] += I[e+r] * W[r];
```

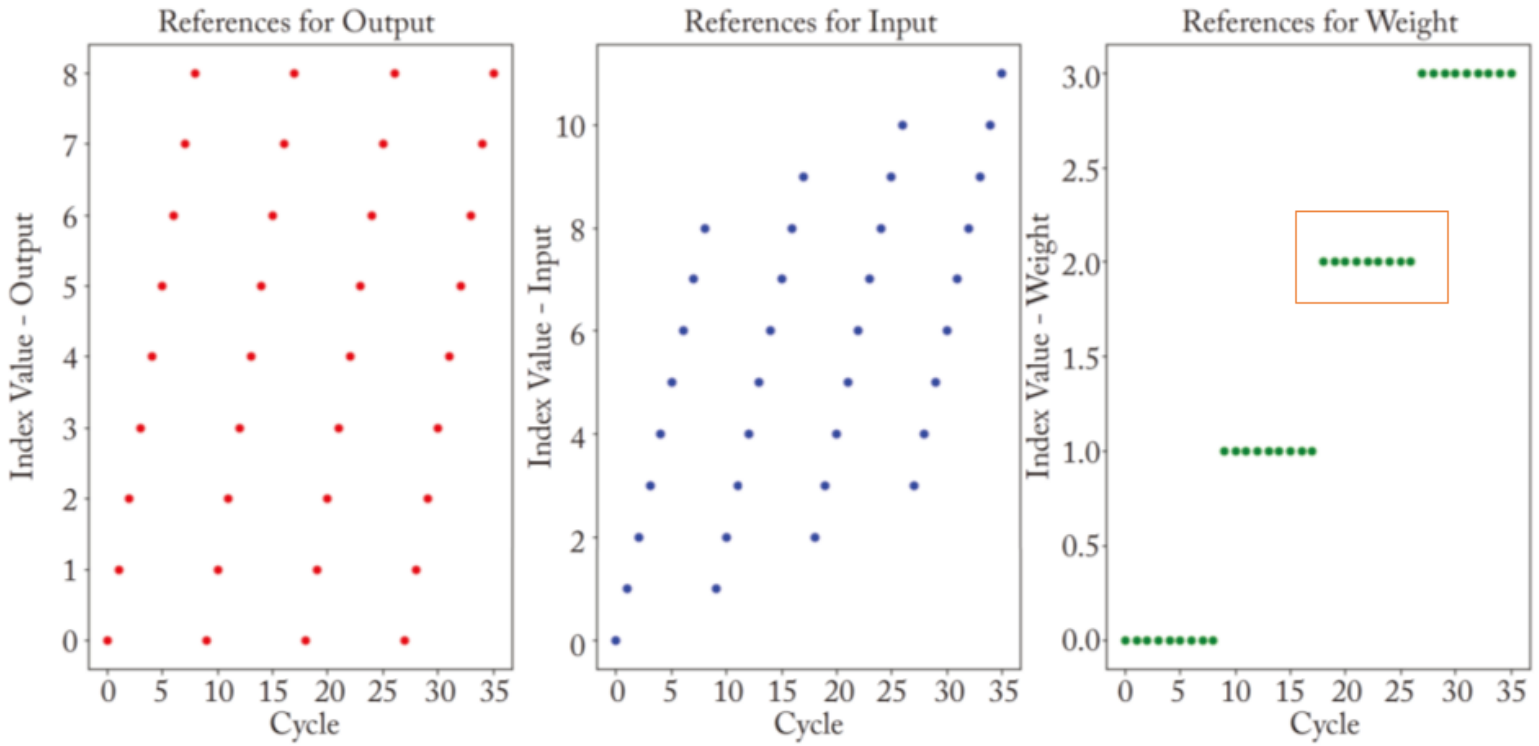
	OS
MACs	E*R
Weight Reads	E*R
Input Reads	E*R
Output Reads	0
Output Writes	E



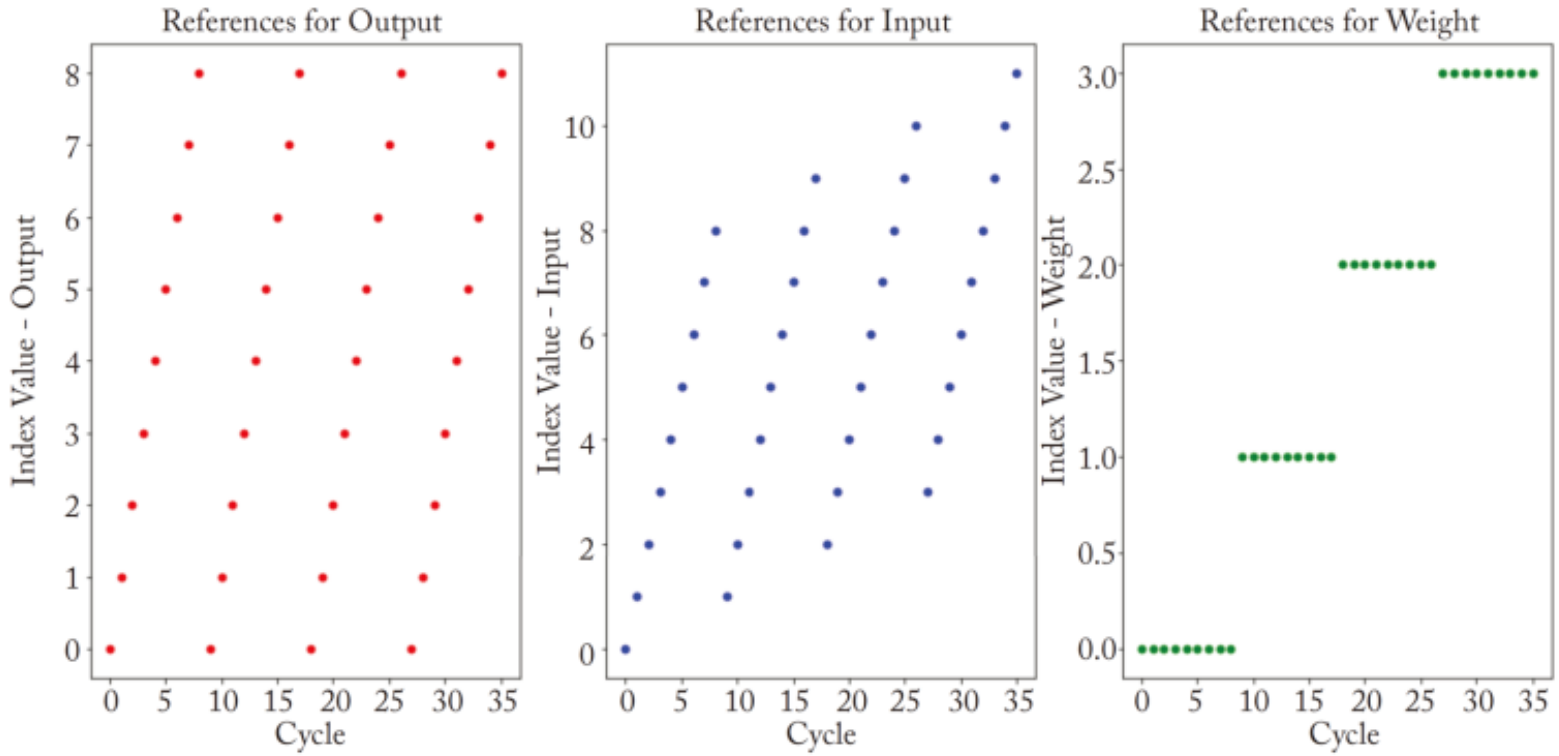
Weight Stationary – Reference Pattern

```
for (e = 0; e < E; e++)
    for (r = 0; r < R; r++)
        O[e] += I[e+r] * W[r];
```

No constraints on loop permutations!

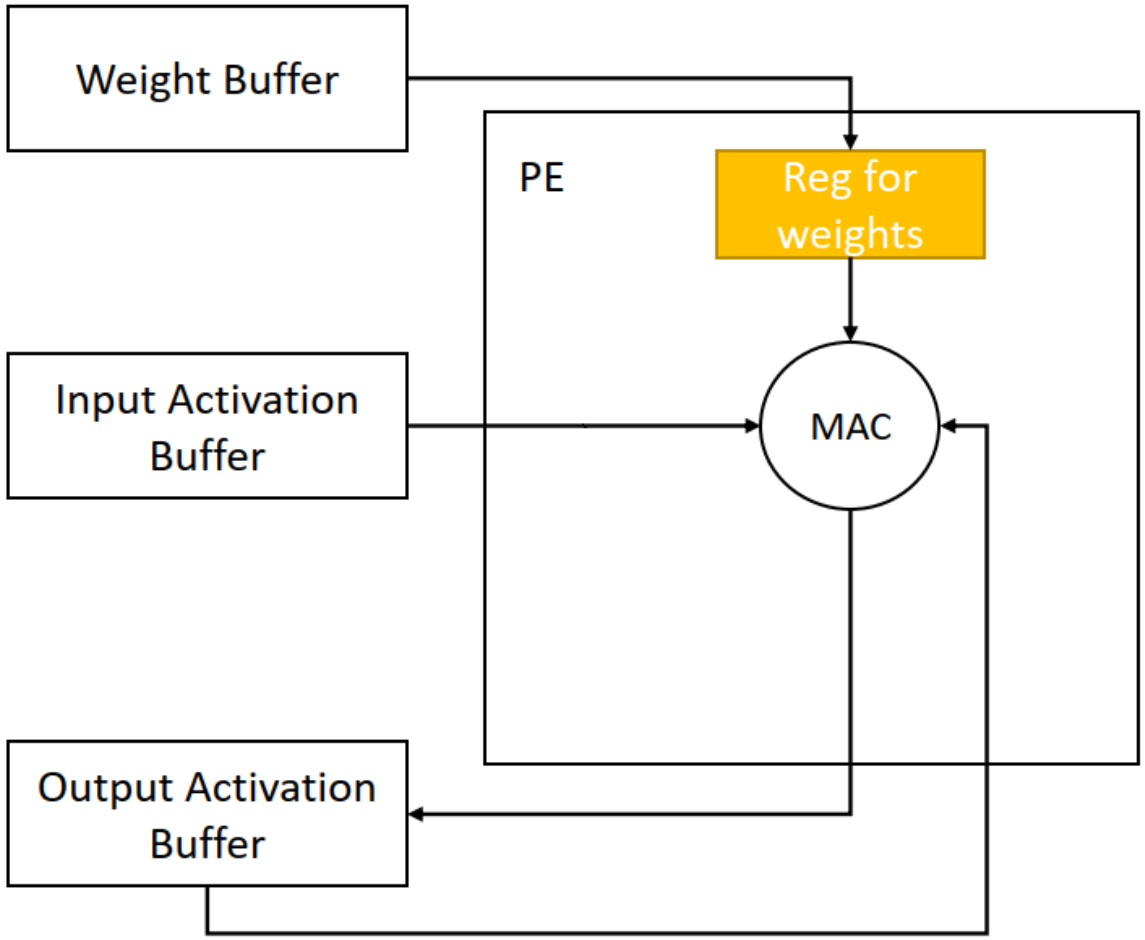


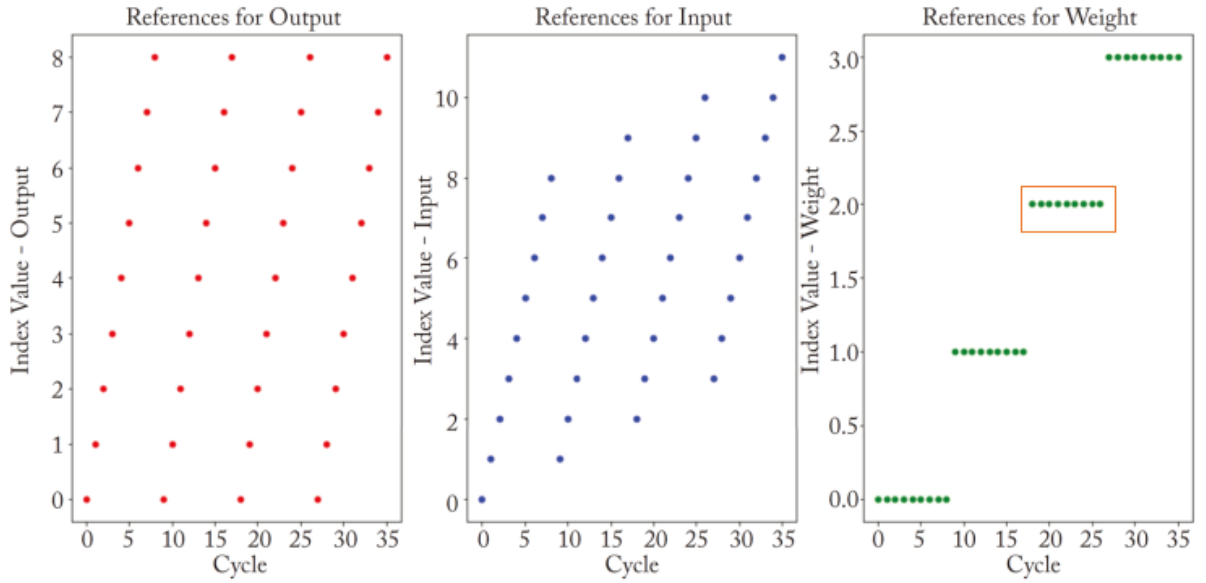
Weight Stationary – Reference Pattern



- Single weight is reused many times (E)
- Large sliding window of inputs (size = E)
- Fixed window of outputs (size = E)

Buffer Access Pattern (WS)





L1 Weight Stationary - Costs

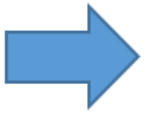
```
for (e = 0; e < E; e++)
    for (r = 0; r < R; r++)
        O[e] += I[e+r] * W[r];
```

	OS	WS	IS	Min
MACs	E*R	E*R		
Weight Reads	E*R	R		
Input Reads	E*R	E*R		
Output Reads	0	E*R		
Output Writes	E	E*R		



Tiled Loop Nest For Weight-stationary

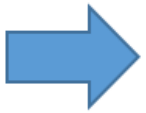
```
# i[W] - input activations
# f[R] - filter weights
# o[E] - output activations
for r in range(R):
    for e in range(E):
        w = e+r
        o[e] += i[w] * f[r]
```



```
# i[W] - input activations
# f[R] - filter weights
# o[E] - output activations
for e1 in range(E1):
    for r in range(R):
        for e0 in range(E0):
            e = e1*e0 + e0
            w = e+r
            o[e] += i[w] * f[r]
```

Parallel Processing for The A Tile Of Filter Weights

```
# i[W] - input activations
# f[R] - filter weights
# o[E] - output activations
for r in range(R):
    for e in range(E):
        w = e+r
        o[e] += i[w] * f[r]
```



```
# i[W] - input activations
# f[R] - filter weights
# o[E] - output activations
for r1 in range(R1):
    for e in range(E):
        spatia_for r0 in range(R0):
            r = r1*r0 + r0
            w = e+r
            o[e] += i[w] * f[r]
```

Reference Pattern for Parallel Processing

```
# i[W] - input activations
# f[R] - filter weights
# o[E] - output activations
for r1 in range(R1):
    for e in range(E):
        spatia_for r0 in range(R0):
            r = r1*r0 + r0
            w = e+r
            o[e] += i[w] * f[r]
```

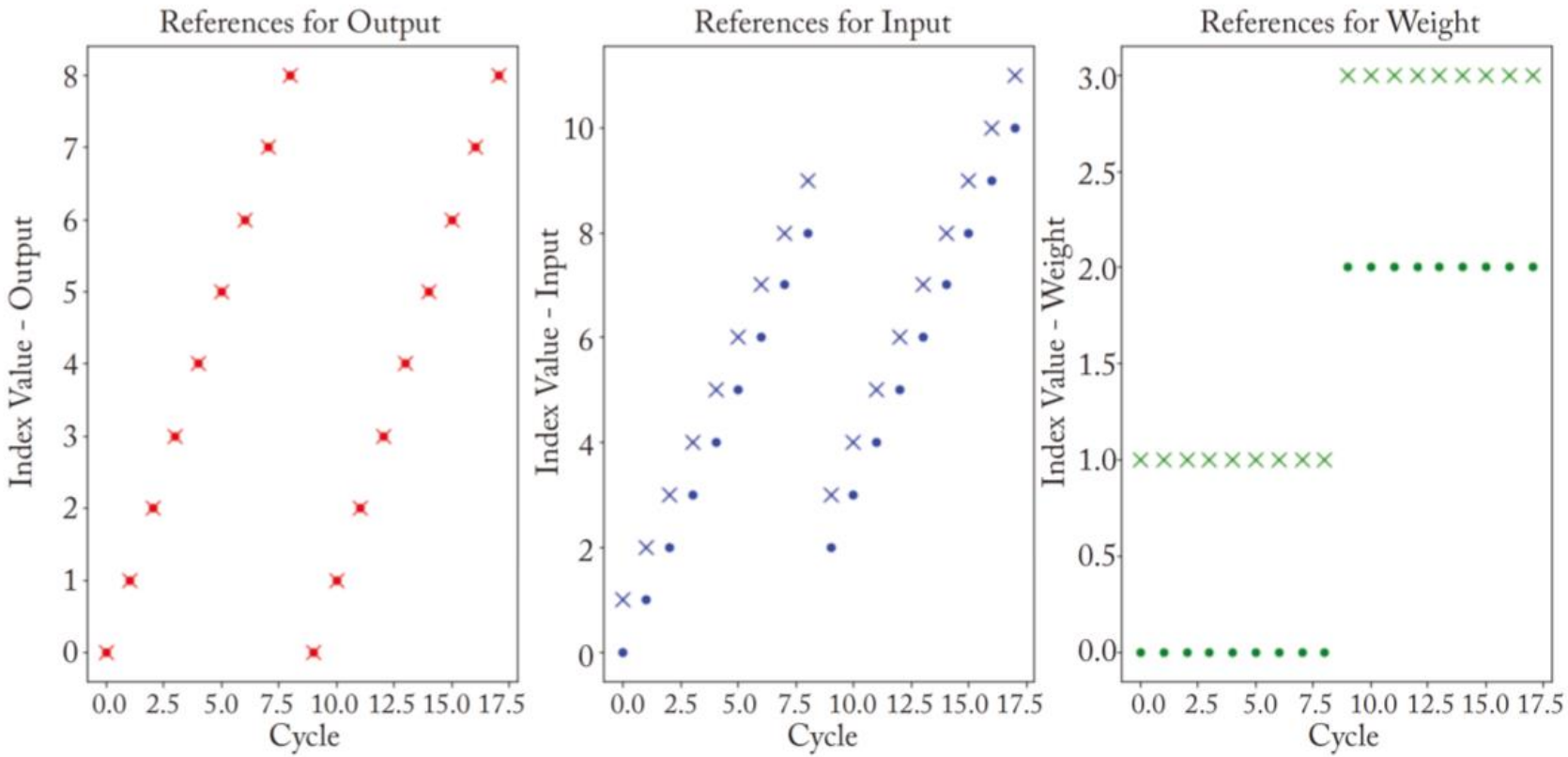
R = 4

E = 9

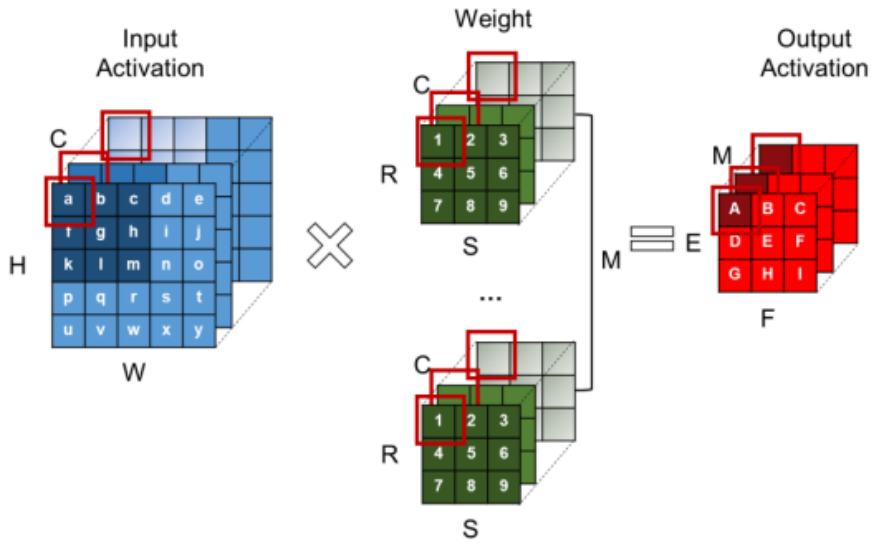
W = 12

R1 = 2

R0 = 2



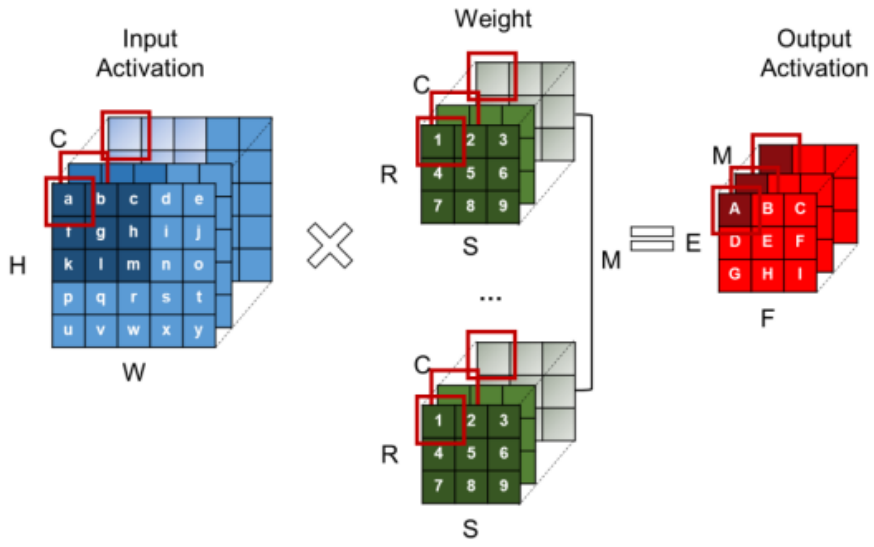
Weight Stationary Dataflow



Original nested loop:

```
for (n=0; n<N; n++) {
  for (m=0; m<M; m++) {
    for (e=0; e<E; e++) {
      for (f=0; f<F; f++) {
        OA[n][m][e][f]= 0;
        for (r=0; r<R; r++) {
          for (s=0; s<S; s++) {
            for (c=0; c<C; c++) {
              h = e * stride - pad + r;
              w = f * stride - pad + s;
              OA[n][m][e][f] +=
                          IA[n][c][h][w]
                             * W[m][c][r][s];
            }
          }
        }
      }
    }
  }
}
```

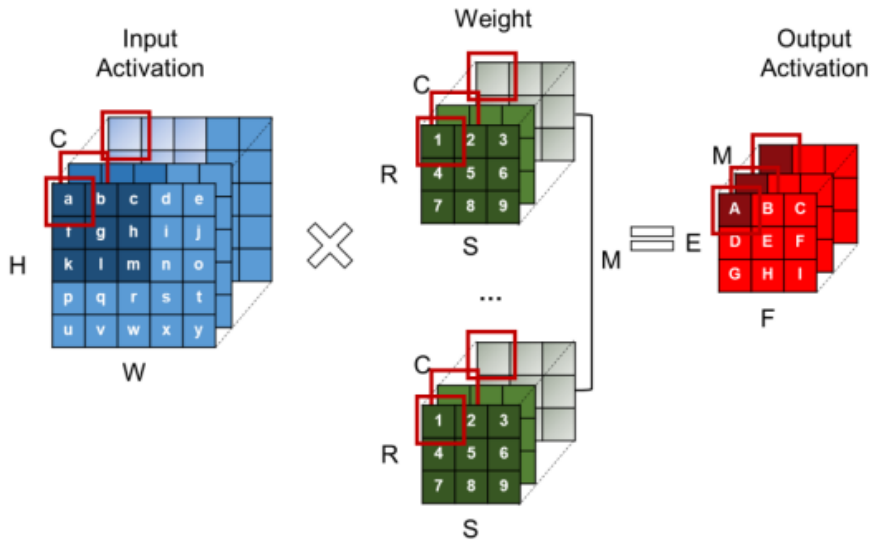
Weight Stationary Dataflow



Change temporal ordering

```
for (n=0; n<N; n++) {
  for (r=0; r<R; r++) {
    for (s=0; s<S; s++) {
      for (c=0; c<C; c++) {
        for (m=0; m<M; m++) {
          float curr_w = W[r][s][c][m];
          for (e=0; e<E; e++) {
            for (f=0; f<F; f++) {
              h = e * stride - pad + r;
              w = f * stride - pad + s;
              OA[n][m][e][f] +=
                        IA[n][c][h][w]
                           * curr_w;
            }
          }
        }
      }
    }
  }
}
```

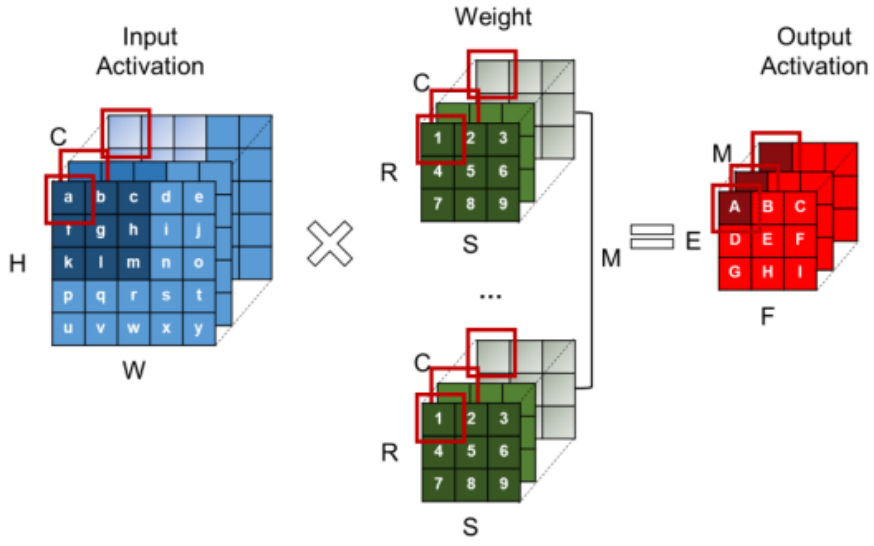
Weight Stationary Dataflow



Apply spatial parallelism

```
for (n=0; n<N; n++) {
  for (r=0; r<R; r++) {
    for (s=0; s<S; s++) {
      spatial_for (c=0; c<C; c++) {
        spatial_for (m=0; m<M; m++) {
          float curr_w = W[r][s][c][m];
          for (e=0; e<E; e++) {
            for (f=0; f<F; f++) {
              h = e * stride - pad + r;
              w = f * stride - pad + s;
              OA[n][m][e][f] +=
                        IA[n][c][h][w]
                           * curr_w;
            }
          }
        }
      }
    }
  }
}
```

Weight Stationary Dataflow



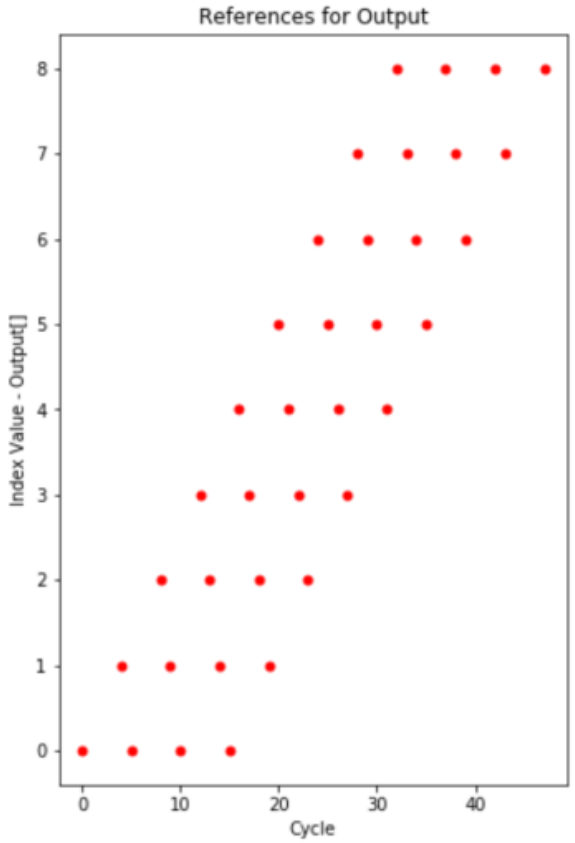
Apply temporal tiling (NVDLA Dataflow (nvdla.org))

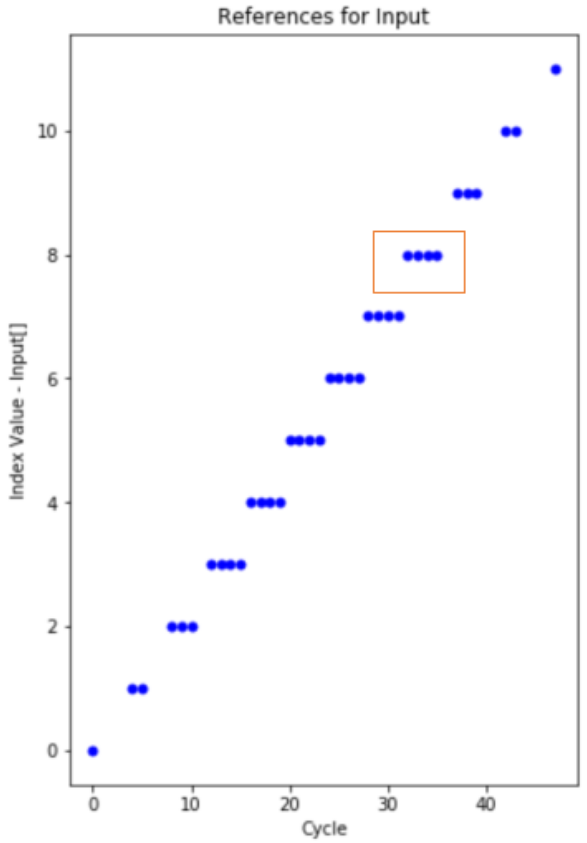
```
for (n=0; n<N; n++) {
  for (r=0; r<R; r++) {
    for (s=0; s<S; s++) {
      for (c_t=0; c_t<C/16; c_t++) {
        for (m_t=0; m_t<M/64; m_t++) {
          spatial_for (c_s=0; c_s<16; c_s++) {
            spatial_for (m_s=0; m_s<64; m_s++) {
              int curr_c = c_t * 16 + c_s;
              int curr_m = m_t * 64 + m_s;
              float curr_w = W[r][s][curr_c][curr_m];
              for (e=0; e<E; e++) {
                for (f=0; f<F; f++) {
                  h = e * stride - pad + r;
                  w = f * stride - pad + s;
                  OA[n][curr_m][e][f] +=
                          IA[n][curr_c][h][w]
                          * curr_w;
      }}}}
    }
  }
}
```

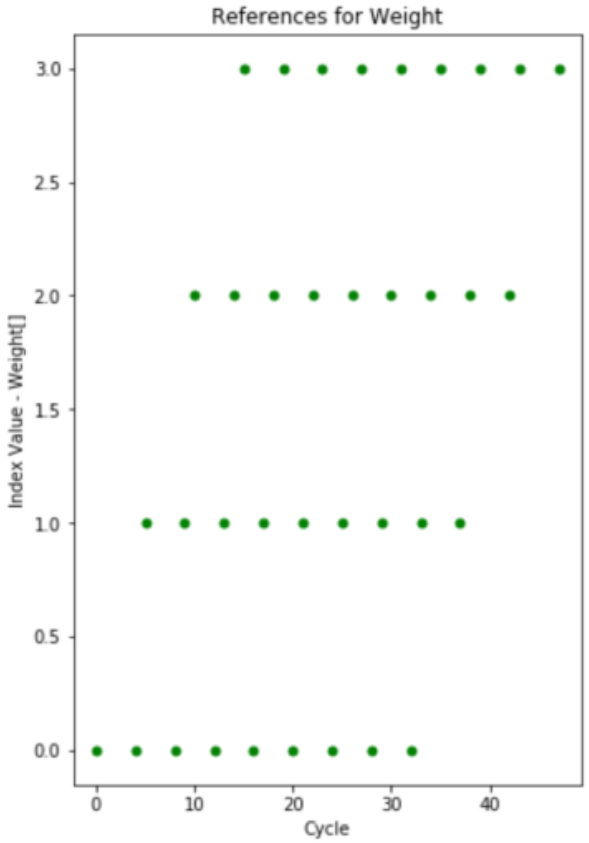
Input Stationary – Reference Pattern

```
for (h = 0; h < H; h++)
    for (r = 0; r < R; r++)
        O[h-r] += I[h] * W[r];
```

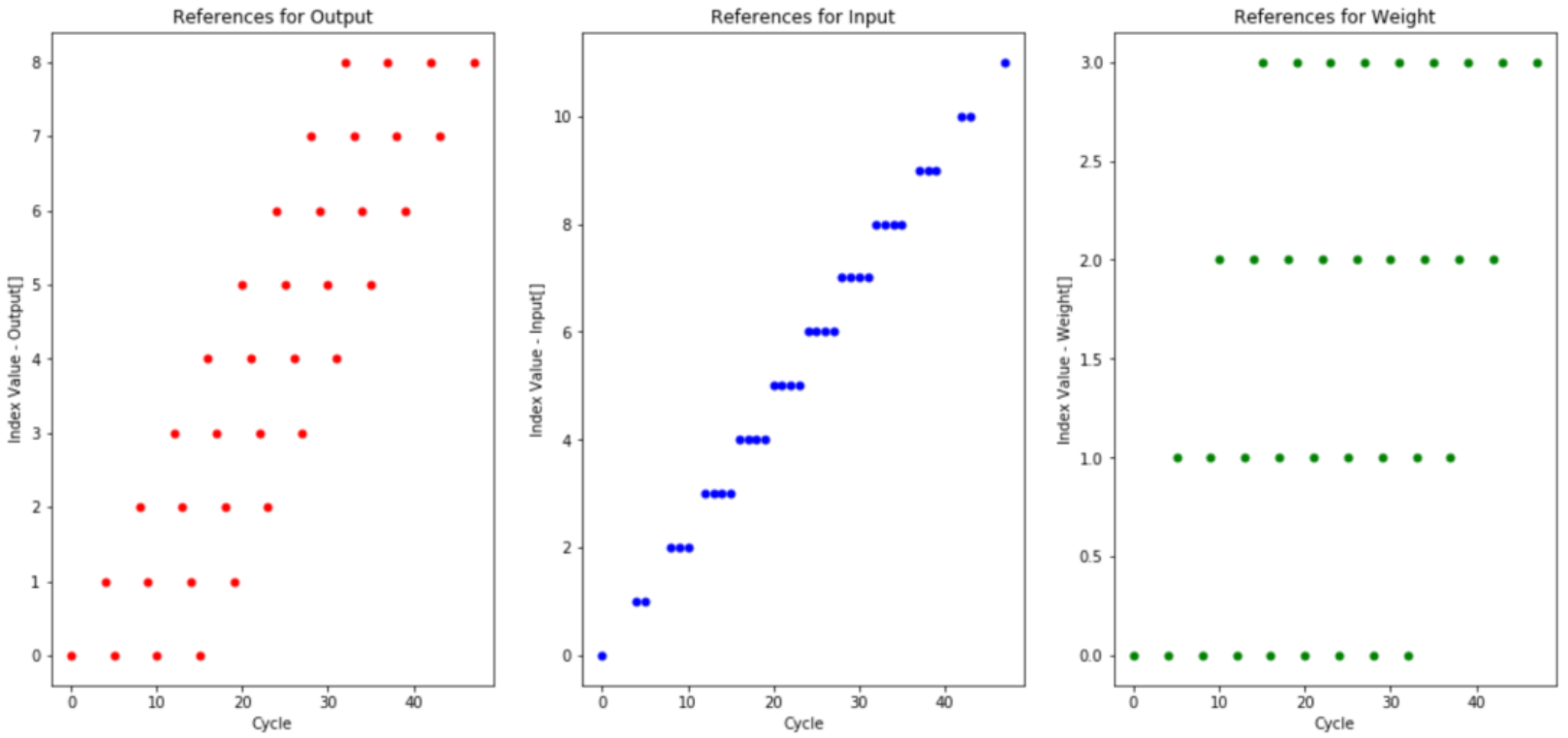
- Implement input stationary with no input index
- Beware w - r must be >= 0 and <E





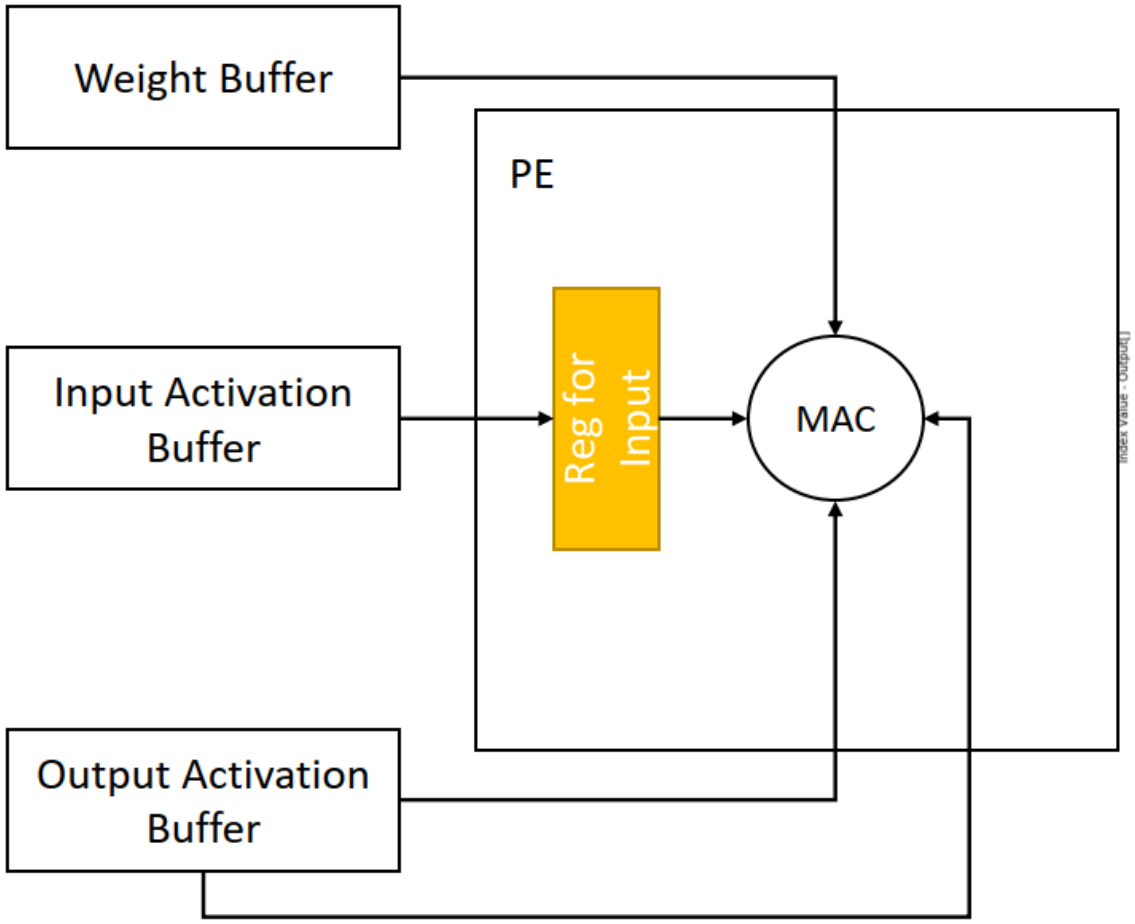


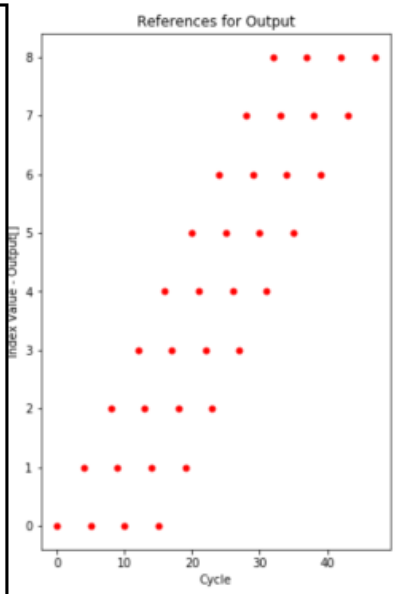
Input Stationary – Reference Pattern

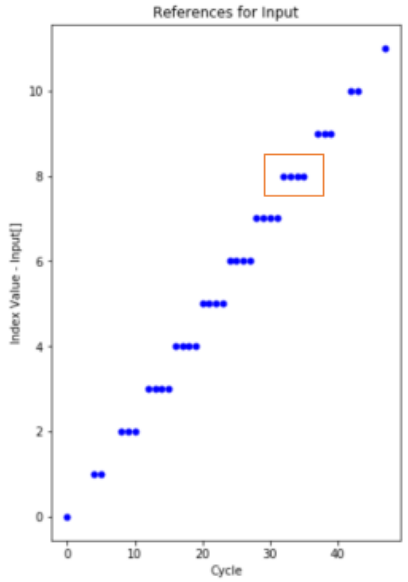


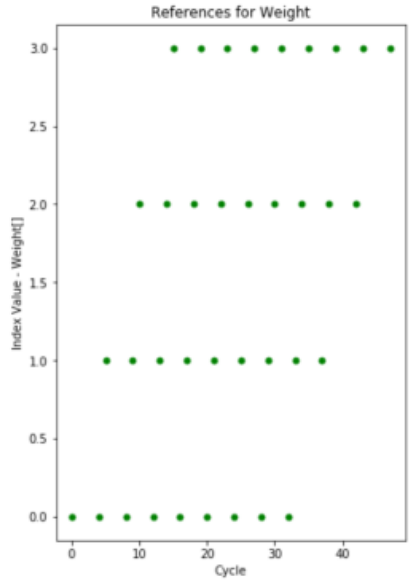
- Inputs used repeatedly (R times)
- Weights reused in large window (size = R)
- Sliding window of outputs (size = R)

Buffer Access Pattern (OS)









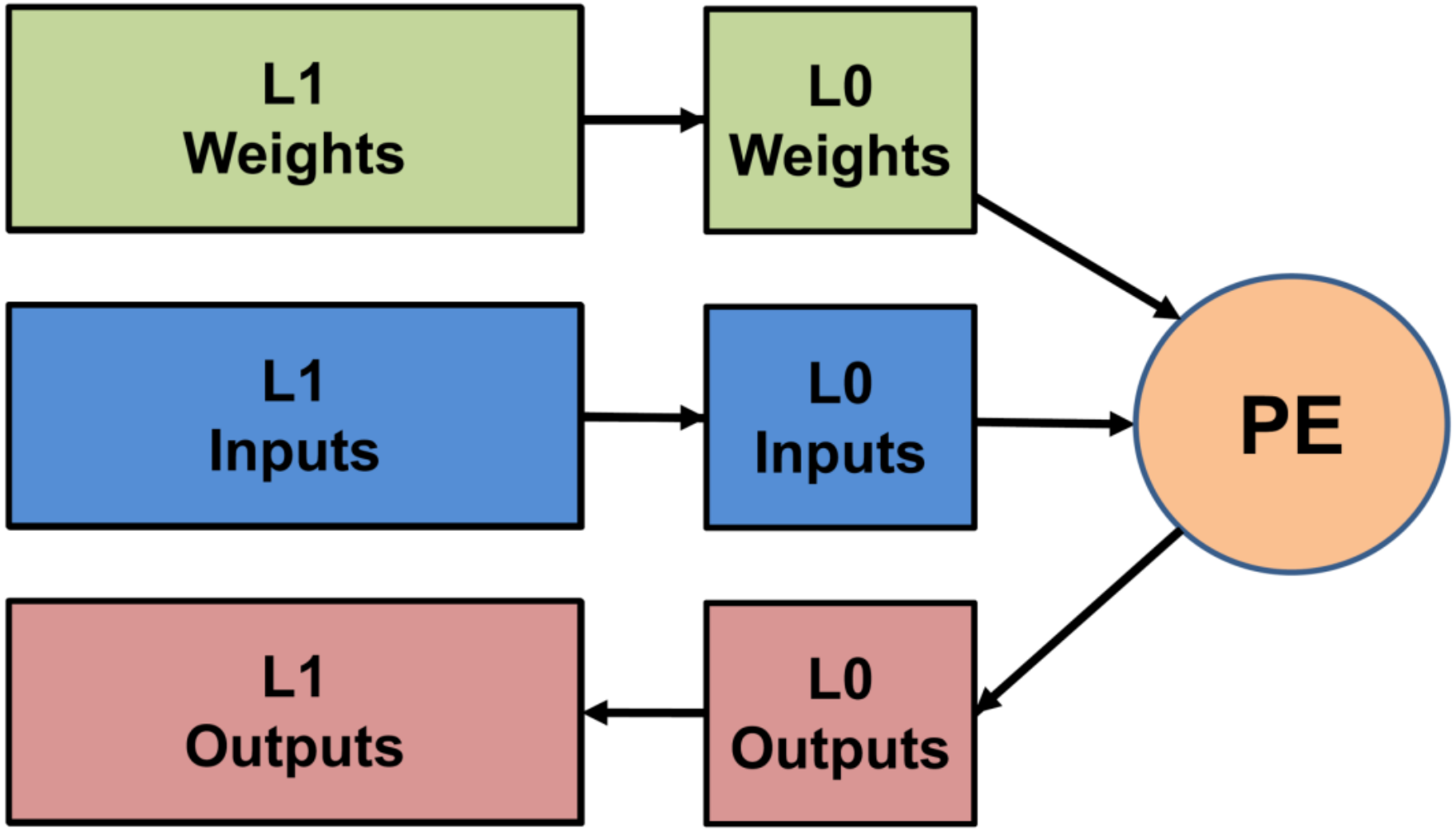
Minimum Costs

	OS	WS	IS	Min
MACs	E*R	E*R	E*R	E*R
Weight Reads	E*R	R	E*R	R
Input Reads	E*R	E*R	E	E
Output Reads	0	E*R	E*R	0
Output Writes	E	E*R	E*R	E

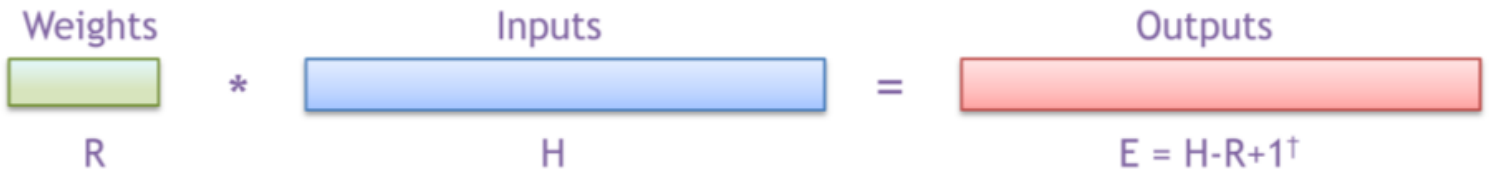
Assume: W ~= E



Intermediate Buffering



1-D Convolution – Buffered




```
int I[H]; // Input activations
int W[R]; // Filter Weights
int O[E]; // Output activations
// Level 1
for (e1 = 0; e1 < E1; e1++)
    for (r1 = 0; r1 < R1; r1++)
        // Level 0
            for (e0 = 0; e0 < E0; e0++)
                for (r0 = 0; r0 < R0; r0++)
                    O[e1*E0+e0] += I[e1*E0+e0 + r1*R0+r0] * W[r1*R0+r0];
```

Note E and R are factored so:

- **E0*E1 = E**
- **R0*R1 = R**

Buffer Sizes

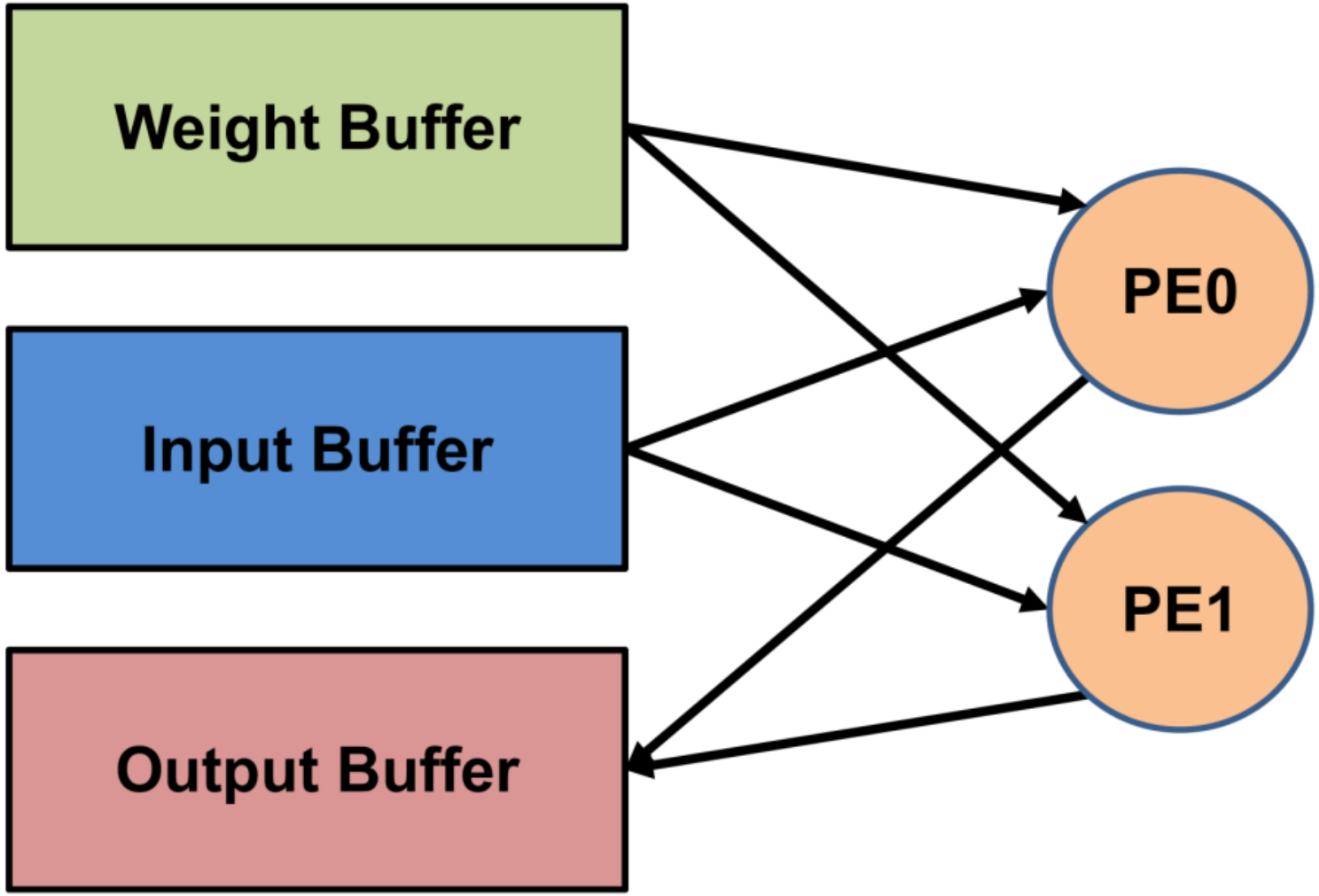
- Level 0 buffer size is volume needed in each Level 1 iteration
- Level 1 buffer size is volume needed to be preserved and redelivered in future (usually successive) Level 1 iterations

```
int I[H]; // Input activations
int W[R]; // Filter Weights
int O[E]; // Output activations
// Level 1
for (e1 = 0; e1 < E1; e1++)
    for (r1 = 0; r1 < R1; r1++)
        // Level 0
            for (e0 = 0; e0 < E0; e0++)
                for (r0 = 0; r0 < R0; r0++)
                    O[e1*E0+e0] += I[e1*E0+e0 + r1*R0+r0] * W[r1*R0+r0];
```

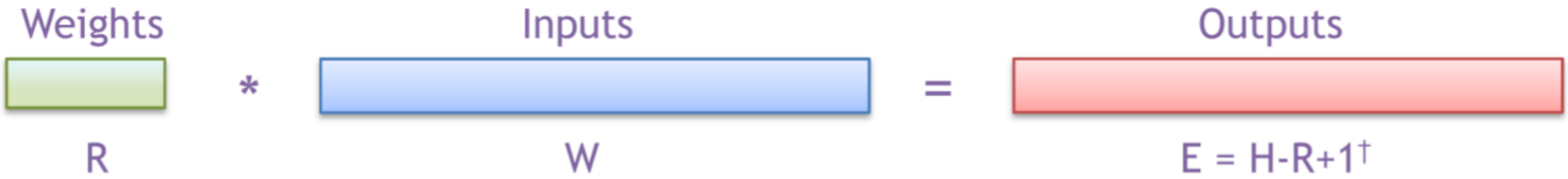
- A legal assignment of loop limits will fit into the hardware's buffer sizes



Spatial PEs



1-D Convolution – Spatial




```
int I[W]; // Input activations
int W[R]; // Filter Weights
int O[E]; // Output activations
// Level 1
for (r1 = 0; r1 < R1; r1++)
    for (e1 = 0; e1 < E1; e1++)
        // Level 0
            spatial_for (r0 = 0; r0 < R0; r0++)
                spatial_for (e0 = 0; e0 < E0; e0++)
                    O[e1*E0+e0] += I[e1*E0+e0 + r1*R0+r0] * W[r1*R0+r0];
```

Note:

- **E0*E1 = E**
- **R0*R1 = R**
- **R0*E0 <= #PEs**

Outline

- Benchmarking Metrics
- GEMM Accelerator Design
- **DNN Accelerator Design**
  - Locality and Data Reuse
  - **Dataflow Taxonomy**
  - Network-on-Chip
  - Optimization
- Energy

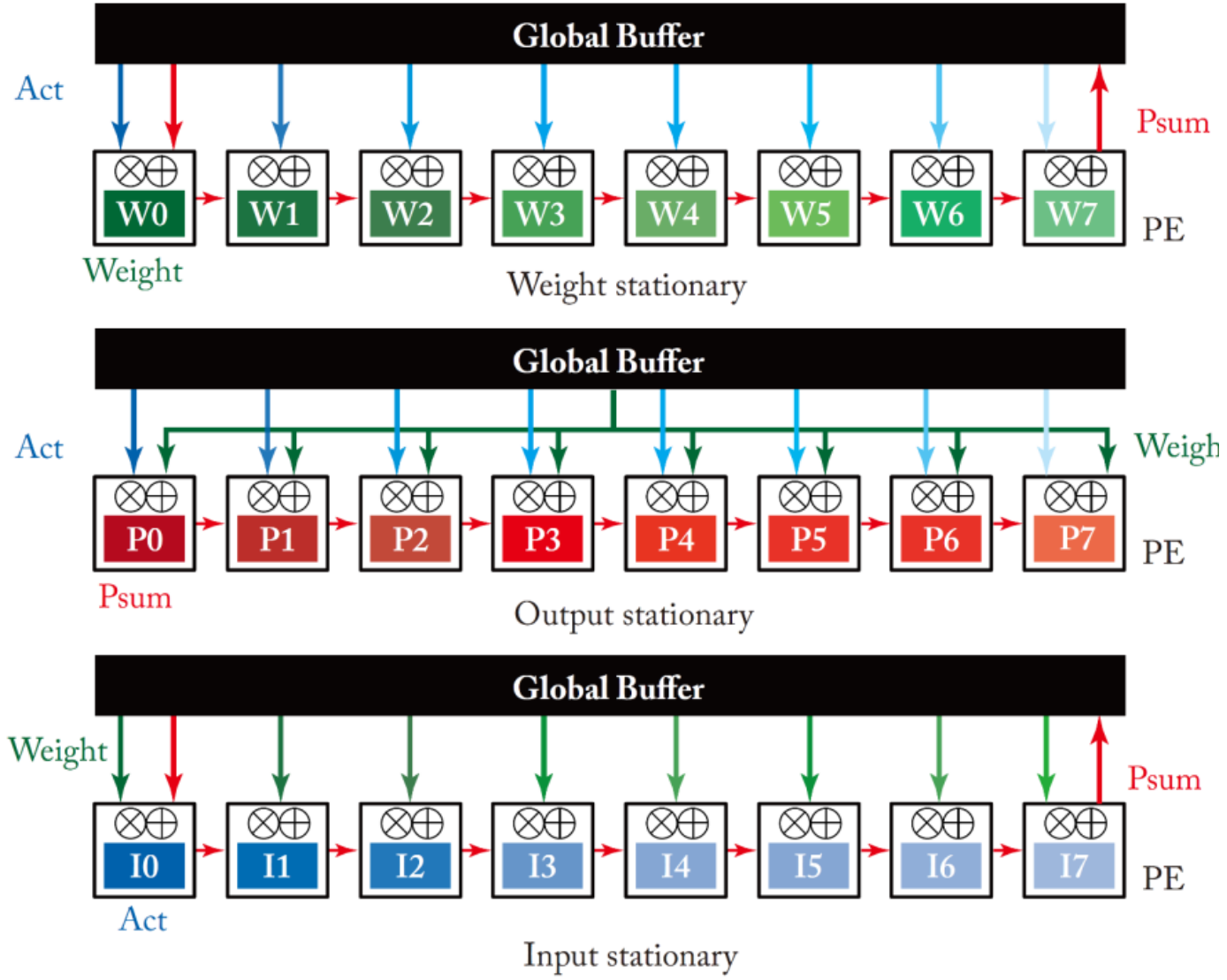


Dataflow Taxonomy

- Minimizing data movement is the key to achieving high energy efficiency for DNN accelerators
- Dataflow taxonomy:
  - Output Stationary: minimize movement of psums
  - Weight Stationary: minimize movement of weights
  - Input Stationary: minimize movement of inputs
- Loop nest provides a compact way to describe various properties of a dataflow, e.g., data tiling in multi-level storage and spatial processing.

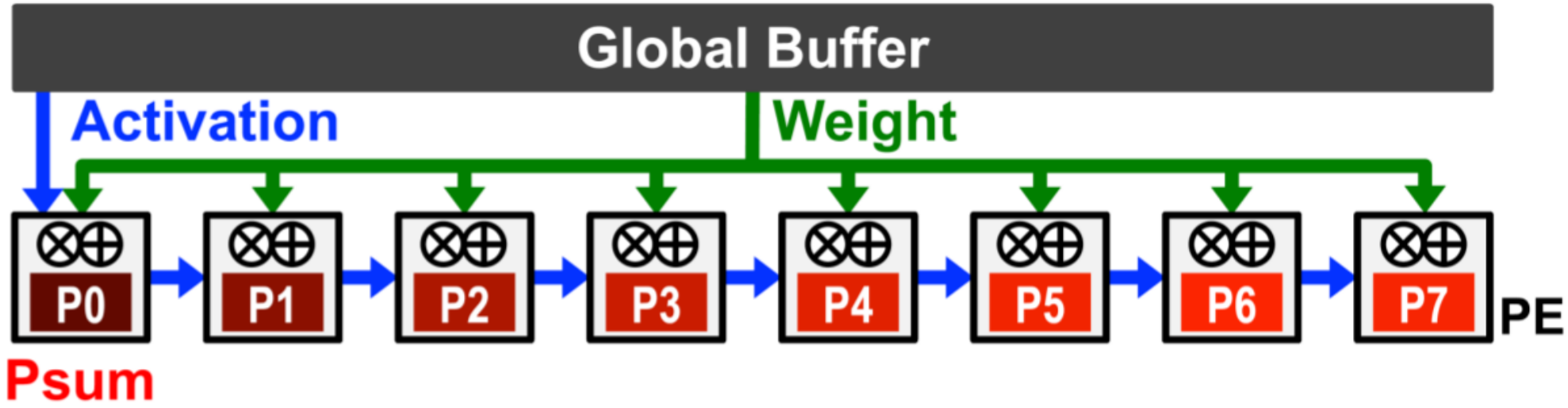


Dataflow Taxonomy

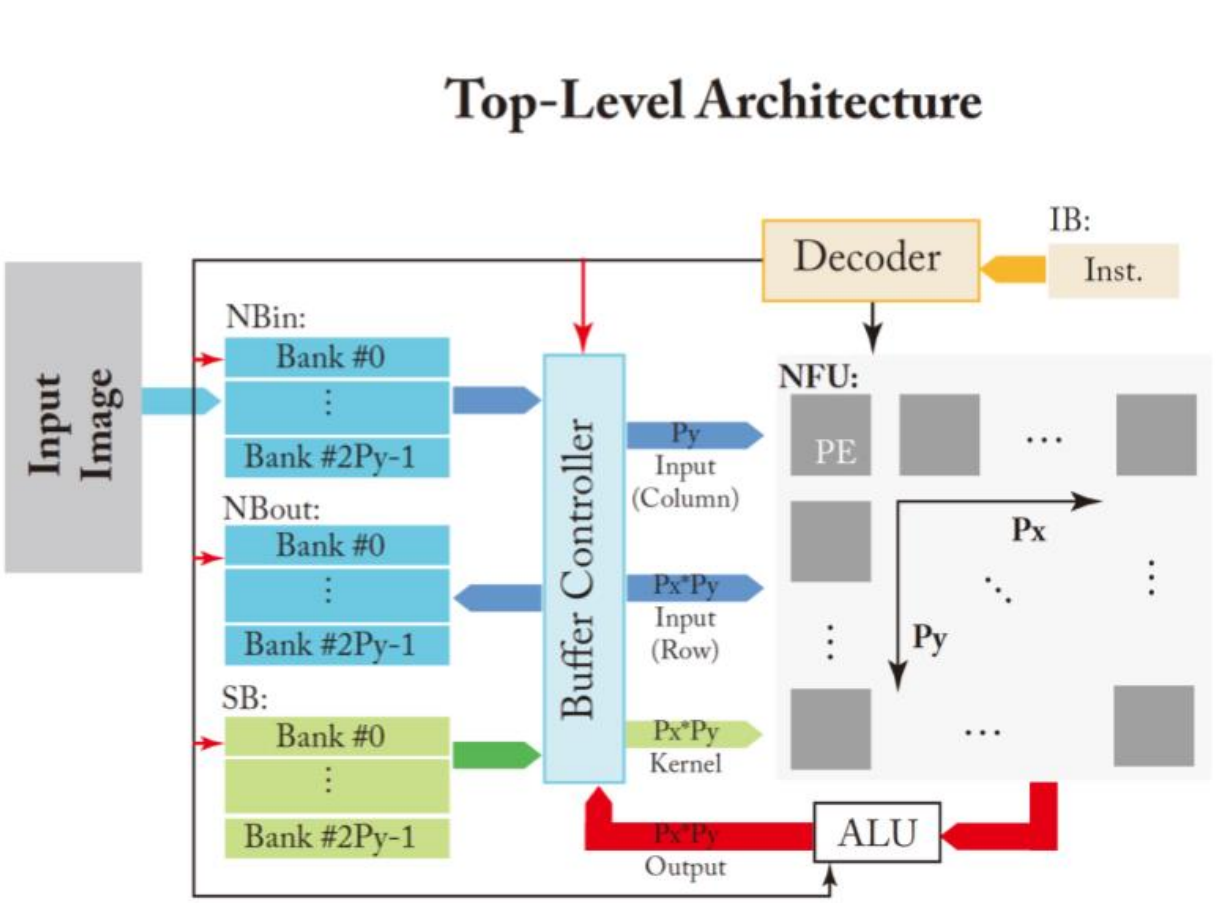


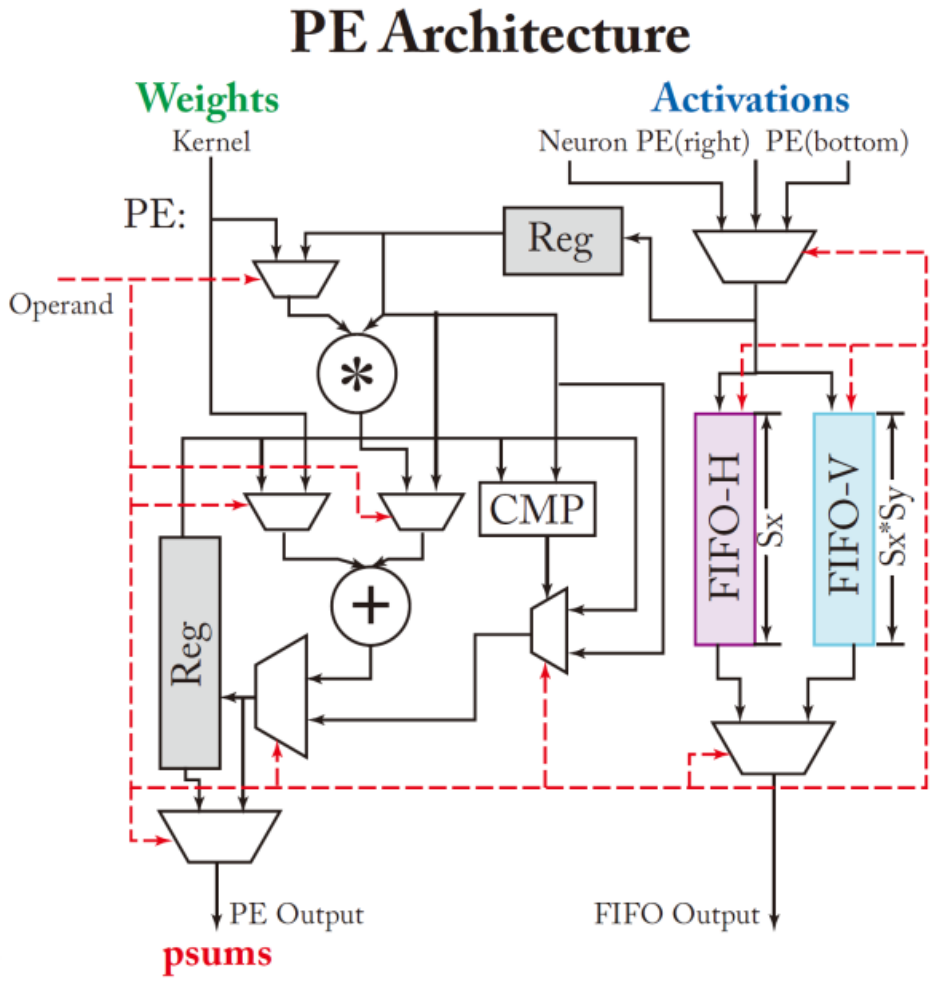
Output Stationary (OS)

- Minimize partial sum R/W energy consumption
  - Maximize local accumulation
- Broadcast/Multicast filter weights and reuse
  - Activations spatially across the PE array

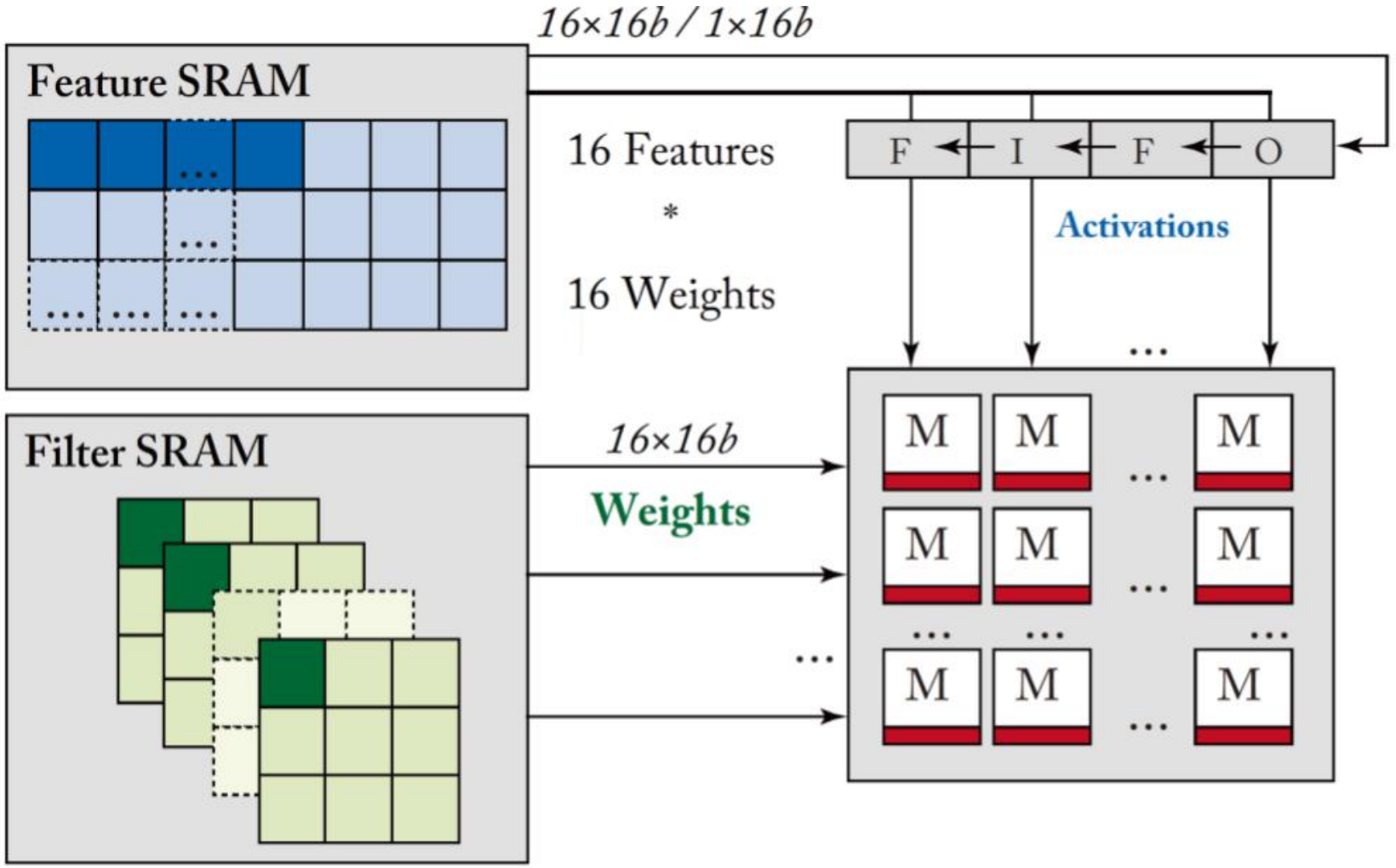


OS Example: ShiDianNao

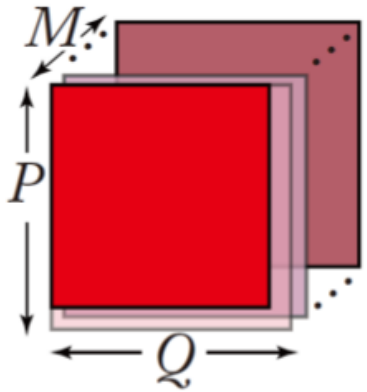
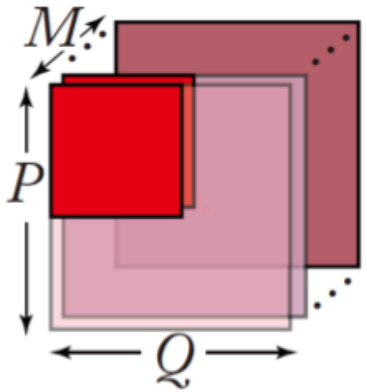
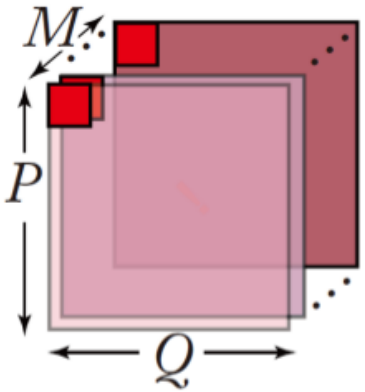




OS Example: ENVISION

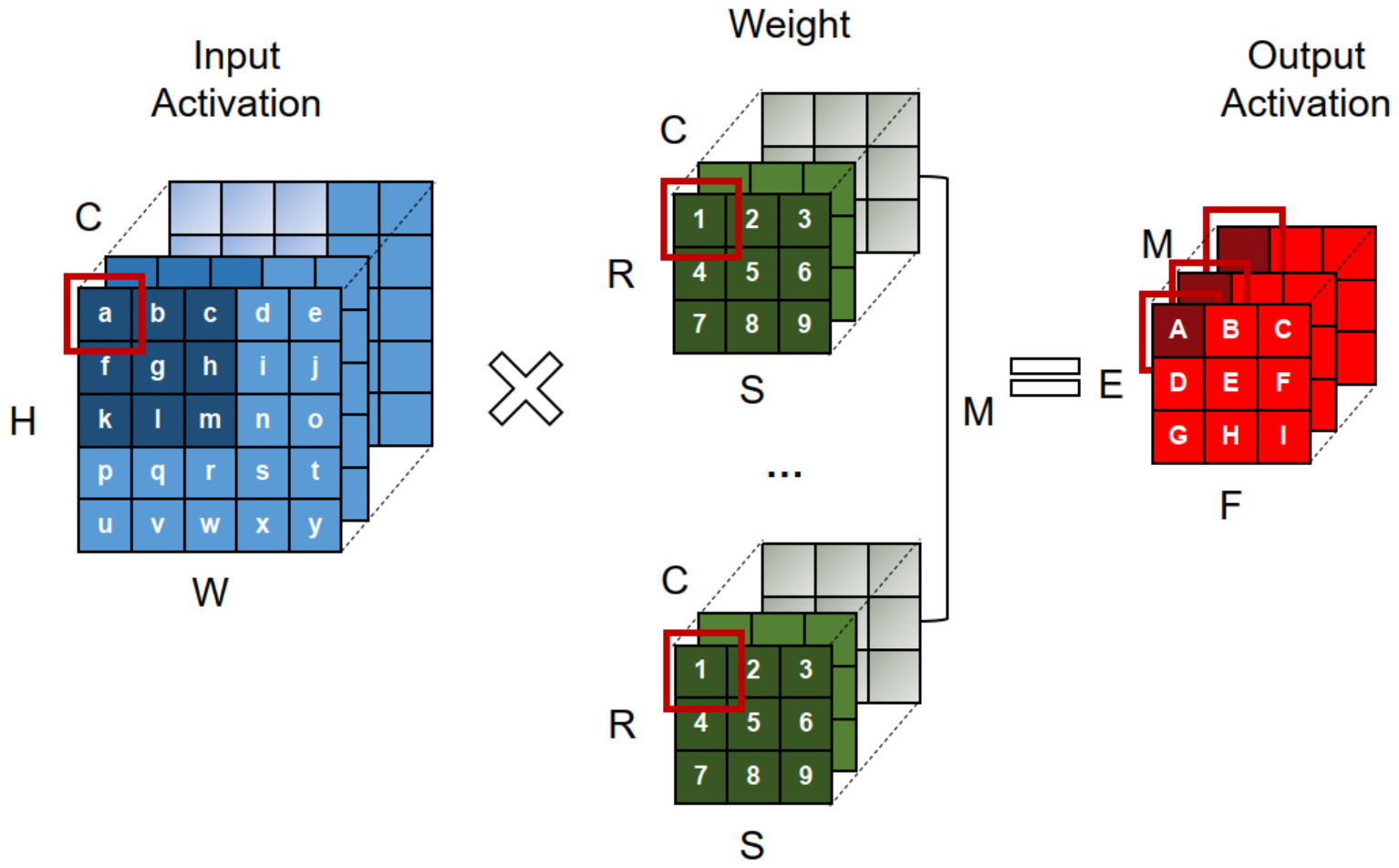


Variations of Output Stationary

	OS_A	OS_B	OS_C
Parallel Output Region			
# Output Channels # Output Activations	Single Multiple	Multiple Multiple	Multiple Single
Notes	Targeting **CONV** Layers		Targeting **FC** Layers

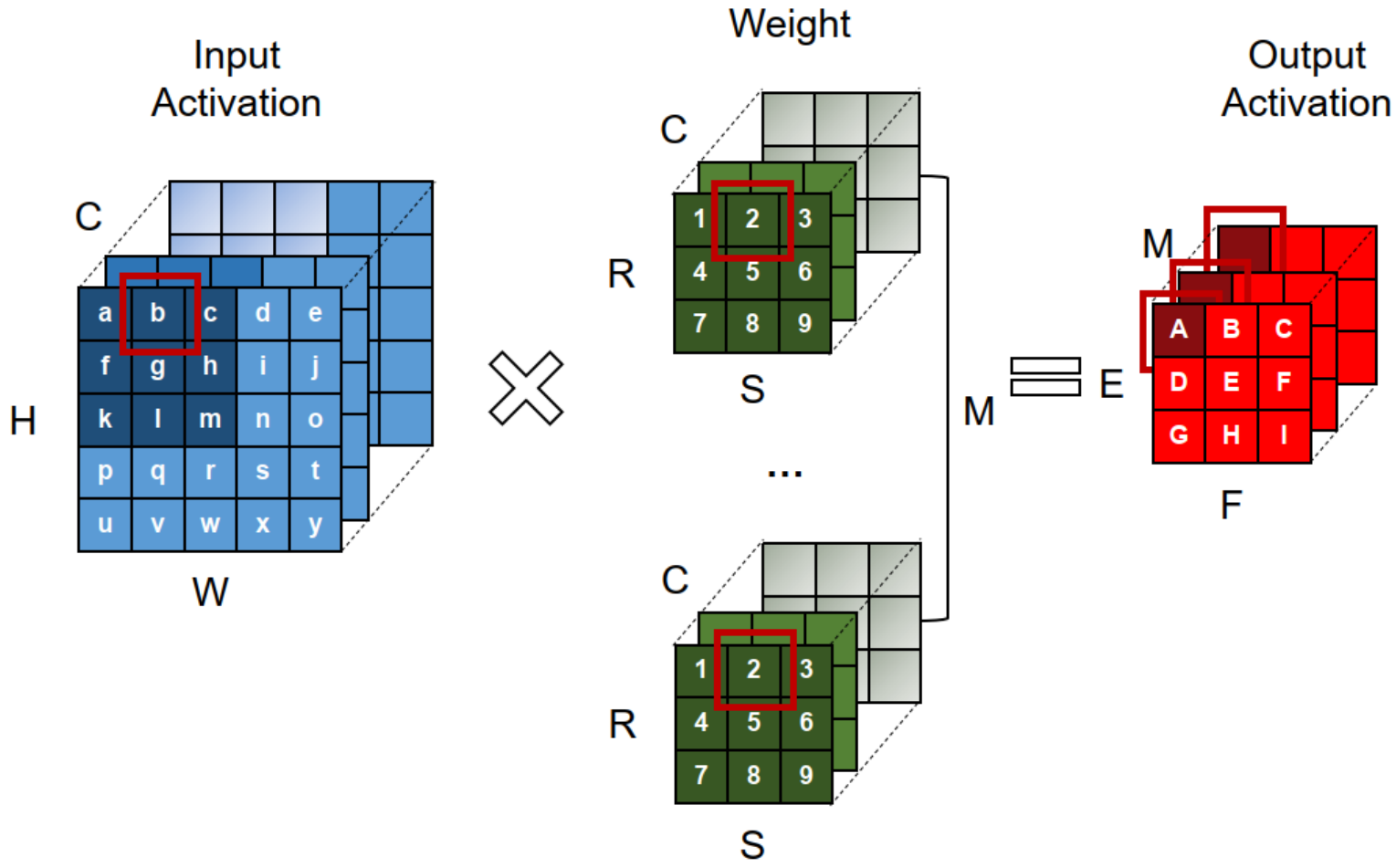
Output Stationary Dataflow

- Cycle through input fmap and weights (hold psum of output fmap)



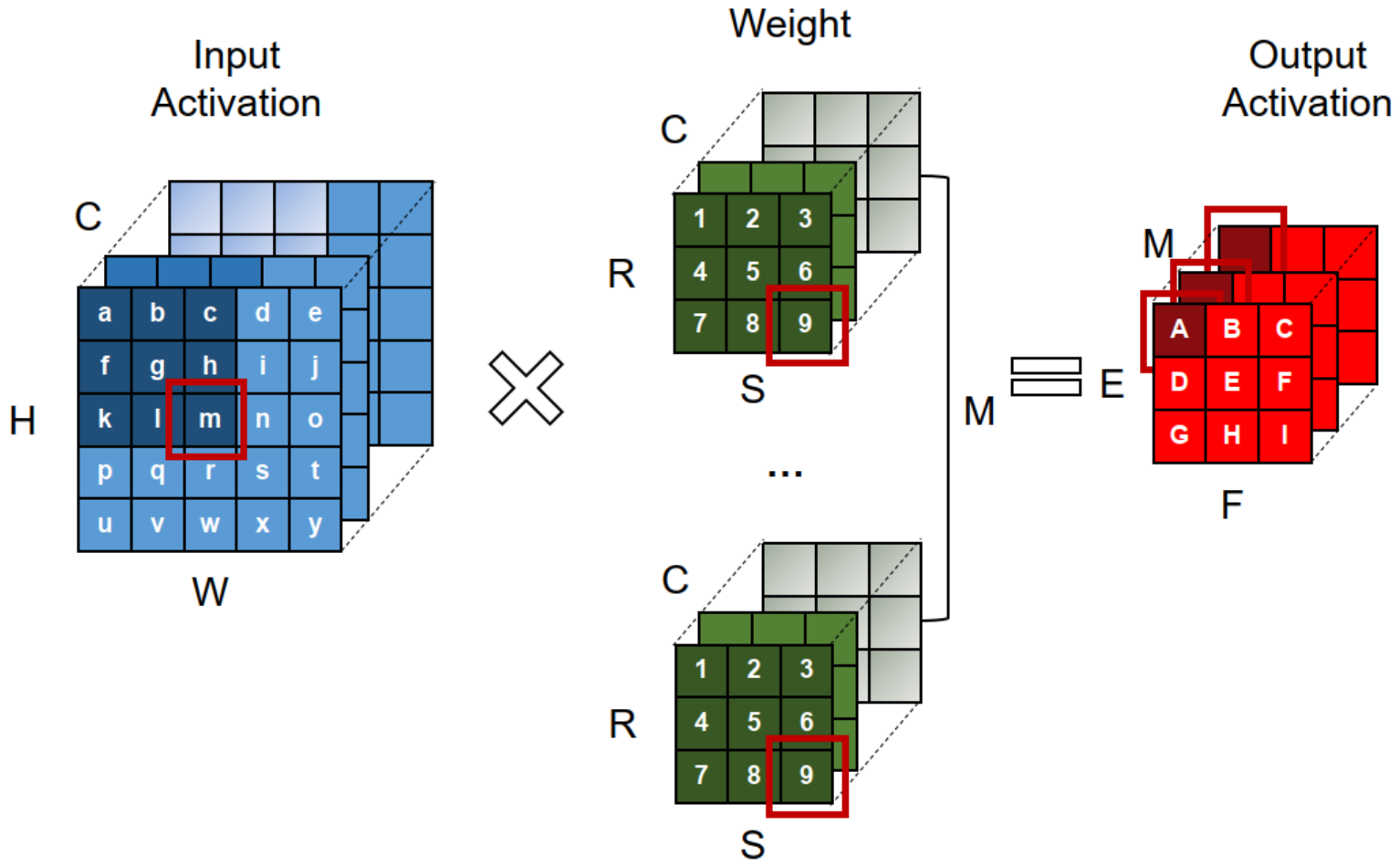
Output Stationary Dataflow

- Cycle through input fmap and weights (hold psum of output fmap)



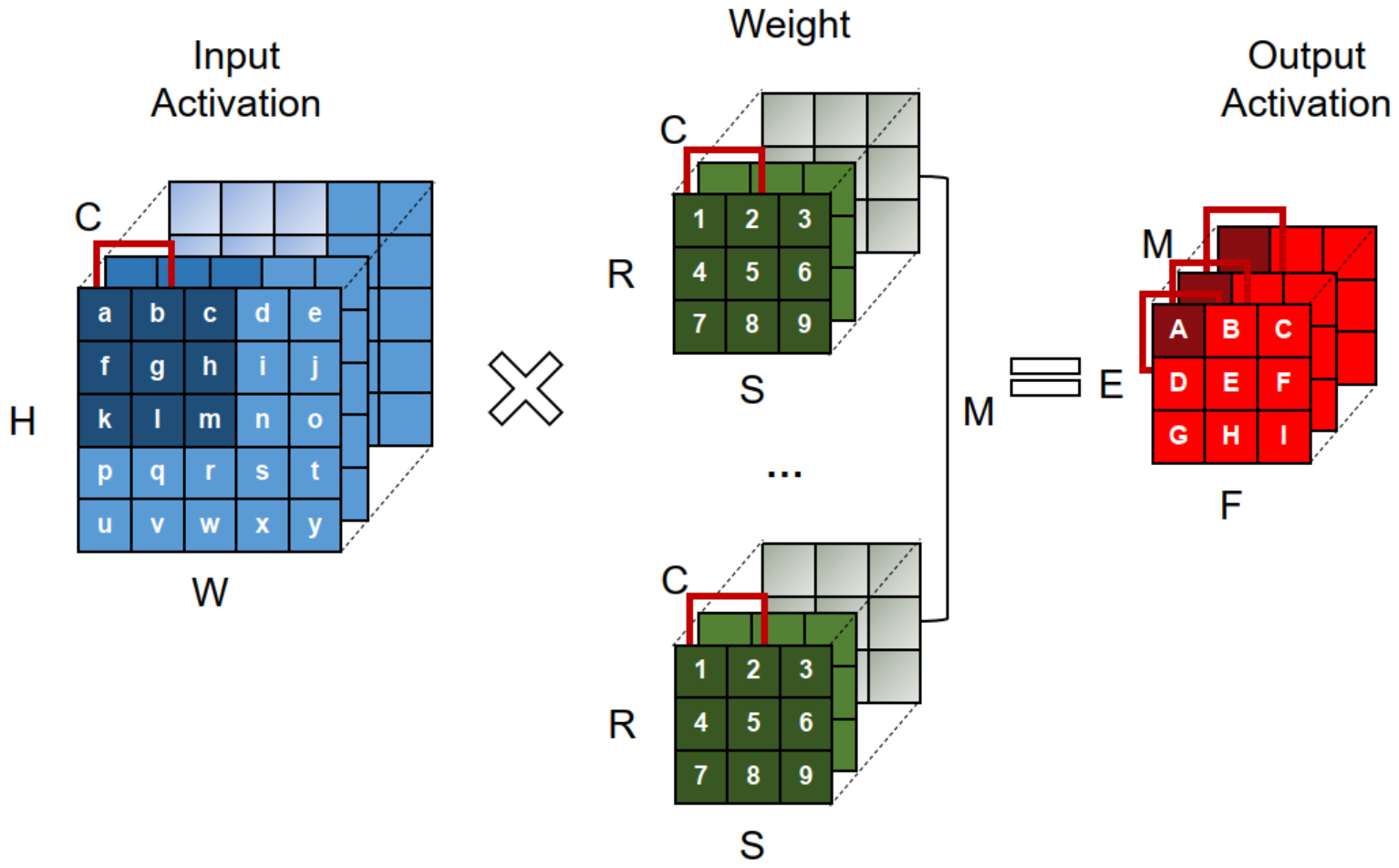
Output Stationary Dataflow

- Cycle through input fmap and weights (hold psum of output fmap)



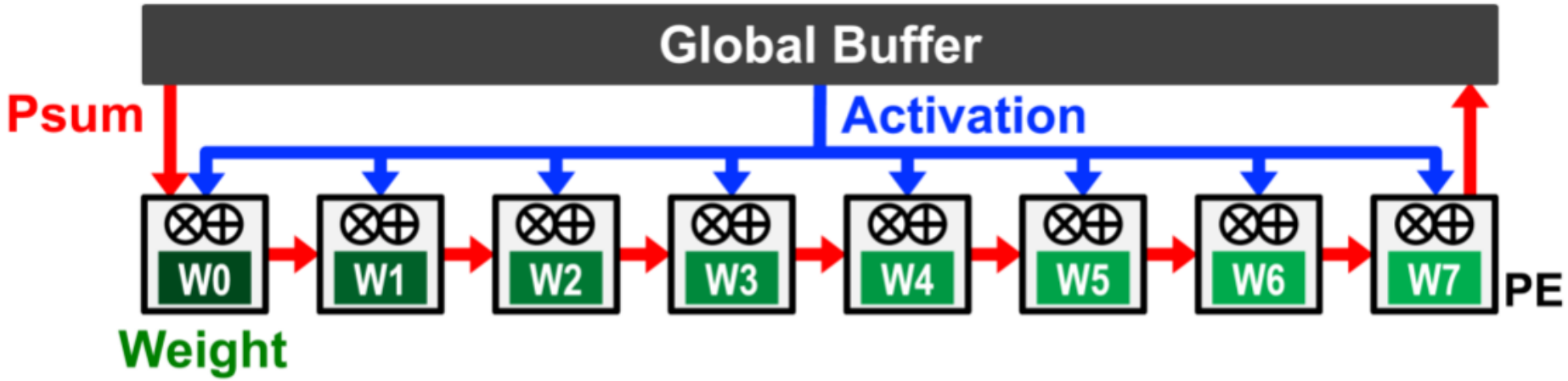
Output Stationary Dataflow

- Cycle through input fmap and weights (hold psum of output fmap)

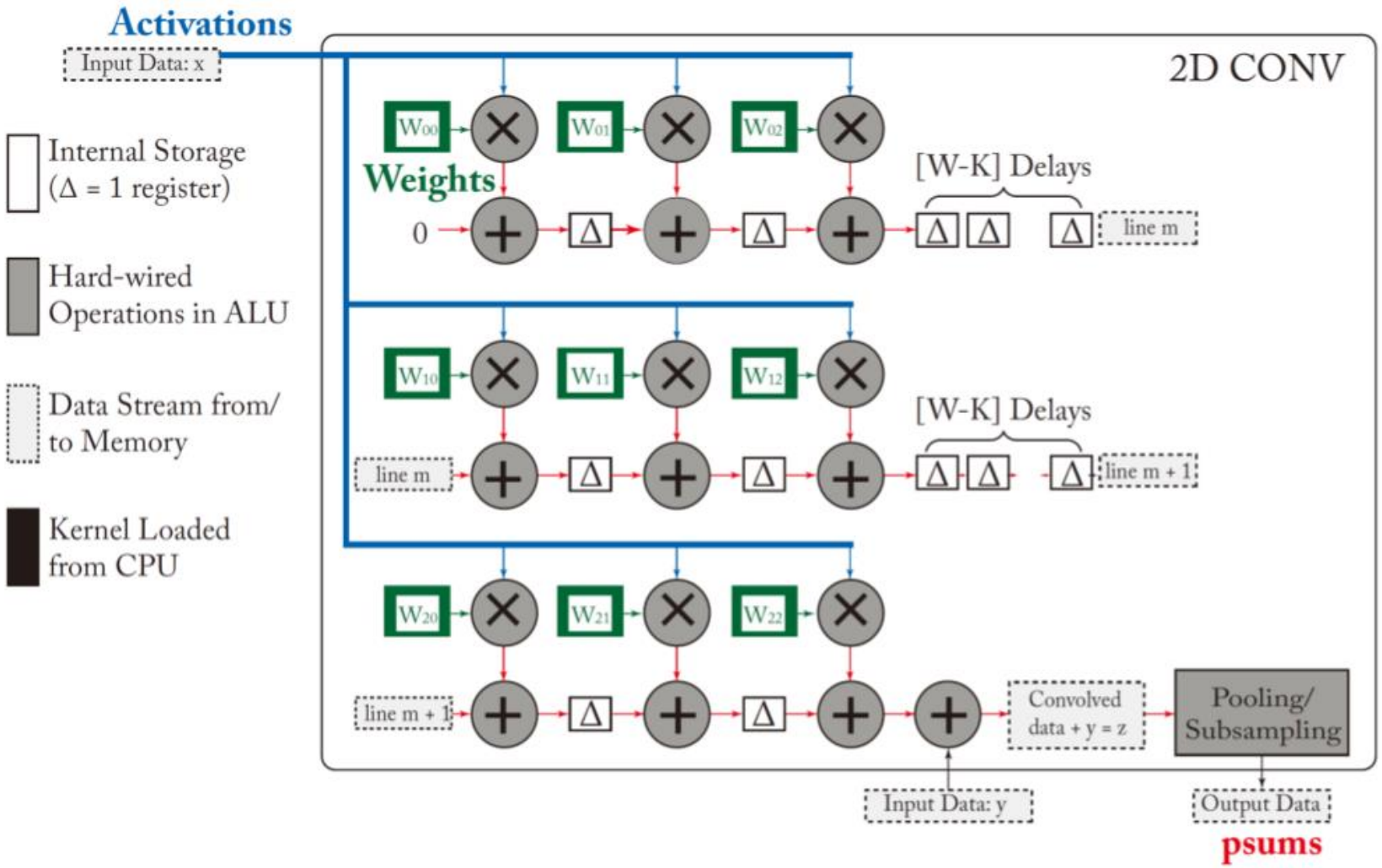


Weight Stationary (WS)

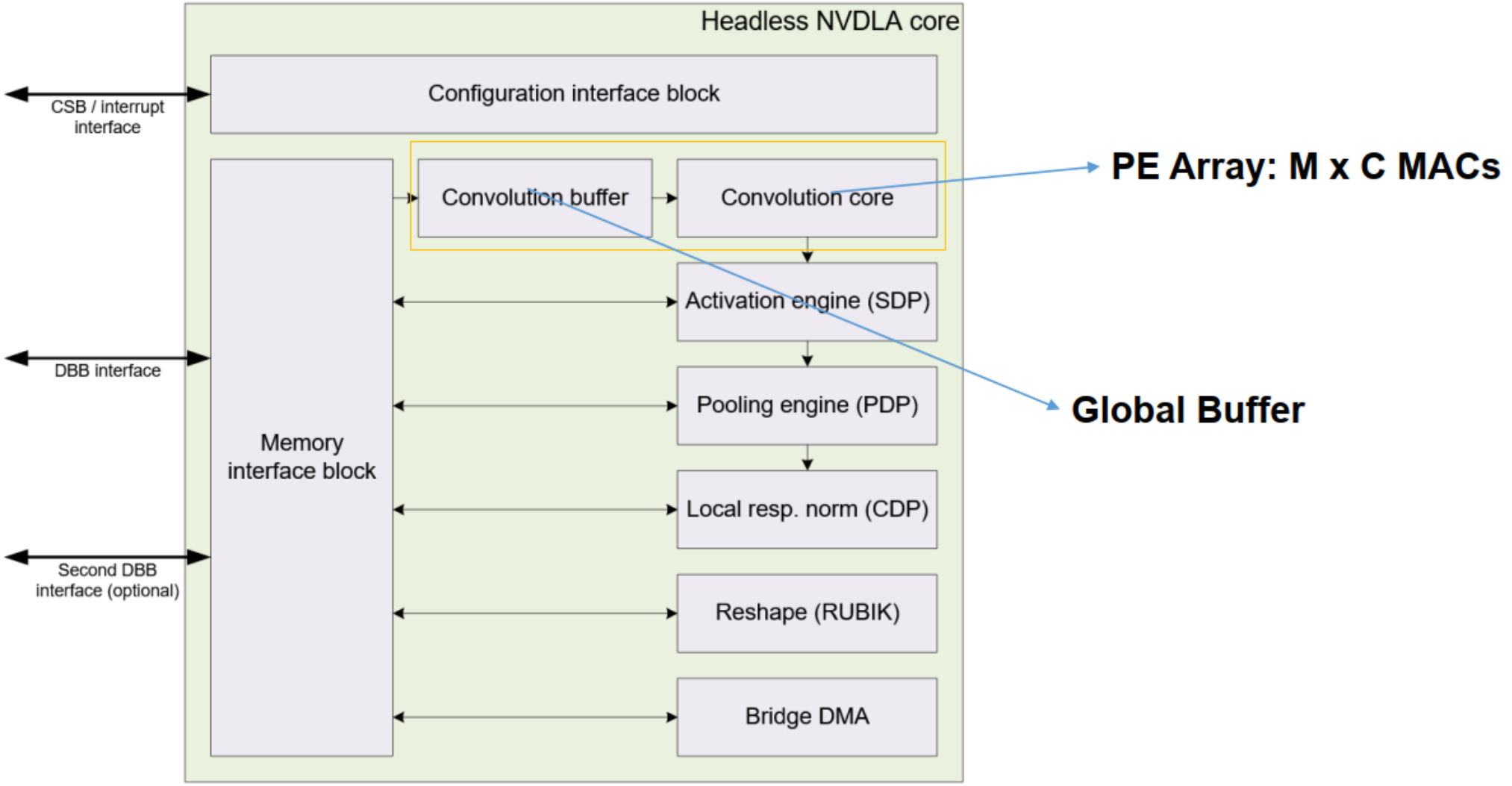
- Minimize weight read energy consumption
  - Maximize convolutional and filter reuse of weights
- Broadcast activations and accumulate psums
  - Spatially across the PE array.



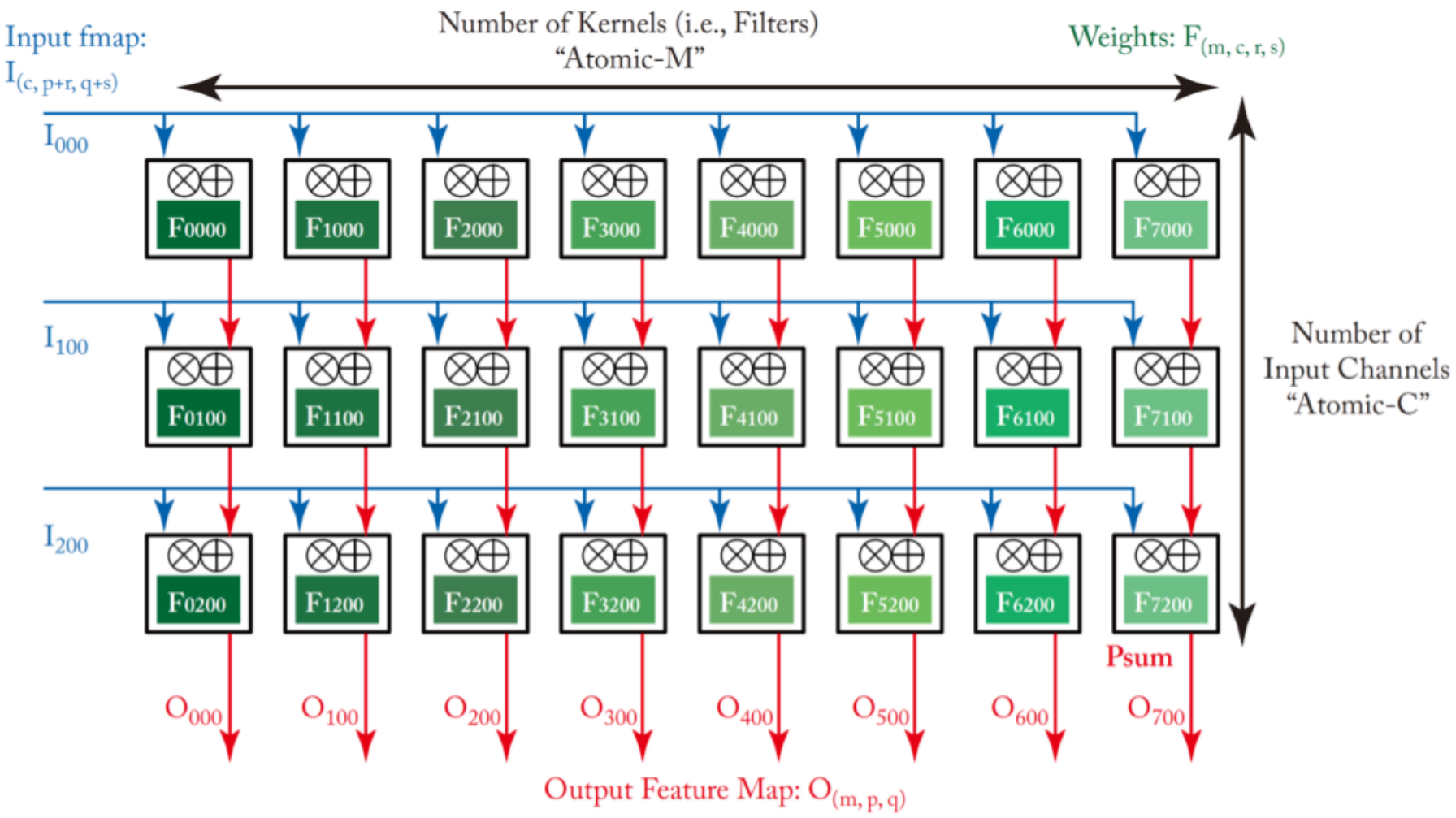
WS Example: nn-X (NeuFlow)



WS Example: NVDLA (Simplified)



WS Example: NVDLA (Simplified)



Loop Nest For Simplified NVDLA

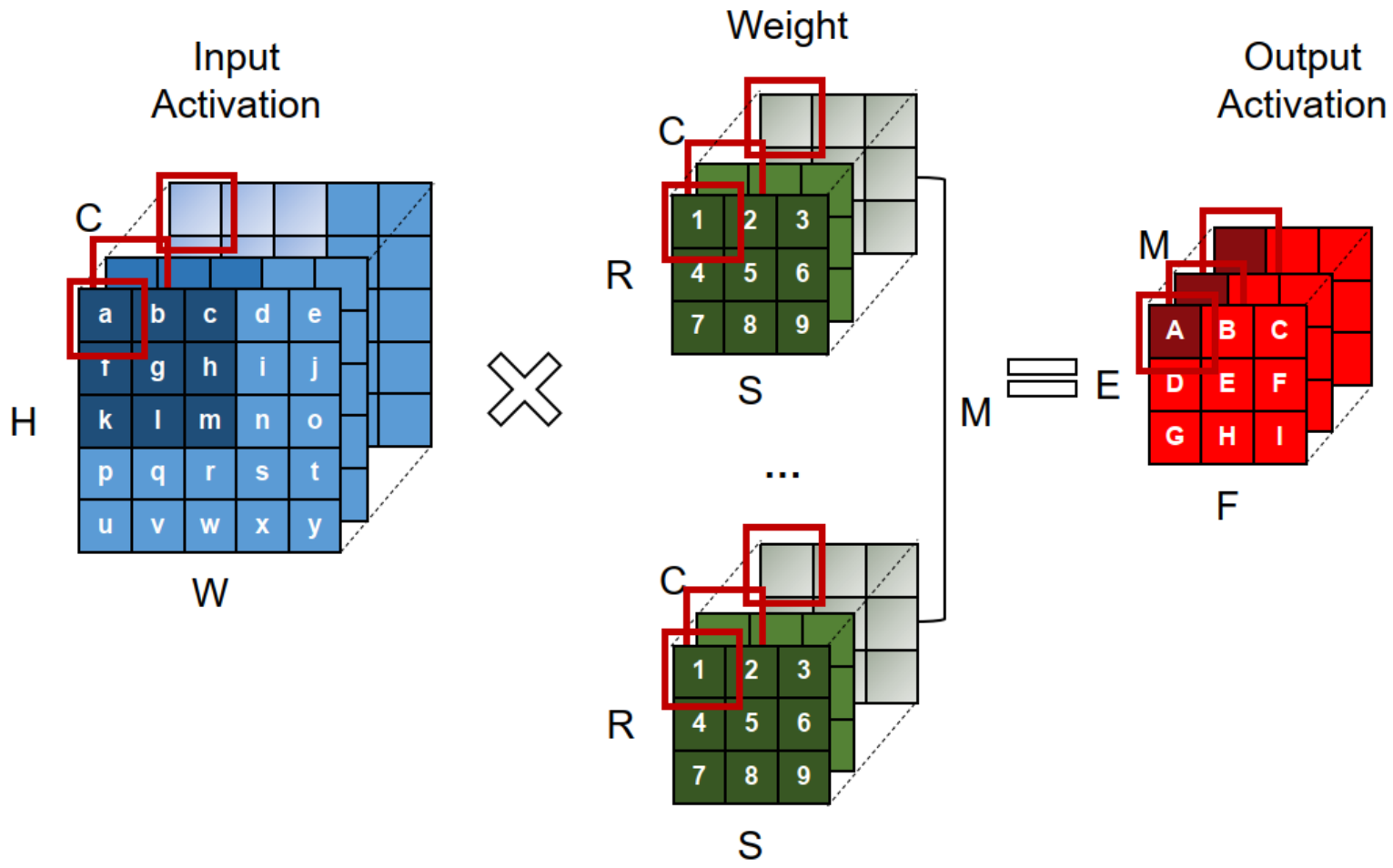
```
# i[C,H,W]     -  Input activations
# f[M,C,R,S]   -  Filter weights
# o[M,P,Q];    -  Output activations

parallel-for m in range(M):
  parallel-for c in range(C):
    for r in range(R):
      for s in range(S):
        for p in range(P):
          for q in range(Q):
            o[m,p,q] += i[c,p+r,q+s] * f[m,c,r,s]
```



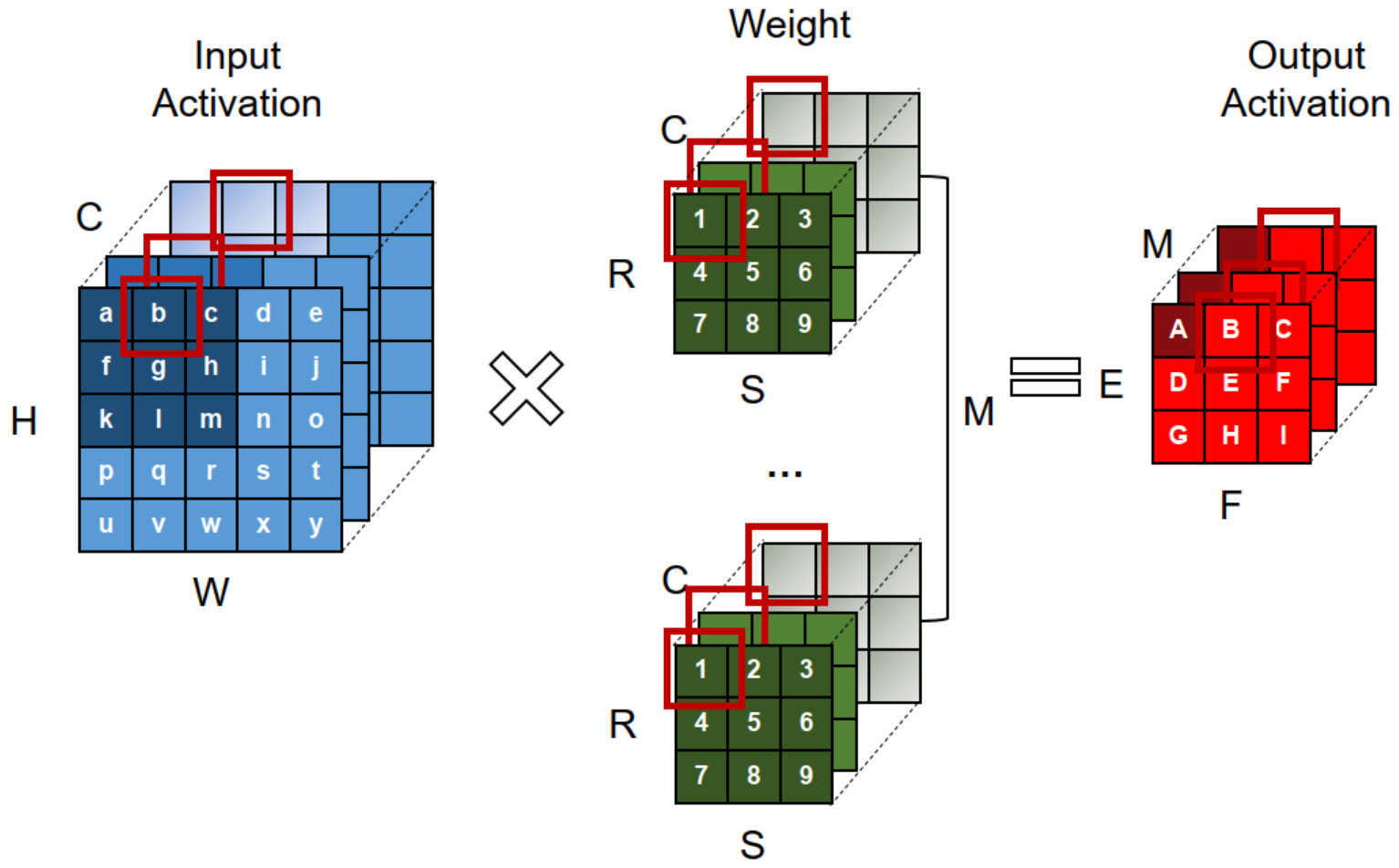
Weight Stationary Dataflow

- Cycle through input and output fmap (hold weights)



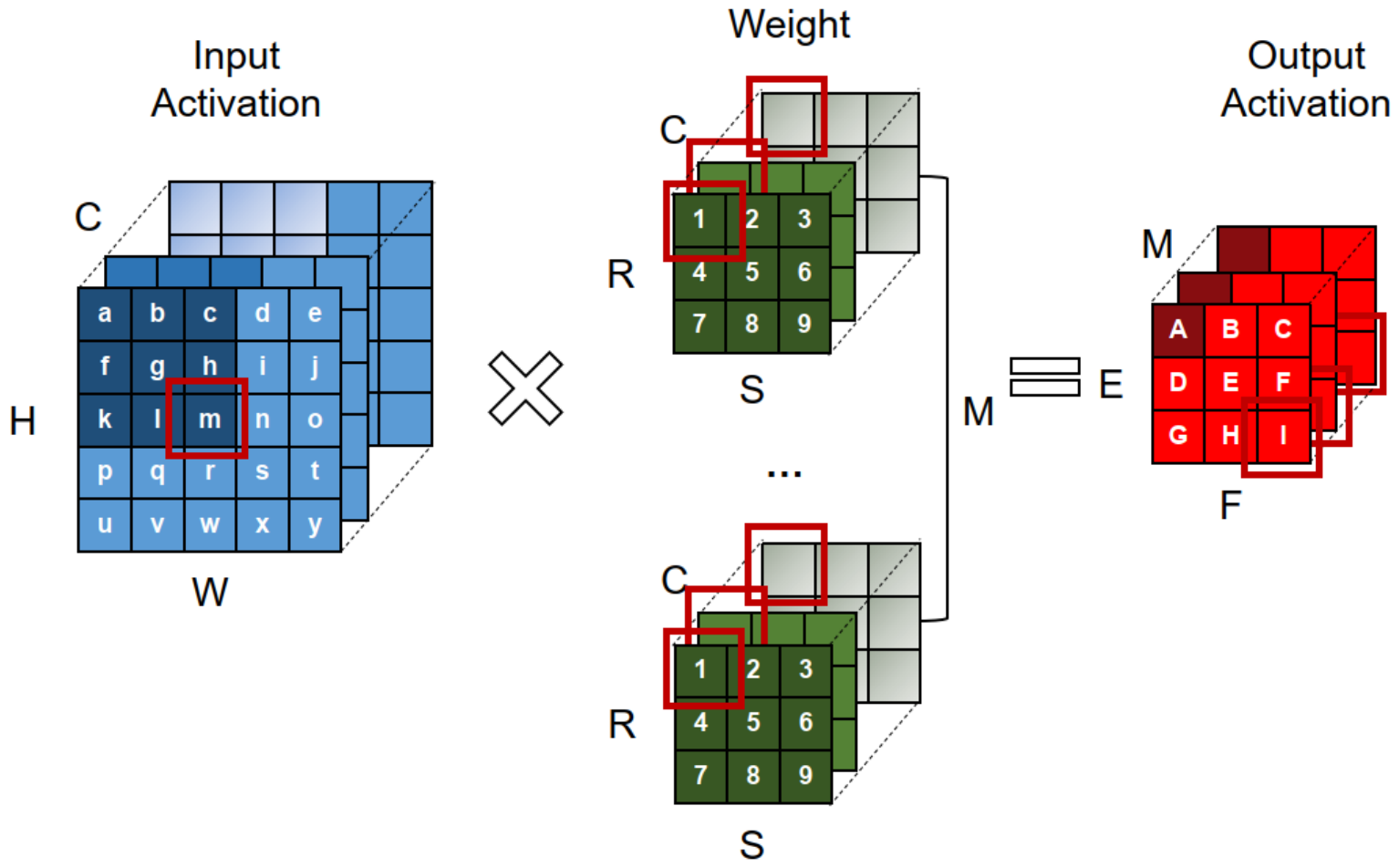
Weight Stationary Dataflow

- Cycle through input and output fmap (hold weights)



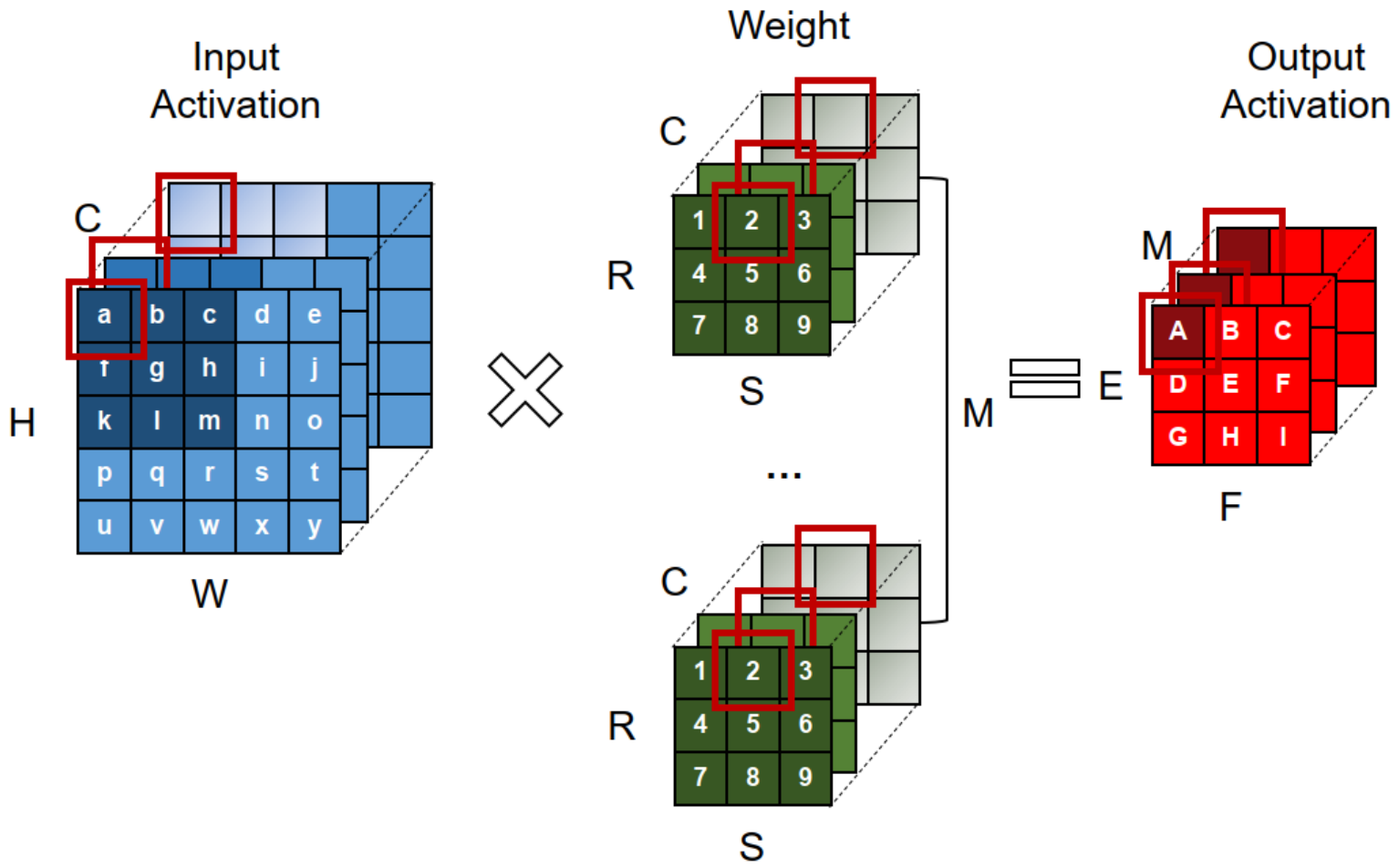
Weight Stationary Dataflow

- Cycle through input and output fmap (hold weights)



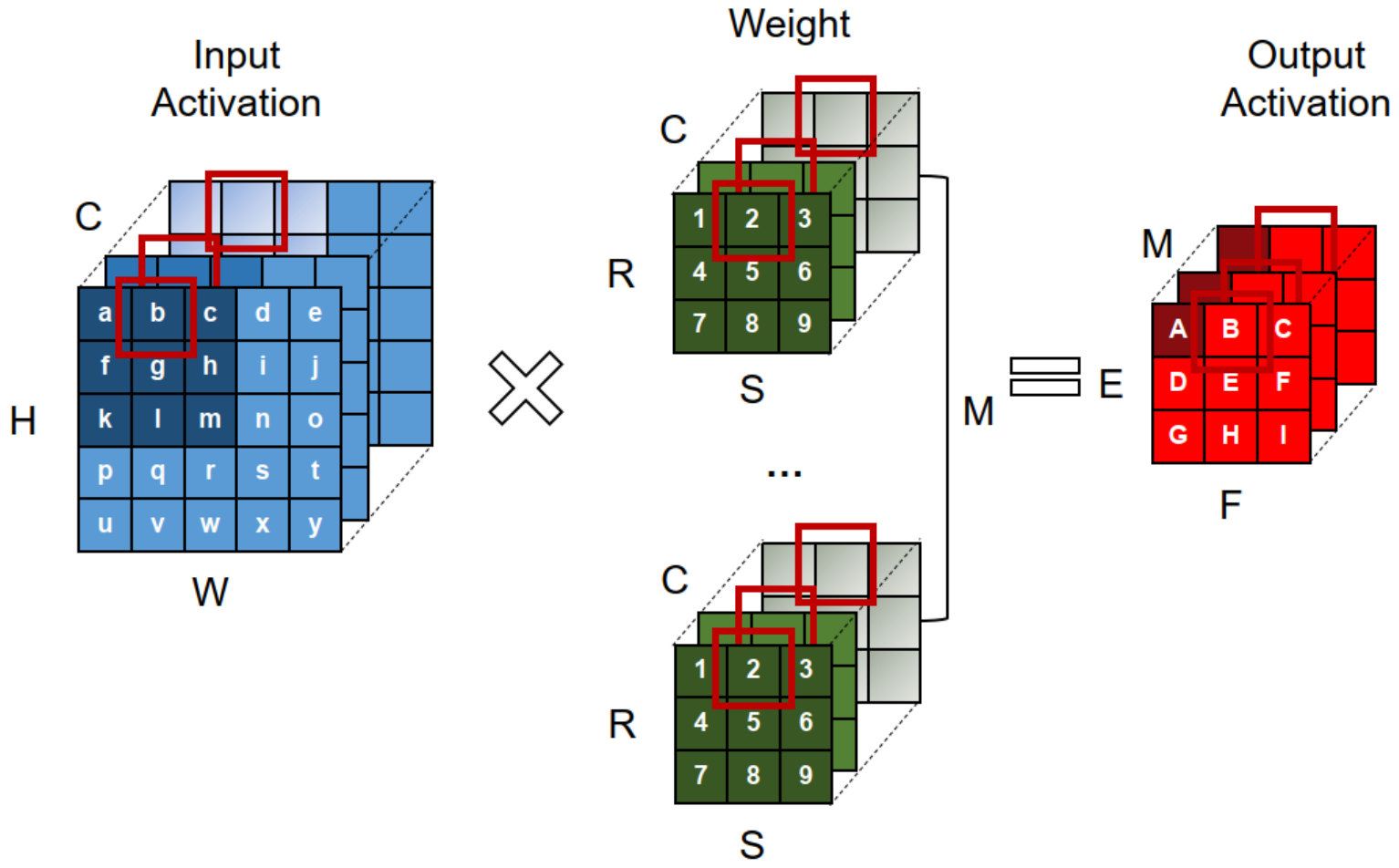
Weight Stationary Dataflow

- **Load new weights**



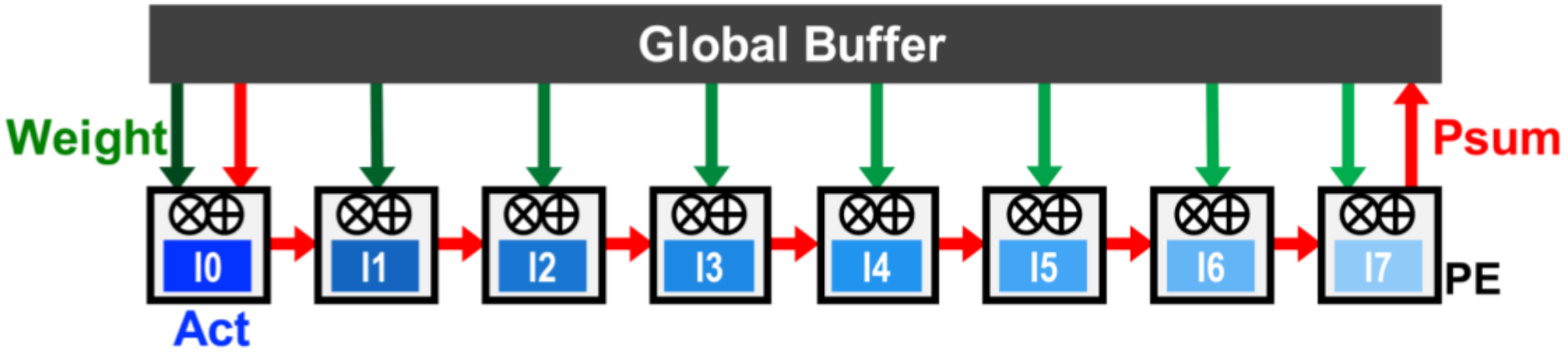
Weight Stationary Dataflow

- Cycle through input and output fmap (hold weights)



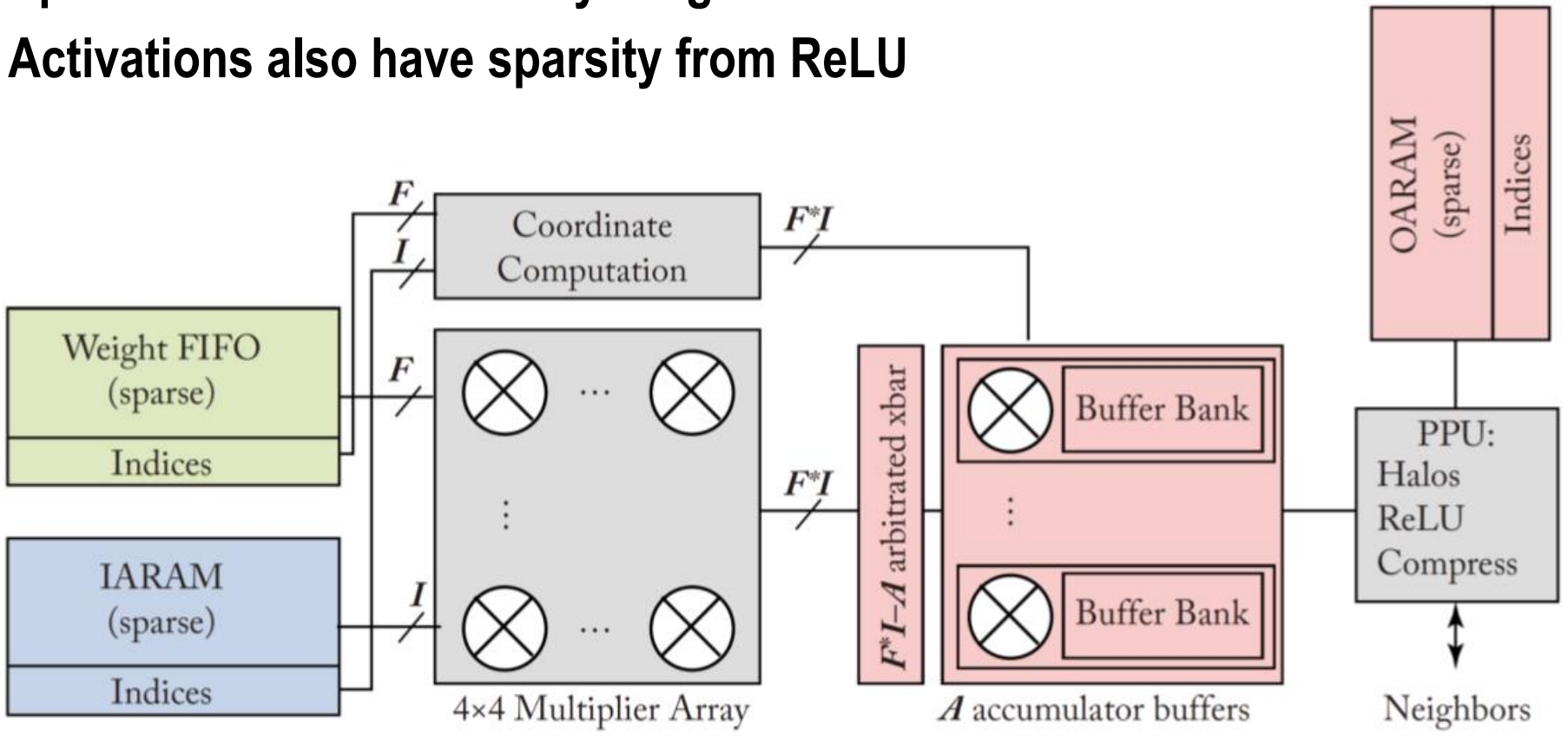
Input Stationary (IS)

- Minimize activation read energy consumption
  - Maximize convolutional and fmap reuse of activations
- Unicast weights and accumulate psums spatially
  - Across the PE array.
- Usually for sparse CNNs



IS Example: SCNN

- Used for sparse CNNs
  - Sparse CNN is where many weights are zeros
  - Activations also have sparsity from ReLU

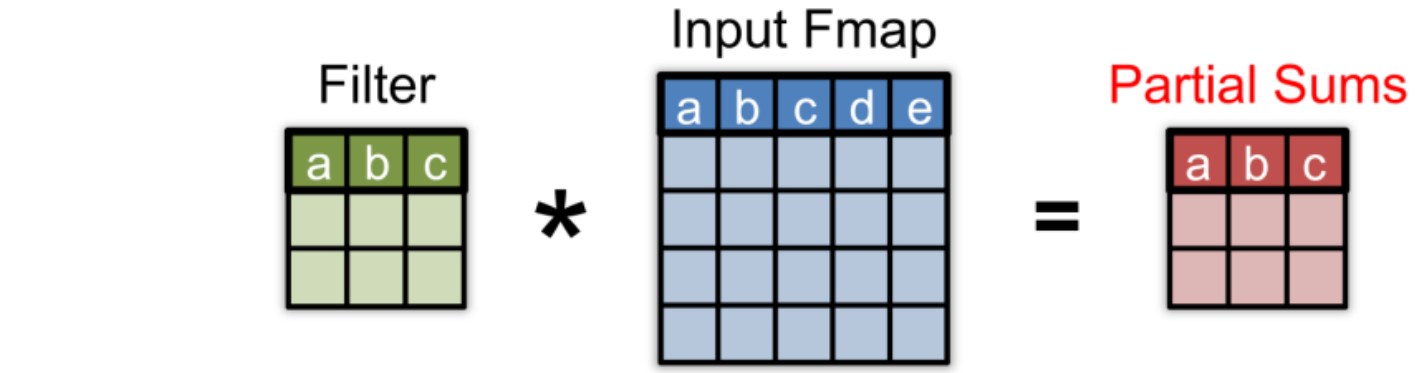


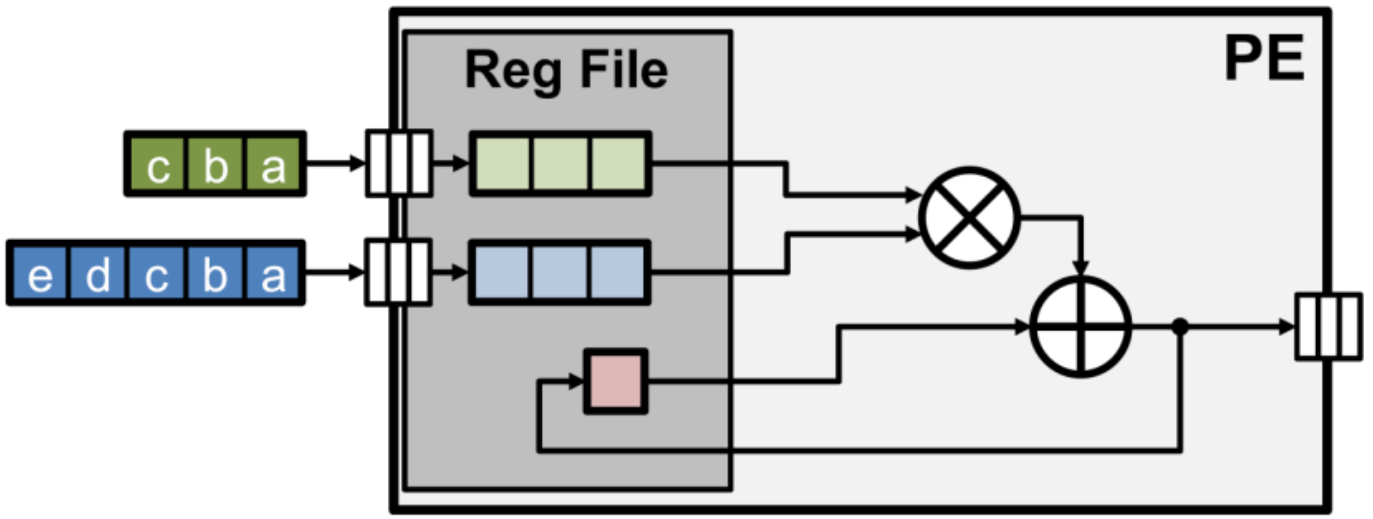
Eyeriss - Row Stationary (RW)

- **Maximize data reuse at Register File**
  - **Maximize row convolutional reuse in Register File**
    - Keep a filter row and fmap sliding window in Register File
  - **Maximize row psum accumulation in Register File**
- **Optimize for overall energy efficiency instead for only a certain data type**

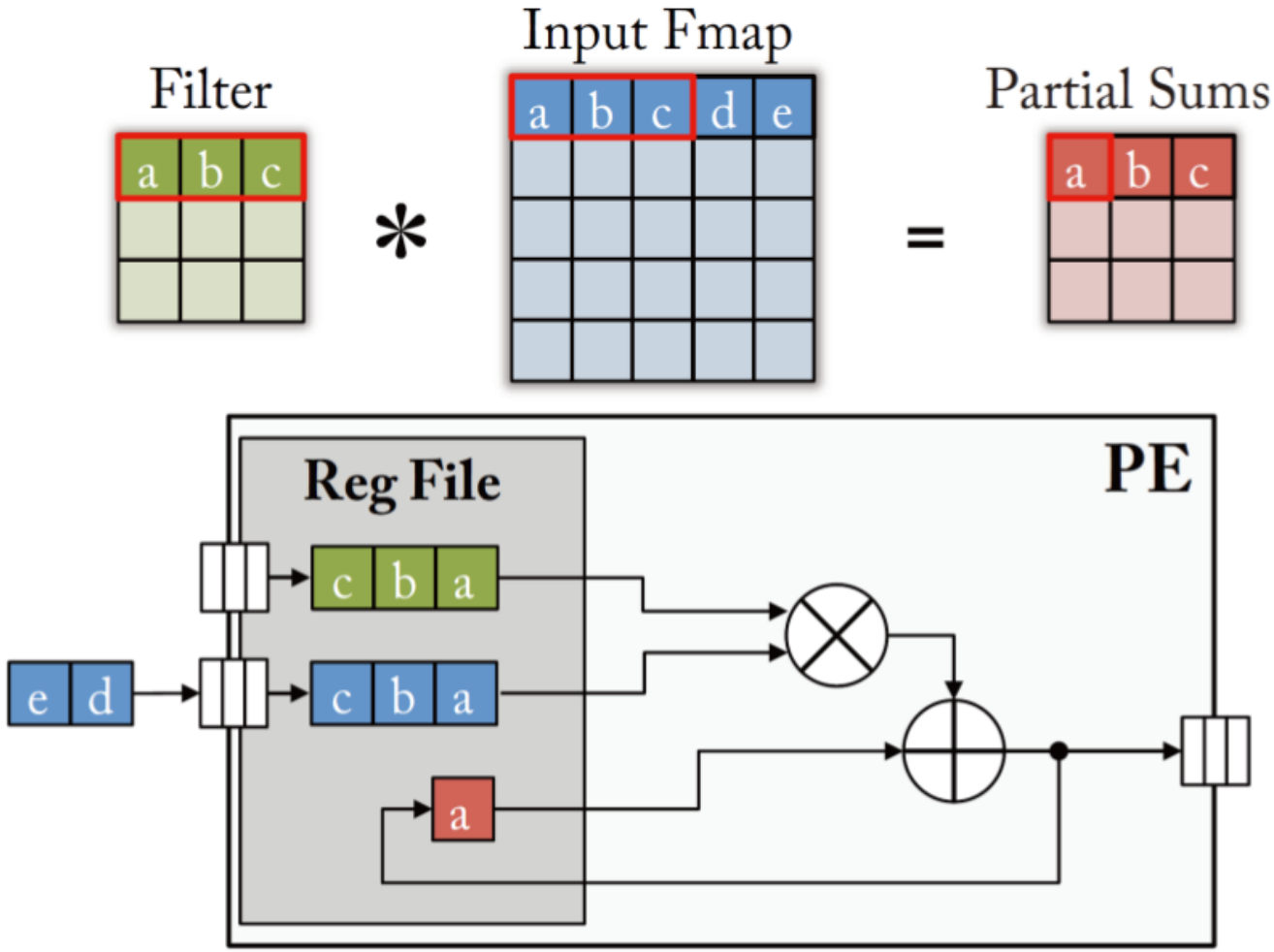


1D Row Convolution in PE

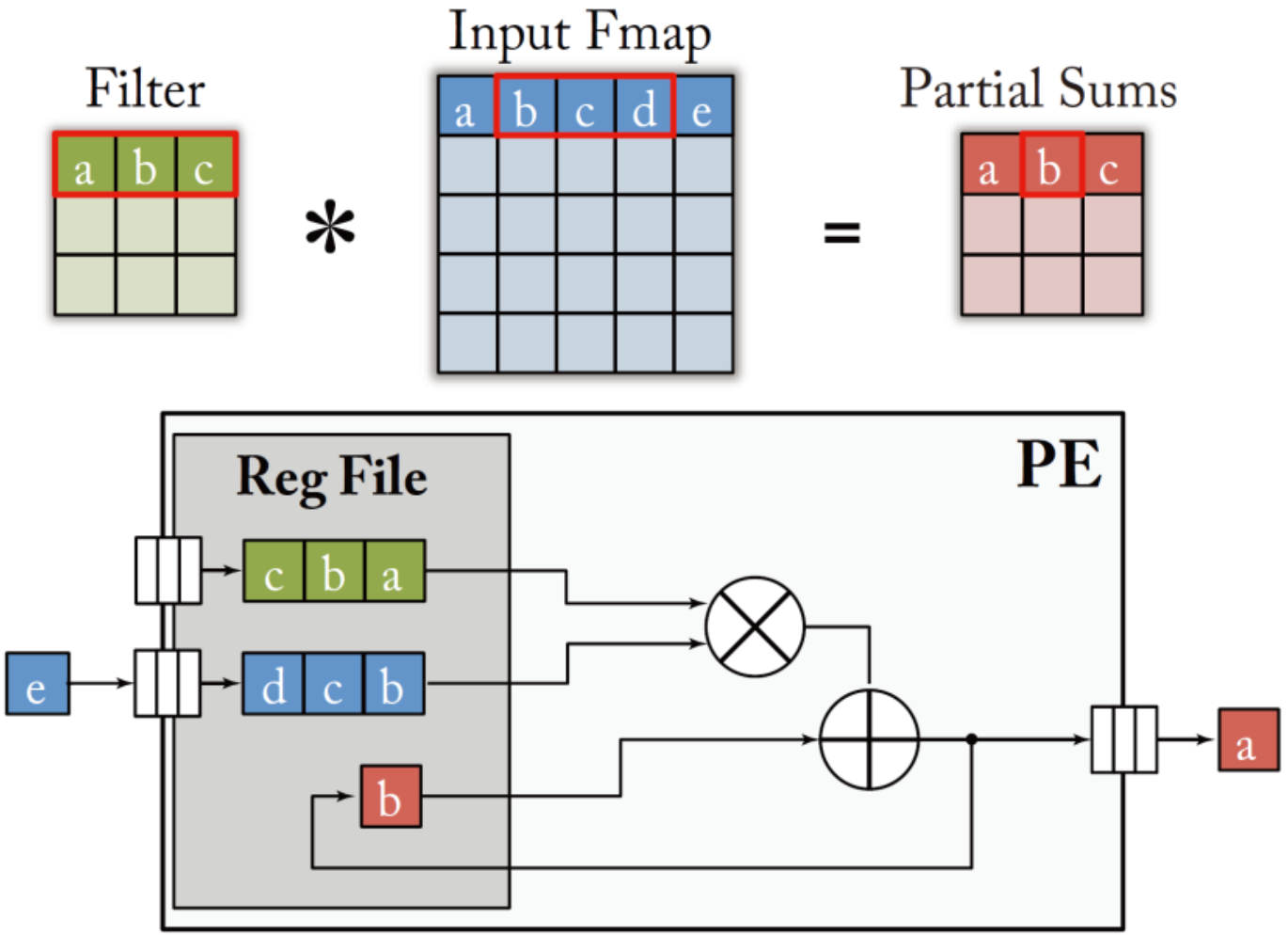




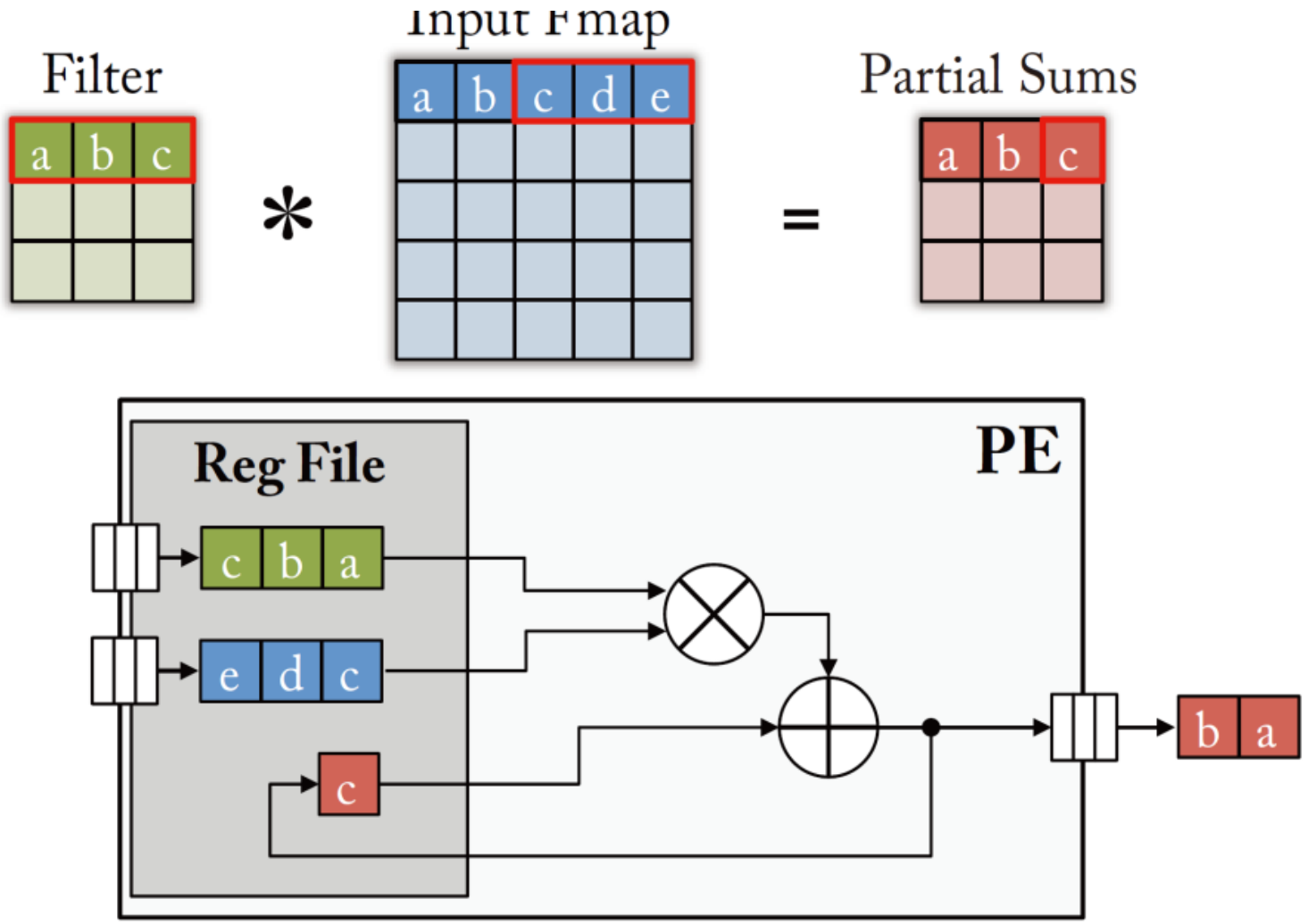
1D Row Convolution in PE



1D Row Convolution in PE

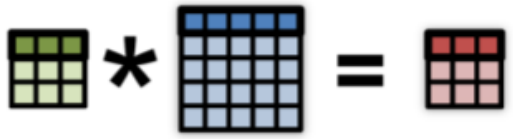


1D Row Convolution in PE

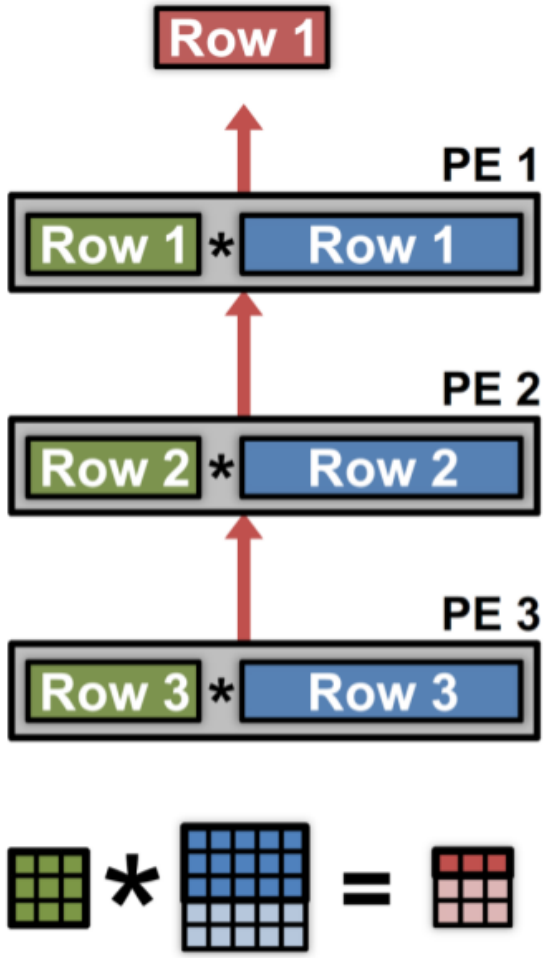


2D Convolution in PE Array

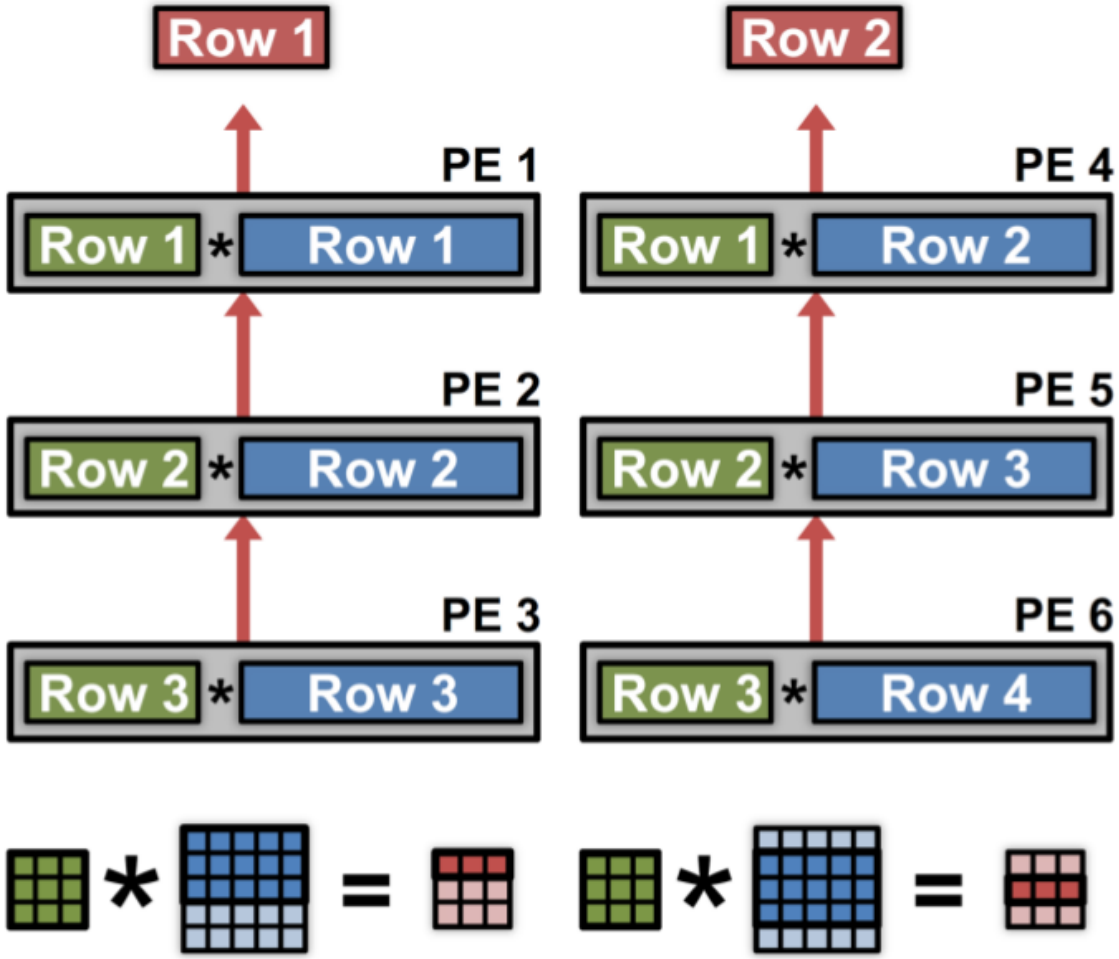




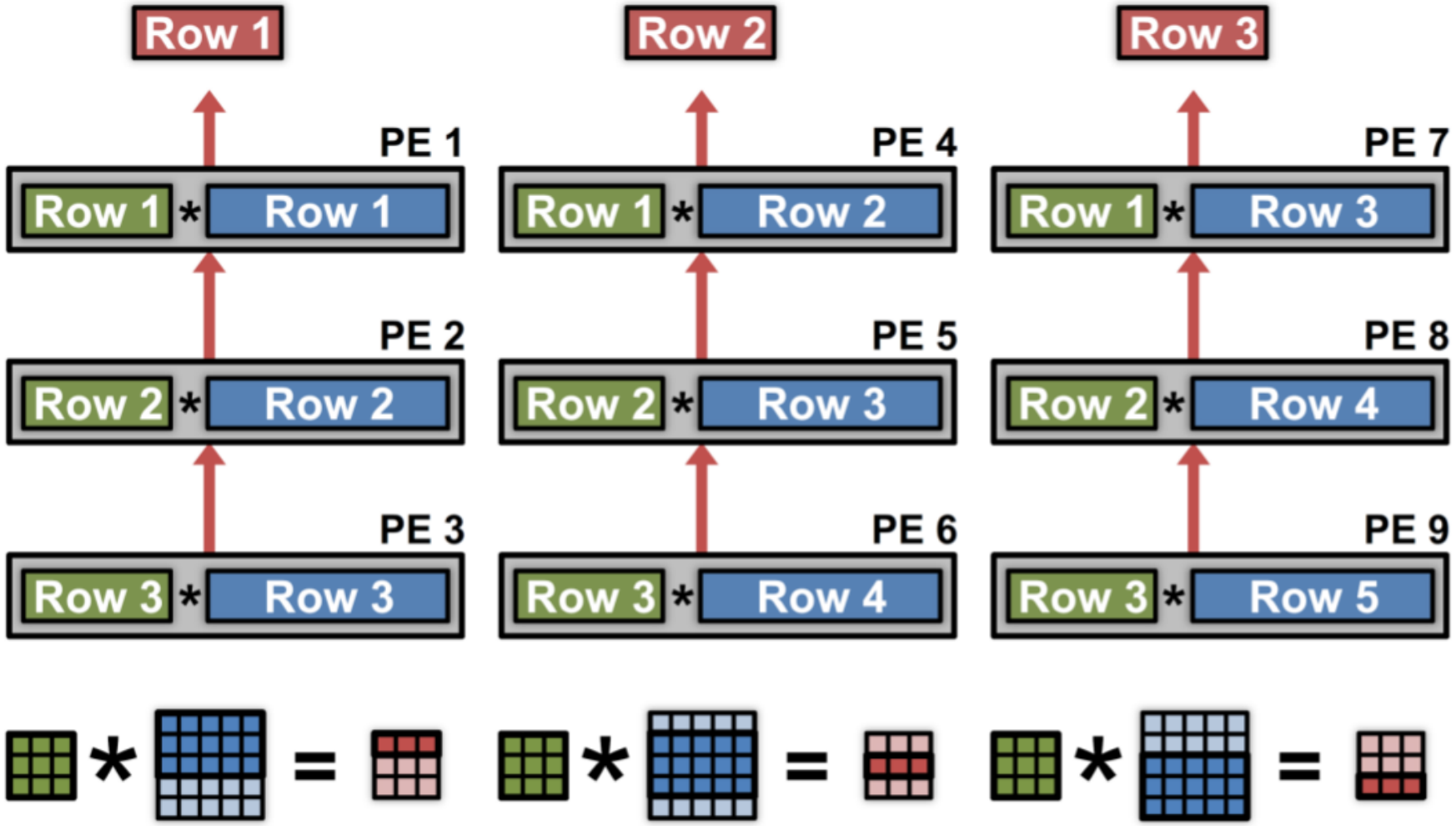
2D Convolution in PE Array



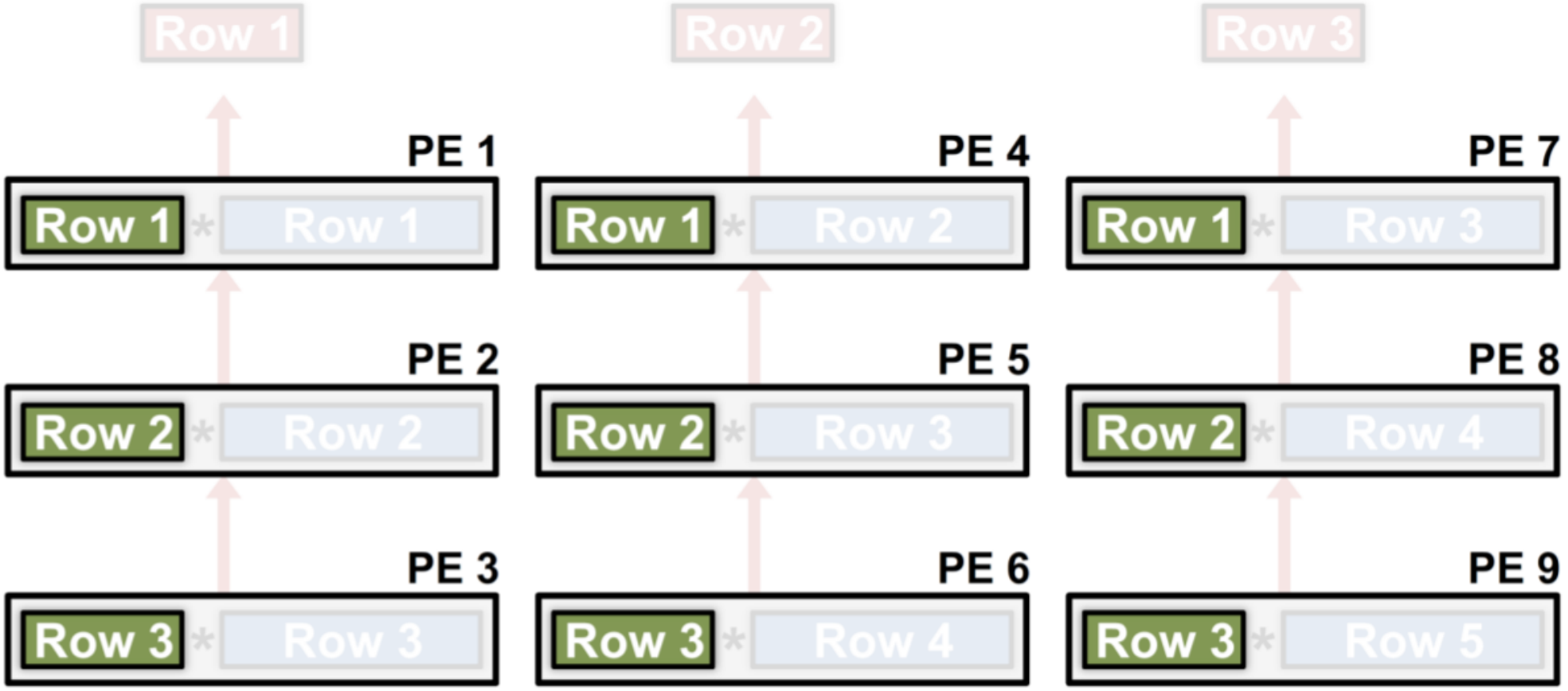
2D Convolution in PE Array



2D Convolution in PE Array

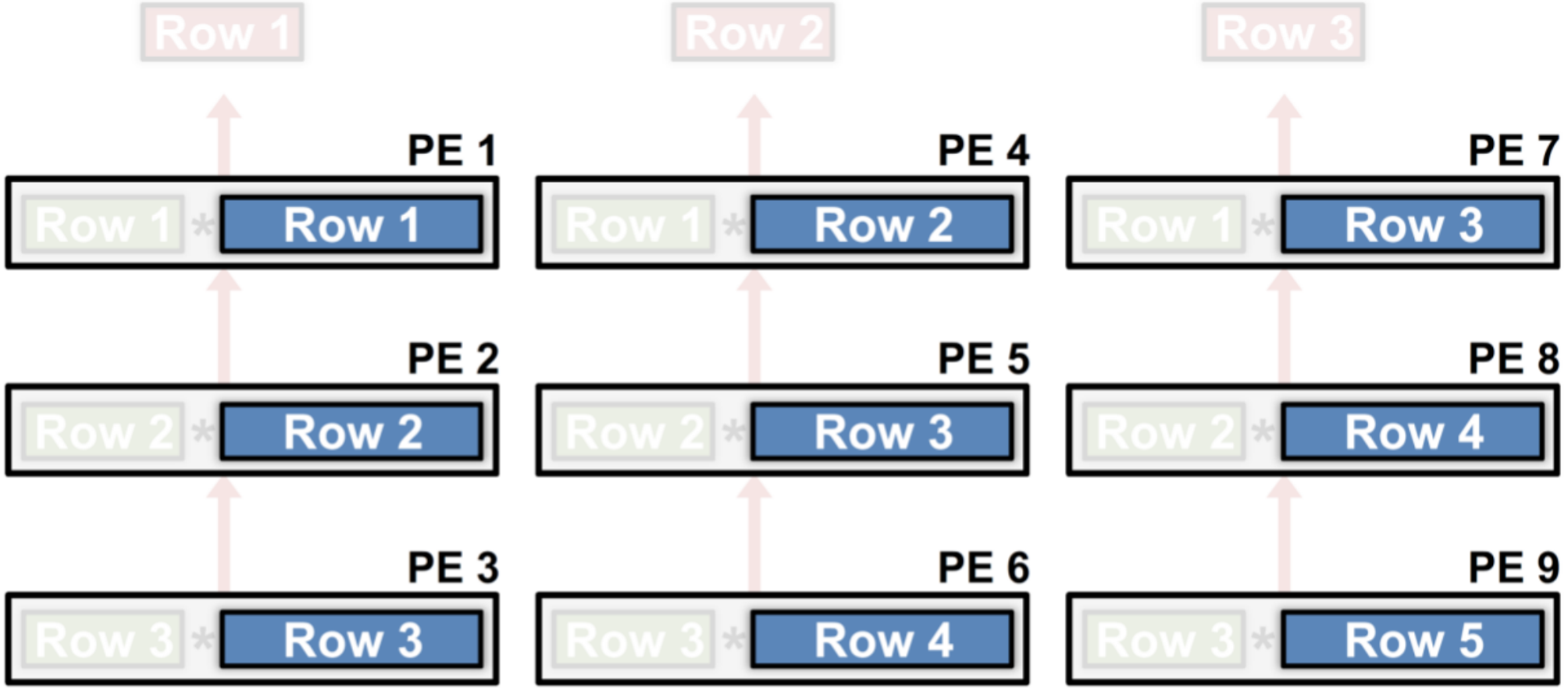


Convolutional Reuse Maximized



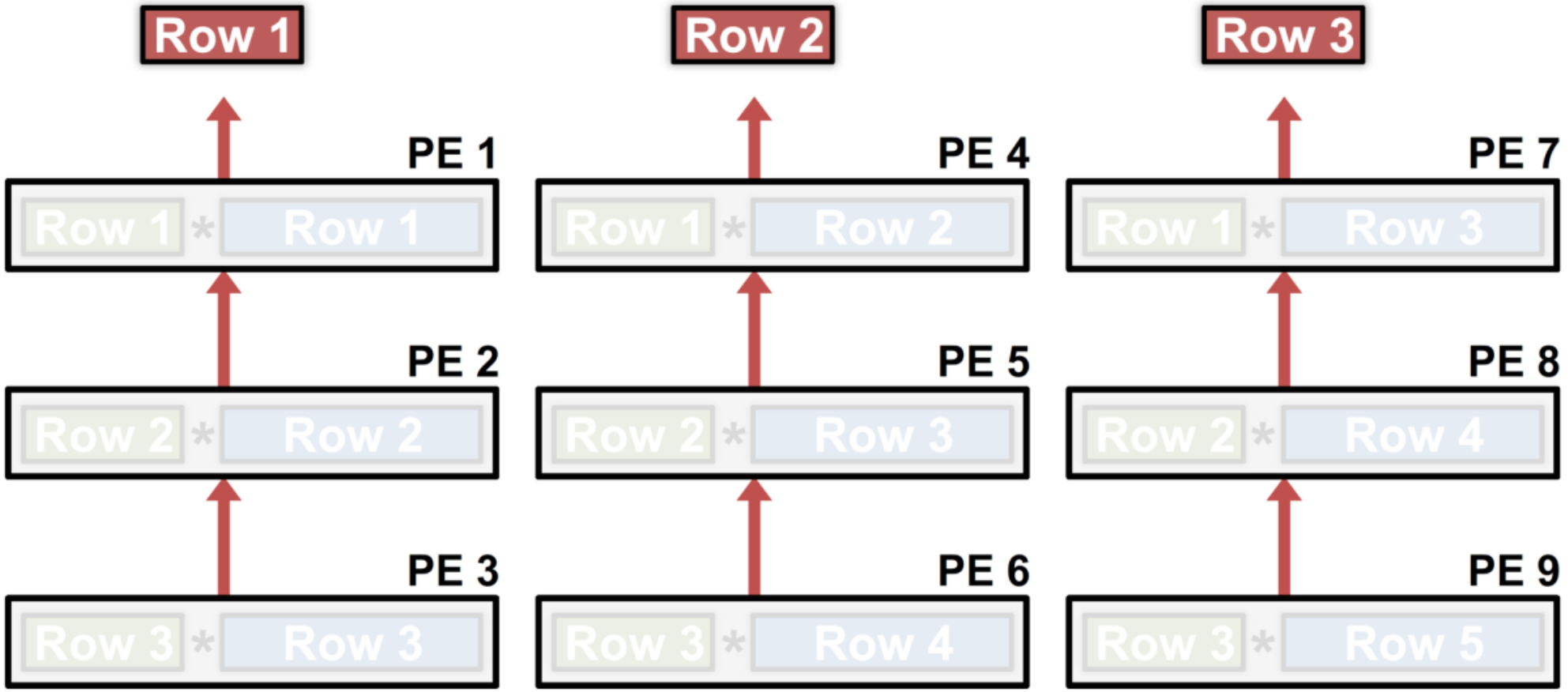
Filter rows are reused across PEs **horizontally**

Convolutional Reuse Maximized



Fmap rows are reused across PEs **diagonally**

Maximize 2D Accumulation in PE Array



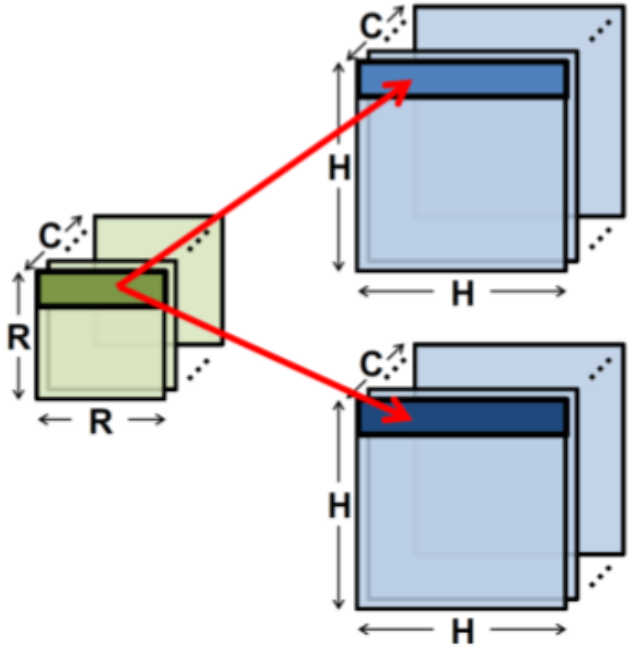
Partial sums accumulate across PEs **vertically**

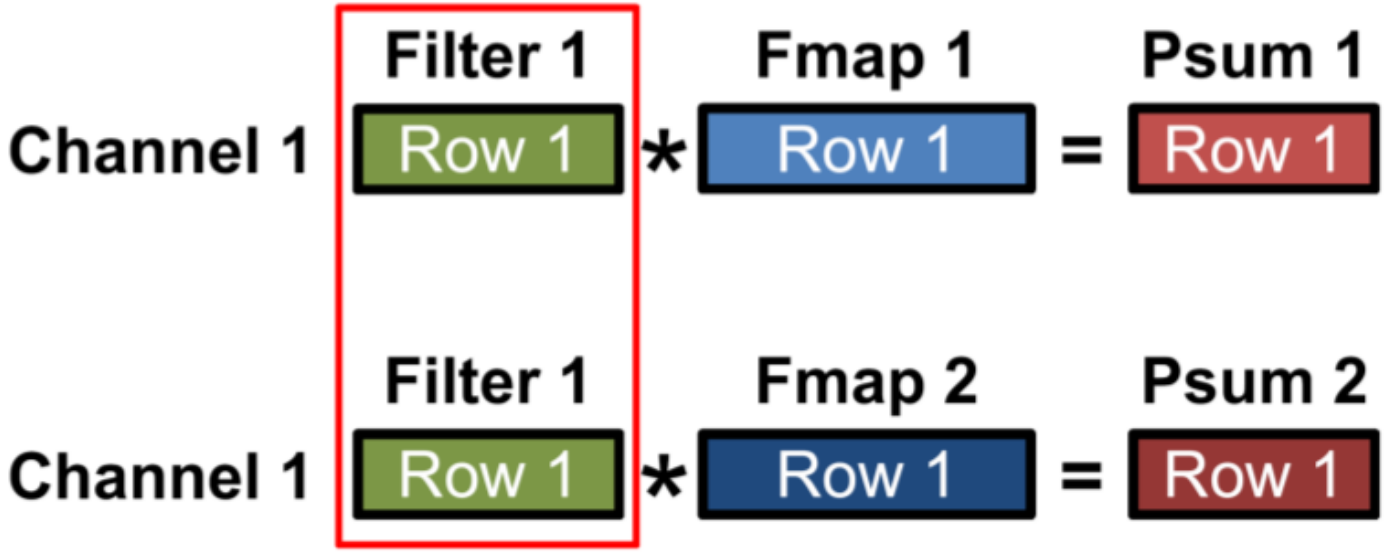
Dimensions Beyond 2D Convolution

- Multiple Fmaps
- Multiple Filters
- Multiple Channels

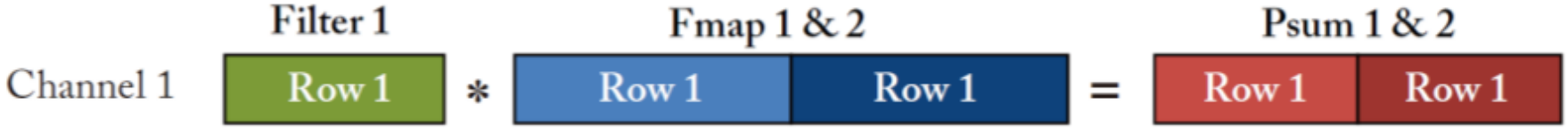


Multiple Filters - Fmap Reuse in PE

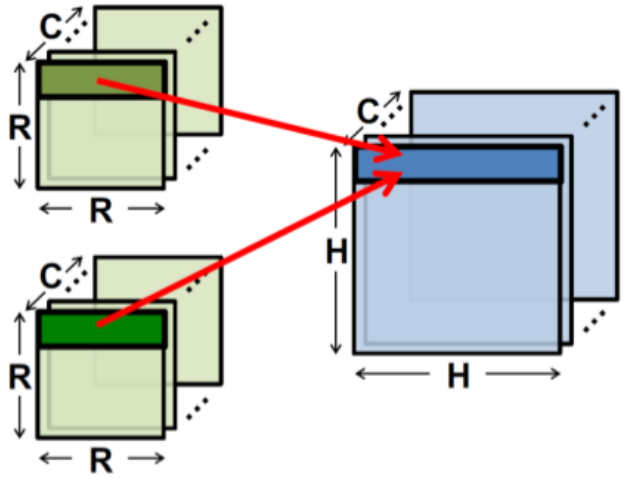


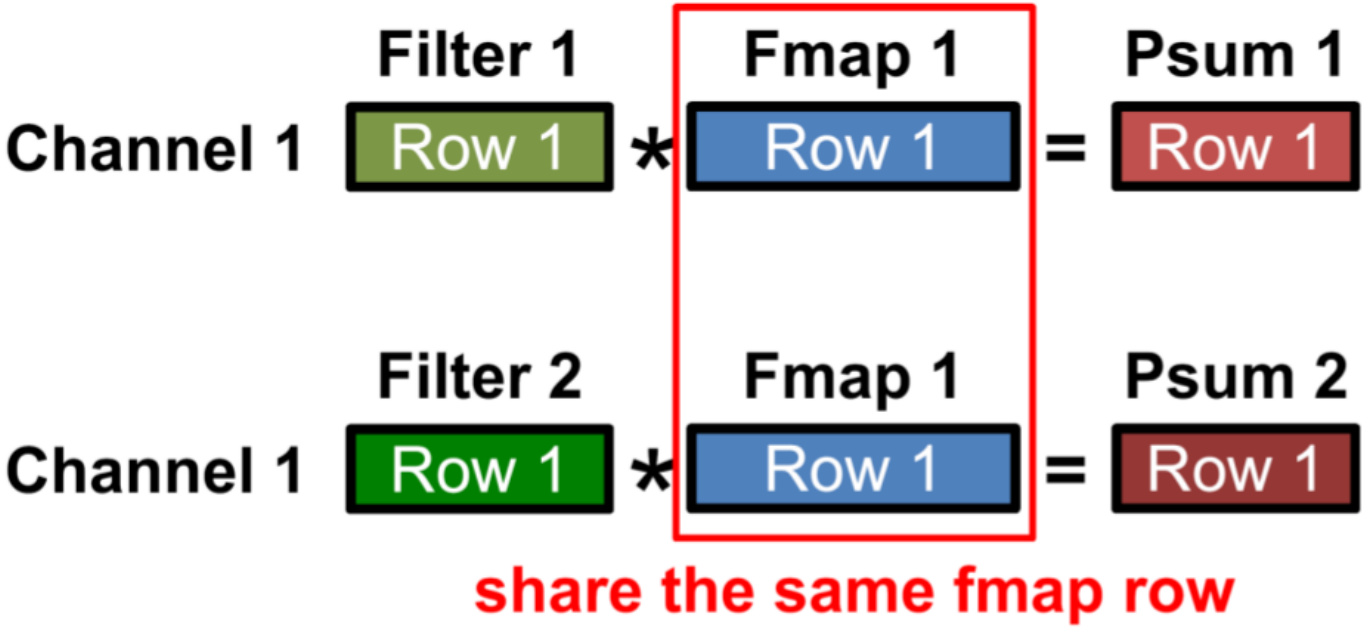


Processing in PE: interleave filter rows



Multiple Filters - Fmap Reuse in PE

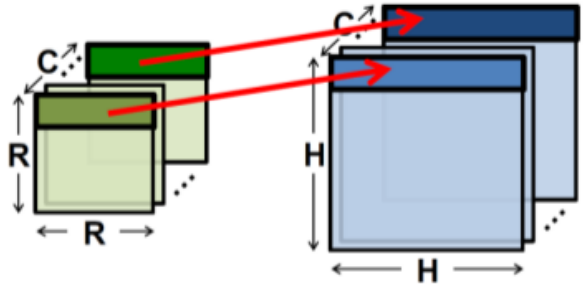


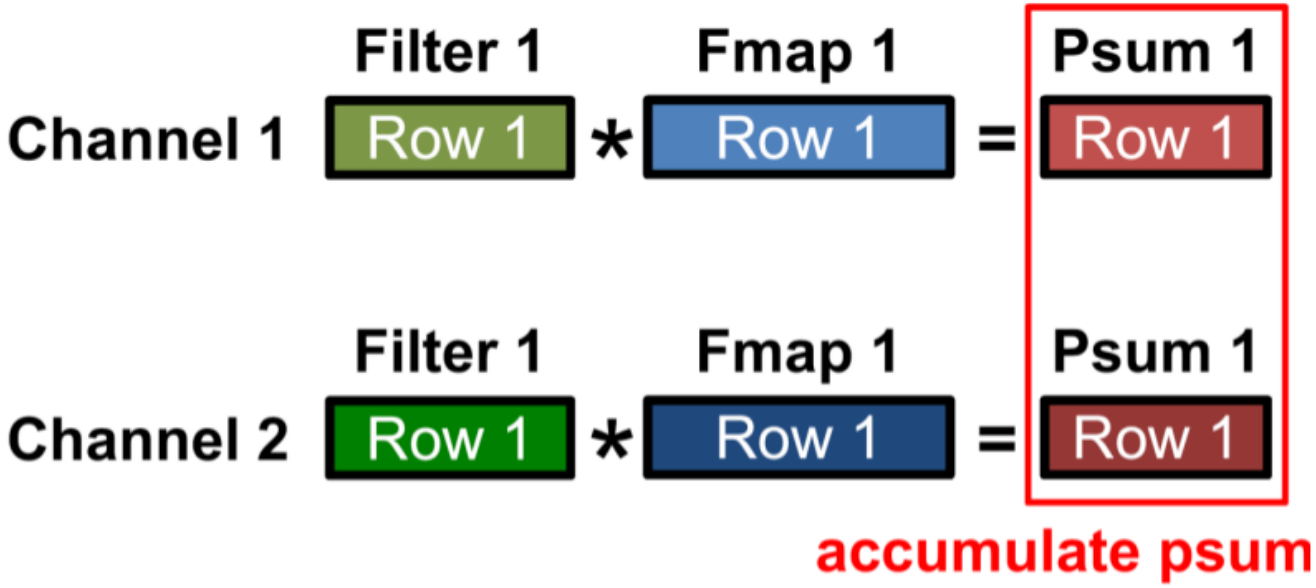


Processing in PE: interleave filter rows



Multiple Channels - Channel Accumulation in PE



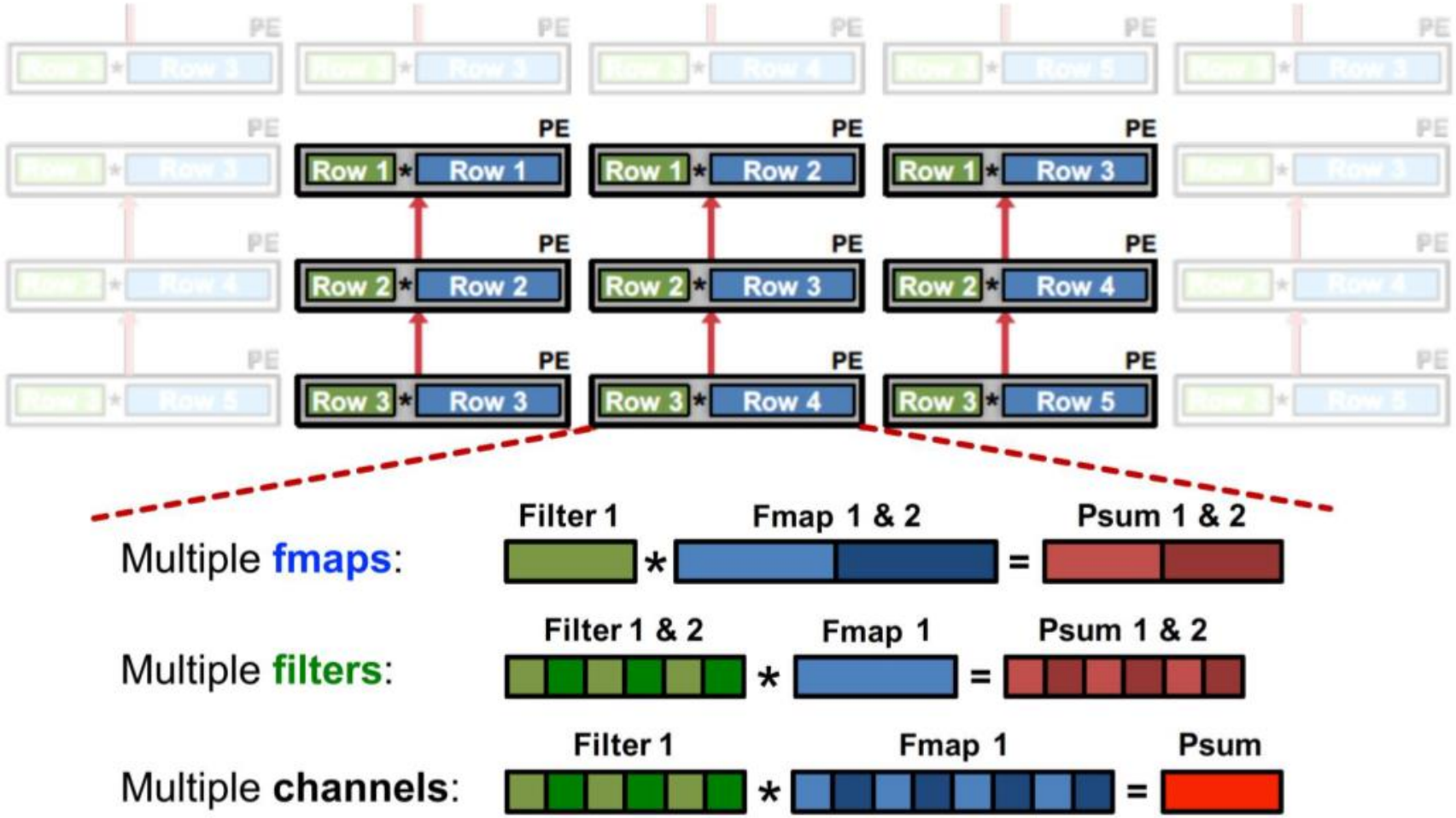


Processing in PE: interleave filter rows



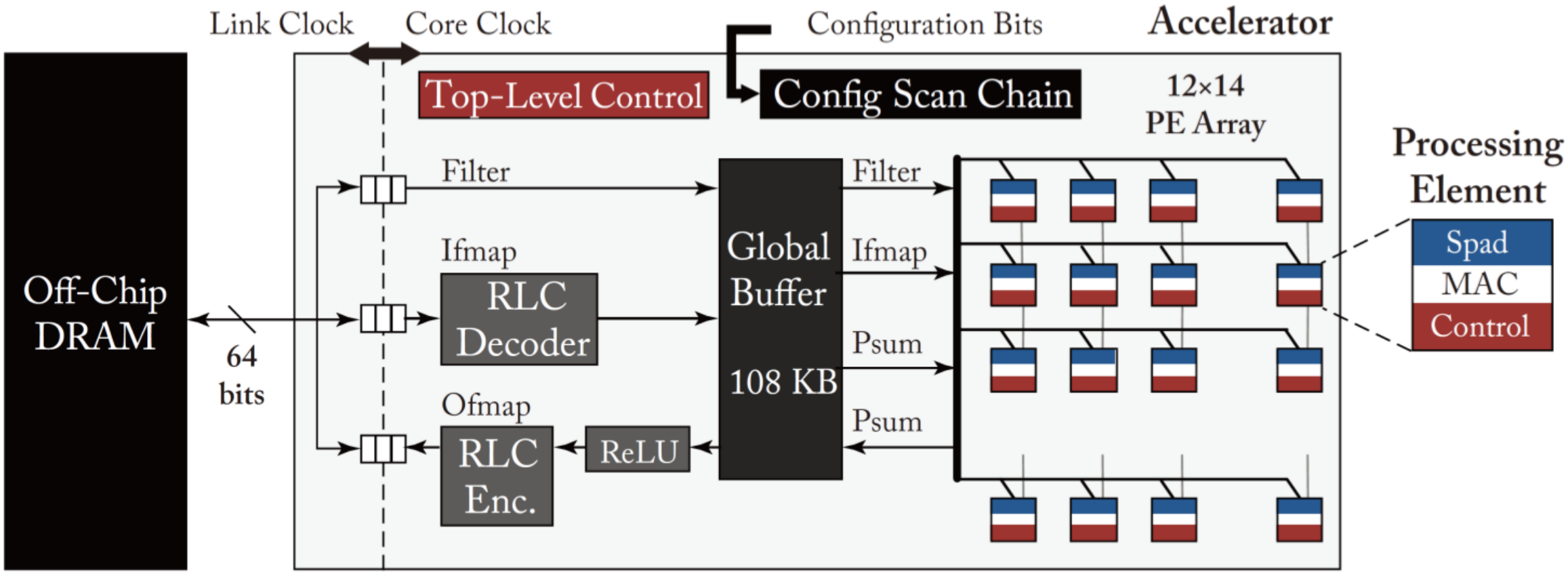


The Full Picture



Map rows from multiple **fmaps**, **filters** and **channels** to same PE to exploit other forms of reuse and local accumulation

Eyeriss DNN accelerator



Taxonomy: More Examples

Dataflow	Recent Work
Weight Stationary	NVDLA, TPU, neuFlow, Sankaradas et al., Park et al., Chakradhar et al., Sriram et al., Origami, ISAAC, PRIME, etc.
Output Stationary	DaDianNao, DianNao, Zhang et al., Moons et al., ShiDianNao, Gupta et al., Peeman et al., Peemen, Moons, Thinker, etc.
Input Stationary	SCNN, etc.
Multiple data-type Stationary	Eyeriss v1(Row Stationary), Eyeriss v2(Row Stationary), etc.

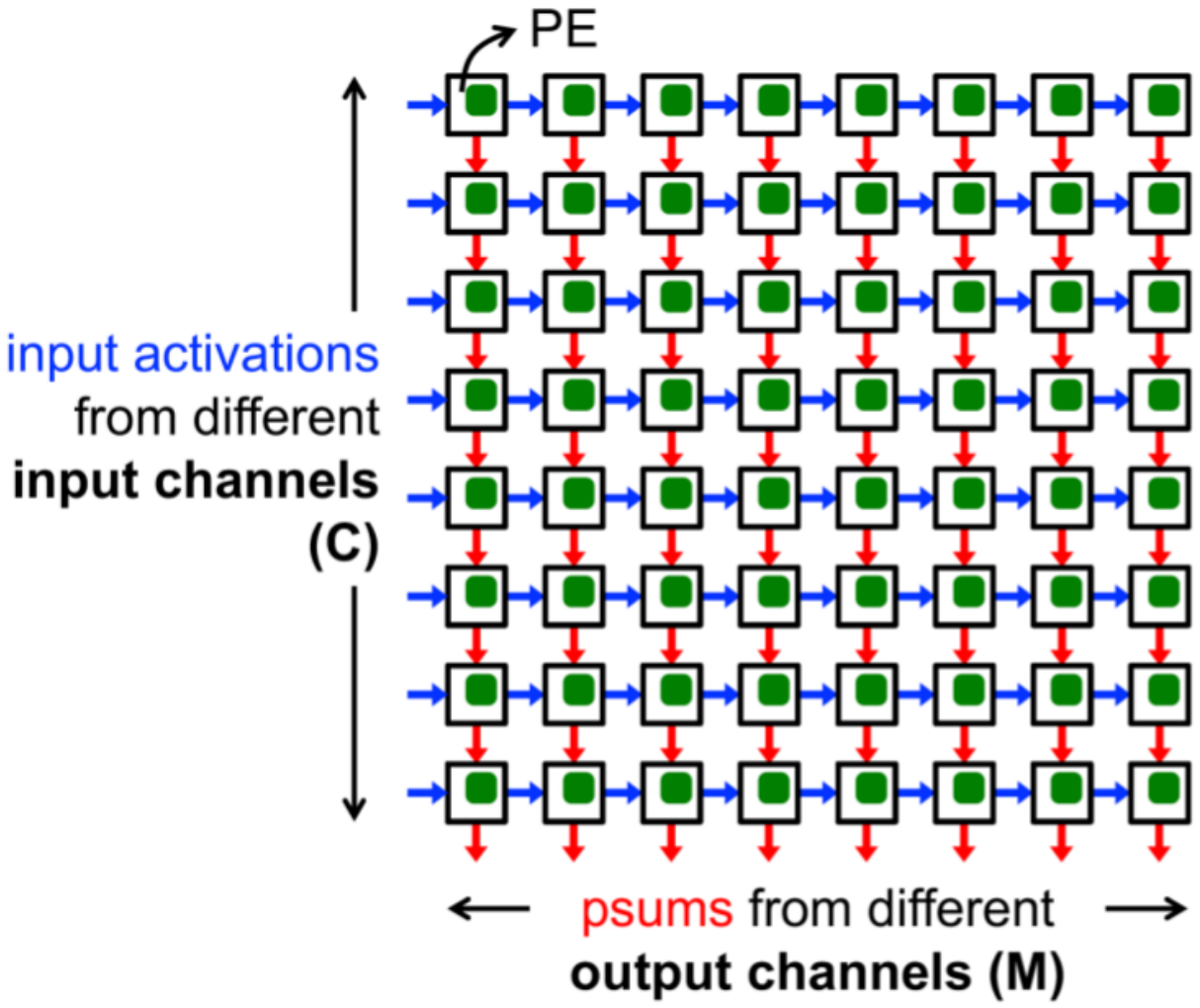


Outline

- Benchmarking Metrics
- GEMM Accelerator Design
- **DNN Accelerator Design**
  - Locality and Data Reuse
  - Dataflow Taxonomy
  - **Network-on-Chip**
  - Optimization
- Energy

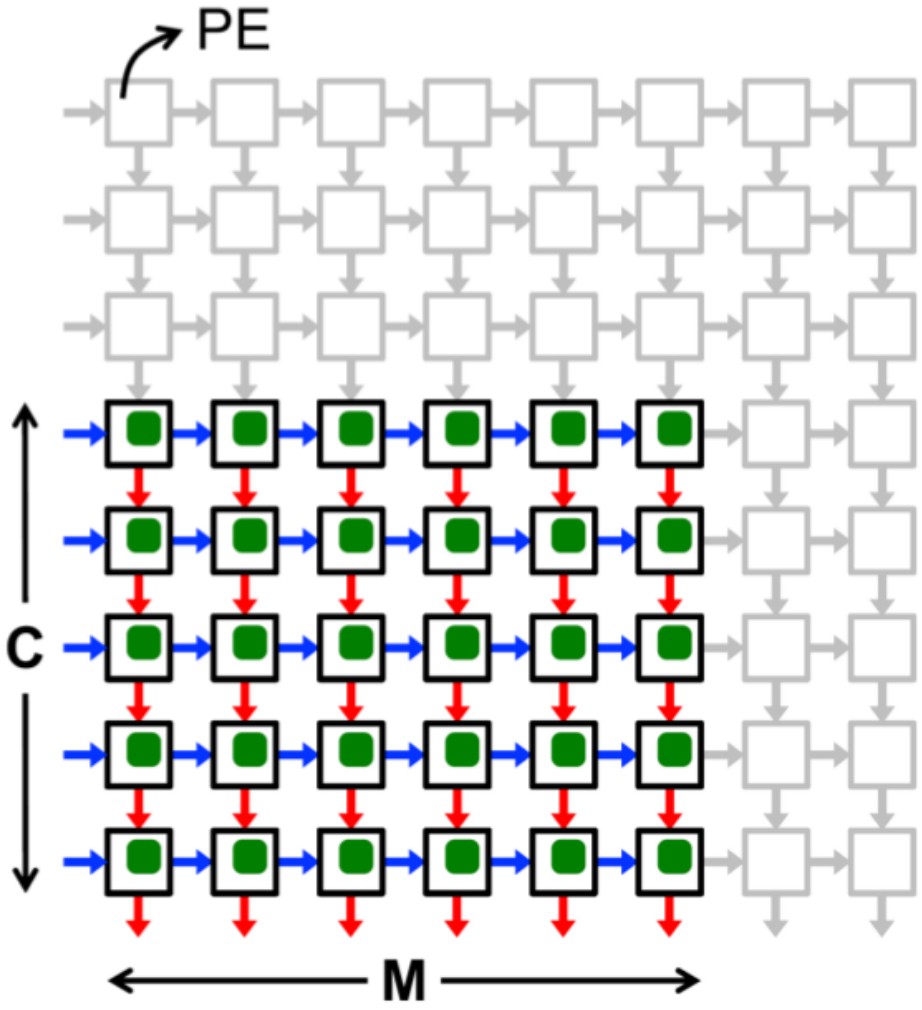


A Common Design Pattern



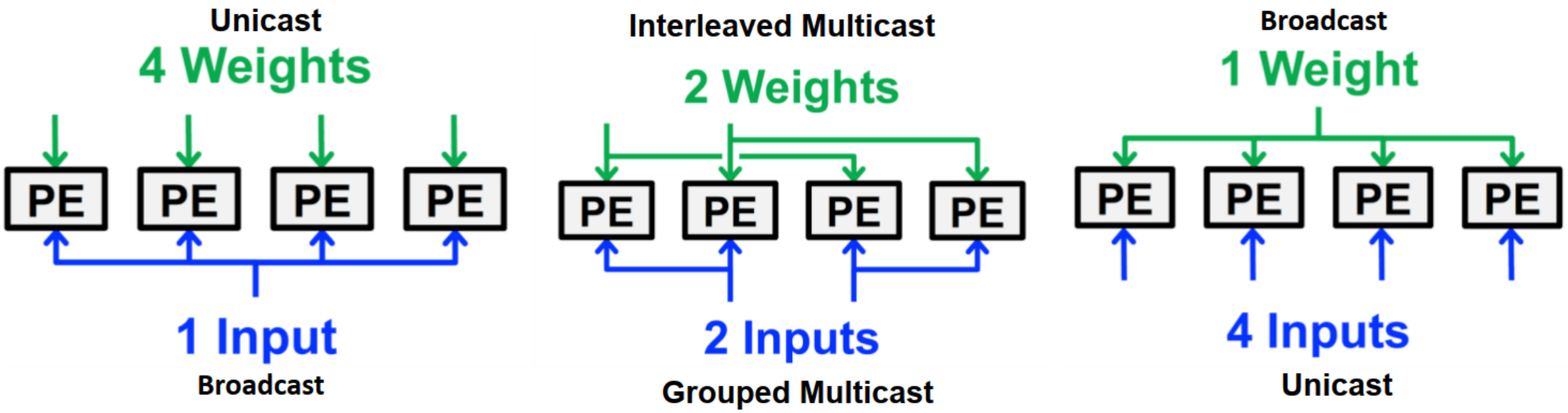
A Common Design Pattern

- PE array underutilized If there are fewer input and/or output channels than the array dimensions
- Effective data delivery BW also becomes lower → further impact performance
- Not scalable → utilization will be worse at larger scales



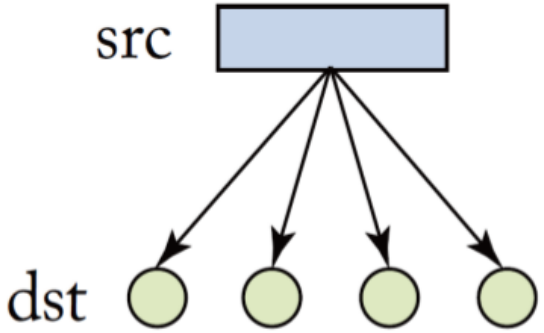
A More Flexible Data Delivery Strategy

- Adapt to the reuse and bandwidth requirements



Common NoC Design

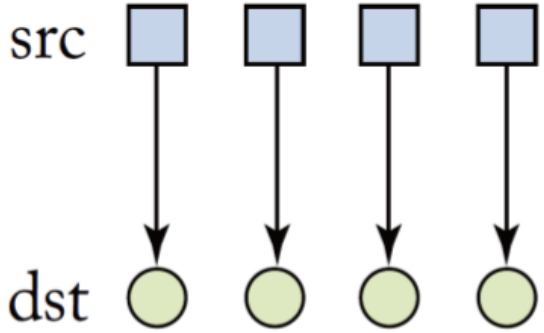
Broadcast Network



High Reuse

Low Bandwidth

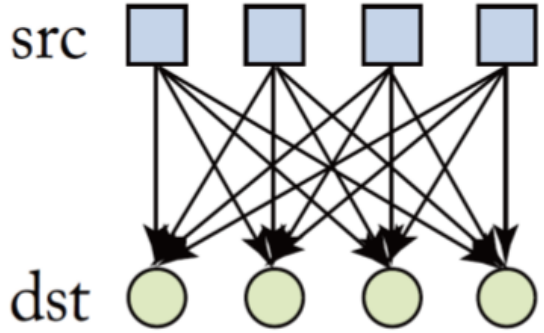
Unicast Network



Low Reuse

High Bandwidth

All-to-All Network

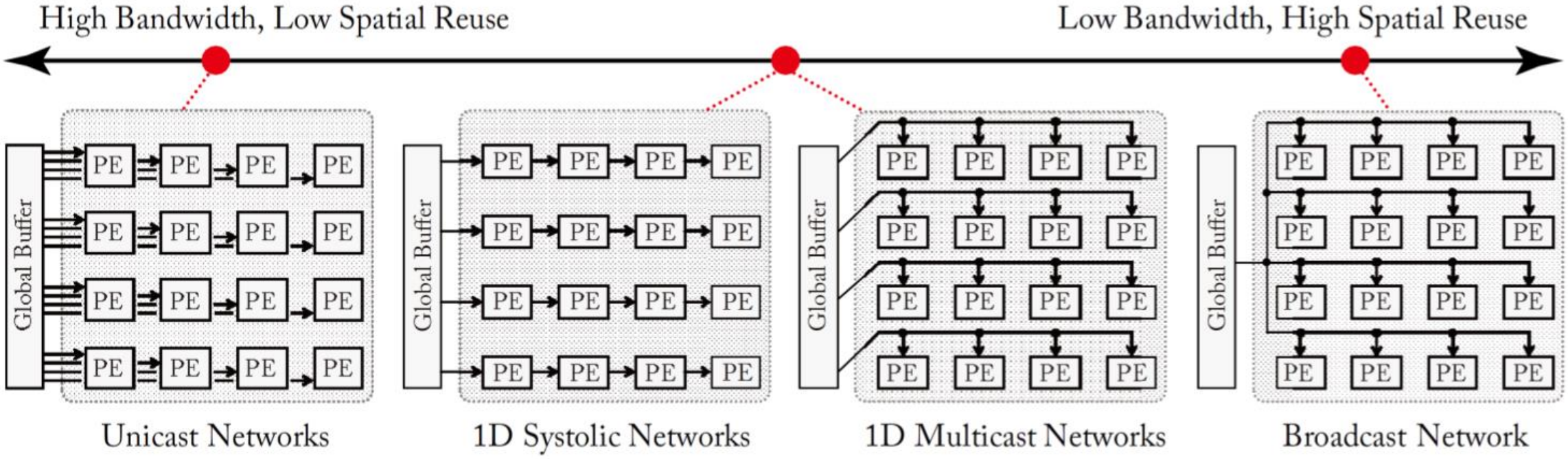


High Reuse

High Bandwidth

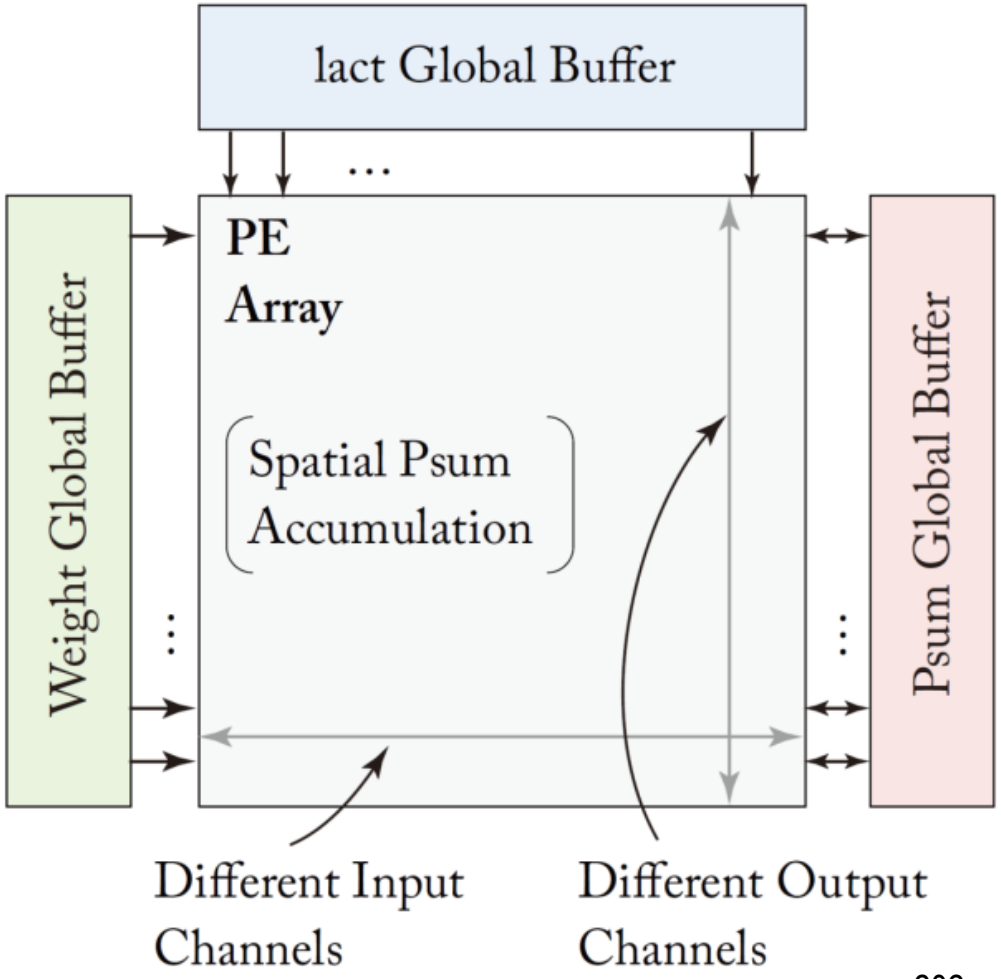
Hard to Scale

Common NoC Designs on DNNs



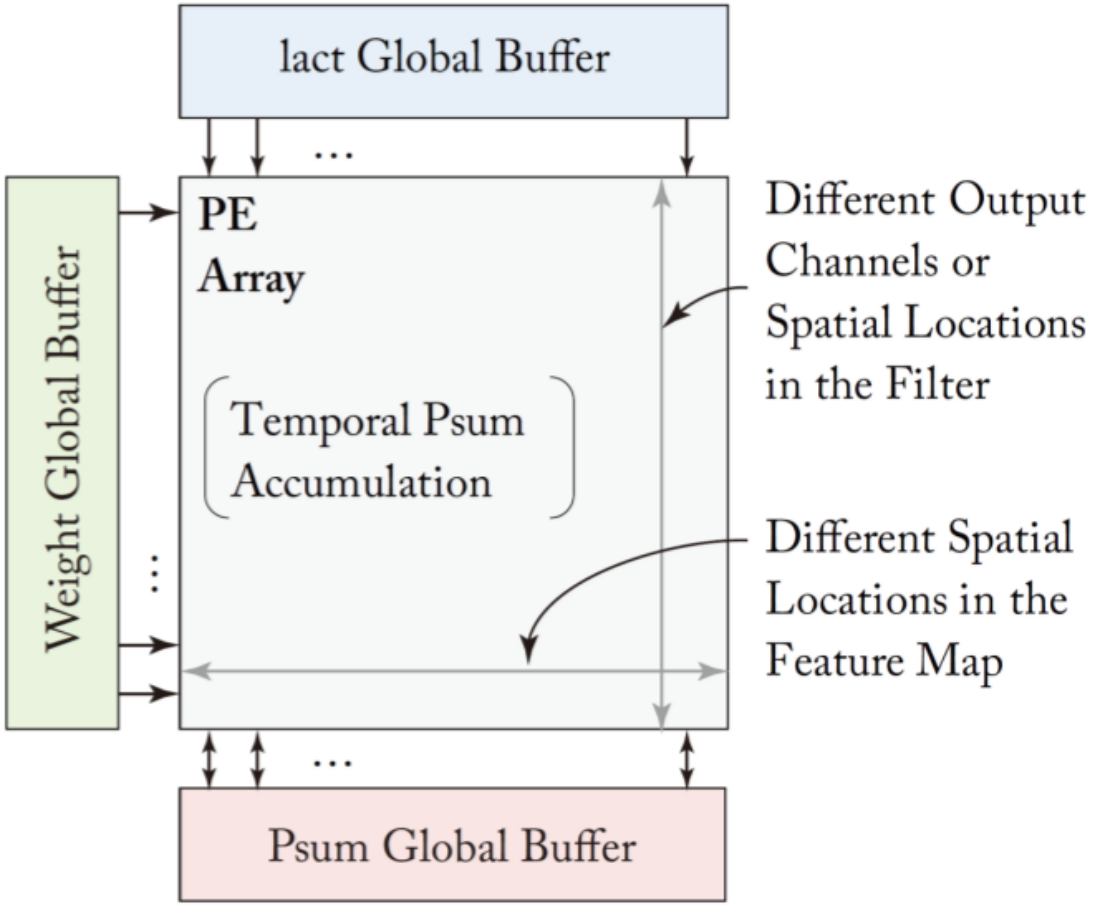
Spatial Accumulation DNN Accelerator Design

- Input activations (Iacts) are reused vertically
- Partial sums are accumulated horizontally
- Often used for a weight-stationary dataflow

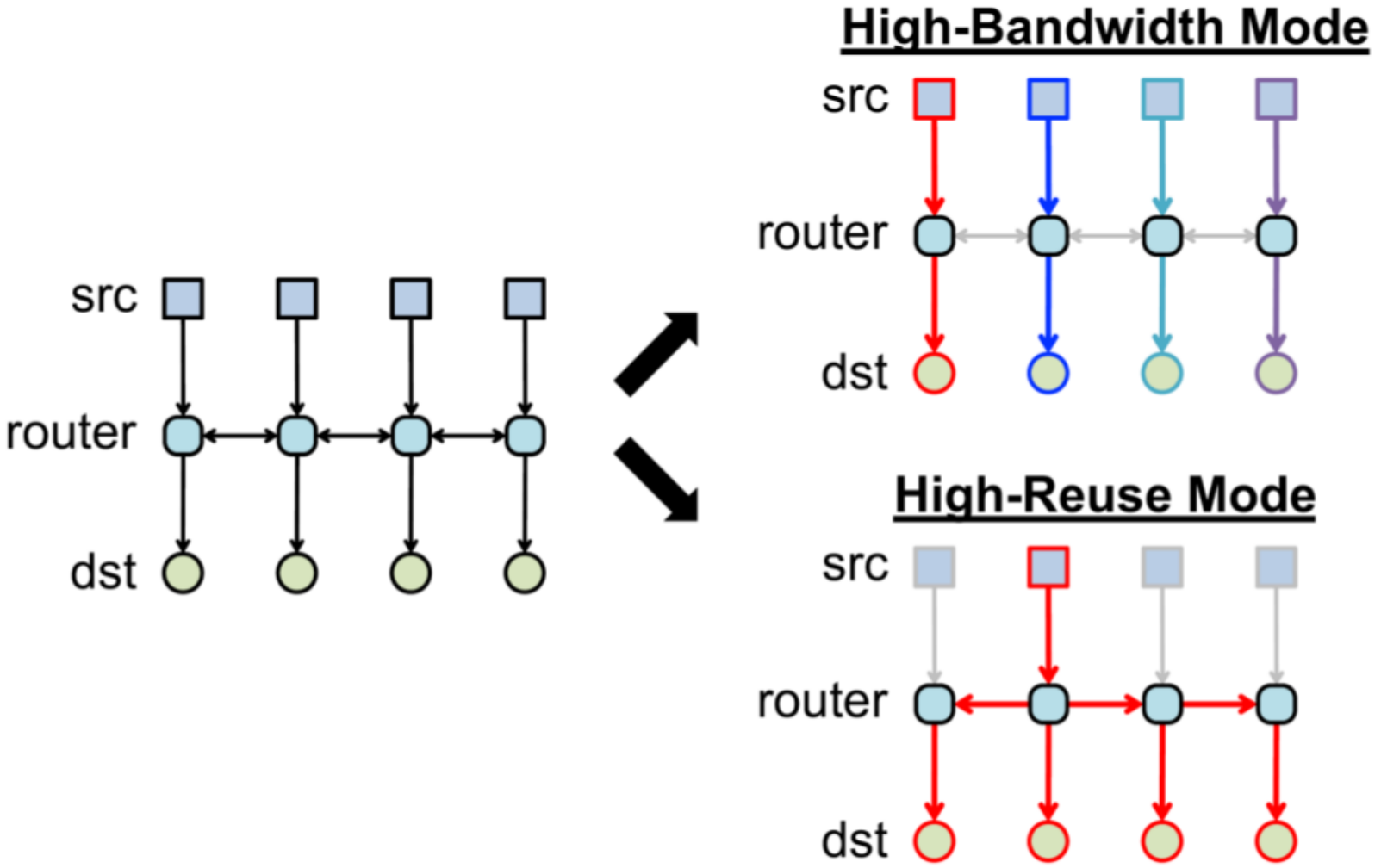


Temporal Accumulation DNN Accelerator Design

- Input activations (Iacts) are reused vertically
- Weights are accumulated horizontally
- Often used for an output-stationary dataflow

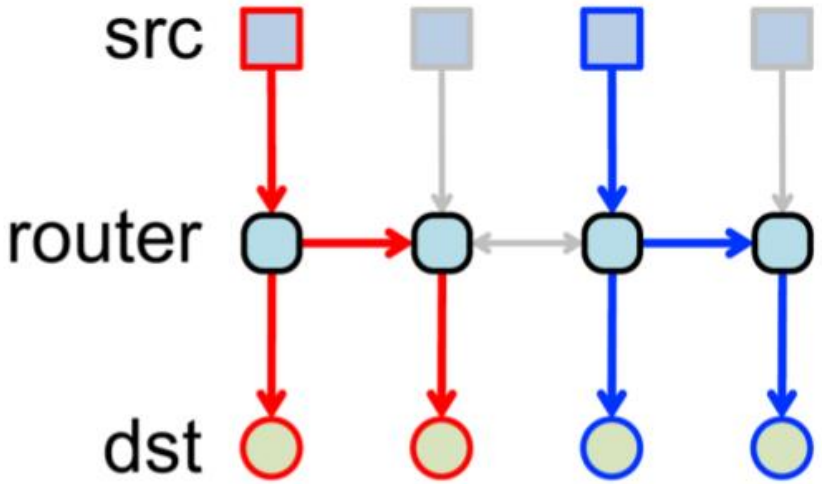


Mesh Network – Best of Both Worlds

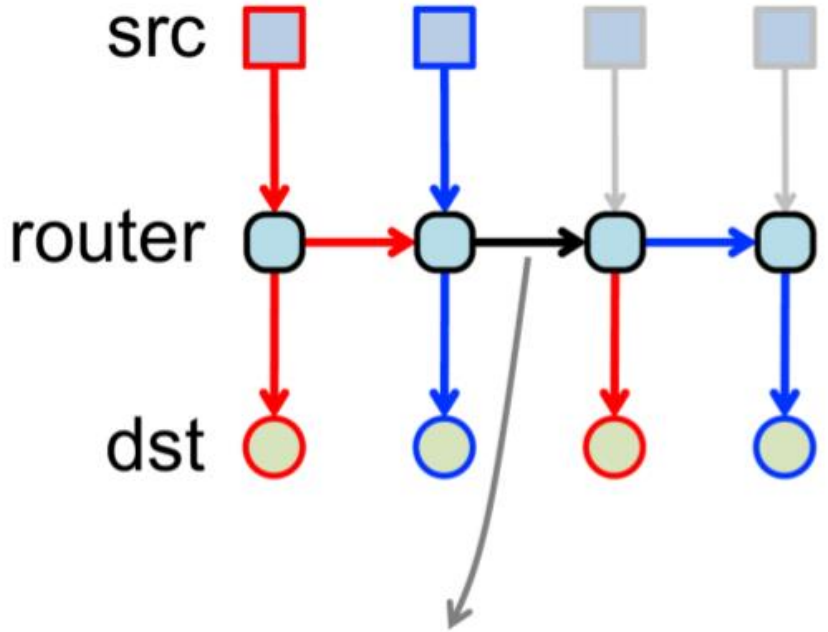


Mesh Network – More Complicated Cases

Grouped-Multicast Mode



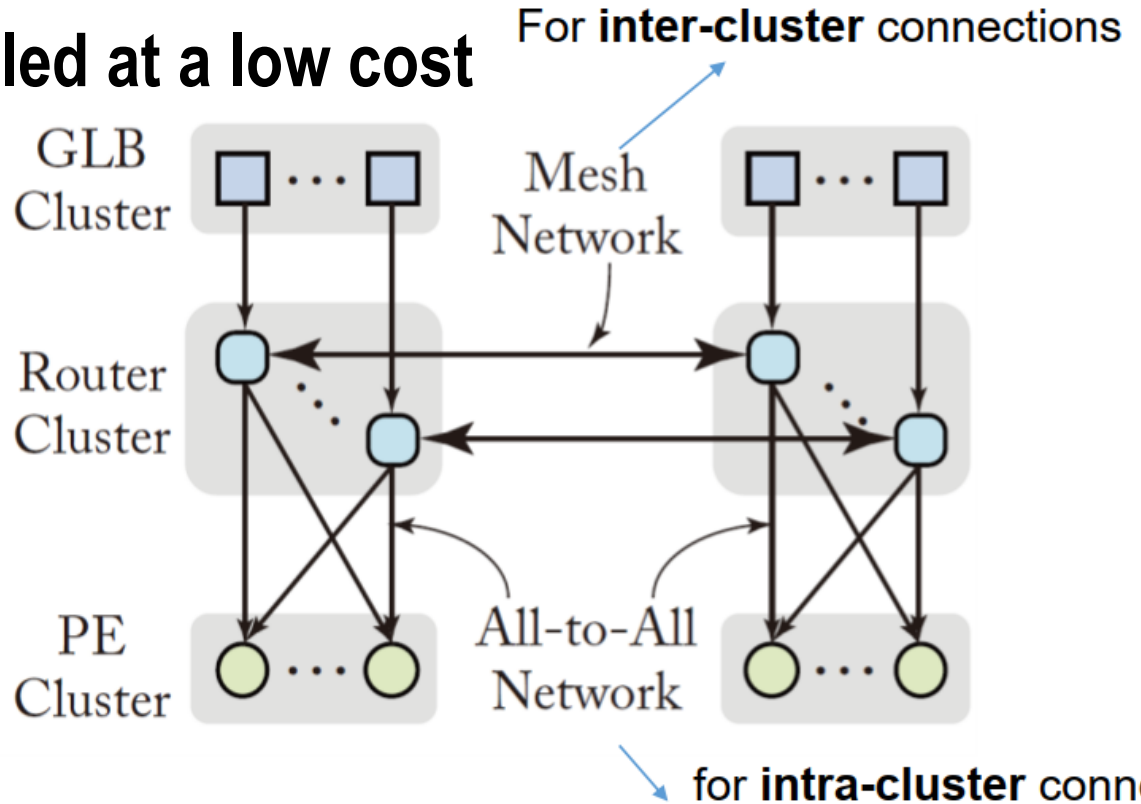
Interleaved-Multicast Mode



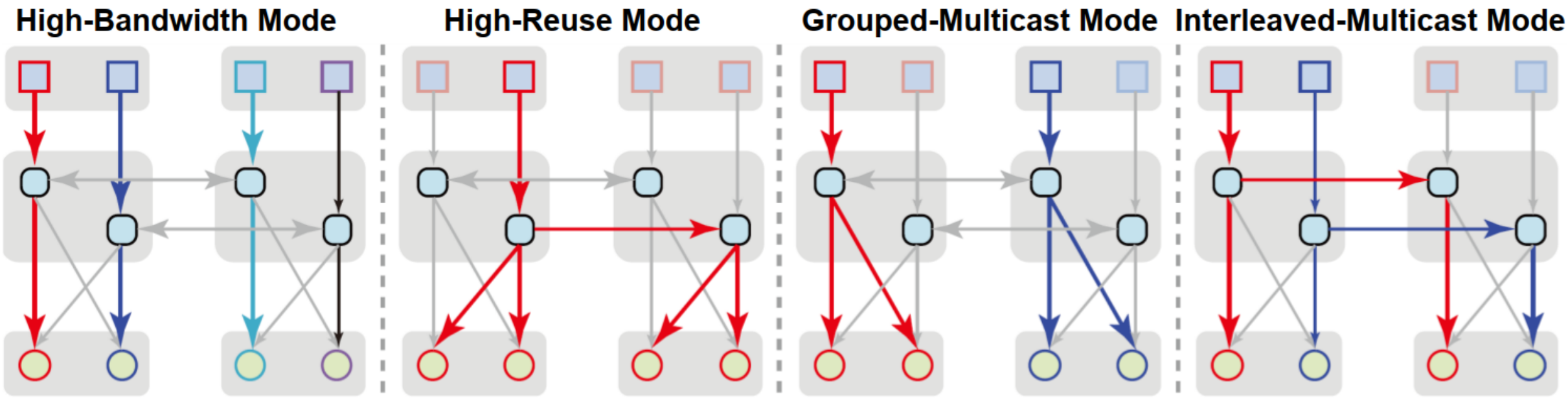
Bandwidth-limited route
(flow control required)

Hierarchical Mesh Network

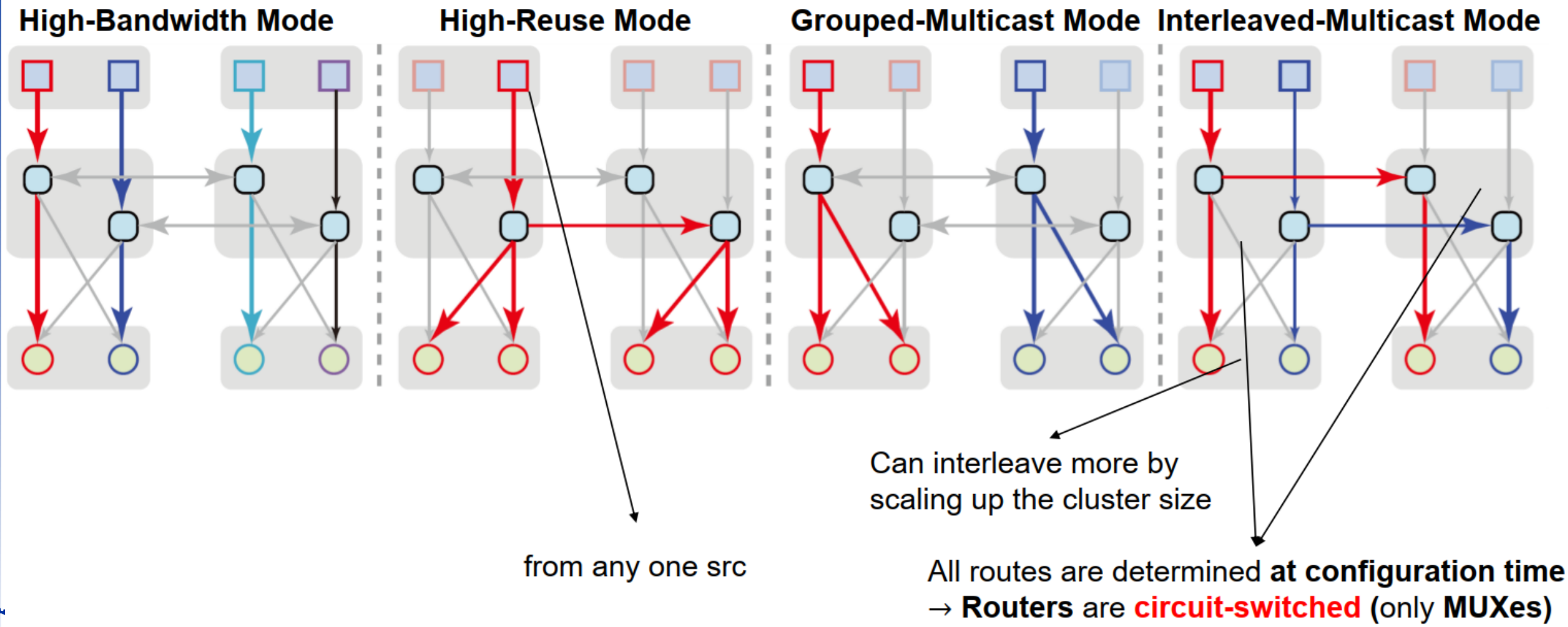
- Flexible to support patterns ranging from high reuse to high bandwidth scenarios
- Can be easily scaled at a low cost



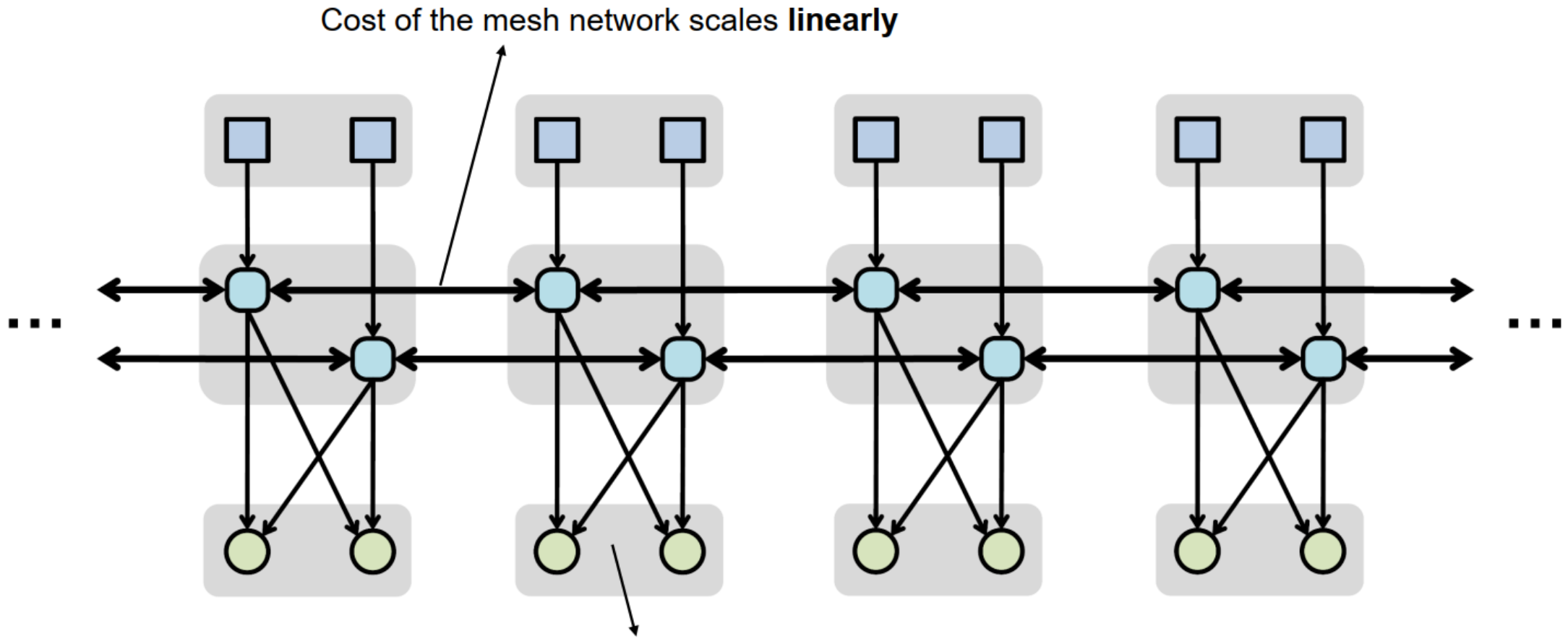
Different Operating Modes



Different Operating Modes



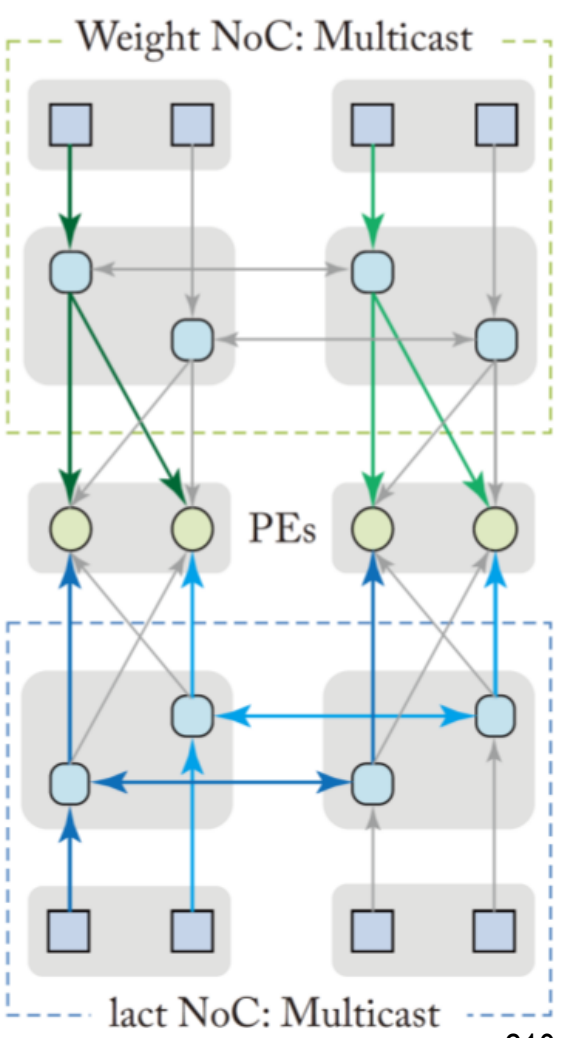
Scaling the Hierarchical Mesh Network



Hierarchical Mesh Networks CONV Layer

- Keep all four PE the lowest bandwidth requirement
  - 2 input activations
  - 2 weight

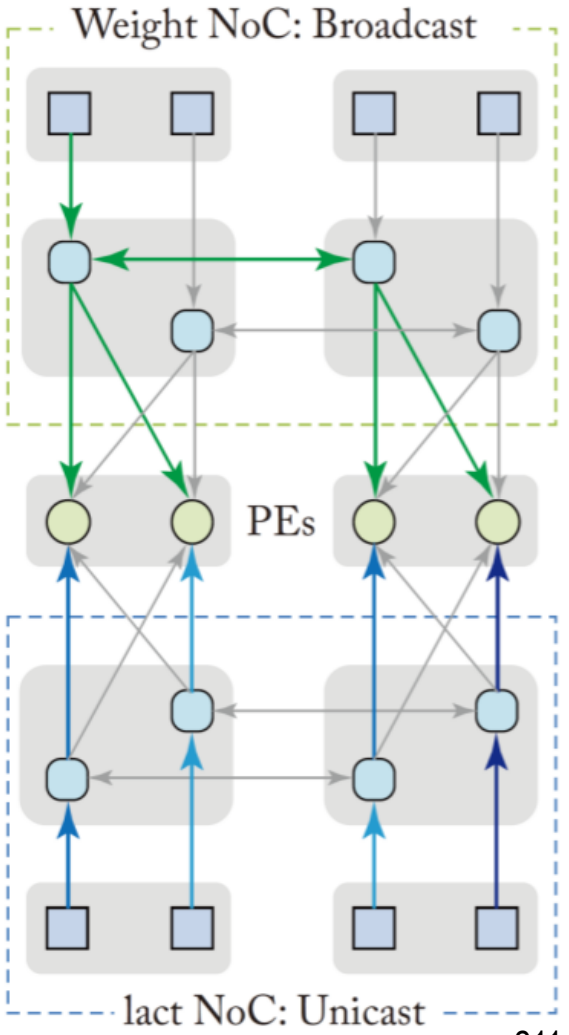
- HM-NoC Configurations (2 options)
  1. Grouped-multicast for input activations
     Interleaved-multicast for weights
  2. Grouped-multicast for weights
     Interleaved-multicast for input activations



Hierarchical Mesh Networks Depth-wise CONV Layer

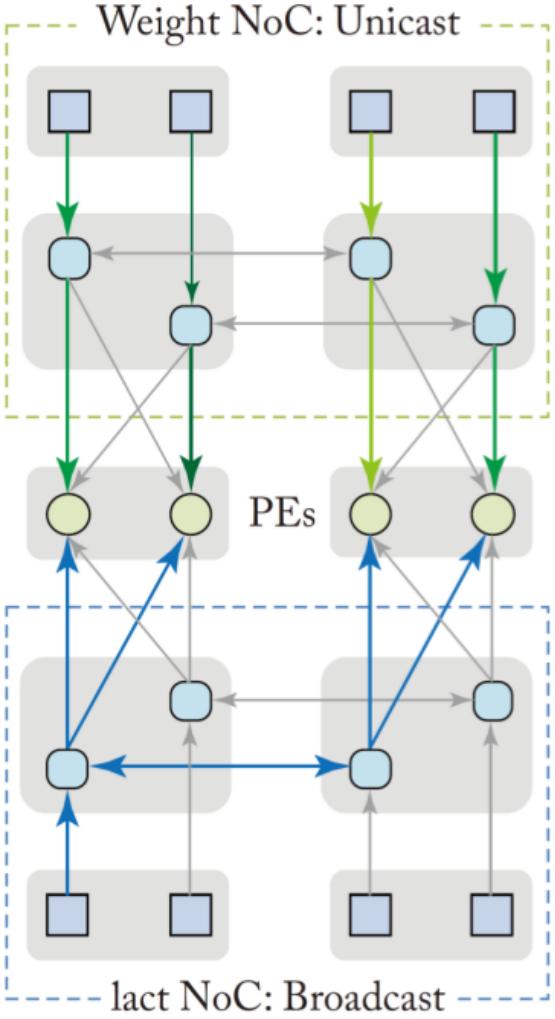
- Nearly no reuse for input activation
- Exploit the reuse of weights

- HM-NoC Configurations
  - Broadcasting the weights to all Pes
  - Fetching unique input activation for each PE



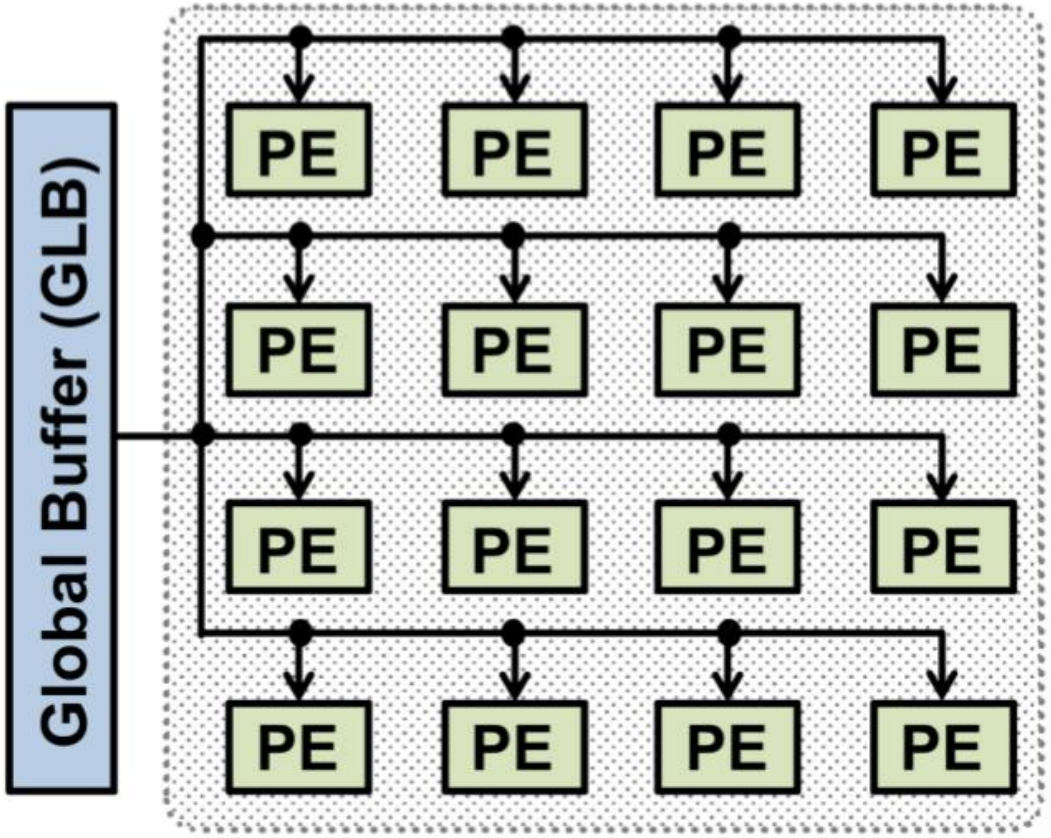
Hierarchical Mesh Networks Depth-wise CONV Layer

- Little reuse for weights
  - Especially when the batch size is limited
- HM-NoC Configurations
  - Weights are now unicast to the Pes
  - Input activations are broadcast to all Pes

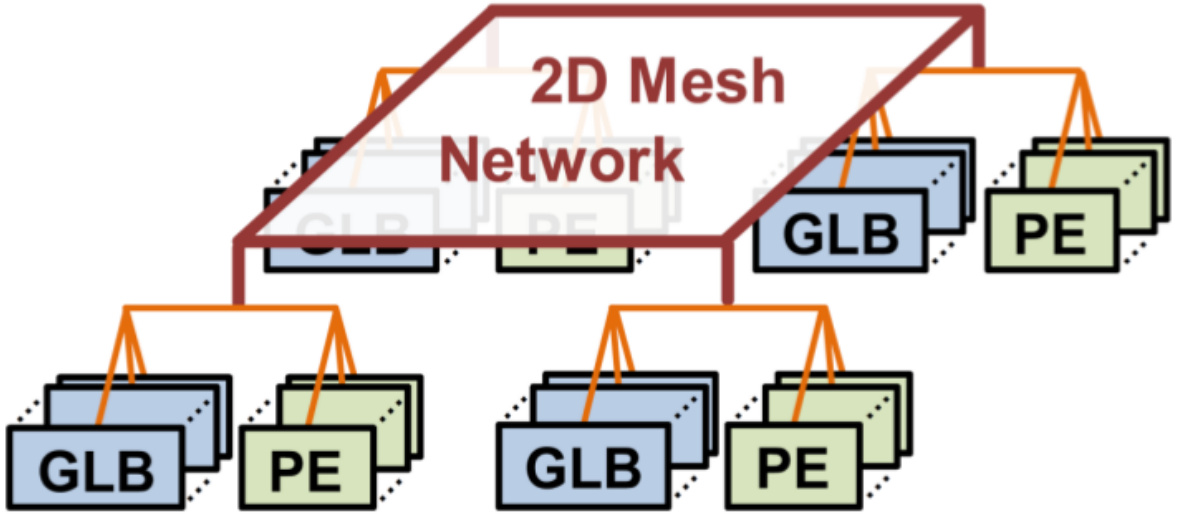


Eyeriss with Hierarchical Mesh Network

Eyeriss v1



Eyeriss v2



Outline

- Benchmarking Metrics
- GEMM Accelerator Design
- **DNN Accelerator Design**
  - Locality and Data Reuse
  - Dataflow Taxonomy
  - Network-on-Chip
  - **Optimization**
- Energy

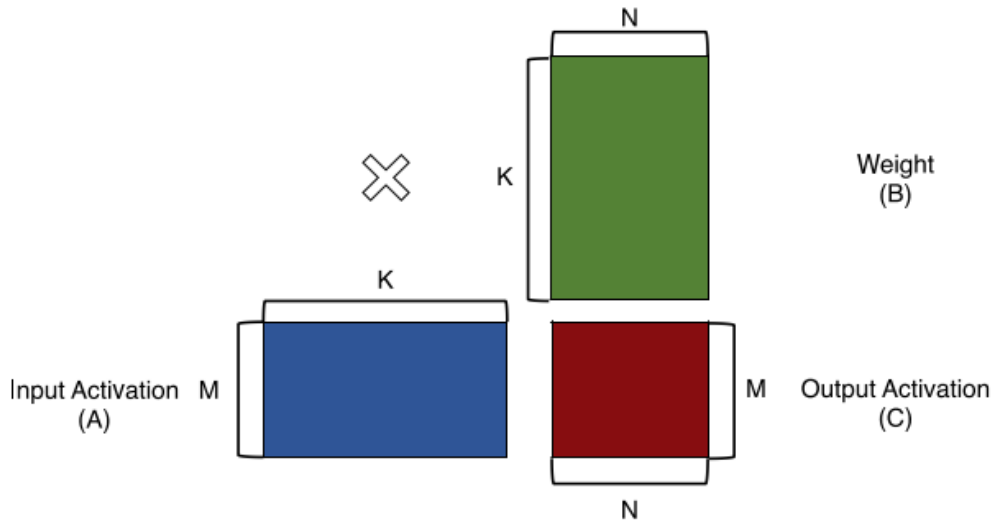


Datapath Optimization

- **spatial_for - to explore parallelism**
  - **Spatial-K: accumulation dimension**
    - Adder tree/dot product
    - Systolic accumulation
  - **Spatial-M/N: data reuse dimension**
    - Direct-wiring multicast
    - Systolic multicast
- **State-of-the-art accelerators typically use a combination of both at different levels of the hierarchy.**

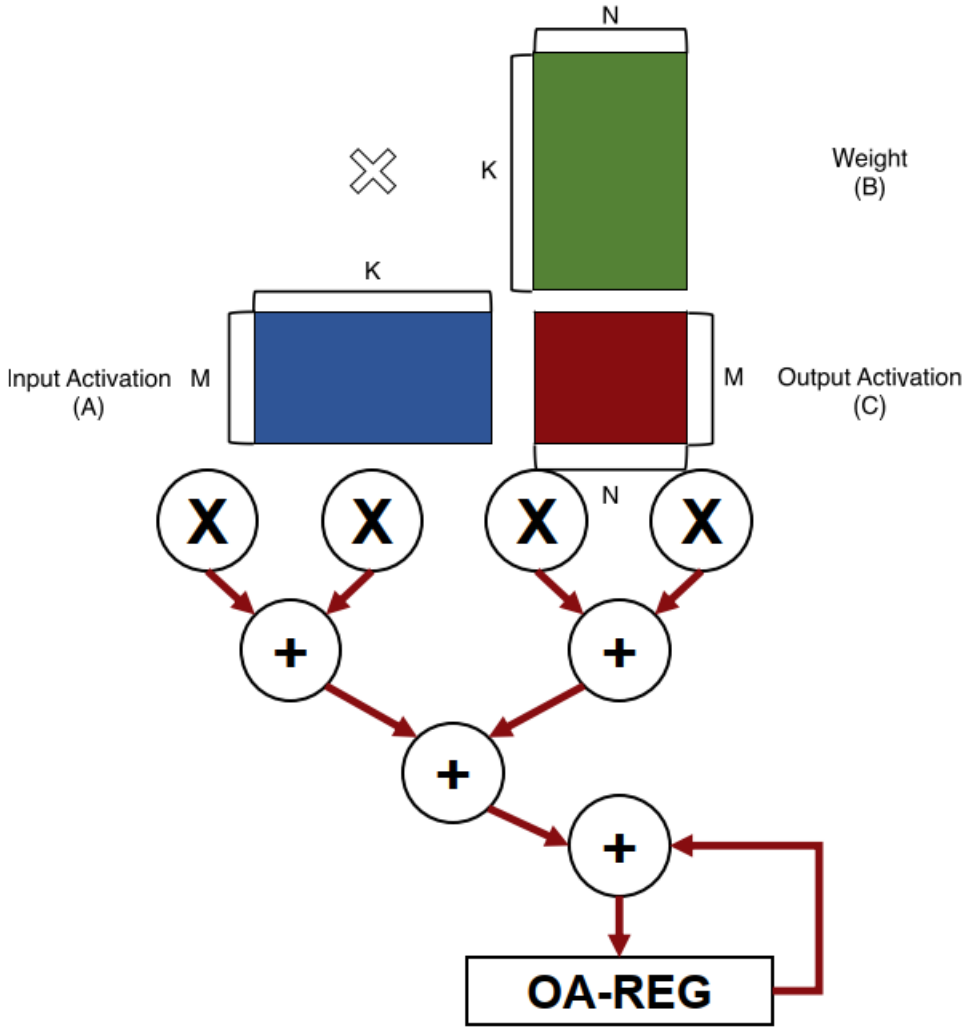


Matrix Multiply




```
for (m=0; m<M; m++) {
    for (n=0; n<N; n++) {
        OA[n,m] = 0;
        for (k=0; k<K; k++) {
            OA[n,m] +=  IA[m, k]* W[k, n];
        }
    OA[n,m] = Activation(OA[n,m]);
    }
}
```

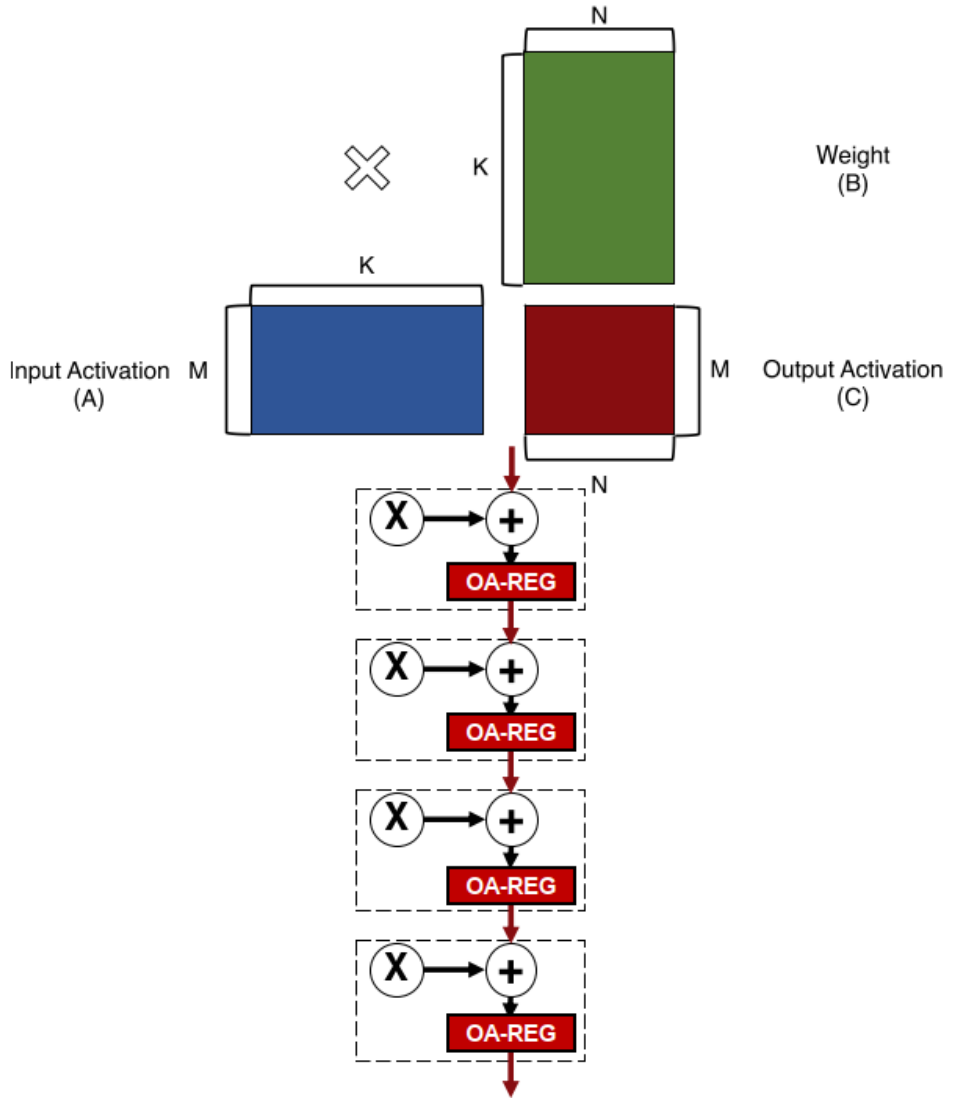
Datapath Optimization - Spatial-K




```
for (m=0; m<M; m++) {
    for (n=0; n<N; n++) {
        OA[n,m] = 0;
        spatial_for (k=0; k<K; k++) {
            OA[n,m] += IA[m, k]* W[k, n];
        }
    OA[n,m] = Activation(OA[n,m]);
    }
}
```

- Type 1: Adder Tree
- Example: NVDLA, DianNao
- Typical width: 8-64
- Applicable to any accumulation dimensions
  - E.g., R, S, C in convolution

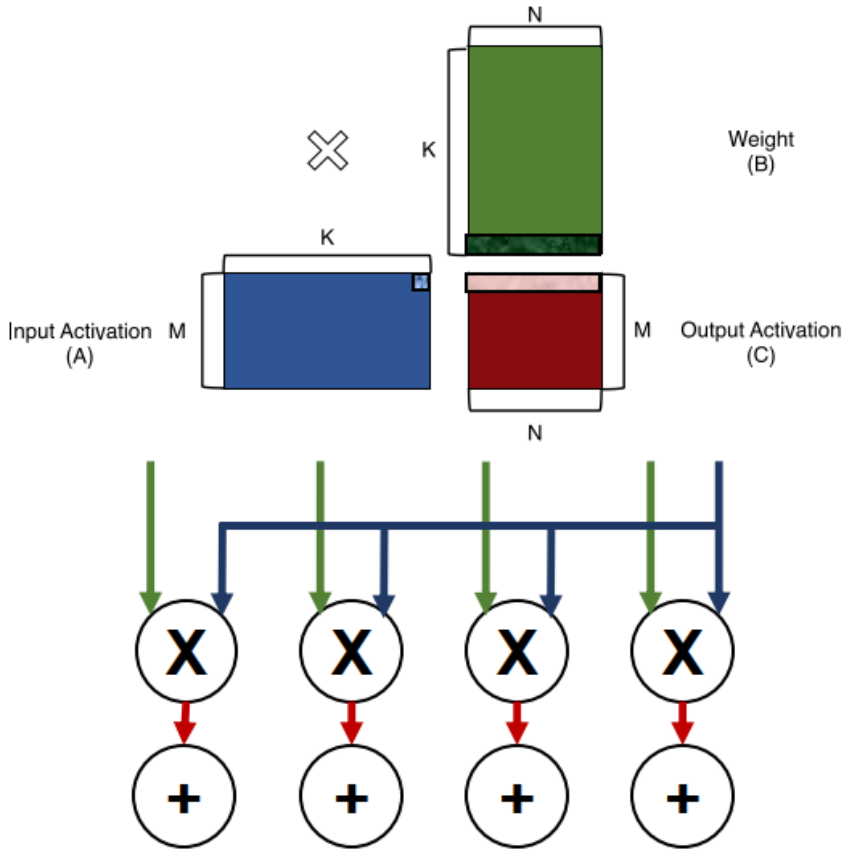
Datapath Optimization - Spatial-K




```
for (m=0; m<M; m++) {
    for (n=0; n<N; n++) {
        OA[n,m] = 0;
        spatial_for (k=0; k<K; k++) {
            OA[n,m] += IA[m, k]* W[k, n];
        }
    OA[n,m] = Activation(OA[n,m]);
    }
}
```

- Type 2: Systolic Accumulation
- Example: TPU, Gemmini
- Typical width: 8-256
- Applicable to any accumulation dimensions
  - E.g., R, S, C in convolution

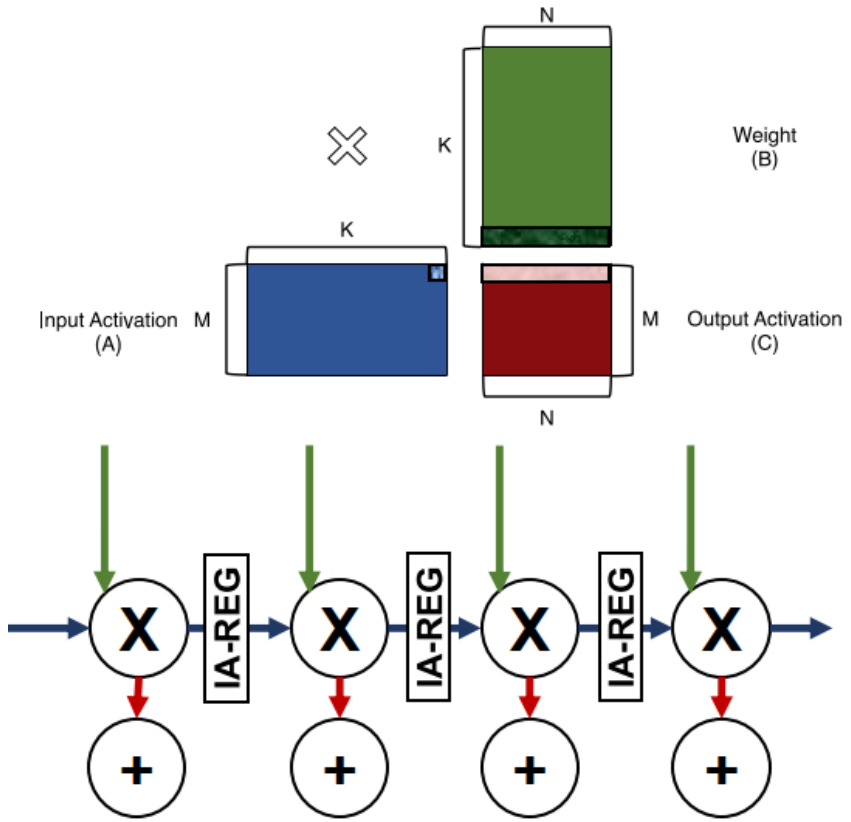
Datapath Optimization - Spatial-N




```
for (m=0; m<M; m++) {
    spatial_for (n=0; n<N; n++) {
        OA[n,m] = 0;
        for (k=0; k<K; k++) {
            OA[n,m] += IA[m, k]* W[k, n];
        }
        OA[n,m] = Activation(OA[n,m]);
    }
}
```

- Type 1: Direct-wiring multicast
- Example: NVDLA, DianNao
- Typical width: 8-16
- Applicable to any non-accumulation dimensions

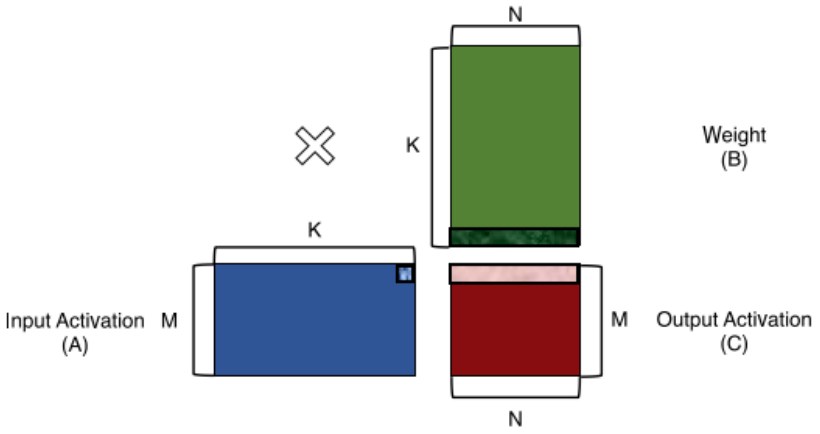
Datapath Optimization - Spatial-N



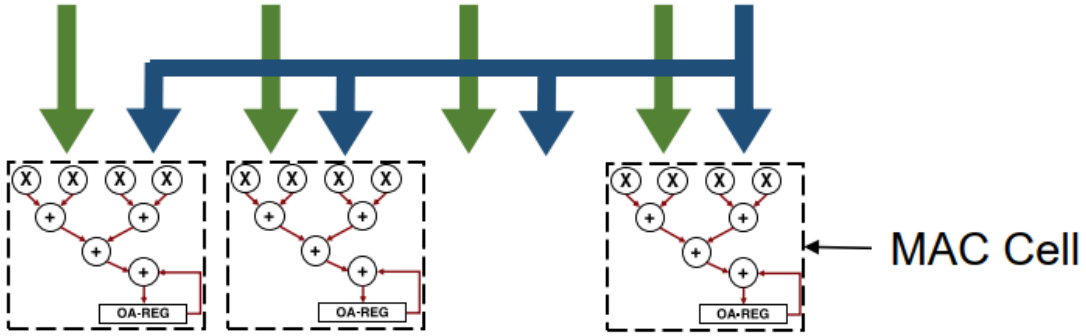

```
for (m=0; m<M; m++) {
    spatial_for (n=0; n<N; n++) {
        OA[n,m] = 0;
        for (k=0; k<K; k++) {
            OA[n,m] += IA[m, k]* W[k, n];
        }
        OA[n,m] = Activation(OA[n,m]);
    }
}
```

- Type 2: Systolic multicast
- Example: TPU, Gemmini
- Typical width: 8-256
- Applicable to any non-accumulation dimensions

Datapath Optimization Combined: NVDLA

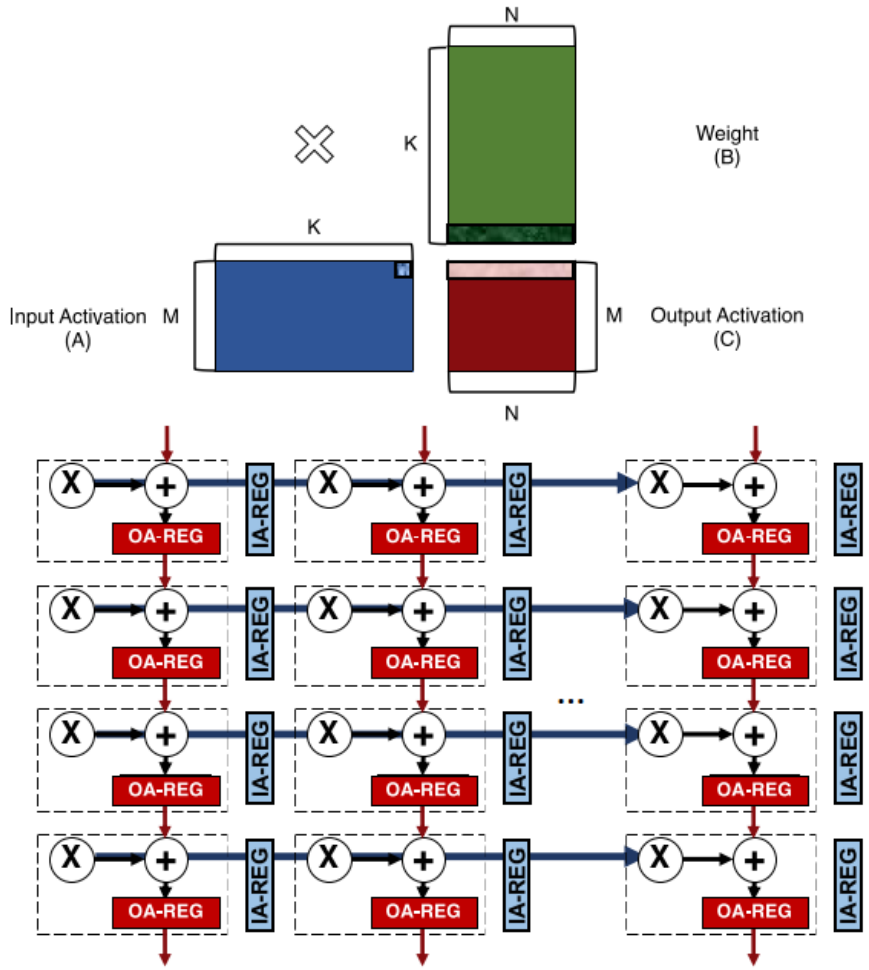



```
for (m=0; m<M; m++) {
    spatial_for (n=0; n<N; n++) {
        OA[n,m] = 0;
        spatial_for (k=0; k<K; k++) {
            OA[n,m] += IA[m, k]* W[k, n];
        }
        OA[n,m] = Activation(OA[n,m]);
    }
}
```



- **Adder-tree accumulation**
- **Direct-wiring multicast**

Datapath Optimization - Spatial-K



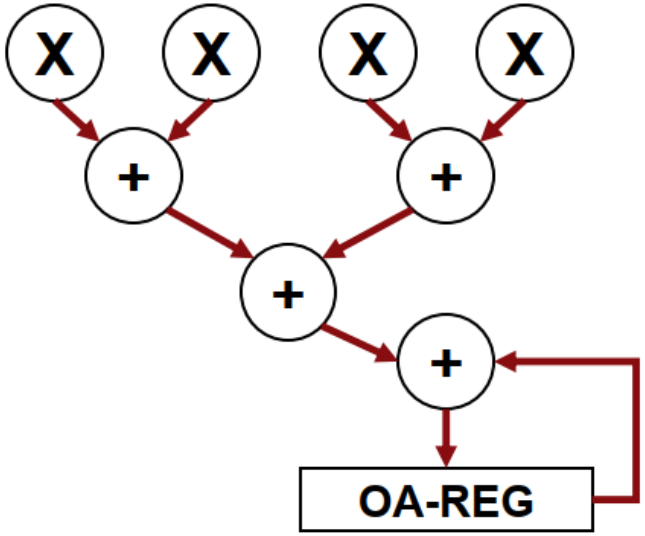

```
for (m=0; m<M; m++) {
    spatial_for (n=0; n<N; n++) {
        OA[n,m] = 0;
        spatial_for (k=0; k<K; k++) {
            OA[n,m] += IA[m, k]* W[k, n];
        }
        OA[n,m] = Activation(OA[n,m]);
    }
}
```

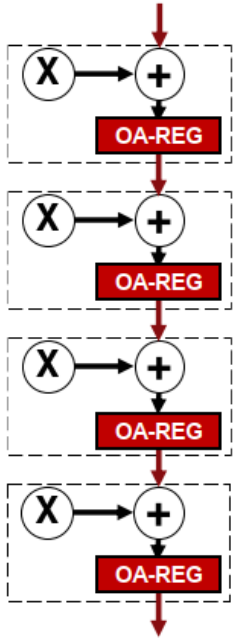
- **Systolic accumulation**
- **Systolic multicast**

Memory Optimization

Short-lived intermediate results

- Instead of accessing SRAM or a shared, large register file
  - Consume the intermediate results directly
- Example: adder tree and systolic accumulation

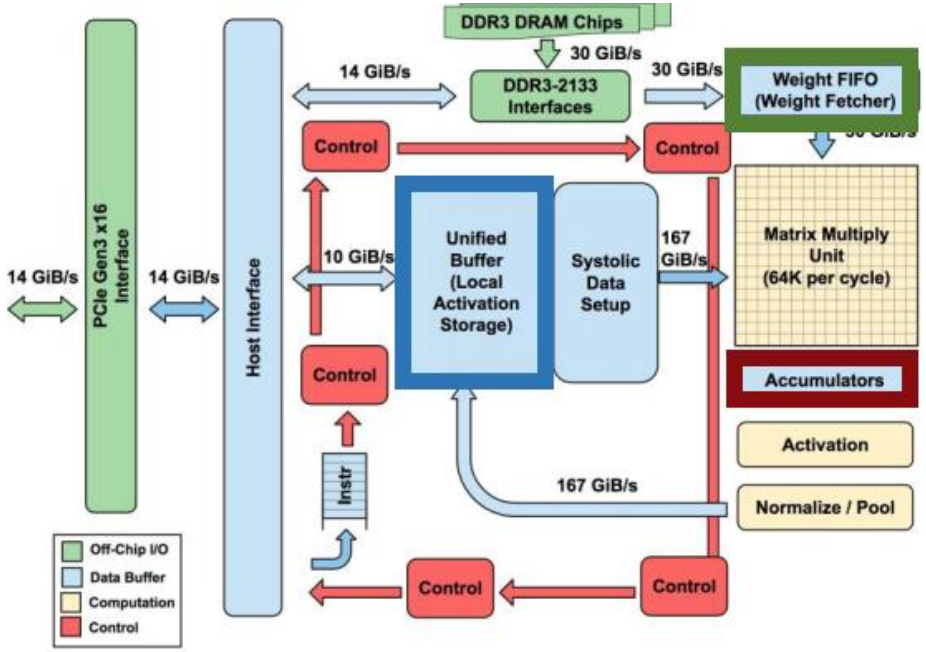




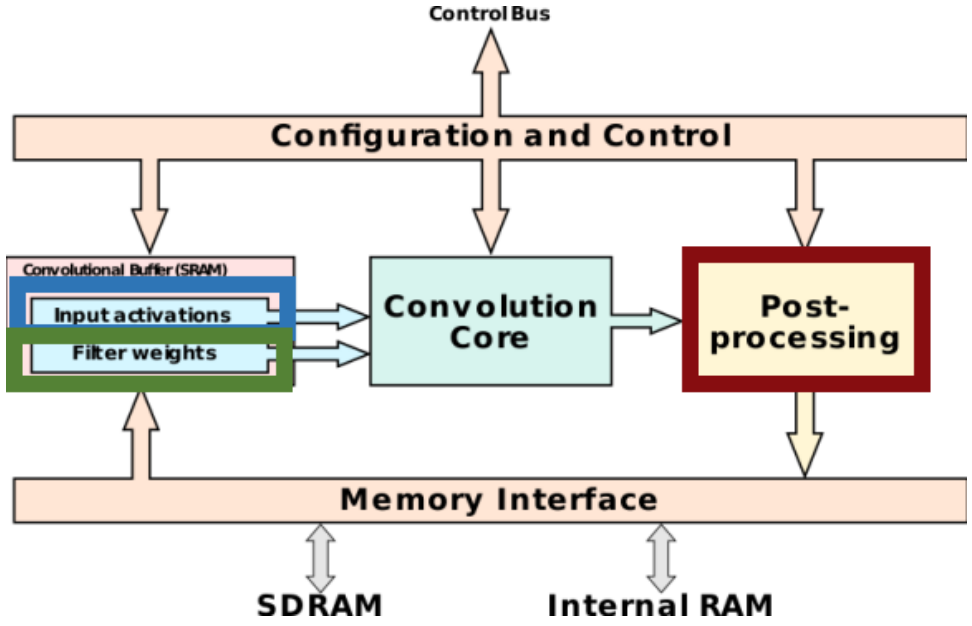
Memory Optimization

App-specific storage size/BW

- Dedicated storage for each operand



TPU

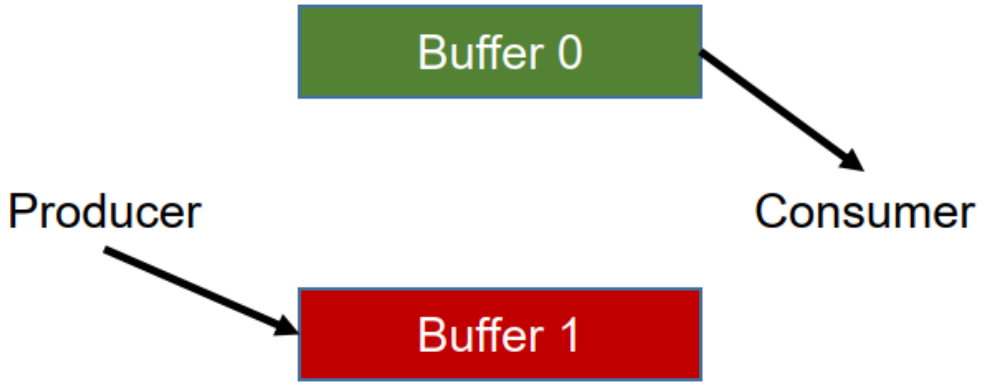


NVDLA

Memory Optimization

App-specific data delivery

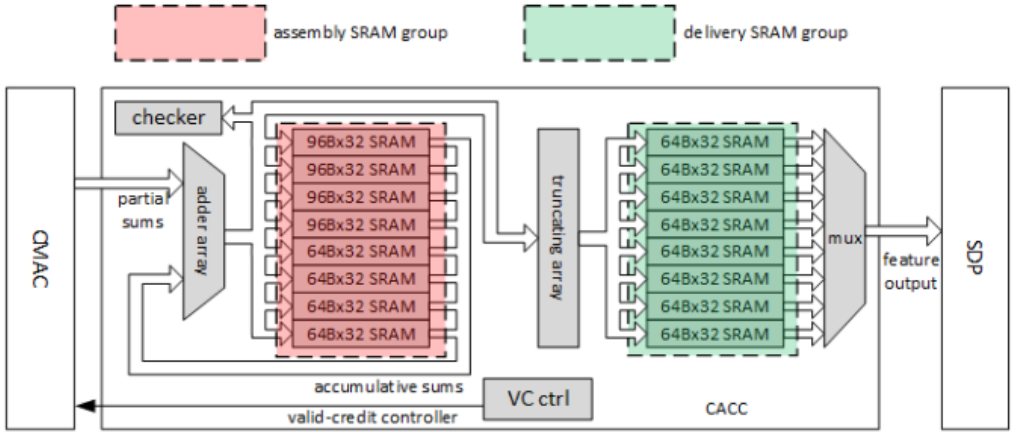
- Double-buffering
  - Goal: Overlap compute and communication by doubling the SRAM sizes.
  - Producer writes to a buffer while Consumer reads from the other buffer



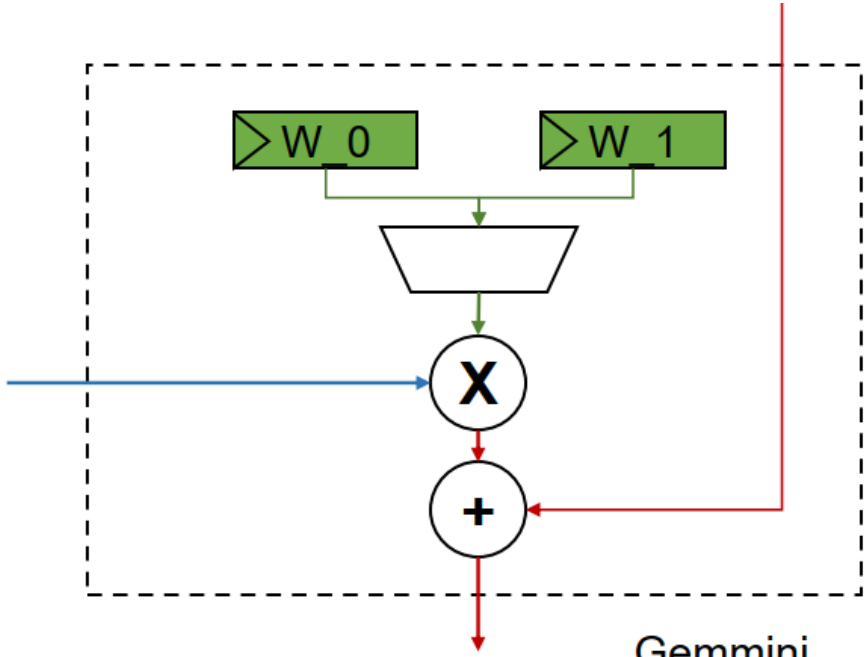
Memory Optimization

App-specific data delivery

- **Double-buffering**
  - Goal: Overlap compute and communication by doubling the SRAM sizes.
  - Producer writes to a buffer while Consumer reads from the other buffer



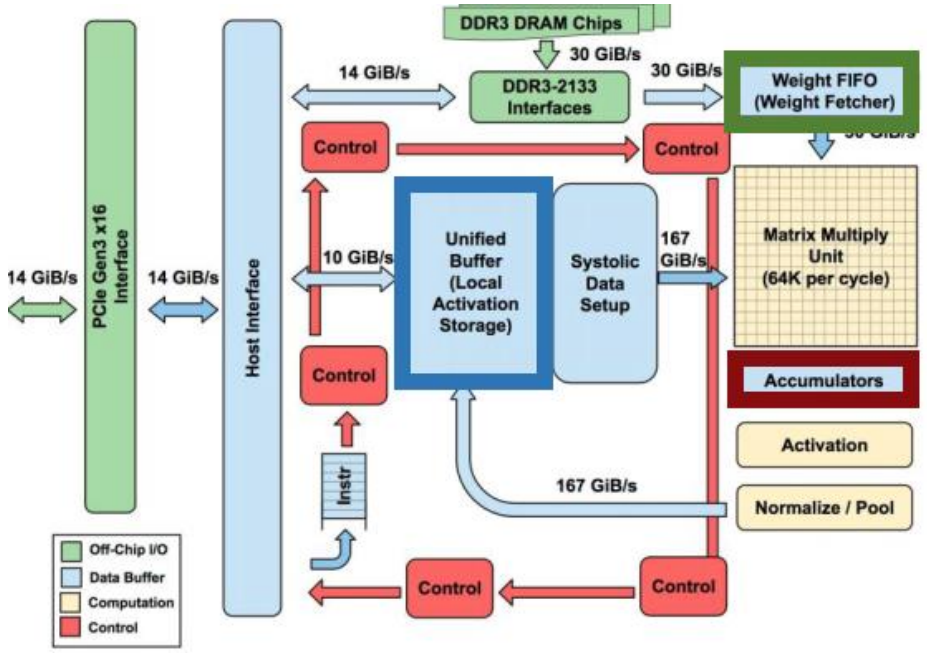
NVDLA



Gemmini

Tensor Processing Unit

- **Datapath:**
  - **Spatial-K: Systolic Accumulation**
    - Multi-cycle with registers
  - **Spatial-N: Systolic multicast**
    - Multi-cycle with registers
  - **Better scalability**
- **Memory**
  - **Custom systolic registers**
  - **Dedicated accumulation and weight buffers**
  - **Double-buffered, weight-stationary dataflow**



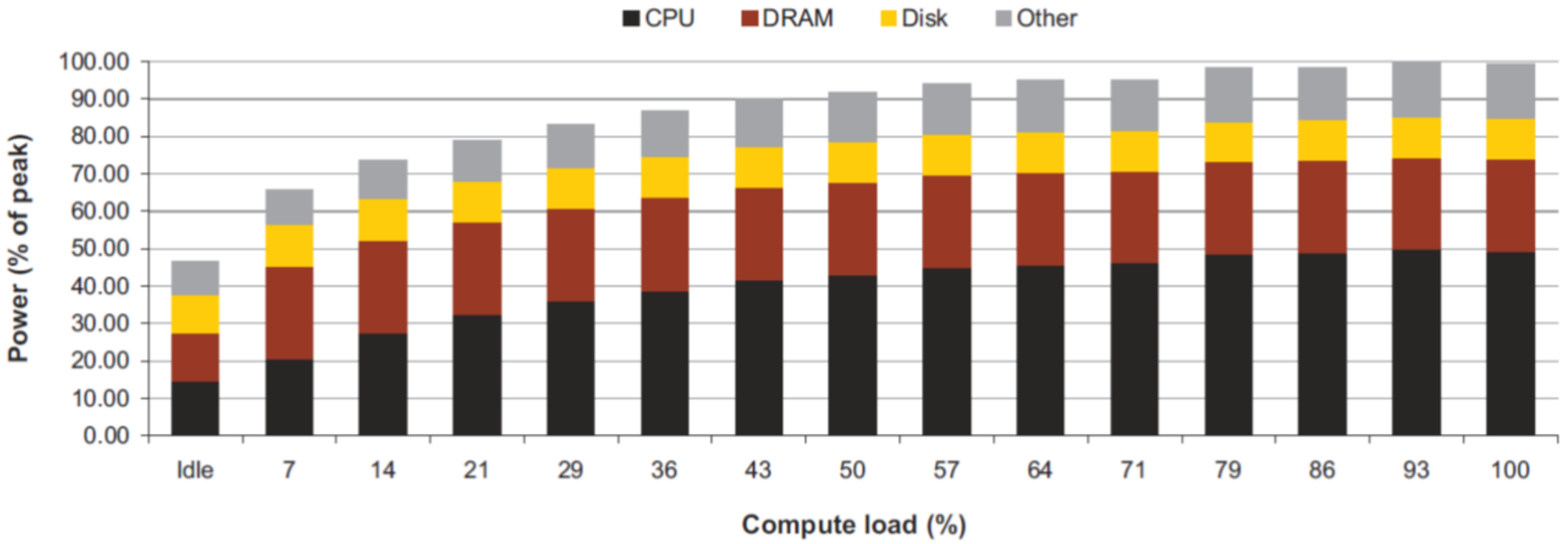
TPU

Outline

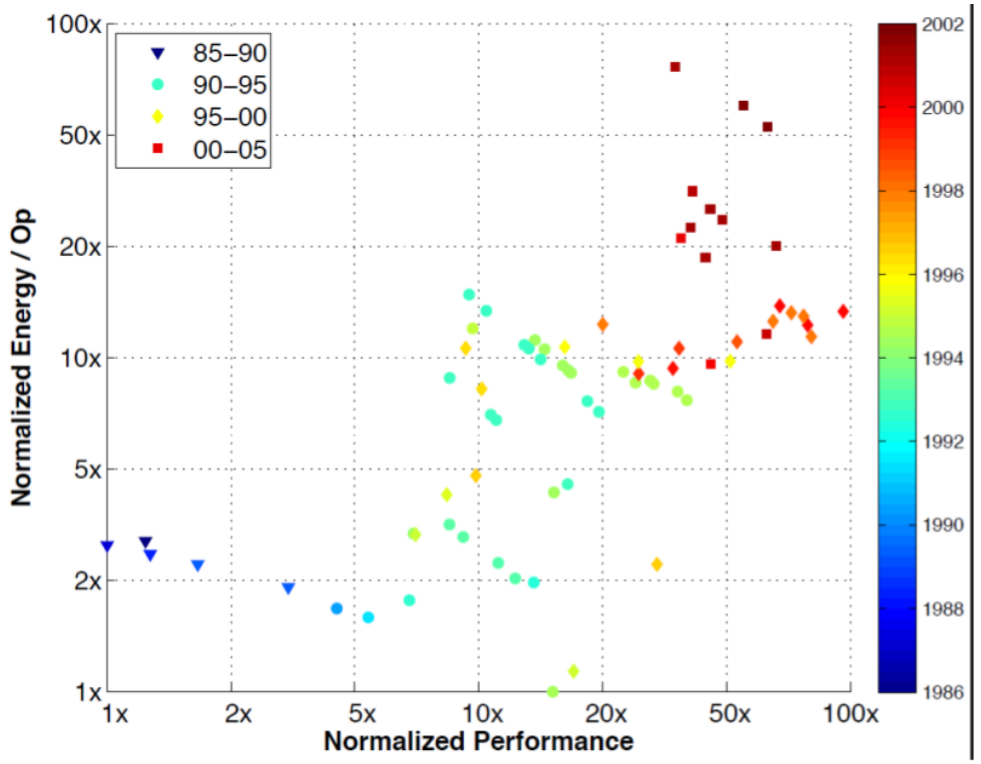
- Benchmarking Metrics
- GEMM Accelerator Design
- DNN Accelerator Design
- **Energy**

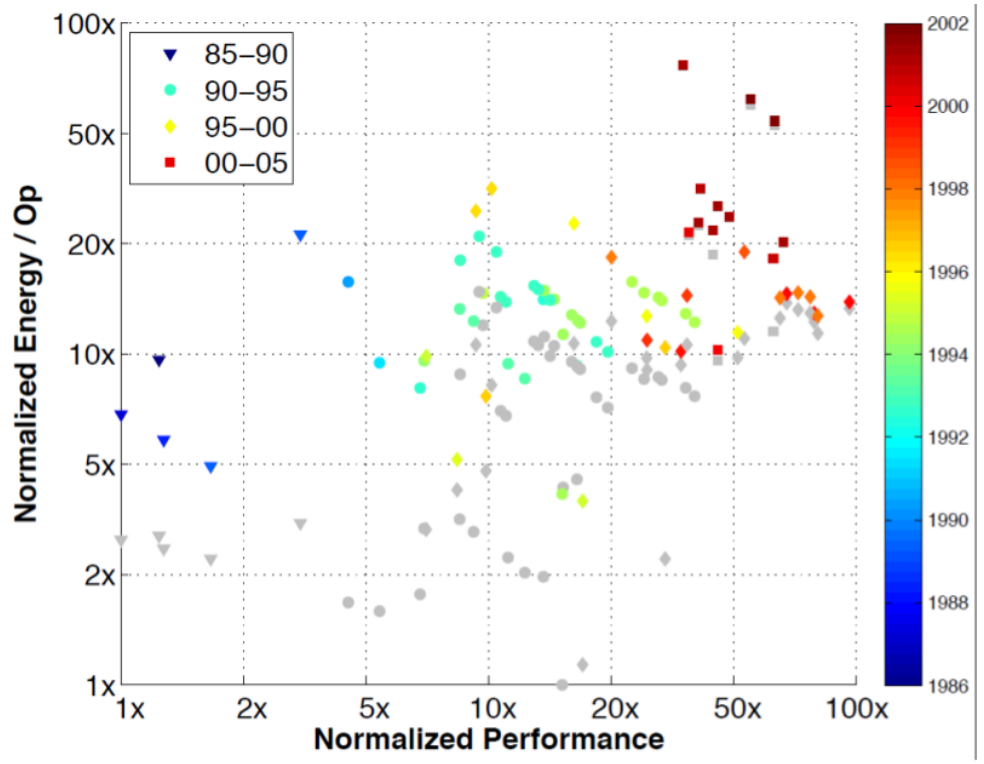


Don’t Forge Memory System Energy



Don’t Forget Cache Energy

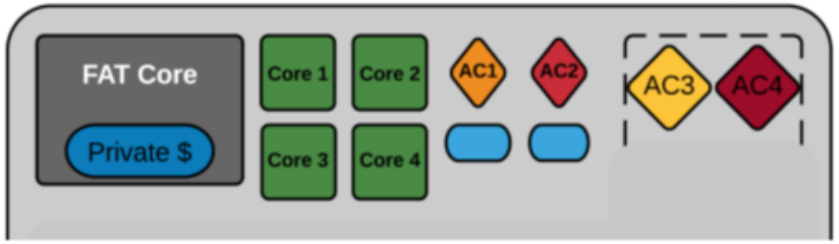




Energy with Corrected Cache Size

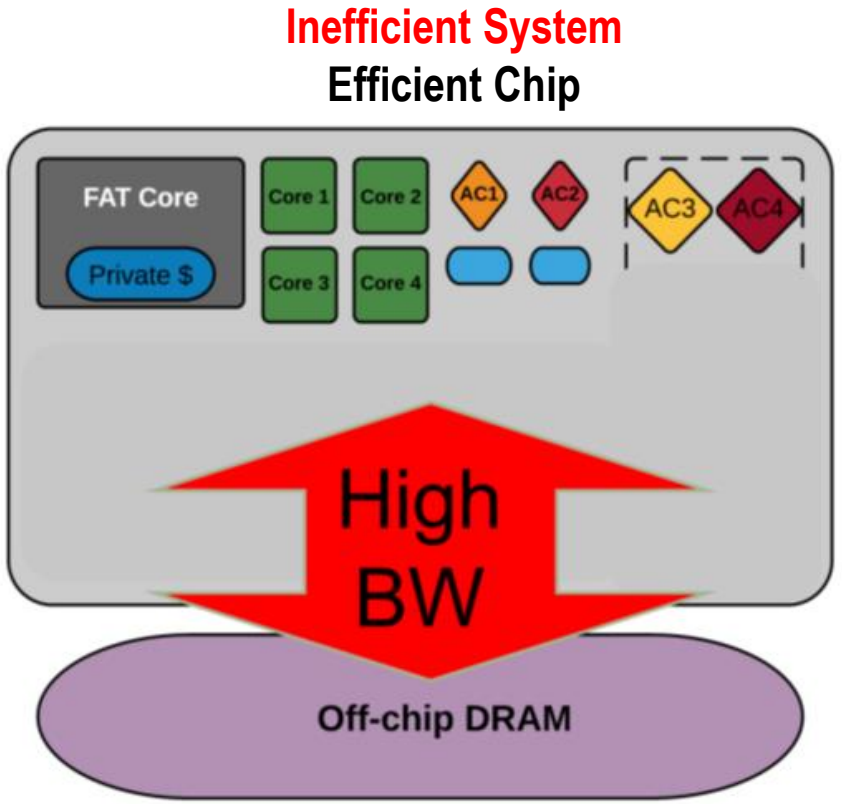
Dark Silicon View

- Efficient Accelerators
- Increase Chip's
  - Energy Efficiency
  - Area Efficiency
- Report 10~1000x improvements



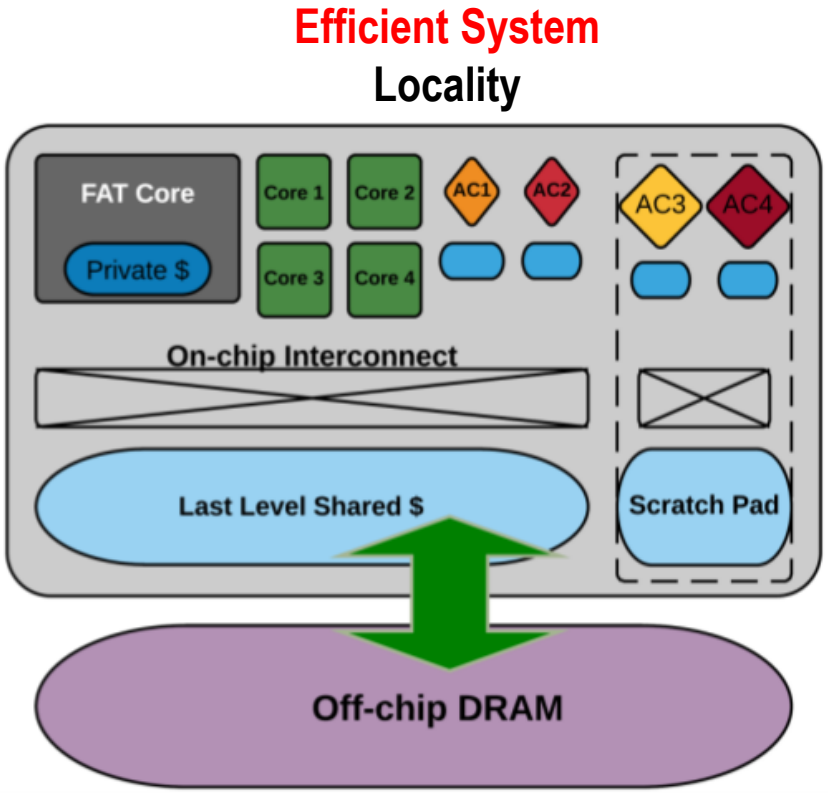
Real Perspectives

- Efficient Accelerators
- Increase ~~Chip's~~
  - Energy Efficiency
  - Area Efficiency
- Report ~~10~1000x~~ improvements



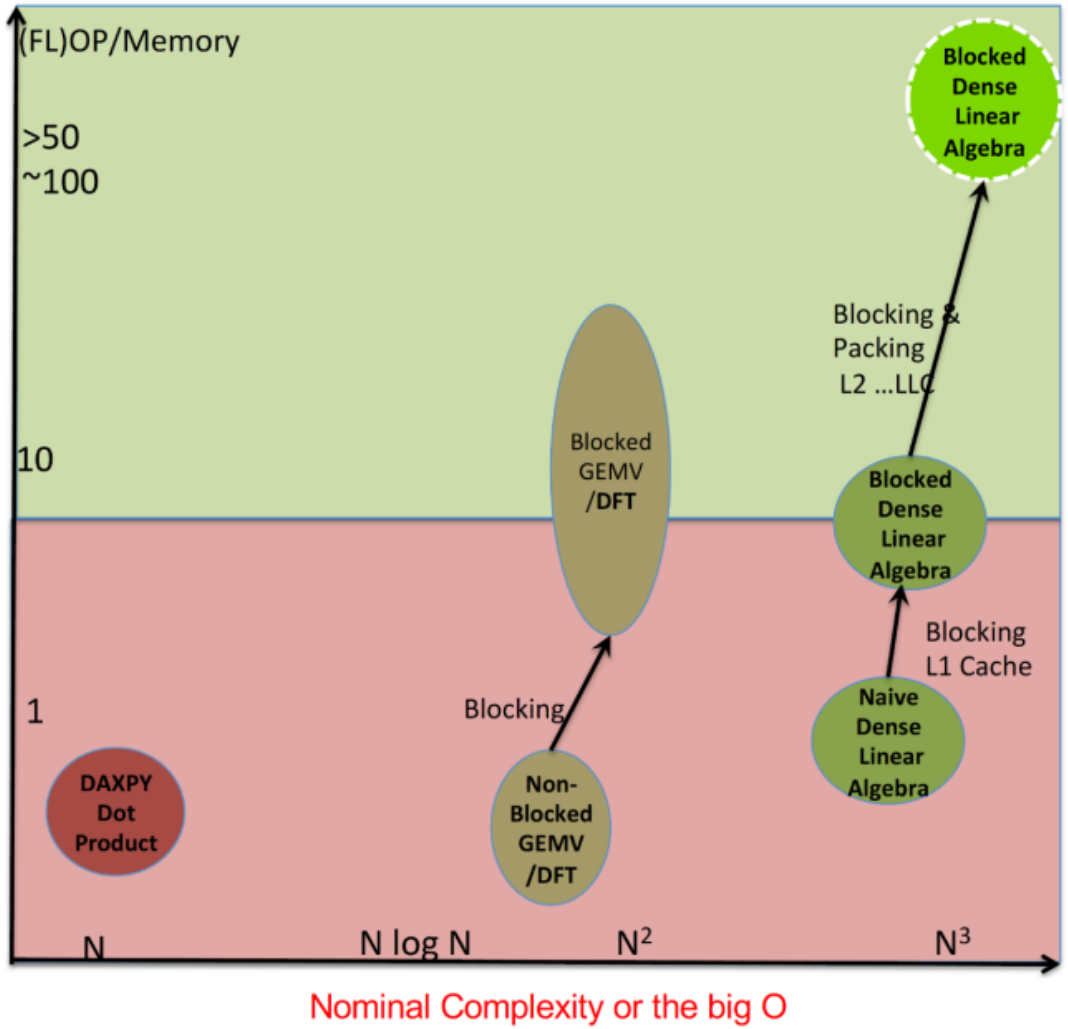
Dark Memory Perspectives

- Efficient System
- On-chip buffers
  - Less efficient chip
- Lower Off-chip Memory Bandwidth



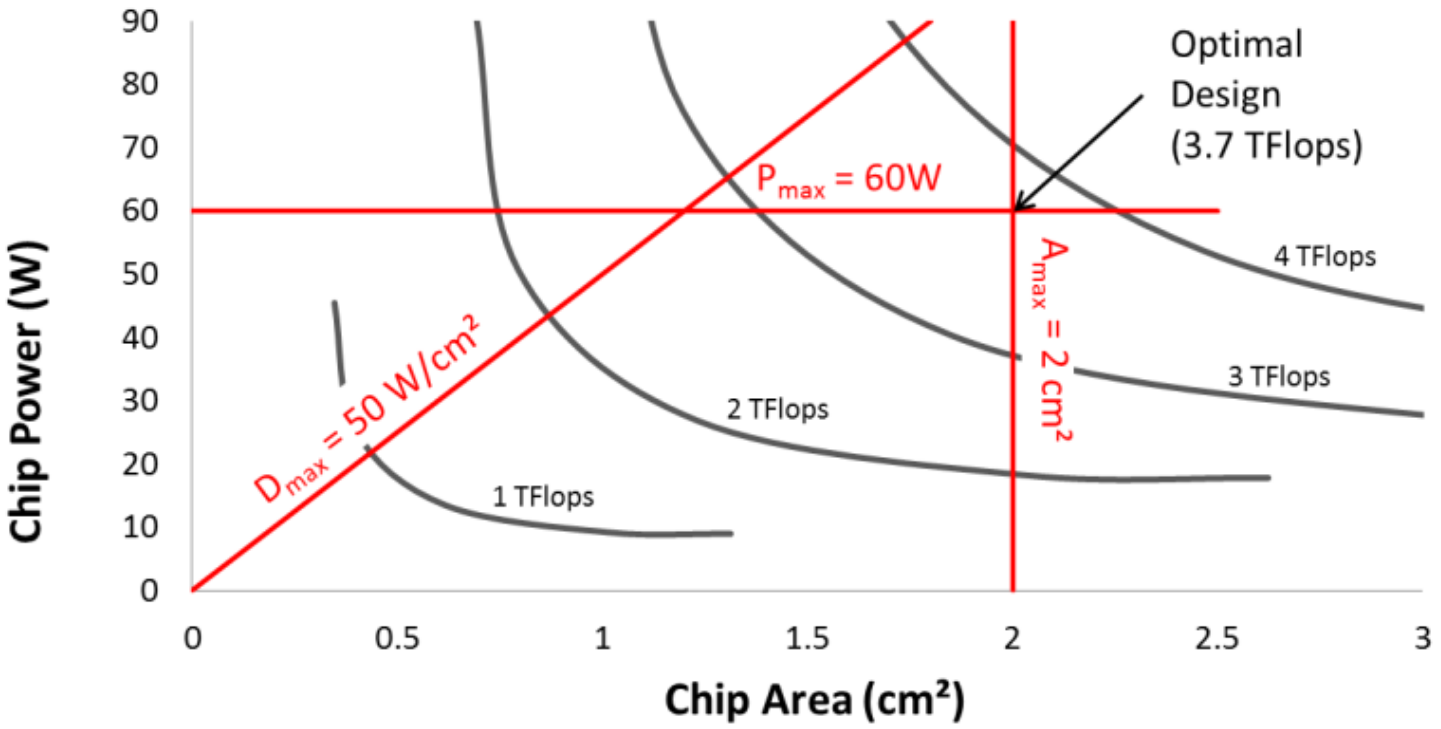
Dark Memory

- Keep DRAM and Memory Hierarchy Dark
- First Focus on Memory Hierarchy
- Algorithm: minimize DRAM access
- High chip level locality when parallelized



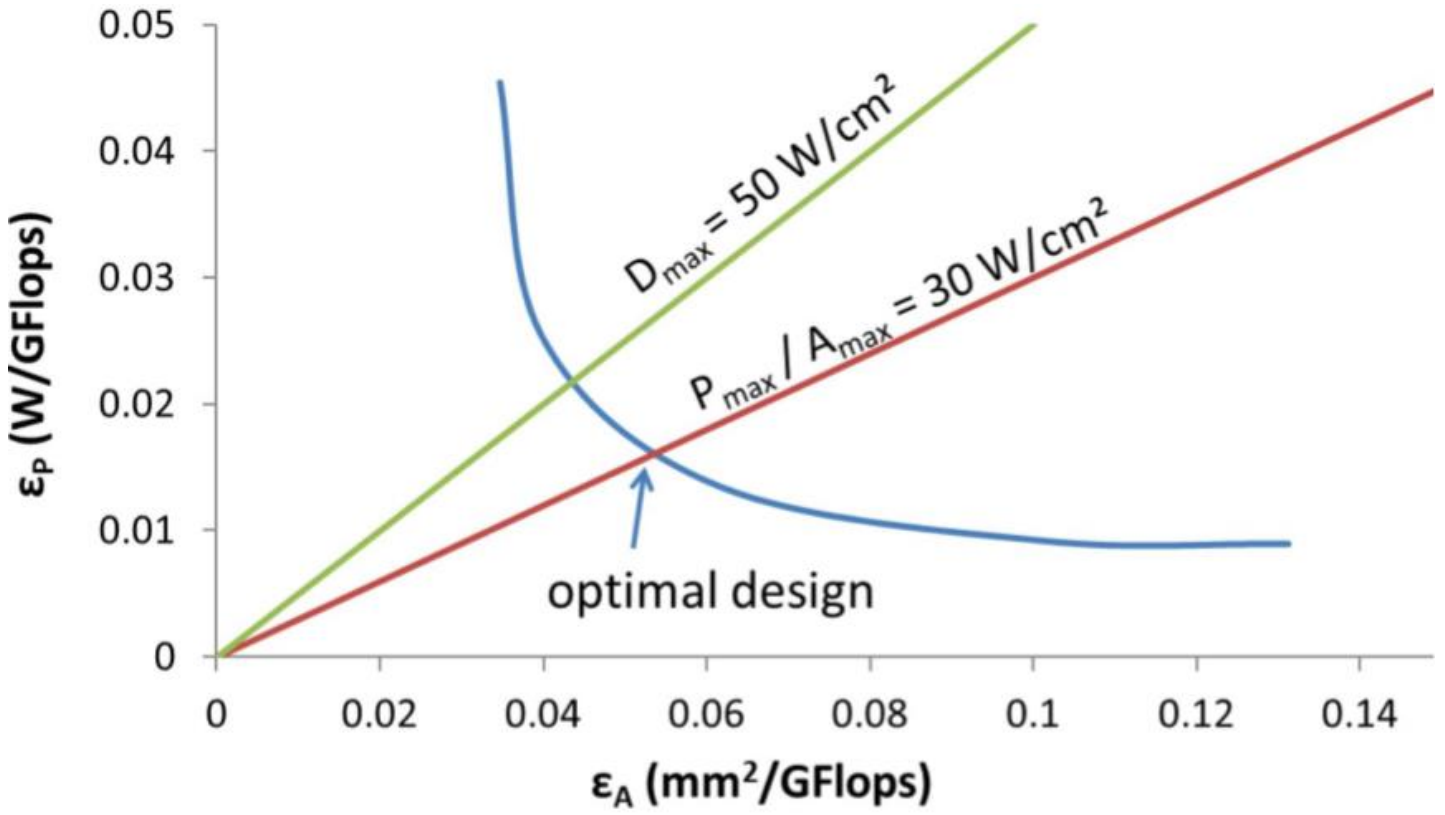
Metrics for Resource Constrained Computing

- Data Parallel Space
- More Performance
- More Hardware
- More Power
- More Area



Scale by Performance

- Energy/OP
- Area/OP
- One Pareto Optimal
- Diming the chip



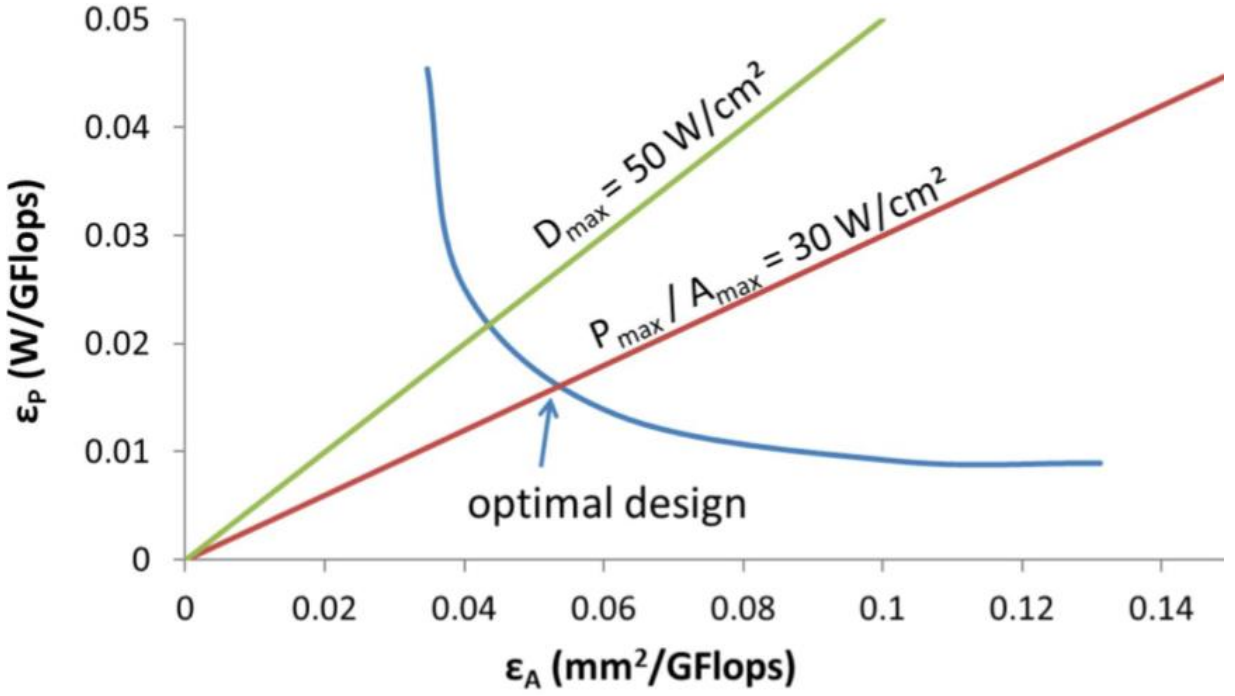
Always (FL)OPs?

- Define op
  - Invariant across the different implementations
- DFT and FFT
  - Op is one Transform
- (Convolutional) Neural Networks
  - Op is one Inference
- Dense and Sparse Solvers
  - Op is a number of solutions



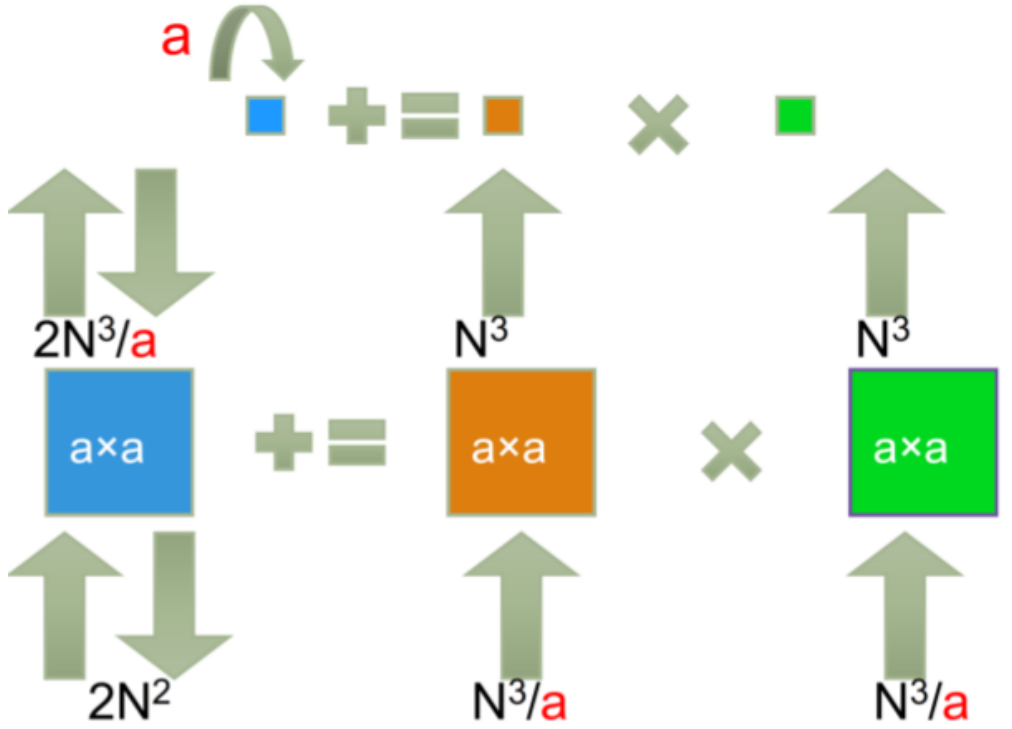
System Design Optimization

- Sensitivity study along the Pareto Curves (Puzzle)
- Trade Area/Performance of one IP with another
- Both Scalable and Non-scalable

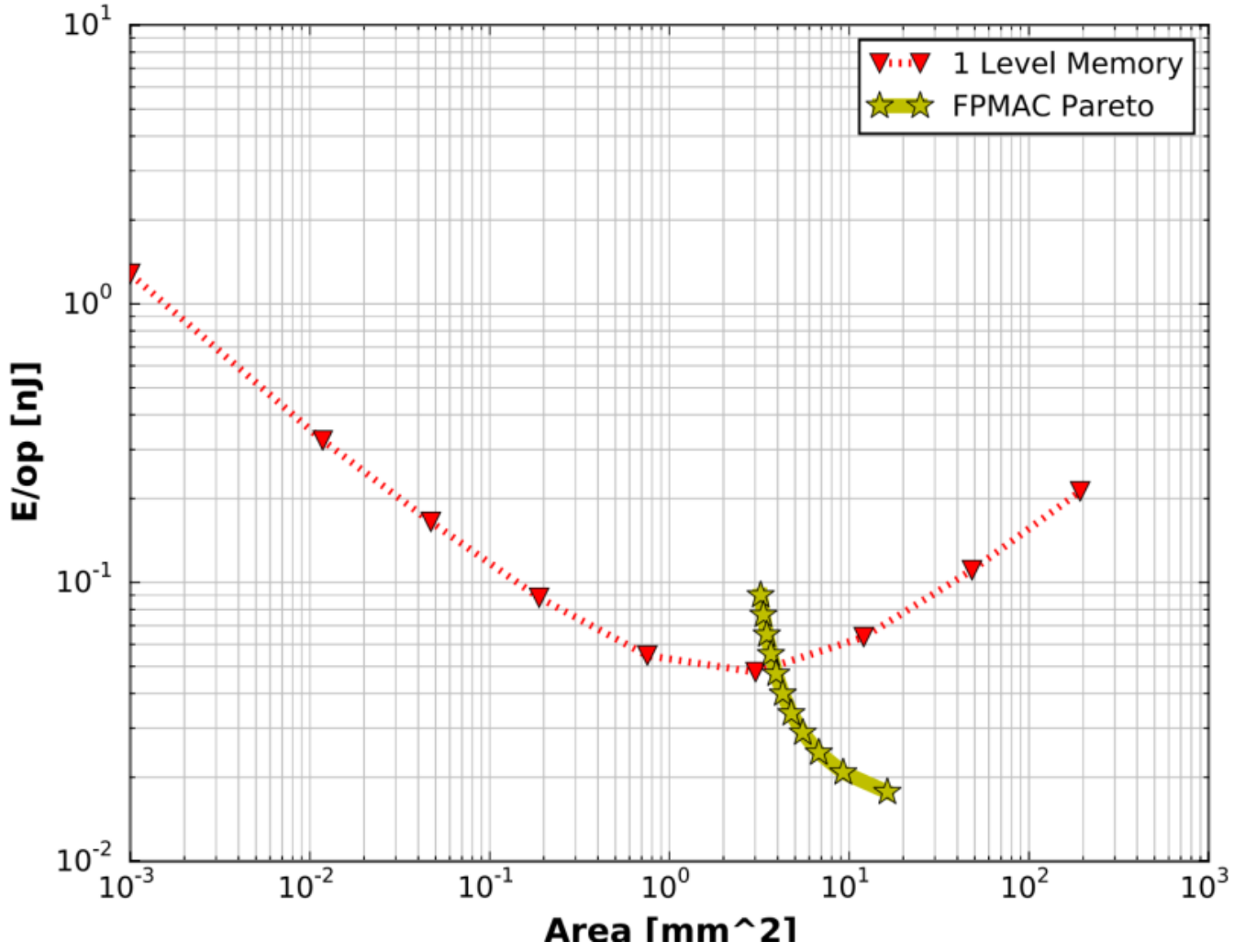


Example: Blocked GEMM

- A or B
  - From a x a buffer
  - N^3 / a
- C
  - Access for last level cache is $2N^2$
- Highest level always suffers from constant N^3 accesses to A and B



One Level Caching DP-FP



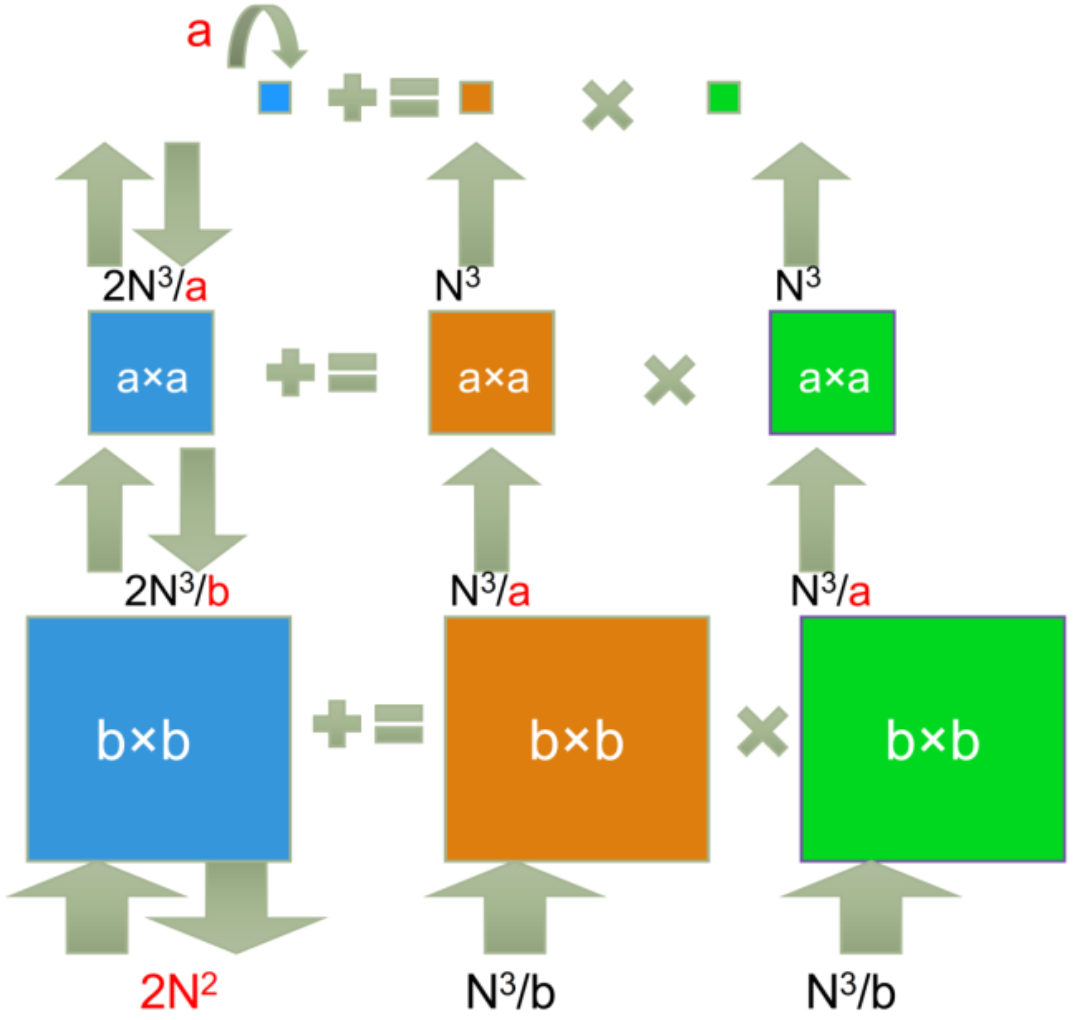
The Energy vs. Size of Top Level

- **Energy Consumption**
  - Boosts significantly as size grows
- **Memory Size:**
  - Does not compensate for the access savings

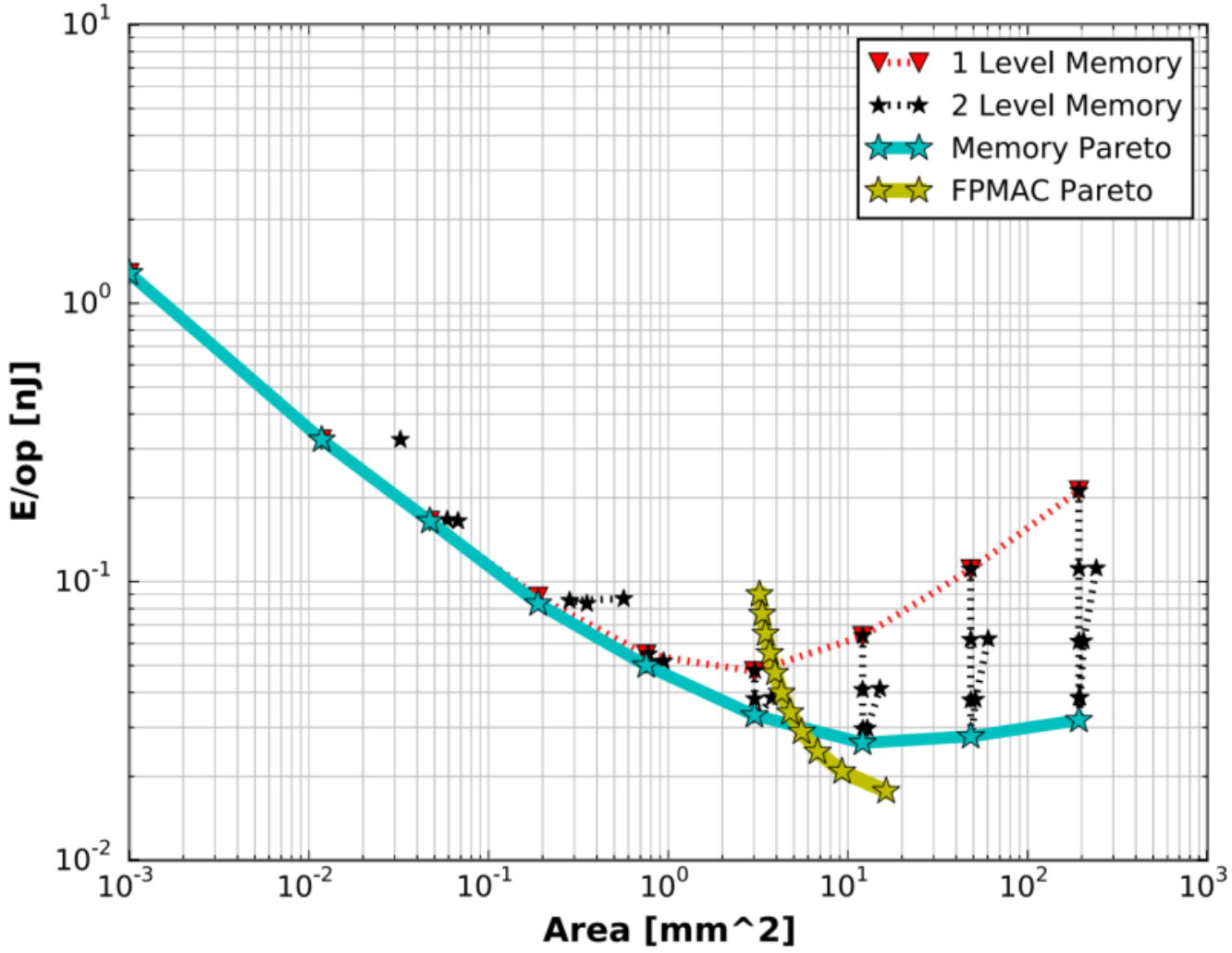


Let's Add Another Layer

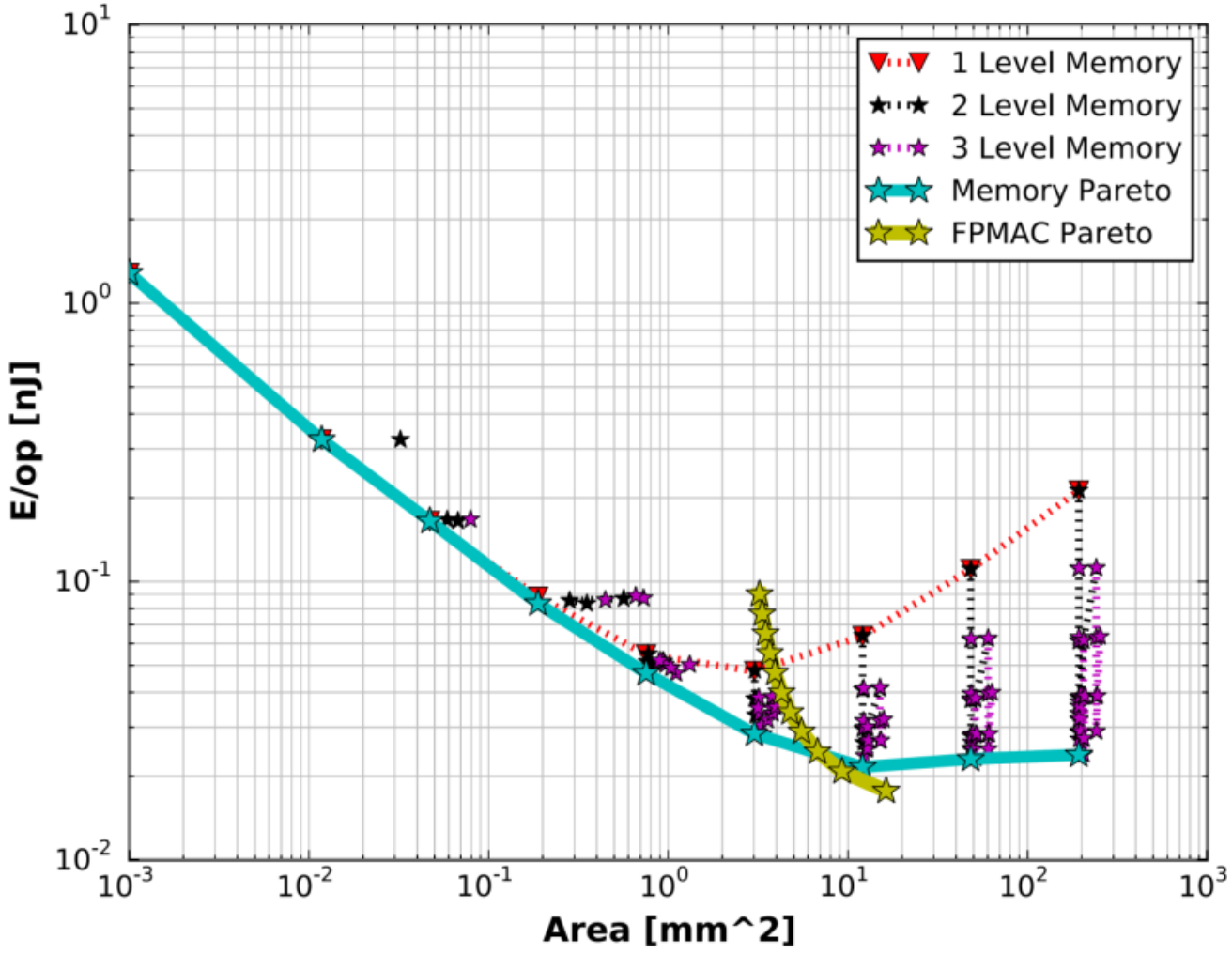
- A or B
  - From b x b buffer
  - N^3 / b
- C
  - Access for last level cache is $2N^2$



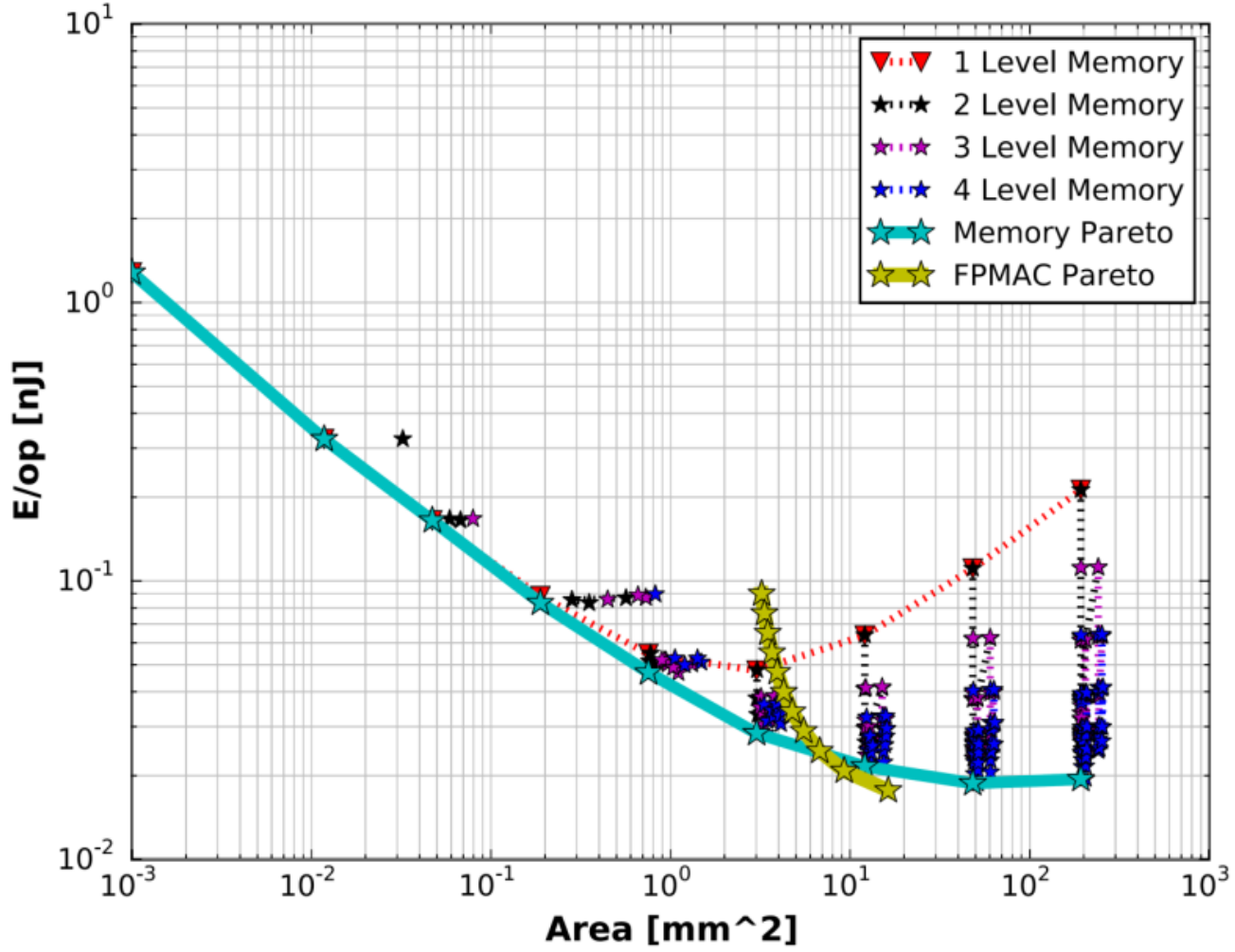
Second-level Caching



Three Levels

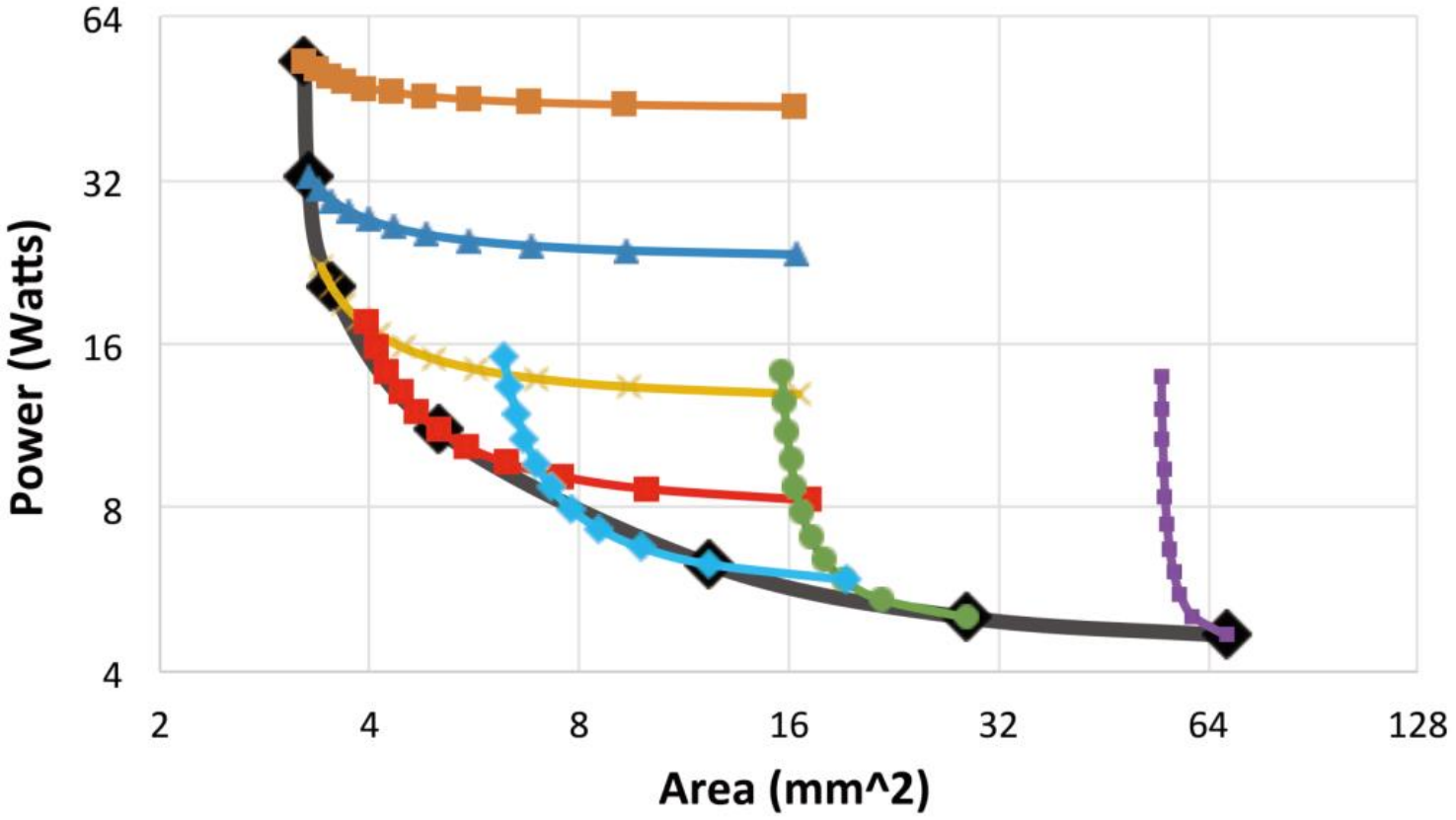


Memory Subsystem = Half Energy



Putting It All Together

- Merging FPU Pareto with Memory Pareto



Algorithm/Architecture Co-design Methodology

